

NEON RIFT

NEON RIFT: NIGHT RUN

COMPLETE TECHNICAL DOCUMENTATION

A Technical Book of the Game's Architecture, Systems, Implementation and Development

Architecture • Systems • Development • Implementation • Testing • Technical Reference

Soyeb Mohammad

Background: the game's own title shot (12_Title), real-time capture, `Docs/Screenshots/Intro2/13_80.5s.jpg`

NEON RIFT

NEON RIFT: NIGHT RUN

COMPLETE TECHNICAL DOCUMENTATION

Architecture • Systems • Development • Implementation • Testing • Technical Reference

A Technical Book of the Game's Architecture, Systems, Implementation and Development

PREPARED FOR

Project Demonstration / Academic Evaluation / Technical Reference

AUTHOR

Soyeb Mohammad

PDF edition compiled from `Docs/NEON_RIFT_BOOK/` · documented code state: commit `602c7130` · Unity 6000.6.4f1 · October 2026

Project Information

Title	**Neon Rift: Night Run** (`productName` in `ProjectSettings/ProjectSettings.asset` , version `0.1.0`)
Genre	Arcade-simulation driving, heist, open-route racing
Engine	Unity **6000.6.4f1**, Universal Render Pipeline 17.6
Key packages	Input System 1.19, Cinemachine 6.6, Timeline 6.6, glTFast 6.20
Platform	Desktop (macOS player builds exist; PC and Mobile URP assets are both configured)
Mission	Night Run (`Assets/_NeonRift/Data/Missions/Mission_NightRun.asset`)
Player cars	SLS AMG, SLS AMG GT3, Terzo Millennio
Rival crews	VEX and KADE, driving the two cars the player did not pick
Code	198 C# files, about 36,000 lines, 13 assemblies
Tests	125 EditMode test cases (last recorded run: 125 passed, 0 failed)
Repository	`https://github.com/soyebmohammad03-dev/NEON-RIFT` (branch `main`)
Documented state	Commit `602c7130` . Later commits change documentation only.
Author	Soyeb Mohammad

How to use this book

Master index. A complete, searchable technical documentation set for the Neon Rift: Night Run Unity project, written from the repository as it stands at commit `602c7130` (Unity 6000.6.4f1, URP 17.6).

How to use it: find your question in **Find It Fast** below, jump to the chapter, and follow the file paths. Paths are repository-relative. In tables, `S/` means `Assets/_NeonRift/Scripts/` and `D/` means `Assets/_NeonRift/Data/` . For viva or interview preparation start with 47 Teacher Q&A. For a one-page map, see 46 Developer Quick Reference.

CONVENTIONS

- `Monospace` marks file paths, class names, methods, ids and asset names. Paths are repository-relative.
- In dense tables, `S/` = `Assets/_NeonRift/Scripts/` , `D/` = `Assets/_NeonRift/Data/` , `P/` = `Assets/_NeonRift/Prefabs/` . Find It Fast and the Viva guide spell paths out in full.
- Shaded Implementation status boxes mark statements about what is **not** implemented. These are deliberate and accurate.
- Chapter and figure references are clickable: the Contents, List of Figures, Find It Fast and every “In this chapter” box link to their targets. The PDF also has bookmarks for every part, chapter and section.
- Line numbers (for example “line 163”) refer to the source at the documented commit.

Contents

Project Information 3
List of Figures 6
Find It Fast 7
Master Index 12

Part I **Foundations** 18
01 Project Overview 19
02 Architecture 24
03 Tech Stack 30
04 Project Structure 33
05 Game Flow 37
06 Bootstrap and Scenes 41

Part II **The World** 44
07 City and World Generation 45
08 Buildings and Environment 51
09 Roads and Route Network 56

Part III **Vehicles** 60
10 Vehicle System 61
11 Vehicle Physics 65
12 Vehicle Audio 71
13 Vehicle Visuals 74
14 Car Catalog and Data 77
15 Car Selection 81
16 Player Input 85

Part IV **AI and Traffic** 88
17 Rival AI 89
18 Traffic System 95

Part V **Missions and the Heist** 99
19 Mission System 100
20 Heist System 106
21 Security Gate 110
22 Data Core 114
23 Heat and Lockdown 120
24 Environmental Interactions 125

Part VI	**Rendering and Effects**	**127**
25	Lighting	128
26	Weather and Atmosphere	131
27	Water and Effects	133
28	Particles and VFX	135
Part VII	**Audio and Music**	**137**
29	Audio System	138
30	Music System	143
Part VIII	**Cinematics**	**145**
31	Cinematic Intro	146
32	Timeline and Camera	155
Part IX	**Interface and Presentation**	**157**
33	UI and HUD	158
34	Minimap	162
35	Objectives and Mission Presentation	165
36	Results and Debrief	168
Part X	**Animation and Camera**	**170**
37	Animation Systems	171
38	Camera System	174
Part XI	**Engineering, Quality and Release**	**176**
39	Performance	177
40	Testing and Validation	180
41	Input Playtesting	184
42	Build and Release	186
43	Git and Repository	189
44	Assets and Licensing	192
45	Known Limitations	196
Part XII	**Reference**	**198**
46	Developer Quick Reference	199
47	Teacher Q&A Guide	203
48	Glossary	216
Back	**Appendices**	**000**
A	Script Index (all 198 C# files)	219
B	Documentation Audit	232

List of Figures

0.1 Overall game architecture: major system dependencies 16
1.1 Core gameplay loop (Night Run) 21
2.1 Assembly dependency graph (13 assemblies) 25
2.2 Game startup flow 26
2.3 Scene and game-state flow 27
2.4 Data flow from Car Select into the mission 28
5.1 Game flow: screens and transitions 38
7.1 City generation flow (NightRunBuilder.Build) 47
9.1 Route network: gate state to routes 58
10.1 Vehicle architecture 62
11.1 Vehicle physics pipeline (VehicleController.Step, 100 Hz) 66
13.1 Vehicle prefab build pipeline 74
14.1 Vehicle catalog data relationships 77
15.1 Car Selection flow 82
16.1 Vehicle input flow 86
17.1 Rival AI architecture 90
17.2 Rival heist flow (per crew) 91
19.1 Objective state machine (MissionProgress) 102
19.2 Night Run mission flow 104
20.1 Multi-stage interaction sequence 108
21.1 Security Gate hack flow (Interaction_HackGate) 112
22.1 Data Core chamber phases (DataCoreChamber) 116
22.2 Data Core heist flow 118
23.1 Heat and lockdown reactions 122
29.1 Audio architecture 139
31.1 Cinematic intro runtime flow 148
31.2 Intro timeline: the 25 shots and the title cards 150
33.1 HUD data flow 160

Find It Fast

Every common question about the project, the chapter that answers it, and the exact place in the repository where it is implemented. Read left to right: **question → chapter → code**.

PART I · FOUNDATIONS

How does the game start up? → 05 Game Flow 06 Bootstrap and Scenes

- → `Assets/_NeonRift/Scripts/Game/BootstrapLoader.cs`
- → `Assets/_NeonRift/Scripts/Game/GameRoot.cs`
- → `Assets/_NeonRift/Scripts/Game/GameFlow.cs`

How are scenes loaded and switched? → 06 Bootstrap and Scenes

- → `GameFlow.TransitionAsync`

PART II · THE WORLD

How is the city generated? → 07 City and World Generation

- → `Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.cs`
- → `Assets/_NeonRift/Scripts/Editor/District/CityLayout.cs`
- → `Assets/_NeonRift/Scripts/Editor/District/NightRunDistrict.cs`

How are buildings made? → 08 Buildings and Environment

- → `NightRunDistrict.BuildStreetWall`
- → `Assets/_NeonRift/Scripts/Editor/World/BuildingPrefabBuilder.cs`
- → `Assets/_NeonRift/Scripts/Editor/District/CityInfill.cs`
- → `Assets/_NeonRift/Scripts/Editor/District/TowerDressing.cs`

Where do yards, car parks, parks come from? → 08 Buildings and Environment

- → `Assets/_NeonRift/Scripts/Editor/District/CityInfill.cs`

Street furniture, signals, signs, CCTV? → 08 Buildings and Environment

- → `Assets/_NeonRift/Scripts/Editor/District/CityProps.cs`

Landmarks (skyway, plaza, cranes)? → 08 Buildings and Environment

- → `Assets/_NeonRift/Scripts/Editor/District/CityLandmarks.cs`

How do roads / routing / GPS work? → 09 Roads and Route Network

- → `Assets/_NeonRift/Scripts/World/RoadGraph.cs`
- → `Assets/_NeonRift/Scripts/Gameplay/Navigation/CityNavigation.cs`

PART III · VEHICLES

How does car physics work? → 11 Vehicle Physics

- → `Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleController.cs`
- → `Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleWheel.cs`
- → `Assets/_NeonRift/Scripts/Vehicles/Physics/TyreModel.cs`

Suspension / tyres / gearbox / steering? → 11 Vehicle Physics

- → `Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleWheel.cs`
- → `Assets/_NeonRift/Scripts/Vehicles/Physics/TyreModel.cs`
- → `Assets/_NeonRift/Scripts/Vehicles/Physics/Drivetrain.cs`
- → `Assets/_NeonRift/Scripts/Vehicles/Physics/Gearbox.cs`
- → `Assets/_NeonRift/Scripts/Vehicles/Physics/SteeringSystem.cs`

Engine sound? → 12 Vehicle Audio

- → `Assets/_NeonRift/Scripts/Audio/VehicleAudio.cs`
- → `Assets/_NeonRift/Scripts/Editor/Audio/EngineSynth.cs`

Wheels, lights, LODs on cars? → 13 Vehicle Visuals

- → `Assets/_NeonRift/Scripts/Vehicles/VehicleRig.cs`
- → `Assets/_NeonRift/Scripts/Vehicles/VehicleLights.cs`
- → `Assets/_NeonRift/Scripts/Editor/Vehicles/VehicleLodSetup.cs`

Which cars exist and their data? → 14 Car Catalog and Data

- → `Assets/_NeonRift/Data/Vehicles/VehicleCatalog.asset`
- → `Assets/_NeonRift/Scripts/Vehicles/VehicleDefinition.cs`

How does car selection work? → 15 Car Selection

- → `Assets/_NeonRift/Scripts/Frontend/CarSelectScreen.cs`
- → `Assets/_NeonRift/Scripts/Frontend/VehicleShowroom.cs`

Controls / input? → 16 Player Input

- → `Assets/_NeonRift/Settings/Input/NeonRiftControls.inputactions`
- → `Assets/_NeonRift/Scripts/Input/PlayerDrivingInput.cs`

PART IV · AI AND TRAFFIC

How does rival AI work? → 17 Rival AI

- → `Assets/_NeonRift/Scripts/Gameplay/Racing/RacerDriver.cs`
- → `Assets/_NeonRift/Scripts/Gameplay/Racing/RivalDirector.cs`

What do the rivals do / qualifying? → 17 Rival AI

- → `Assets/_NeonRift/Scripts/Missions/RivalTask.cs`
- → `RivalDirector.BeginTask`

How does traffic work? → 18 Traffic System

- → `Assets/_NeonRift/Scripts/Gameplay/City/CityTraffic.cs`
- → `Assets/_NeonRift/Scripts/Gameplay/City/TrafficCar.cs`

PART V · MISSIONS AND THE HEIST

How are objectives tracked? → 19 Mission System

- → `Assets/_NeonRift/Scripts/Missions/MissionProgress.cs`
- → `Assets/_NeonRift/Scripts/Gameplay/Mission/MissionDirector.cs`

Mission data (objectives, timers, announcements)? → 19 Mission System

→ `Assets/_NeonRift/Data/Missions/Mission_NightRun.asset` (built by `NightRunBuilder.CreateMissionData`)

How do hacks / multi-stage interactions work? → 20 Heist System

→ `Assets/_NeonRift/Scripts/Missions/InteractionRun.cs`

→ `Assets/_NeonRift/Scripts/Gameplay/Mission/Interactable.cs`

How does the security gate work? → 21 Security Gate

→ `Assets/_NeonRift/Scripts/Gameplay/Mission/GateLockSystem.cs`

→ `Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityBarrier.cs`

How does the Data Core work? → 22 Data Core

→ `Assets/_NeonRift/Scripts/Gameplay/Mission/DataCoreChamber.cs`

→ `Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.Heist.cs`

Heat, alert, lockdown? → 23 Heat and Lockdown

→ `MissionProgress.AddHeat`

→ `Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityLightGroup.cs`

→ `Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityAlarm.cs`

Security cameras? → 23 Heat and Lockdown

→ `Assets/_NeonRift/Scripts/Gameplay/City/SecurityCamera.cs`

Drones? → 23 Heat and Lockdown 37 Animation Systems

→ `Assets/_NeonRift/Scripts/Gameplay/City/SecurityDrones.cs`

→ `Assets/_NeonRift/Scripts/Gameplay/City/HoverSentry.cs`

→ `Assets/_NeonRift/Scripts/Editor/World/DroneModels.cs`

PART VI · RENDERING AND EFFECTS

How does lighting work? → 25 Lighting

→ `NightRunBuilder.BuildLighting`

→ `Assets/_NeonRift/Scripts/Gameplay/City/LightBudget.cs`

→ `Assets/_NeonRift/Settings/Rendering/`

Fog / haze? → 26 Weather and Atmosphere

→ `Assets/_NeonRift/Scripts/Rendering/GroundHazeFeature.cs`

→ `Assets/_NeonRift/Scripts/Rendering/GroundHaze.cs`

Where is the rain system? → 26 Weather and Atmosphere

→ **Not implemented.** Wet look: `DistrictTextures.Asphalt`

Particles? → 28 Particles and VFX

→ **None**; additive meshes + `Assets/_NeonRift/Art/Shaders/NeonRift_AdditiveGlow.shader`

PART VII · AUDIO AND MUSIC

Audio mixer / sounds? → 29 Audio System

- → `Assets/_NeonRift/Scripts/Audio/AudioMixerService.cs`
- → `Assets/_NeonRift/Scripts/Gameplay/Mission/MissionAudio.cs`

Music / M toggle? → 30 Music System

- → `Assets/_NeonRift/Data/Missions/MissionAudio_NightRun.asset`
- → `Assets/_NeonRift/Scripts/Editor/Audio/ScoreComposer.cs`
- → `Assets/_NeonRift/Scripts/Game/GameSettings.cs`

PART VIII · CINEMATICS

How does the cinematic work? → 31 Cinematic Intro

- → `Assets/_NeonRift/Scripts/Intro/IntroSceneEntry.cs`
- → `Assets/_NeonRift/Scripts/Editor/Intro/IntroBuilder.cs`
- → `Assets/_NeonRift/Data/Intro/Intro_NightRun.playable`

Timeline / cue track? → 32 Timeline and Camera

- → `Assets/_NeonRift/Scripts/Intro/IntroCue.cs`
- → `Assets/_NeonRift/Scripts/Intro/IntroCueTrack.cs`
- → `Assets/_NeonRift/Scripts/Intro/IntroCueClip.cs`

PART IX · INTERFACE AND PRESENTATION

HUD? → 33 UI and HUD

- → `Assets/_NeonRift/Scripts/Gameplay/Mission/MissionHud.cs`
- → `Assets/_NeonRift/UI/MissionHud.uxml`

How does the minimap work? → 34 Minimap

- → `Assets/_NeonRift/Scripts/Gameplay/Mission/CityMinimap.cs`
- → `Assets/_NeonRift/Scripts/Gameplay/Mission/MinimapProjection.cs`

Briefing card, splits, announcements? → 35 Objectives and Mission Presentation

- → `MissionDirector.BuildBriefing`
- → `MissionHud.StampObjective`

Grade / debrief? → 36 Results and Debrief

- → `Assets/_NeonRift/Scripts/Missions/MissionGrade.cs`
- → `MissionDirector.BuildDebrief`

PART X · ANIMATION AND CAMERA

Where is this animation implemented? → 37 Animation Systems

- → catalogue table

Gameplay camera? → 38 Camera System

- → `Assets/_NeonRift/Scripts/Gameplay/VehicleChaseCamera.cs`

PART XI · ENGINEERING, QUALITY AND RELEASE

Performance work? → 39 Performance

- → `Assets/_NeonRift/Scripts/Editor/Validation/RoutePerf.cs`
- → `Assets/_NeonRift/Scripts/Editor/Validation/GpuBench.cs`
- → `Assets/_NeonRift/Scripts/Editor/Validation/PerfBuild.cs`

Tests? → 40 Testing and Validation

- → `Assets/_NeonRift/Scripts/Tests/EditMode/`

Mission auto-playtest? → 40 Testing and Validation

- → `Assets/_NeonRift/Scripts/Gameplay/Mission/NightRunValidator.cs`
- → `Assets/_NeonRift/Scripts/Editor/Validation/MissionSeries.cs`

How to build / open the project? → 42 Build and Release

- → `ProjectSettings/EditorBuildSettings.asset`

Asset licences? → 44 Assets and Licensing

- → `Assets/ThirdParty/**/ATTRIBUTION.txt`
- → `Docs/ThirdParty.md`
- → `LICENSE.md`

Master Index

Scenes, prefabs, data assets, important classes, interfaces, packages, the system dependency map, tests and assets, each pointing to its chapter. The full script index is Appendix A.

IN THIS INDEX

C. Scene index
D. Prefab index
E. ScriptableObject / data asset index
F. Important class index
G. Interface index
H. Package / framework index
I. Major system dependency map
J. Testing index
K. Asset / licensing index

C. Scene index

Scene	Path	Generated by	Entry point	Chapter
Bootstrap	`Assets/_NeonRift/Scenes/Bootstrap.unity`	hand-made	– (`GameRoot`)	06
Frontend	`Assets/_NeonRift/Scenes/Frontend.unity`	hand-made	`TitleScreen`	05
CarSelect	`Assets/_NeonRift/Scenes/CarSelect.unity`	`GarageBuilder`	`CarSelectScreen`	15
TestTrack (dev)	`Assets/_NeonRift/Scenes/Dev/TestTrack.unity`	`ValidationRouteBuilder` (route)	`MissionSceneEntry`	11
NightRun	`Assets/_NeonRift/Scenes/NightRun.unity`	`NightRunBuilder` + `IntroBuilder`	`MissionSceneEntry` , `IntroSceneEntry`	07, 31

D. Prefab index

Folder	Prefabs	Made by	Chapter
`Assets/_NeonRift/Prefabs/Vehicles/`	`PF_Vehicle_MercedesSLS` , `PF_Vehicle_MercedesSLSGT3` , `PF_Vehicle_LamborghiniTerzo` , `PF_Vehicle_MercedesAMGGT3` (not in the catalog)	`VehiclePrefabBuilder`	13
`…/Prefabs/Traffic/`	`PackCar_Compact` , `_Coupe` , `_Hatchback` , `_Minivan` , `_Offroad` , `_Pickup` , `_Sedan` , `_Sport` , `_SUV` , `_Wagon`	`TrafficCarBuilder`	18
`…/Prefabs/Drones/`	`PF_Drone_Mech` , `PF_Drone_Buster`	`DroneModels`	23
`…/Prefabs/Buildings/`	`PF_Bld_LondonSkyscraper` , `PF_Bld_SingaporeOffice` , `PF_Bld_NightSkylineCluster` , `PF_Bld_AsianNight_B1…B2_008` (19)	`BuildingPrefabBuilder` / `ThirdPartyIntake`	08
`…/Prefabs/District/`	`PF_Prop_StreetLight_Led` , `_Sodium` , `_Warm` , `PF_Prop_JerseyBarrier` , `PF_Prop_SecurityBeacon`	`DistrictKit.CreatePrefabs`	08

E. ScriptableObject / data asset index

Asset	Type	Purpose	Chapter
`D/Config/GameConfig.asset`	`GameConfig`	Scenes, catalog, missions, mixer, fade	06
`D/Config/IntroSettings.asset`	`IntroSettings`	Intro policy, mode, skippable	31
`D/Vehicles/VehicleCatalog.asset`	`VehicleCatalog`	Player cars in order	14
`D/Vehicles/Vehicle_*.asset` (4)	`VehicleDefinition`	Identity, prefabs, profiles, stats, licence	14
`D/Vehicles/Physics/VehiclePhysics_*.asset` (3)	`VehiclePhysicsProfile`	Driving tuning	11
`D/Vehicles/Audio/VehicleAudio_*.asset` (3)	`VehicleAudioProfile`	Car sound	12
`D/Vehicles/ImportSetups/Setup_*.asset` (4)	`VehicleModelSetup`	Prefab build recipes	13
`D/Missions/Mission_NightRun.asset`	`MissionDefinition`	Night Run objectives, announcements, rival tasks, par, qualifying	19
`D/Missions/Mission_DevTestTrack.asset`	`MissionDefinition`	Test-track mission	11
`D/Missions/Interaction_CoreSecurity.asset`, `Interaction_DataCoreUplink.asset`, `Interaction_HackGate.asset`	`InteractionDefinition`	Heist devices	20
`D/Missions/MissionAudio_NightRun.asset`	`MissionAudioSet`	Mission sounds and music	29
`D/Racing/Racer_Vex.asset`, `Racer_Kade.asset`	`RacerProfile`	AI personalities	17
`D/World/RoadNetwork_NightRun.asset`	`RoadNetwork`	Road graph	09
`D/World/BuildingCatalog.asset`, `Building_*.asset` (22)	`BuildingCatalog`, `BuildingDefinition`	Building models	08
`D/Audio/AudioMixerConfig.asset`	`AudioMixerConfig`	Mixer groups, snapshots, parameters	29
`D/Intro/Intro_NightRun.playable`	`TimelineAsset`	Intro Timeline	31
`D/Intro/Intro_Surveillance.asset`	`VolumeProfile`	CCTV grade	31
`D/Physics/PM_VehicleBody.asset`	`PhysicsMaterial`	Low-friction car body	11
`Assets/_NeonRift/Settings/Rendering/*.asset`	URP assets, Volume profiles	Rendering	25

F. Important class index

Class	Role	File
`GameRoot`	Composition root	`S/Game/GameRoot.cs`
`GameFlow`	Scene state machine	`S/Game/GameFlow.cs`
`RunSession`	Selected car, rivals, mission, result	`S/Game/RunSession.cs`

Class	Role	File
`VehicleController`	The car	`S/Vehicles/Physics/VehicleController.cs`
`VehicleWheel`, `TyreModel`, `Drivetrain`, `Gearbox`, `SteeringSystem`	Physics parts	`S/Vehicles/Physics/`
`VehicleAudio`	Car sound	`S/Audio/VehicleAudio.cs`
`VehicleSpawnPoint`	Spawning	`S/Gameplay/VehicleSpawnPoint.cs`
`VehicleChaseCamera`	Gameplay camera	`S/Gameplay/VehicleChaseCamera.cs`
`MissionSceneEntry`	Mission scene entry, pause	`S/Gameplay/MissionSceneEntry.cs`
`MissionDirector`	Runs a mission attempt	`S/Gameplay/Mission/MissionDirector.cs`
`MissionProgress`	Mission state machine	`S/Missions/MissionProgress.cs`
`MissionWorld`	Per-attempt event hub	`S/Gameplay/Mission/MissionWorld.cs`
`Interactable`, `InteractionRun`	Multi-stage hacks	`S/Gameplay/Mission/Interactable.cs`, `S/Missions/InteractionRun.cs`
`DataCoreChamber`	Data Core presentation	`S/Gameplay/Mission/DataCoreChamber.cs`
`SecurityBarrier`, `GateLockSystem`	Gates	`S/Gameplay/Mission/`
`RivalDirector`, `RacerDriver`, `RacingLine`	Rival AI	`S/Gameplay/Racing/`
`CityTraffic`, `TrafficCar`	Traffic	`S/Gameplay/City/`
`CityNavigation`, `RoadGraph`	Routing	`S/Gameplay/Navigation/`, `S/World/`
`MissionHud`, `CityMinimap`, `TerminalReadout`	HUD	`S/Gameplay/Mission/`
`CarSelectScreen`, `VehicleShowroom`	Car Select	`S/Frontend/`
`IntroSceneEntry`, `IntroCityLife`, `IntroOverlay`	Intro	`S/Intro/`
`NightRunBuilder`, `NightRunDistrict`, `CityLayout`	City generator	`S/Editor/District/`
`IntroBuilder`, `GarageBuilder`	Intro and garage generators	`S/Editor/Intro/`, `S/Editor/Frontend/`

G. Interface index

Interface	File	Implemented by
`ISceneEntryPoint`	`S/Game/GameContext.cs`	`TitleScreen`, `CarSelectScreen`, `MissionSceneEntry`, `IntroSceneEntry` (via `IIntroEntryPoint`)
`IIntroEntryPoint`	`S/Game/IGameFlow.cs`	`IntroSceneEntry`
`IGameFlow`	`S/Game/IGameFlow.cs`	`GameFlow`
`IVehicleInputSource`	`S/Vehicles/DrivingInput.cs`	`PlayerDrivingInput`, `RacerDriver`, `WaypointDriver`, `RouteAutopilot`, `ScriptedDrivingInput`

Interface	File	Implemented by
`IVehicleInputReceiver`	`S/Vehicles/DrivingInput.cs`	`VehicleController`
`IMissionWorldComponent`	`S/Gameplay/Mission/IMissionWorldComponent.cs`	`MissionZone`, `Interactable`, `SecurityBarrier`, `GateLockSystem`, `SecurityLightGroup`, `SecurityAlarm`, `SecurityPostEffects`, `DataCoreChamber`, `SecurityCamera`, `TrafficSignalNetwork`, `LockdownScreens`, `CityTraffic`, `CrowdGroups`, `SecurityDrones`, `RivalDirector`
`IMissionTarget`	`S/Gameplay/Mission/IMissionTarget.cs`	`MissionZone`, `Interactable`
`IIntroCueHandler`	`S/Intro/IntroCue.cs`	`IntroSceneEntry`

H. Package / framework index

Package	Version	Used for	Chapter
Universal RP	17.6.0	Rendering, Volumes, Render Graph haze	25
Input System	1.19.0	All input	16
Cinemachine	6.6.0	Chase, Car Select and intro cameras	38
Timeline	6.6.0	Intro	32
glTFast	6.20.0	GLB import	13
Test Framework	1.8.0	EditMode tests	40
Video module	1.0.0	Optional pre-rendered intro	31
uGUI	2.6.0	EventSystem only	03
Splines, ProBuilder, AI Navigation	–	**Installed, not used**	03

I. Major system dependency map

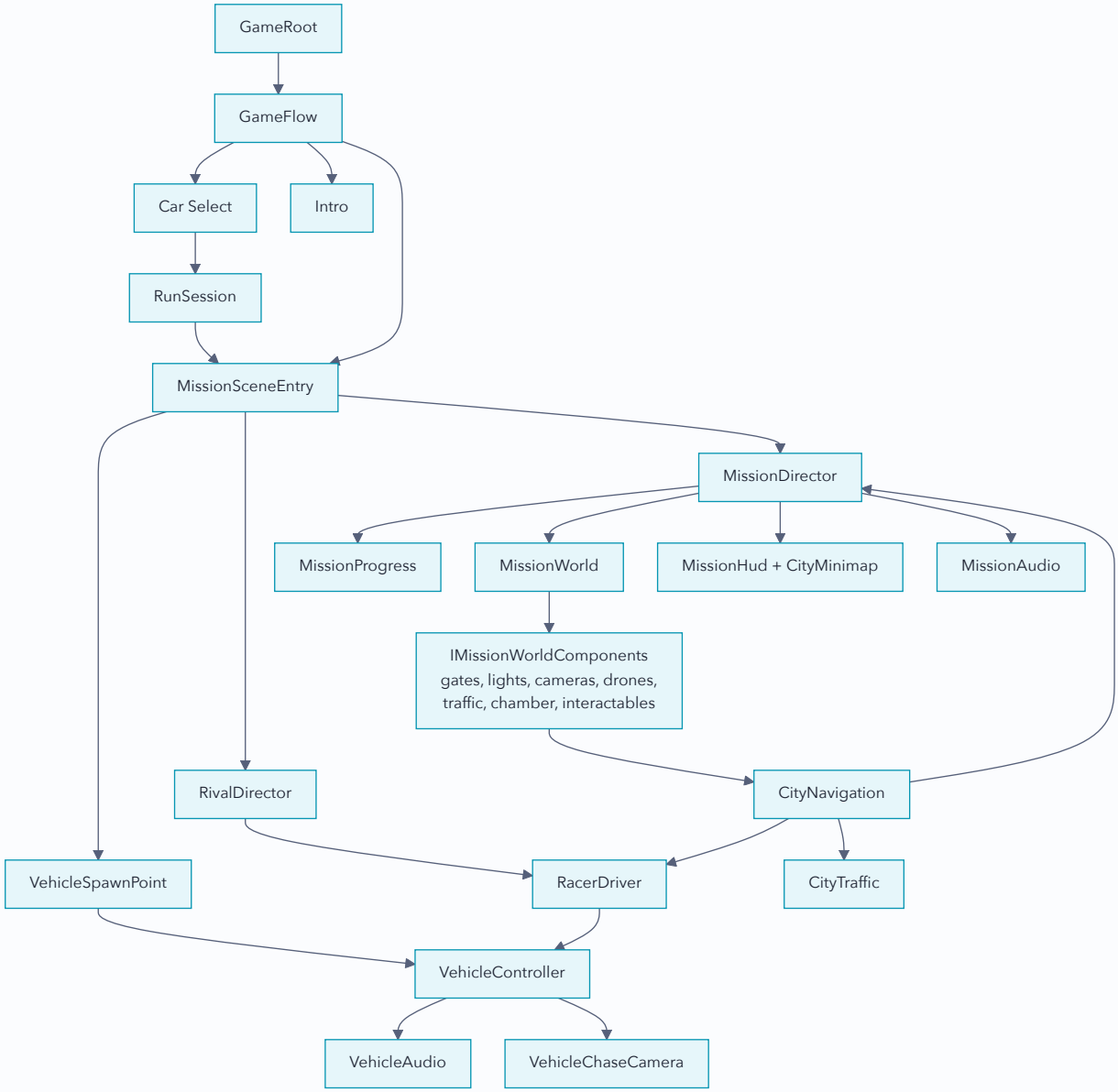


Figure 0.1 Overall game architecture: major system dependencies

J. Testing index

Test file	Area	Chapter
`S/Tests/EditMode/FoundationTests.cs`	Catalog, session, wiring	40
`S/Tests/EditMode/InputTests.cs`	Input bindings	16
`S/Tests/EditMode/VehicleModelTests.cs`	Tyres, steering, gearbox, drivetrain	11

Test file	Area	Chapter
`S/Tests/EditMode/VehiclePhysicsTests.cs`	Driving on the bench (per car)	11
`S/Tests/EditMode/VehicleAudioTests.cs`	Audio model, mixer, score, limiter	12, 29
`S/Tests/EditMode/CityTests.cs`	City, graph, AI line, rubber band, traffic	07, 17
`S/Tests/EditMode/MissionTests.cs`	State machine, data, scene wiring, grade	19
`S/Tests/EditMode/InteractionTests.cs`	Multi-stage interactions	20
`S/Tests/EditMode/IntroTests.cs`	Intro	31
`S/Tests/EditMode/MinimapTests.cs`	Minimap projection	34
`S/Tests/EditMode/CarSelectTests.cs`	Car Select	15
Play Mode tools	`NightRunValidator`, `MissionPlaytest`, `MissionSeries`, `InputPlaytest`, `RoutePerf`, `GpuBench`, `IntroCapture`, `VehicleAudioValidator`	40

K. Asset / licensing index

Asset group	Licence	Location	Chapter
SLS AMG, SLS GT3	CC-BY-4.0	`Assets/ThirdParty/Sketchfab/Vehicles/`	44
Terzo Millennio	CC-BY-NC-4.0 (non-commercial)	same	44
AMG GT3 Red Bull (unused)	CC-BY-NC-SA-4.0	same	44
Generic passenger car pack	not recorded	same	44
99.Miles buildings	CC-BY-4.0	`Assets/ThirdParty/Sketchfab/Buildings/`	44
Mech Drone, Buster Drone	CC BY 4.0	`Assets/ThirdParty/Sketchfab/Drones/`	44
Drone Design (unused)	CC BY-NC-ND 4.0	same	44
Music tracks	not recorded	`Assets/ThirdParty/SoundAudio/`	44
Everything else (code, generated art, audio)	First-party, all rights reserved (`LICENSE.md`)	`Assets/_NeonRift/`	44

PART I

Foundations

What the game is, how it is built, and how it starts.

01 Project Overview

02 Architecture

03 Tech Stack

04 Project Structure

05 Game Flow

06 Bootstrap and Scenes

01 Project Overview

IN THIS CHAPTER

1.1 What Neon Rift is
1.2 Player fantasy
1.3 Core gameplay loop (Night Run)
1.4 Design philosophy: racing + heist + environmental interaction + dynamic consequences
1.5 Major systems (one line each)
1.6 Project scale (measured from the repository)
1.7 Current state

1.1 What Neon Rift is

Neon Rift: Night Run (`productName` in `ProjectSettings/ProjectSettings.asset`, version `0.1.0`) is a cyberpunk **racing + heist** game built in **Unity 6 (6000.6.4f1)** with the **Universal Render Pipeline**, the **Input System**, **Cinemachine** and **Timeline**.

The player picks a car in a crew garage, drives into a generated night city of about 1.1 × 1.4 km, breaks into the **Sector 7 Data Core**, and then escapes through a city that locks down around them. Two rival crews run their own heists at the same time. Only the first two crews out qualify.

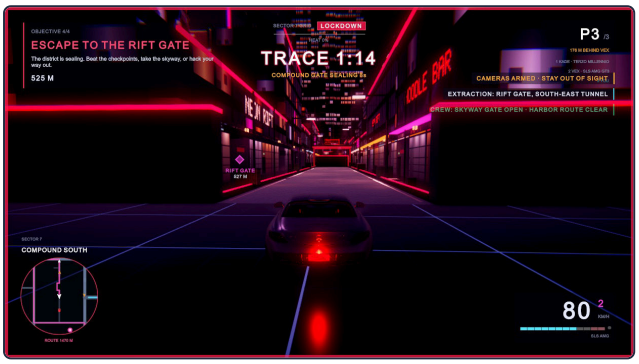
Genre	Arcade-simulation driving, heist, open-route racing
Platform	Desktop (macOS player builds exist; PC and Mobile URP assets are both configured)
Mission	**Night Run** (`Assets/_NeonRift/Data/Missions/Mission_NightRun.asset`)
Player cars	SLS AMG, SLS AMG GT3, Terzo Millennio (`Assets/_NeonRift/Data/Vehicles/VehicleCatalog.asset`)
Rival crews	VEX and KADE (`Assets/_NeonRift/Data/Racing/Racer_Vex.asset`, `Racer_Kade.asset`), driving the two cars the player did not pick
Code	198 C# files, about 36,000 lines, 13 assemblies (`Assets/_NeonRift/Scripts/*/NeonRift.*.asmdef`)
Tests	125 EditMode test cases (last run: 125 passed, 0 failed), in `Assets/_NeonRift/Scripts/Tests/EditMode/`



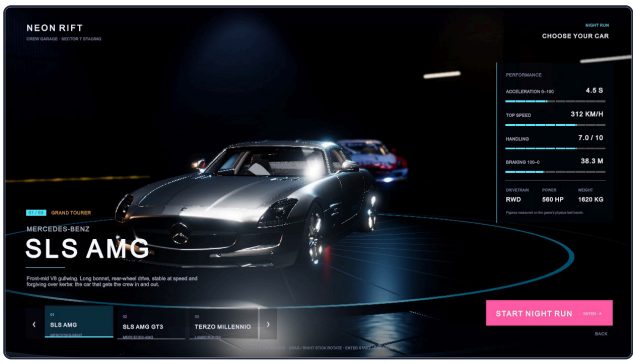
Spire Boulevard during a run
`Docs/Screenshots/Detail/04_spire_boulevard.jpg`



The Data Core uplink at DATA ACQUIRED
`Docs/Screenshots/Heist/core_acquired.jpg`



Escape to the Rift Gate with the race panel
`Docs/Screenshots/Crews/escape_race_hud.jpg`



Car Select in the crew garage
`Docs/Screenshots/CarSelect2/hero.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

1.2 Player fantasy

You are the driver of a small crew. Your crew's hidden garage on W Avenue opens its roller door, you drive out into the neon city, steal a data package from a corporate grid, and outrun both the city's lock-down and the other crews.

1.3 Core gameplay loop (Night Run)

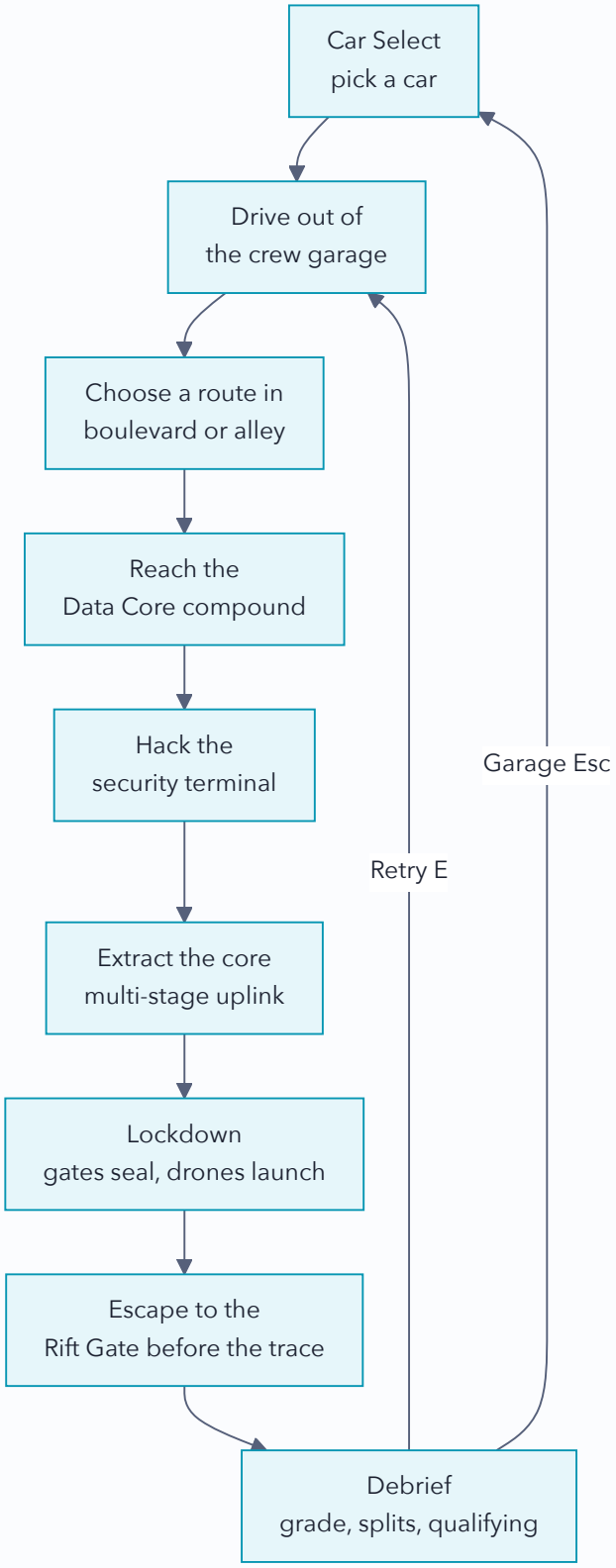


Figure 1.1 Core gameplay loop (Night Run)

The four mission objectives are data (see 19 Mission System):

1. `reach_core`: INFILTRATE THE DATA CORE FACILITY.
2. `disable_security`: DISABLE FACILITY SECURITY.
3. `hack_core`: EXTRACT THE DATA CORE.
4. `escape`: ESCAPE TO THE RIFT GATE (80 s trace timer, shortened by heat).

1.4 Design philosophy: racing + heist + environmental interaction + dynamic consequences

Pillar	How the project does it	Main code
Racing	A physical car model and two AI rival crews on the same road graph	`VehicleController`, `RacerDriver`, `RivalDirector`
Heist	Multi-stage "stop and work the device" interactions	`InteractionRun`, `Interactable`, `DataCoreChamber`
Environmental interaction	Gates with physical bolts, terminals with screens, a chamber with robotic arms	`GateLockSystem`, `SecurityBarrier`, `TerminalDisplay`
Dynamic consequences	Heat, Alert/Lockdown, sealing gates, cameras, drones, signal grids and screens that react	`MissionProgress`, `SecurityLightGroup`, `SecurityCamera`, `SecurityDrones`, `TrafficSignalNetwork`, `LockdownScreens`

How it differs from a conventional racing game:

- **There is no fixed track.** Routes are your choice on a city road graph (`RoadNetwork`), and they change when gates close.
- **Risk has a price.** The short alley costs **heat** (a gate hack logs +35 %). Heat raises security, shortens the escape timer and makes checkpoints seal sooner.
- **The opponents are working, not only racing.** Rivals stop to hack relays and tap the core's trunk line (17 Rival AI).
- **The world reacts to string events, not to scripts.** Objectives fire event ids such as `core.breached`, and every reactive object listens for ids (02 Architecture).

1.5 Major systems (one line each)

System	Chapter
Boot, scenes and game flow	05, 06
Generated city, buildings, roads	07, 08, 09
Vehicles: physics, audio, visuals, data	10 to 14
Car Select garage	15
Input	16
Rival AI, traffic	17, 18
Mission, heist, gate, Data Core, lockdown	19 to 24
Lighting, atmosphere, effects	25 to 28

System	Chapter
Audio and music	29, 30
Cinematic intro	31, 32
UI, HUD, minimap, presentation, debrief	33 to 36
Animation, camera, performance	37 to 39
Testing, build, repository, licensing	40 to 44

1.6 Project scale (measured from the repository)

Item	Count	Source
C# scripts	198 (≈36 k lines)	`Assets/_NeonRift/Scripts/`
Scenes in the build	5	`ProjectSettings/EditorBuildSettings.asset`
Night Run scene	≈15,200 GameObjects, 862 lights, 63 reflection probes, 9,046 box colliders (29 MB YAML)	`Assets/_NeonRift/Scenes/NightRun.unity`
Generated district meshes	8,880 files	`Assets/_NeonRift/Art/District/Meshes/`
Road graph	72 nodes, 107 edges (last build log)	`Assets/_NeonRift/Data/World/RoadNetwork_NightRun.asset`
Generated audio clips	≈97	`Assets/_NeonRift/Audio/Generated/`

1.7 Current state

The game is complete for its scope: one mission (Night Run), three player cars, two rival crews, a full front end (cinematic intro, title, Car Select, pause and settings), a graded debrief, and automated tests and validation tools. Known limits are listed honestly in 45 Known Limitations.

02 Architecture

IN THIS CHAPTER

2.1 High-level shape
2.2 Assemblies (actual .asmdef references)
2.3 Runtime flow
2.4 Scene flow
2.5 Data flow for one run
2.6 Mission architecture
2.7 Vehicle architecture
2.8 AI architecture
2.9 UI architecture
2.10 Audio architecture
2.11 Environment architecture
2.12 Persistence

2.1 High-level shape

Neon Rift is built around four rules. They appear everywhere in the code:

1. **One composition root.** `GameRoot` (`Assets/_NeonRift/Scripts/Game/GameRoot.cs`) lives in the always-loaded **Bootstrap** scene and creates every long-lived service. There are no singletons (see Decision D3 in `Docs/Decisions.md`).
2. **Context injection.** Each content scene has exactly one root component implementing `ISceneEntryPoint` (`Game/GameContext.cs`). `GameFlow` calls `Enter(GameContext)` after loading the scene and `Exit()` before unloading it. Scenes never look services up globally.
3. **Data in ScriptableObjects.** Vehicles, physics profiles, audio profiles, missions, interactions, AI profiles and the road network are all assets in `Assets/_NeonRift/Data/`.
4. **Missions talk through string event ids.** Objectives fire ids (`core.arrive`, `core.breached`, `lockdown` …). Scene objects implement `IMissionWorldComponent`, receive a per-attempt `MissionWorld`, and react to those ids. The director contains no mission-specific rules.

Domain reload on entering Play Mode is disabled (`ProjectSettings/EditorSettings.asset`: `m_EnterPlayModeOptionsEnabled: 1`, `m_EnterPlayModeOptions: 3`). This is why the code avoids static mutable state.

2.2 Assemblies (actual `.asmdef` references)

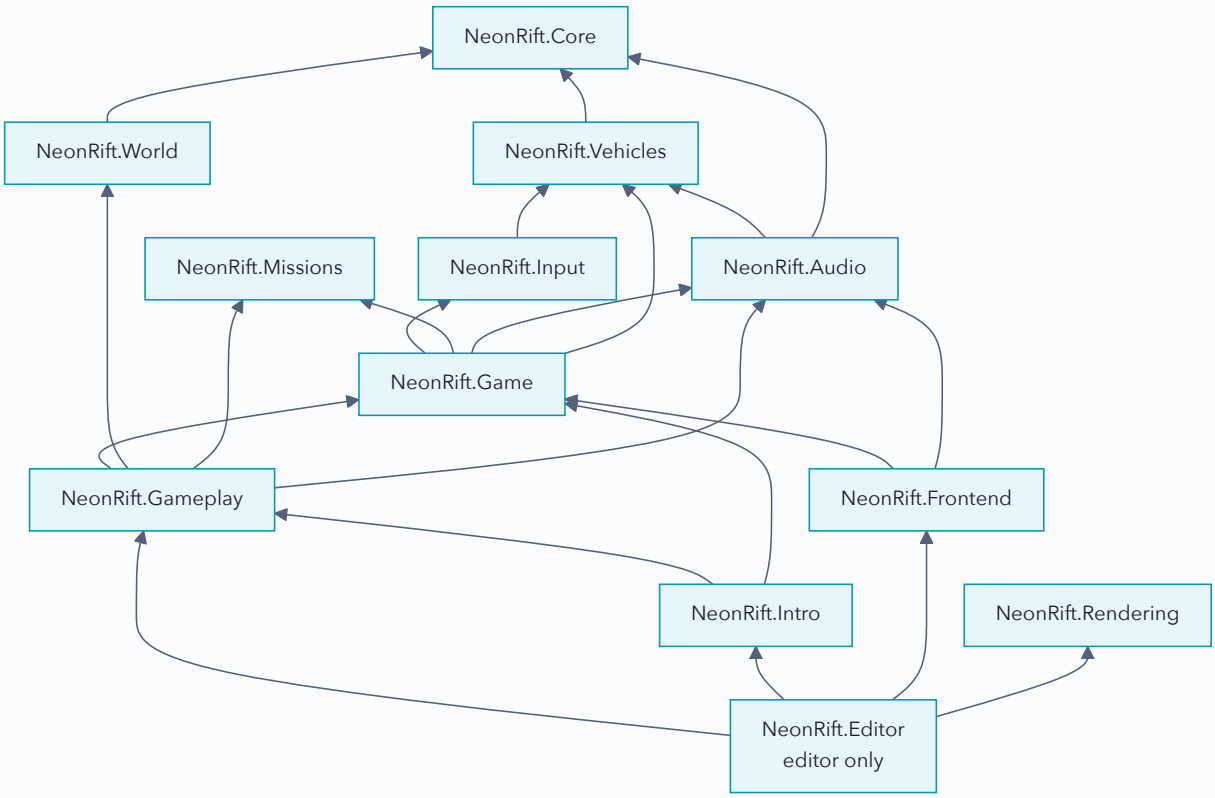


Figure 2.1 Assembly dependency graph (13 assemblies)

Assembly	Folder	Owns
`NeonRift.Core`	`Scripts/Core`	`AssetLicense`
`NeonRift.Vehicles`	`Scripts/Vehicles`	`VehicleController`, `VehicleWheel`, `TyreModel`, `Drivetrain`, `Gearbox`, `SteeringSystem`, `VehiclePhysicsProfile`, `VehicleDefinition`, `VehicleCatalog`, `VehicleRig`, `VehicleLights`, `DrivingInput`, `IVehicleInputSource`, audio profile data
`NeonRift.World`	`Scripts/World`	`RoadNetwork`, `RoadGraph`, `RoadPath`, `BuildingDefinition`, `BuildingCatalog`
`NeonRift.Missions`	`Scripts/Missions`	`MissionDefinition`, `ObjectiveDefinition`, `MissionProgress` (pure state machine), `InteractionStep`, `InteractionRun`, `RivalTask`, `MissionGrade`, `MissionRecords`, `ScoreMix`
`NeonRift.Input`	`Scripts/Input`	Generated `NeonRiftControls`, `PlayerDrivingInput`
`NeonRift.Audio`	`Scripts/Audio`	`VehicleAudio`, `EngineSoundModel`, `TyreSoundModel`, `AudioMixerService`, `AudioMixerConfig`, `MasterLimiter`
`NeonRift.Game`	`Scripts/Game`	`GameRoot`, `GameFlow`, `GameContext`, `RunSession`, `GameConfig`, `GameSettings`, `IntroSettings`, `OptionsMenu`, `LoadingOverlay`, `BootstrapLoader`
`NeonRift.Gameplay`	`Scripts/Gameplay`	Mission runtime, city systems, racing AI, traffic, camera, spawn
`NeonRift.Frontend`	`Scripts/Frontend`	`TitleScreen`, `CarSelectScreen`, `VehicleShowroom`, `ShowroomEngine`

Assembly	Folder	Owns
`NeonRift.Intro`	`Scripts/Intro`	`IntroSceneEntry`, Timeline cue track, `IntroOverlay`, `IntroCityLife`
`NeonRift.Rendering`	`Scripts/Rendering`	`GroundHaze` Volume override, `GroundHazeFeature` (Render Graph)
`NeonRift.Editor`	`Scripts/Editor`	Every generator, builder, validator and test harness (editor only)
`NeonRift.Tests.EditMode`	`Scripts/Tests/EditMode`	EditMode unit and integration tests

2.3 Runtime flow

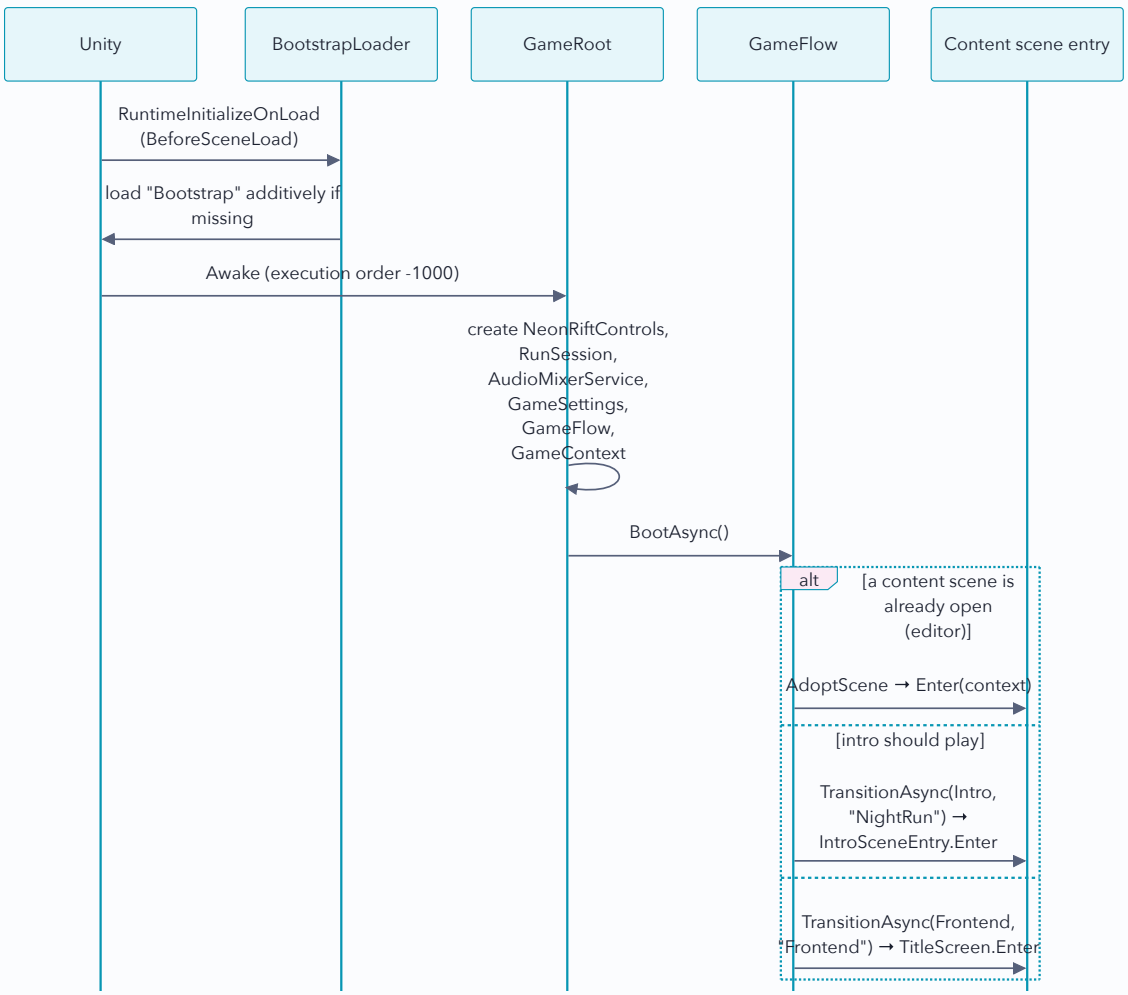


Figure 2.2 Game startup flow

`GameFlow.TransitionAsync` (`Game/GameFlow.cs:122`) always runs in the same order: fade in (`LoadingOverlay.ShowAsync`) → `Exit()` → unload the old scene → load the new one additively → set it active → `Resources.UnloadUnusedAssets` → find the entry point (`FindEntryPoint`, which matches `IIntroEntryPoint` only for `GameState.Intro`) → `Enter(context)` → fade out. Requests made during a transition are ignored (`Request`).

2.4 Scene flow

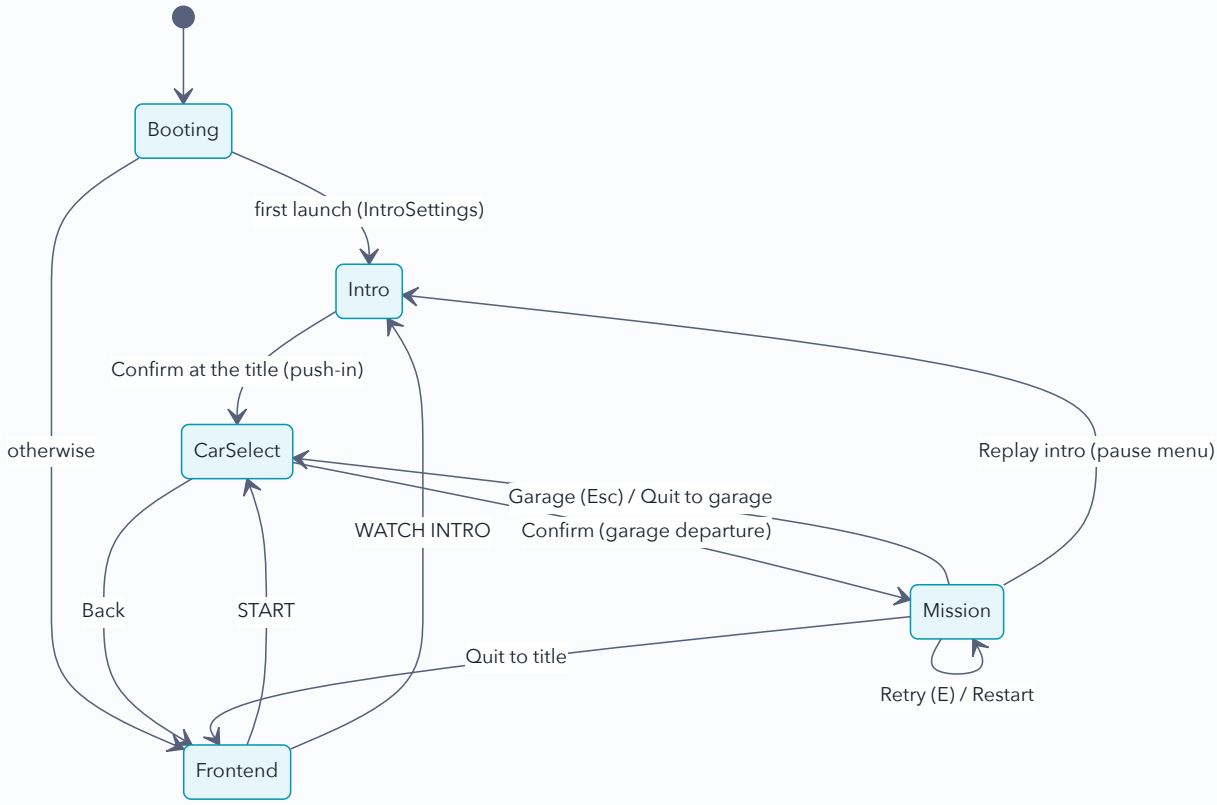


Figure 2.3 Scene and game-state flow

`GameState` values (`Game/IGameFlow.cs`): `None`, `Booting`, `Intro`, `Frontend`, `CarSelect`, `Mission`.

2.5 Data flow for one run

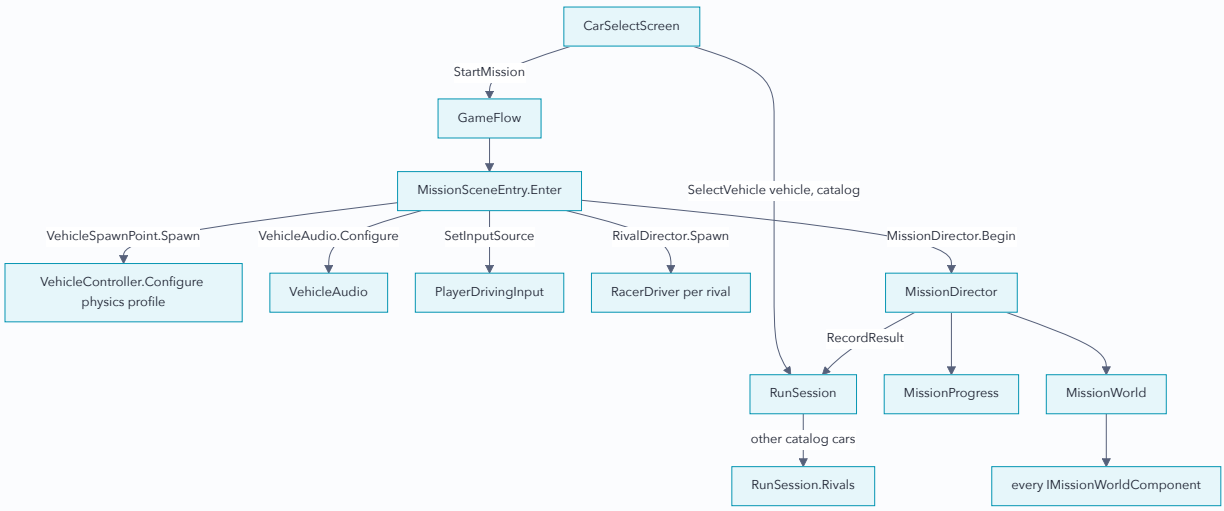


Figure 2.4 Data flow from Car Select into the mission

2.6 Mission architecture

Layer	Type	File
Data	`MissionDefinition`, `ObjectiveDefinition`, `MissionAnnouncement`, `RivalTask`	`Scripts/Missions/`
Pure state	`MissionProgress` (objectives, heat, security, timers, phases)	`Scripts/Missions/MissionProgress.cs`
Hub	`MissionWorld` (player, events, countdowns, announcements)	`Scripts/Gameplay/Mission/MissionWorld.cs`
Runner	`MissionDirector` (binds components, drives HUD, audio, mixer, results)	`Scripts/Gameplay/Mission/MissionDirector.cs`
Participants	`IMissionWorldComponent` implementers: `MissionZone`, `Interactable`, `SecurityBarrier`, `GateLockSystem`, `SecurityLightGroup`, `SecurityAlarm`, `SecurityPostEffects`, `DataCoreChamber`, `SecurityCamera`, `TrafficSignalNetwork`, `LockdownScreens`, `CityTraffic`, `CrowdGroups`, `SecurityDrones`, `RivalDirector`	various

Full details: 19 Mission System.

2.7 Vehicle architecture

`VehicleController` is the only driving component. It implements `IVehicleInputReceiver` and reads `DrivingInput` from any `IVehicleInputSource`: the player (`PlayerDrivingInput`), AI (`RacerDriver`, `WaypointDriver`, `RouteAutopilot`) or scripts (`ScriptedDrivingInput`). Tuning comes from `Vehicle`

`PhysicsProfile`, geometry from the prefab's `VehicleRig`. See 10 Vehicle System.

2.8 AI architecture

Rivals use the player's input interface: `RacerDriver : IVehicleInputSource`. `RivalDirector` owns task lists, standings and qualifying. Traffic is separate and simpler (`CityTraffic` + `TrafficCar`). See 17 Rival AI and 18 Traffic.

2.9 UI architecture

All screens are **UI Toolkit** (UXML/USS in `Assets/_NeonRift/UI/`, one `NeonRiftPanelSettings.asset`). Views are passive: `MissionHud`, `CarSelectScreen`, `IntroOverlay`, `OptionsMenu` and `TitleScreen` receive state from their controllers. The minimap draws with `Painter2D`. See 33 UI and HUD.

2.10 Audio architecture

One mixer (`Assets/_NeonRift/Audio/Mixer/NeonRiftMixer.mixer`), described by `AudioMixerConfig` and driven at runtime by `AudioMixerService` (snapshots plus user volumes). Vehicles play telemetry-driven loops (`VehicleAudio`), and missions play ambience, cues, music and the adaptive score (`MissionAudio`). See 29 Audio.

2.11 Environment architecture

The whole city scene is generated by editor code (`NightRunBuilder` + `NightRunDistrict` + `CityLayout` + helpers). The generated scene is the build artefact: **change the builder, not the scene.** See 07 City and World Generation.

2.12 Persistence

What	Where	Key
Settings (volumes, music on/off, quality, fullscreen, V-sync)	`GameSettings` → PlayerPrefs	prefix `NeonRift.Settings.`
Intro seen	`IntroSettings` → PlayerPrefs	`NeonRift.IntroSeen`
Best time / best score / qualified per mission	`MissionRecords` → PlayerPrefs	`NeonRift.Best.<id>.time`, `.grade`, `.qualified`
Session state (selected car, rivals, mission, last result)	`RunSession` (memory only)	–

03
Tech Stack

IN THIS CHAPTER

3.1 Engine
3.2 Language
3.3 Packages actually installed (Packages/manifest.json)
3.4 Source control
3.5 Internal / custom frameworks

Every entry below was checked against `ProjectSettings/ProjectVersion.txt`, `Packages/manifest.json`, the `.asmdef` references and `using` statements in `Assets/_NeonRift/Scripts/`.

3.1 Engine

WHAT	Unity **6000.6.4f1** (revision 12bfff696524)
USED FOR	Everything
IMPORTANT SETTINGS	Fixed timestep **0.01 s (100 Hz)** (`ProjectSettings/TimeManager.asset`, stored as rational 1411200/141120000); max allowed timestep 0.1 s; Enter Play Mode options: **no domain reload, no scene reload** (`EditorSettings.asset`, options 3); active input handler = Input System only (`ProjectSettings.asset`, `activeInputHandler: 1`); real voices **64**, virtual 512 (`AudioManager.asset`); gravity −9.81

3.2 Language

C# in 13 assemblies (see 02 Architecture). Async flow uses Unity's `Awaitable` (`GameFlow.TransitionAsync`).

3.3 Packages actually installed (`Packages/manifest.json`)

Package	Version	Used by the code?	What it is used for / where
`com.unity.render-pipelines.universal` (URP)	17.6.0	**Yes**	Rendering. Assets: `Assets/_NeonRift/Settings/Rendering/PC_RPAsset.asset` (HDR, 4× MSAA, shadow distance 90 m, 2 cascades, soft-shadow quality Low, SRP Batcher, GPU Resident Drawer) + `PC_Renderer.asset` (rendering mode Forward+, SSAO and `GroundHazeFeature` renderer features); `Mobile_RPAsset.asset` / `Mobile_Renderer.asset`. Volumes: `NightRun_Base.asset`, `NightRun_Alert.asset`, `NightRun_Lockdown.asset`, `Garage_Volume.asset`. Custom passes: `Scripts/Rendering/GroundHazeFeature.cs` (Render Graph)
`com.unity.inputsystem`	1.19.0	**Yes**	`Settings/Input/NeonRiftControls.inputactions` → generated `Scripts/Input/NeonRiftControls.cs`; `PlayerDrivingInput`, menus, `InputPlaytest`
`com.unity.cinemachine`	6.6.0	**Yes**	`CinemachineCamera` + `CinemachineBrain`: chase camera (`VehicleChaseCamera`), Car Select cameras (`VehicleShowroom`), the intro's shot cameras and push-in (`IntroSceneEntry`, `IntroBuilder`)
`com.unity.timeline`	6.6.0	**Yes**	The intro Timeline `Data/Intro/Intro_NightRun.playable`; custom track `IntroCueTrack` / `IntroCueClip` / `IntroCueBehaviour` (`Scripts/Intro/`)
`com.unity.cloud.gltfast`	6.20.0	**Yes** (import)	Imports the Sketchfab `.glb` cars, buildings and the Buster Drone. `Editor/GltfMaterialFixer.cs` remaps glTFast clearcoat materials (Decision D12)
`com.unity.modules.video`	1.0.0	**Yes**	`VideoPlayer` path for a pre-rendered intro (`IntroSettings.Mode.Video`, `IntroSceneEntry.PlayVideo`). Implemented but no clip is supplied
`com.unity.ugui`	2.6.0	Indirectly	The Bootstrap scene's `EventSystem` with `InputSystemUIInputModule`. No uGUI screens: all UI is UI Toolkit
`com.unity.test-framework`	1.8.0	**Yes**	EditMode tests (`Scripts/Tests/EditMode/`), `Editor/TestRunReporter.cs`
`com.unity.splines`	2.9.1	**No**	Installed, but no script references `UnityEngine.Splines`. Routes use the project's own `DrivingRoute`, `RacingLine` and `RoadPath`
`com.unity.probuilder`	6.1.2	**No**	Installed (listed in older docs as "blockout"), but no code or scene uses it. The city geometry comes from the project's own `MeshBuilder`
`com.unity.ai.navigation`	2.0.12	**No**	Installed, but no NavMesh is used. AI plans on the project's road graph (`RoadGraph`, `CityNavigation`)
`com.unity.collab-proxy`	2.12.4	No	Unity Version Control client; Git is the source of truth (Decision D10)
`com.unity.ai.assistant`, `com.unity.ai.inference`, `com.unity.pipeline`	–	No (tooling)	Unity AI and editor tooling used during development (MCP bridge); not used by game code
`com.unity.ide.rider`, `com.unity.ide.visualstudio`	–	No	IDE integration

Unity built-in features used directly: **UI Toolkit** (`UnityEngine.UIElements`; `Painter2D` in `CityMinimap`), **AudioMixer**, **PlayerPrefs**, **legacy** `Animation` (Mech Drone hover loop, set up in `Editor/World/DroneModels.cs`), **Animator** (on the intro's shot cameras, driven by Timeline animation tracks), **ReflectionProbe** (baked, box-projected), **LODGroup**, **Unity.Collections** (`Mesh.AcquireReadOnlyMeshData` in `TrafficCarBuilder`).

IMPLEMENTATION STATUS

Not used: `WheelCollider` (deliberately; Decision D15), `ParticleSystem` (no particle systems exist in any scene or script; see 28), NavMesh, uGUI screens.

3.4 Source control

Tool	Use	Files
Git	Repository `https://github.com/soyebmohammad03-dev/NEON-RIFT` (`main`)	`.gitignore`
Git LFS	Models, textures, audio, EXR probes, screenshots, generated district meshes	`.gitattributes`
UnityYAMLMerge	Merge driver for scenes, prefabs and assets (`merge=unityyamlmerge`)	`.gitattributes`, `Docs/SourceControl.md`

3.5 Internal / custom frameworks

Framework	What	Where
City generator	Data-driven layout → blocks → meshes, props, lights, probes, road graph	`Scripts/Editor/District/`
Vehicle physics	Raycast suspension, combined-slip tyres, drivetrain, aero	`Scripts/Vehicles/Physics/`
Mission framework	Data + pure state machine + event-id world	`Scripts/Missions/`, `Scripts/Gameplay/Mission/`
Multi-stage interactions	`InteractionStep` / `InteractionRun` / `Interactable`	`Scripts/Missions/`, `Scripts/Gameplay/Mission/`
Racing AI	Road-graph planner + racing line + traffic sweep	`Scripts/Gameplay/Racing/`
Procedural audio	Offline synthesis (engines, heist, mission, intro, score)	`Scripts/Editor/Audio/`
Procedural textures and fonts	Facades, asphalt, signage, 5×7 pixel font	`Editor/District/DistrictTextures.cs`, `PixelFont.cs`
Custom shaders	`NeonRift/AdditiveGlow`, `GroundHaze`, `NightSky`, `SkylineBackdrop`, `TerminalScreen`, `TowerFacade`	`Assets/_NeonRift/Art/Shaders/`
Validation harnesses	Vehicle bench, mission validator and series, route profiler, GPU bench, input playtest	`Scripts/Editor/Vehicles/`, `Scripts/Editor/Validation/`, `Scripts/Gameplay/Mission/NightRunValidator.cs`

04
Project Structure

IN THIS CHAPTER

4.1 Top level
4.2 Assets/_NeonRift/
4.3 Assets/ThirdParty/
4.4 Docs/
4.5 Naming conventions (from Docs/Conventions.md)

Only folders that exist in the repository are listed. All first-party content lives under `Assets/_NeonRift/`. Vendor content lives under `Assets/ThirdParty/` and is never edited in place (Decision D9).

4.1 Top level

```
Neon Rift/
├─ Assets/
│  ├─ _NeonRift/          first-party content (code, data, art, audio, scenes, UI)
│  └─ ThirdParty/         vendor models and music, each at its own import root
├─ Docs/                  design and technical notes (this book: Docs/NEON_RIFT_BOOK/)
├─ Packages/manifest.json Unity packages
├─ ProjectSettings/       Unity project settings
├─ README.md
├─ LICENSE.md
├─ .gitattributes         Git LFS + UnityYAMLMerge rules
└─ .gitignore             Library/, Temp/, Logs/, Builds/, UserSettings/, SourceArchives/ …
```

Local-only folders that are **not committed** (in `.gitignore`): `Library/`, `Temp/`, `Logs/` (test and playtest reports), `Builds/` (e.g. `Builds/PerfMac`), `UserSettings/`, `SourceArchives/` (original downloaded archives), `My project/`.

4.2 Assets/_NeonRift/

```
_NeonRift/
├─ Art/
│  ├─ Buildings/          URP materials and packed textures for the building packs
│  ├─ District/
│  │  ├─ Materials/       ≈120 generated district materials (District_*, Drone_*, Crowd_* …)
│  │  ├─ Meshes/          ≈8,880 generated mesh assets (blocks, props, beacons) — Git LFS
│  │  └─ Textures/        generated PNG textures (asphalt, facades, signage, glow shapes)
│  ├─ Garage/             Car Select garage materials, meshes, textures (planning wall)
│  ├─ Materials/DevTrack/ test-track materials
│  ├─ Shaders/            NeonRift_*.shader (6 custom shaders)
│  ├─ Traffic/            traffic car materials (Traffic_Pack*)
│  └─ Vehicles/PF_Vehicle_*/  split wheel meshes per car
├─ Audio/
│  ├─ Generated/          synthesised clips: Engine_V8Road, Engine_V8Race, Engine_EV, Engine_
Pops,
│  │                      Tyres, Road, Mechanical, Impacts, Mission, Intro
│  └─ Mixer/NeonRiftMixer.mixer
├─ Data/                  ScriptableObjects (see the data index in 00_INDEX.md)
│  ├─ Audio/AudioMixerConfig.asset
│  ├─ Config/GameConfig.asset, IntroSettings.asset
│  ├─ Intro/Intro_NightRun.playable, Intro_Surveillance.asset
│  ├─ Missions/Mission_NightRun.asset, Mission_DevTestTrack.asset, Interaction_*.asset, Mission
Audio_NightRun.asset
│  ├─ Physics/PM_VehicleBody.asset
│  ├─ Racing/Racer_Vex.asset, Racer_Kade.asset
│  ├─ Vehicles/VehicleCatalog.asset, Vehicle_*.asset, Physics/, Audio/, ImportSetups/
│  └─ World/RoadNetwork_NightRun.asset, BuildingCatalog.asset, Building_*.asset
├─ Prefabs/
│  ├─ Buildings/          22 PF_Bld_* (Asian Night pack, London, Singapore, skyline cluster)
│  ├─ District/           PF_Prop_StreetLight_{Led,Sodium,Warm}, PF_Prop_JerseyBarrier, PF_Prop_
SecurityBeacon
│  ├─ Drones/             PF_Drone_Mech, PF_Drone_Buster
│  ├─ Traffic/            PackCar_{Compact,Coupe,Hatchback,Minivan,Offroad,Pickup,Sedan,Sport,
SUV,Wagon}
│  └─ Vehicles/           PF_Vehicle_MercedesSLS, _MercedesSLSGT3, _LamborghiniTerzo, _
MercedesAMGGT3
├─ Scenes/
│  ├─ Bootstrap.unity     always loaded (GameRoot, LoadingOverlay, EventSystem)
│  ├─ Frontend.unity      title screen
│  ├─ CarSelect.unity (+ CarSelect/ probe data)  generated crew garage
│  ├─ NightRun.unity (+ NightRun/ 63 reflection-probe EXRs)  generated city + mission + intro
│  └─ Dev/TestTrack.unity physics validation track
├─ Scripts/               13 assemblies, one folder each (see 02_ARCHITECTURE.md)
│  ├─ Audio/ Core/ Frontend/ Game/ Gameplay/{City,Mission,Navigation,Racing}/ Input/ Intro/
│  │  Missions/ Rendering/ Vehicles/{Audio,Physics}/ World/
│  ├─ Editor/{Audio,District,Frontend,Intro,TestTrack,Validation,Vehicles,World}/
│  └─ Tests/EditMode/
├─ Settings/
│  ├─ Input/NeonRiftControls.inputactions
│  └─ Rendering/          PC_/Mobile_ RP assets and renderers, NightRun_* and Garage volumes,
│                         NightRun_Lighting.lighting, Garage_CameraBlends.asset
└─ UI/                    UXML/USS: TitleScreen, CarSelect, MissionHud, IntroOverlay, Loading
Overlay,
                          NeonRift.uss (shared theme), NeonRiftTheme.tss, NeonRiftPanelSettings.
asset
```

4.3 Assets/ThirdParty/

```
ThirdParty/
├─ Sketchfab/
│  ├─ Vehicles/   Mercedes_SLS_AMG_2010, Mercedes_SLS_GT3, Lamborghini_Terzo_Millennio,
│  │              Mercedes_AMG_GT3_RedBull (excluded from the catalog), generic-passenger-car-
pack
│  ├─ Buildings/  London_Skyscraper, Singapore_Office_Skyscraper, Asian_Night_City_Buildings,
Night_City_Building_Skyline
│  └─ Drones/     Mech_Drone, Buster_Drone, Drone_Design (not used)
└─ SoundAudio/    machine-speed-sport-aggressive-electronic.mp3, danger-chaos-cinematic-trailer-
hybrid.mp3
```

Every Sketchfab model folder except the passenger car pack has an `ATTRIBUTION.txt`. See 44 Assets and Licensing.

4.4 Docs/

File	Topic
`Architecture.md`, `Decisions.md` (D1–D27), `Conventions.md`	Architecture notes, decision records, coding rules
`City.md`, `Environment.md`	City generation, night look, LODs
`NightRun.md`, `Interactions.md`	Mission, heist, gate, rivals
`VehiclePhysics.md`, `Audio.md`	Vehicle model, audio
`Intro.md`	Cinematic intro and Car Select
`Performance.md`	Profiling and optimisation
`ThirdParty.md`, `Licenses/`	Asset audit and licences
`SourceControl.md`	Git, LFS, authorship
`Screenshots/`, `Audio/`	Captures and recorded mix excerpts
`NEON_RIFT_BOOK/`	This book

The older notes were written phase by phase. Where they disagree with the code (for example the rival "orders" that were replaced by rival tasks), this book follows the code and says so.

4.5 Naming conventions (from `Docs/Conventions.md`)

Kind	Pattern	Example
Vehicle definition	`Vehicle_<Name>`	`Vehicle_MercedesSLS`
Mission	`Mission_<Name>`	`Mission_NightRun`
Prefab	`PF_<Category>_<Name>`	`PF_Vehicle_MercedesSLS`, `PF_Drone_Mech`
Material	`<Area>_<Surface>`	`District_BeaconCyan`
Namespace	assembly name	`NeonRift.Gameplay`

Units are metres, kilograms and seconds (km/h for display only). Vehicle prefabs face +Z with the pivot at ground centre.

05
Game Flow

IN THIS CHAPTER

5.1 What the player experiences
5.2 How the flow is implemented
5.3 Mixer state per game state
5.4 Related

5.1 What the player experiences

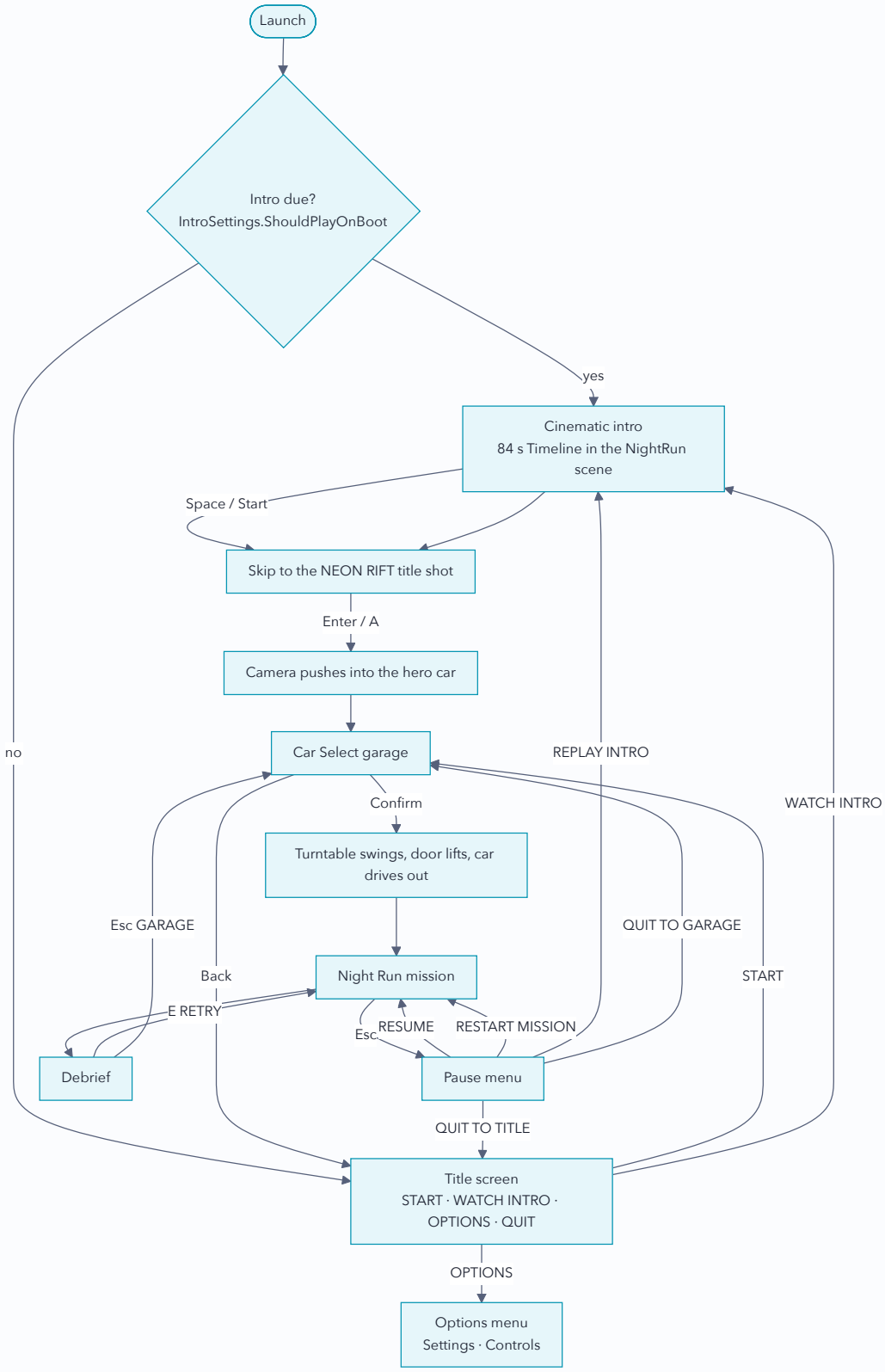


Figure 5.1 Game flow: screens and transitions

5.2 How the flow is implemented

Step	Code	File
Boot	`GameFlow.BootAsync`	`Assets/_NeonRift/Scripts/Game/GameFlow.cs:34`
Decide intro or title	`IntroSettings.ShouldPlayOnBoot()` (policy `FirstLaunchOnly` / `Always` / `Never`)	`Scripts/Game/IntroSettings.cs:31`
Play the intro	`GameFlow.PlayIntro()` → `TransitionAsync(GameState.Intro, config.IntroScene)` (scene `NightRun`) → `IntroSceneEntry.Enter`	`GameFlow.cs:66`, `Scripts/Intro/IntroSceneEntry.cs:87`
Title	`TitleScreen.Enter` binds `start-button`, `intro-button`, `options-button`, `quit-button`	`Scripts/Frontend/TitleScreen.cs`, `UI/TitleScreen.uxml`
Go to Car Select	`GameFlow.GoToCarSelect()`	`GameFlow.cs:74`
Choose car and mission	`RunSession.SelectVehicle(vehicle, catalog)`, `SelectMission`	`Scripts/Game/RunSession.cs`
Start the mission	`GameFlow.StartMission()`: checks `session.IsReadyToLaunch` and `GameConfig.IsPlayable`, clears the last result, loads `SelectedMission.SceneName`	`GameFlow.cs:76`
Mission entry	`MissionSceneEntry.Enter`: spawns the player, binds input and camera, spawns rivals, starts the `MissionDirector`	`Scripts/Gameplay/MissionSceneEntry.cs:39`
Pause	`MissionSceneEntry.OpenPause` builds an `OptionsMenu` with seven rows	`MissionSceneEntry.cs:115`
Retry / Garage on the debrief	`MissionDirector.Retry` (E) → `Flow.StartMission()`; `Continue` (Esc) → `Flow.GoToCarSelect()`	`Scripts/Gameplay/Mission/MissionDirector.cs:731`, `:738`
Quit	`GameFlow.QuitGame` (stops Play Mode in the editor, `Application.Quit` in a player)	`GameFlow.cs:92`

Pause details (verified in `MissionSceneEntry.OpenPause`)

- `Time.timeScale = 0`, `AudioListener.pause = true` (the music track ignores the listener pause and keeps playing, ducked).
- The mixer goes to the **Ducked** snapshot over 0.3 s, the Driving action map is disabled, and `MissionDirector.Paused = true`.
- Rows: RESUME · RESTART MISSION · CONTROLS · SETTINGS · REPLAY INTRO · QUIT TO GARAGE · QUIT TO TITLE.
- `ResumeFromPause` restores the previous time scale and mixer state.

Development shortcuts

You press Play from	What happens	Code
`Bootstrap`	Normal boot: intro or title	`GameFlow.BootAsync`
`NightRun` (or any mission scene)	The open scene is **adopted**: the first catalog car is selected, the mission whose `sceneName` matches is selected, and the mission starts directly	`GameFlow.AdoptScene`, `PrepareDevelopmentSession`

You press Play from	What happens	Code
`CarSelect` or `Frontend`	Adopted as that state	`GameFlow.StateForScene`
A player build with `-perfroute <folder>`	Boots straight into the default mission with the first car, and the performance runner drives the route	`Scripts/Game/PerfRouteArgs.cs`, `Scripts/Gameplay/PerfRouteRunner.cs`

5.3 Mixer state per game state

`GameRoot.OnStateChanged` switches the mixer to **Gameplay** for `Mission` and `Intro`, and to **Menu** for everything else. The mission director then switches to **Lockdown** and **Results** itself. See 29 Audio.

5.4 Related

- Scene loading and entry points: 06 Bootstrap and Scenes
- Intro: 31 Cinematic Intro
- Car Select: 15 Car Selection
- Mission: 19 Mission System

06
Bootstrap and Scenes

IN THIS CHAPTER

6.1 Why a Bootstrap scene
6.2 BootstrapLoader: entering from any scene
6.3 The scenes (build order from ProjectSettings/EditorBuildSettings.asset)
6.4 GameRoot: what it creates (Scripts/Game/GameRoot.cs)
6.5 GameConfig (Assets/_NeonRift/Data/Config/GameConfig.asset)
6.6 LoadingOverlay
6.7 Generated scenes: do not hand-edit
6.8 Layers (ProjectSettings/TagManager.asset)

6.1 Why a Bootstrap scene

The project never uses `DontDestroyOnLoad` (Decision D2). Long-lived objects live in a scene that stays loaded for the whole session, and exactly **one content scene** is loaded additively on top of it.

6.2 `BootstrapLoader`: entering from any scene

`Assets/_NeonRift/Scripts/Game/BootstrapLoader.cs` has a `[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]` method, `EnsureBootstrapLoaded`. If no scene named `Bootstrap` is loaded, it loads `Assets/_NeonRift/Scenes/Bootstrap.unity` additively. If Bootstrap is not in the build list, it logs a warning and the game flow is disabled. So pressing Play in **any** scene still goes through the game flow.

6.3 The scenes (build order from `ProjectSettings/EditorBuildSettings.asset`)

#	Scene	Path	Contents (from the scene file)	Entry point
0	**Bootstrap**	`Assets/_NeonRift/Scenes/Bootstrap.unity`	`GameRoot`, `LoadingOverlay` (UI Toolkit, `UI/LoadingOverlay.uxml`), `EventSystem` with `InputSystemUIInputModule`	– (composition root)
1	**Frontend**	`Scenes/Frontend.unity`	Camera, `UIDocument` with `UI/TitleScreen.uxml`, `TitleScreen`	`TitleScreen`
2	**CarSelect**	`Scenes/CarSelect.unity` (+ `Scenes/CarSelect/` probe data)	The generated crew garage: 69 GameObjects, 21 lights, 1 reflection probe, `VehicleShowroom`, `ShowroomEngine`, Cinemachine cameras, `CarSelectScreen`	`CarSelectScreen`

#	Scene	Path	Contents (from the scene file)	Entry point
3	**TestTrack** (dev)	`Scenes/Dev/TestTrack.unity`	Physics test surfaces and the 1.55 km validation route	`MissionSceneEntry` (mission `Mission_DevTestTrack`)
4	**NightRun**	`Scenes/NightRun.unity` (+ `Scenes/NightRun/` 63 probe EXRs)	The generated city, mission rig, intro rig: ≈15,236 GameObjects, 862 lights, 63 reflection probes, 1 `PlayableDirector`	`MissionSceneEntry` (missions) **and** `IntroSceneEntry` (`IIntroEntryPoint`)

The NightRun scene hosts **two** entry points. `GameFlow.FindEntryPoint` picks the `IIntroEntryPoint` one only for `GameState.Intro`. The intro rig is inert during missions (shots, overlay and stage lights are off), and `IntroTests.NightRunScene_HasAWiredIntro` checks this.

6.4 `GameRoot`: what it creates (`Scripts/Game/GameRoot.cs`)

`[DefaultExecutionOrder(-1000)]`, `Awake`:

Object	Purpose
`NeonRiftControls`	Generated Input System wrapper (Driving and Menu maps)
`RunSession`	Selected car, rivals, mission, last result
`AudioMixerService`	Mixer snapshots and user volumes (mixer set to unscaled time so fades work while paused)
`GameSettings`	Loads and applies PlayerPrefs settings
`GameFlow`	Scene state machine
`GameContext`	Bundle handed to every scene: `Config`, `Session`, `Flow`, `Controls`, `Audio`, `Settings`
Music toggle `InputAction`	`<Keyboard>/m` and `<Gamepad>/buttonNorth` → `GameSettings.ToggleMusic()`, enabled globally

`Start` applies the saved audio and graphics settings, then awaits `GameFlow.BootAsync`.

6.5 `GameConfig` (`Assets/_NeonRift/Data/Config/GameConfig.asset`)

Field	Value
`frontendScene`	`Frontend`
`carSelectScene`	`CarSelect`
`introScene`	`NightRun`
`introSettings`	`Data/Config/IntroSettings.asset`
`vehicleCatalog`	`Data/Vehicles/VehicleCatalog.asset`
`missions`	`Mission_NightRun`, `Mission_DevTestTrack`
`defaultMission`	`Mission_NightRun`
`audioMixer`	`Data/Audio/AudioMixerConfig.asset`
`fadeSeconds`	0.35

Class: `Scripts/Game/GameConfig.cs` (`IsPlayable`, `FindMissionForScene`, `ReferencedScenes`). `MissionDefinition.developmentOnly` hides dev missions in release builds.

6.6 `LoadingOverlay`

`Scripts/Game/LoadingOverlay.cs`: a full-screen UI Toolkit fade that starts opaque (so the empty Bootstrap scene is never seen). `ShowAsync` and `HideAsync` are awaited by `GameFlow.TransitionAsync`.

6.7 Generated scenes: do not hand-edit

Scene	Generated by	Menu
NightRun	`NightRunBuilder.Build(bool bakeProbes)` (`Scripts/Editor/District/NightRunBuilder.cs:81`)	**Neon Rift ▸ Night Run ▸ Build Night Run (district + mission)**
CarSelect	`GarageBuilder.Build` (`Scripts/Editor/Frontend/GarageBuilder.cs:57`)	**Neon Rift ▸ Car Select ▸ Build Garage**
TestTrack validation route	`ValidationRouteBuilder.Build` (`Scripts/Editor/TestTrack/ValidationRouteBuilder.cs:91`)	**Neon Rift ▸ Test Track ▸ Build Validation Route**
Intro rig (inside NightRun)	`IntroBuilder.Build(Scene)` (also run by every NightRun build)	**Neon Rift ▸ Intro ▸ Build Intro (open Night Run scene)**

6.8 Layers (`ProjectSettings/TagManager.asset`)

#	Layer	Used for
6	Vehicle	Player and rival car bodies
7	Drivable	Road and ground surfaces (wheel rays, camera collision)
8	Environment	Buildings, props, walls (camera collision)
9	Trigger	Mission zones and interactables (collide with vehicles only)
10	Showroom	Car Select presentation (collides with nothing)
11	Detail	Small street furniture, culled beyond 180 m by `CameraCullDistances`
12	Clutter	Infill furniture and parked props, culled beyond 90 m
13	Traffic	Pack traffic cars (Traffic ignores Traffic: `Physics.IgnoreLayerCollision` in `CityTraffic.Start`)

PART II

The World

The generated night city, its buildings and its road network.

07 City and World Generation

08 Buildings and Environment

09 Roads and Route Network

07
City and World Generation

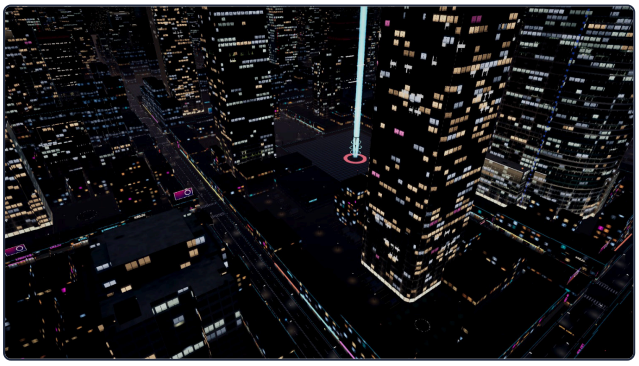
IN THIS CHAPTER

7.1 The short answer
7.2 The city at a glance
7.3 Generation pipeline (order verified in NightRunBuilder.Build)
7.4 Where to find each piece
7.5 Validation

7.1 The short answer

The city is **not modelled by hand**. An editor tool generates the whole `NightRun` scene from data in about 40 s: every block, street wall, prop, lamp, texture, material, mesh, reflection probe, the road graph, the mission data and the intro. The layout data lives in `CityLayout.cs`. **Change the builder, not the scene** (Decisions D23 and D24).

Menu: **Neon Rift ▸ Night Run ▸ Build Night Run (district + mission)** → `NightRunBuilder.BuildFromMenu` → `NightRunBuilder.Build(bakeProbes: true)` (`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.cs:81`).



Aerial view, Sector 7
`Docs/Screenshots/CityInfill/aerial_sector7.jpg`

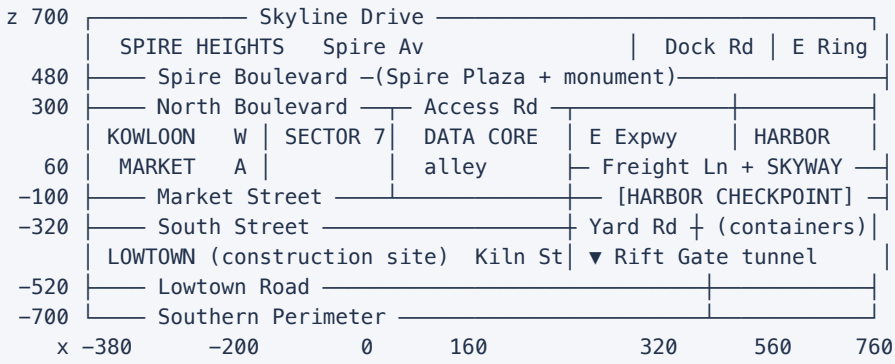


Aerial view, Lowtown
`Docs/Screenshots/CityInfill/aerial_lowtown.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

7.2 The city at a glance

About **1.14 × 1.40 km**, five districts, data in `CityLayout.Districts`:



District (`CityLayout.Districts`)	Area	Character (`NightRunDistrict.Style`, `ZoneStyle`)
SECTOR 7 (hero)	x 0…320, z −320…300	Shopfront podiums, neon awnings, blade signs, kerb neon, curated hero towers
SPIRE HEIGHTS	north of z 300	Corporate high-rise, tall podiums, curtain-wall glass, LED fins
KOWLOON MARKET	x −400…0	Dense, short podiums, signs, overhead cables, lantern strings
HARBOR YARDS	x 320…780	Industrial warehouses, sodium lamps, container yard
LOWTOWN	south of z −320	Worn mid-rise, fire escapes, construction site

7.3 Generation pipeline (order verified in `NightRunBuilder.Build`)

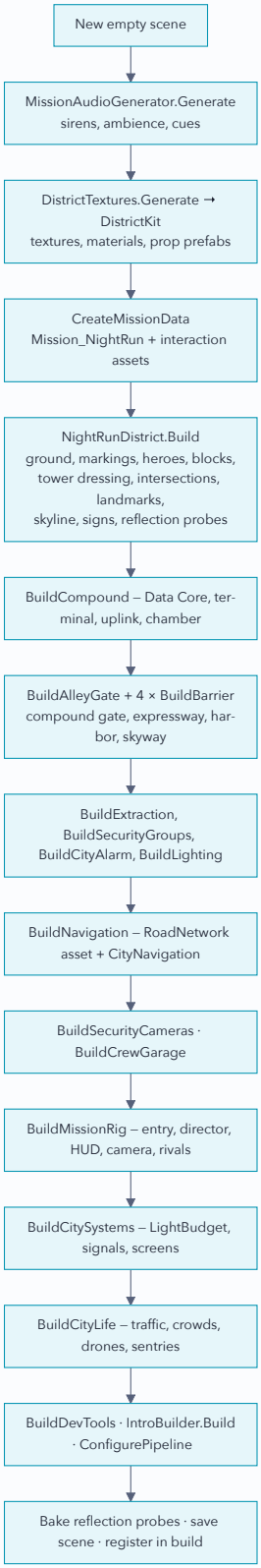


Figure 7.1 City generation flow (NightRunBuilder.Build)

Step 1: layout data → blocks (`CityLayout.Decompose`)

`CityLayout` is plain data:

Data	Meaning	Code
`Roads` (`Road` struct)	Axis-aligned roads with name, sign text, half-width, lanes, sidewalk, `RoadClass`, median, ground flag, optional blocker id	`CityLayout.cs`, factory `R(...)` at line 85
`Blockers`	Gate positions; the graph edge under each gets the gate id	`CityLayout.Blockers`
`Reserved`	Areas the generator leaves alone: Data Core compound, Rift Gate tunnel, crew garage	`CityLayout.Reserved`
`Lots`	Special lots: S7 Car Park, Spire Plaza, Night Market, Harbor Container Yard, Lowtown Construction	`CityLayout.Lots`
`Heroes`	Hand-placed catalog buildings (Singapore offices, London tower, Asian Night towers)	`CityLayout.Heroes`
`Districts`	Name, rectangle, HUD colour	`CityLayout.Districts`

Algorithm. Every ground road contributes its x (vertical roads) or z (horizontal roads) line. The sorted lines cut the world into a grid of cells. Neighbouring cells with **no road between them** are merged with a union-find (`Find` in `Decompose`), so each merged group becomes a **block**. Adding a street reforms the blocks around it automatically. `SlabPieces` and `Subtract` cut the reserved areas and lots out of the pavement slabs.

Step 2: blocks → geometry (`NightRunDistrict`)

`NightRunDistrict.Build` (`Editor/District/NightRunDistrict.cs:126`):

1. `BuildGround`: one asphalt ground plane. **Roads are the gaps between raised pavement blocks** (kerb 15 cm), so junctions need no special geometry.
2. `BuildMarkings`: edge, centre and lane lines, double solid lines on arterials, zebras, stop lines.
3. `PlaceHeroes`: the `CityLayout.Heroes` prefabs, with `HeroLod`.
4. `BuildBlock` per block: pavement slab, kerb, **podium street walls** (`BuildStreetWall`), street lights, towers (`FillTowers` from the `BuildingCatalog`), props and infill (`CityInfill.Fill`).
5. `TowerDressing.Emit`: window shader, crown strips, rooftop plant, aviation lights.
6. `CityProps.BuildIntersections`: signals on mast arms, street-name blades, bollards, CCTV.
7. `CityLandmarks.Build`: skyway, Lantern Arcade, car park, plaza and monument, night market, container yard with gantry cranes, construction site with a tower crane, expressway median.
8. `SkylineBackdrop.Build`: three rings of tower silhouettes beyond the city.
9. `BuildNavigationSigns`: district gateway gantries and escape gantries.
10. `BuildReflectionProbes`: box-projected probes along the arterials.

Mesh strategy (cost control)

Technique	Where
`MeshBuilder` accumulates quads per material; UVs in metres ÷ texture tile size	`Editor/District/MeshBuilder.cs`

Technique	Where
`BlockMeshes` : one combined renderer **per material per block** (the batching unit)	`Editor/District/BlockMeshes.cs`
`DistrictKit.SaveMesh` overwrites mesh assets in place, so GUIDs and scene references stay stable	`Editor/District/DistrictKit.cs:224`
Static batching / occluder flags on district geometry	`DistrictKit.Renderer`
Detail and Clutter layers culled at 180 m and 90 m	`Gameplay/CameraCullDistances.cs`

Determinism

Every generator uses fixed seeds (`System.Random`) and deterministic textures, so the same data produces the same city. Each block's infill has its own seed, so changing the infill never reshuffles street walls.

7.4 Where to find each piece

Feature	Builder	Runtime component (if any)
Layout data	`Editor/District/CityLayout.cs`	–
Master builder	`Editor/District/NightRunBuilder.cs` (+ `.Heist.cs` , `.Garage.cs` , `.CityLife.cs`)	–
Blocks, street walls, towers, lamps	`Editor/District/NightRunDistrict.cs`	`Gameplay/City/LightBudget.cs`
Block interiors	`Editor/District/CityInfill.cs`	–
Street furniture and intersections	`Editor/District/CityProps.cs`	`Gameplay/City/TrafficSignalNetwork.cs`
Landmarks	`Editor/District/CityLandmarks.cs`	–
Tower night look	`Editor/District/TowerDressing.cs` + `Art/Shaders/NeonRift_TowerFacade.shader`	–
Far skyline	`Editor/District/SkylineBackdrop.cs` + `NeonRift_SkylineBackdrop.shader`	–
Textures	`Editor/District/DistrictTextures.cs` , `PixelFont.cs`	–
Materials and prop prefabs	`Editor/District/DistrictKit.cs`	–
Road graph	`CityLayout.BuildNetwork` → `Data/World/RoadNetwork_NightRun.asset`	`Gameplay/Navigation/CityNavigation.cs`

Details: 08 Buildings and Environment, 09 Roads and Route Network, 25 Lighting.

7.5 Validation

- `CityTests` (`Scripts/Tests/EditMode/CityTests.cs`): `CityNetwork_IsValidAndConnected` , `CityIsMuchLargerThanTheSlice` , `EveryBlockHasAZoneAndPavement` , `HeroBuildingsAvoidRoadsAnd`

`ReservedAreas`, `Path_SpawnToCore_FollowsRoads`, `Skyway_IsElevatedAndReachable`, `NightRun_CityLifeIsSetUp`.

- `LookDevShots` (`Editor/District/LookDevShots.cs`) captures fixed viewpoints for before/after comparison (**Neon Rift ▸ Night Run ▸ Capture Look-Dev Shots (Play Mode)**).
- The builder log (`Logs/NightRunBuild.txt` when written) prints counts of podiums, towers, lamps, signals, infill parcels and LODs.

08
Buildings and Environment

IN THIS CHAPTER

8.1 Podium street walls
8.2 Supplied building models (catalog)
8.3 Tower night dressing (skyscrapers)
8.4 Block interiors (CityInfill)
8.5 Street furniture and infrastructure (CityProps)
8.6 Landmarks (CityLandmarks)
8.7 Skyline beyond the city
8.8 Data Core compound, gates and tunnel
8.9 The crew garage in the city
8.10 Generated textures and materials
8.11 Related

The city's buildings come from **three sources**:

1. **Procedural street walls** (podiums) generated per block. They are most of what you see from the car.
2. **Supplied building models** (Sketchfab packs), wrapped in prefabs and placed from a catalog.
3. **Procedural infill** behind the street walls (back buildings, yards, car parks …).

All three are generated by editor code during **Build Night Run**.

8.1 Podium street walls

PURPOSE	A continuous, lit street frontage on every block side: shopfront ground floor, lit upper floors, neon or fabric awnings, crown strip, facade and blade signs, rooftop billboards and plant
IMPLEMENTATION	`NightRunDistrict.BuildStreetWall` (`Assets/_NeonRift/Scripts/Editor/District/NightRunDistrict.cs:406`) walks each block side, splits it into podium segments around heroes and junctions (`OpenSpans`), and emits boxes and panels into `BlockMeshes` (one renderer per material per block). One box collider per podium. Zone styles come from `NightRunDistrict.Style` / `ZoneStyle`
TEXTURES	Facades, shopfronts, signage atlas and billboards from `DistrictTextures` (`Assets/_NeonRift/Scripts/Editor/District/DistrictTextures.cs`); sign text is baked with the in-house 5×7 `PixelFont`
SCENE	`NightRun` → `Environment/District/…` (combined block meshes)
DATA	Zone of each block (`CityLayout.ZoneAt`)
DEPENDENCIES	`MeshBuilder`, `BlockMeshes`, `DistrictKit` materials

Details added in the city-detail pass (`Docs/Environment.md`): fabric awnings on 38 % of non-Spire shopfront segments, closed shutters on 12 %, regulatory kerb signs, CCTV boxes, loading bays, parking-bay runs and steam vents on Kowloon/Lowtown manholes.

8.2 Supplied building models (catalog)

PURPOSE	Landmark towers and inner-block towers
INTAKE	`ThirdPartyIntake.BuildBuildings` / `BuildAsianPack` / `BuildSkylinePack` (`ThirdPartyIntake.cs`) call `BuildingPrefabBuilder` (`Assets/_NeonRift/Scripts/Editor/World/BuildingPrefabBuilder.cs`): ground-centre pivot, URP materials with packed smoothness and alpha (`PackSmoothness`, `PackAlpha`), optional box collider, Environment layer. Menu **Neon Rift ▸ Assets ▸ Run Third-Party Intake**
PREFABS	`Assets/_NeonRift/Prefabs/Buildings/`: `PF_Bld_LondonSkyscraper`, `PF_Bld_SingaporeOffice`, 19 × `PF_Bld_AsianNight_*`, `PF_Bld_NightSkylineCluster`
DATA	`BuildingDefinition` per prefab (`Assets/_NeonRift/Data/World/Building_*.asset`), grouped in `BuildingCatalog.asset`. `BuildingTier`: Hero / Midground / Skyline (`Scripts/World/BuildingDefinition.cs`)
PLACEMENT	Heroes: `CityLayout.Heroes` → `NightRunDistrict.PlaceHeroes` (podiums shrink around them so the street wall stays closed). Generated towers: `NightRunDistrict.FillTowers` picks catalog towers that fit free footprints (`FootprintRect`, `Overlaps`)
LOD	`NightRunDistrict.HeroLod`: hero interiors, lobbies, furniture and curtains cast no shadows and drop out below 15 % screen height; the shell is never culled
SOURCE	`Assets/ThirdParty/Sketchfab/Buildings/` (see 44 Licensing)

The six skyline clusters that once stood about 1.25 km out were removed (`CityLayout.Skyline` is empty). They changed under 0.2 % of pixels in A/B captures. The catalog entry `night_skyline_cluster` is kept.

8.3 Tower night dressing (skyscrapers)

PURPOSE	Stop the low-poly kit towers reading as black masses at night
IMPLEMENTATION	`TowerDressing` (`Assets/_NeonRift/Scripts/Editor/District/TowerDressing.cs`): `ApplyFacade` swaps kit materials for `NeonRift/TowerFacade` (`Assets/_NeonRift/Art/Shaders/NeonRift_TowerFacade.shader`). The shader draws a procedural office-window grid in **world space** (3.6 m floors, 3.2 m bays) with lights on by suite, so it survives static batching. `DressTower` raycasts each tower's own mesh to find its roof edge and adds `Crown` strips plus wash, `Rooftop` plant with status LEDs, and `Aviation` lights on towers over 110 m. Geometry is merged per 400 m cell (`Cell`) and material
COUNTS (Environment.md)	11,229 crown segments, 915 rooftop items, 245 aviation lights

8.4 Block interiors (`CityInfill`)

PURPOSE	Fill the inside of blocks (about 139 ha that used to be bare slab) with fabric that has a reason to be there

ALGORITHM	`CityInfill.Fill` (`Assets/_NeonRift/Scripts/Editor/District/CityInfill.cs:74`): rasterise free space on a 2 m grid (excluding podiums, towers, heroes, lots, reserved areas, sidewalks); take the largest free rectangle (histogram method, `Grow`), split to ≤ 58 m (70 m in the harbor, `Split`), repeat until parcels are under 120 m²; choose a use per parcel from the zone mix (`Choose`), then `Build`
USES	`Buildings` / `Mass` (back buildings with parapets and rooftop plant), `ServiceYard`, `Parking` (surface car park), `Garage` (multi-storey decks), `Park` (pocket park), `Courtyard` (fountain), `Utility` (fenced substation), `Warehouse`, `Storage` (container stacks); slivers become `Annex` or `BackOfHouse`
PROPS	`Fence`, `YardLamp` (emissive only), `Bollard`, `Bench`, `Tree`, `Dumpster`, `Crates`, `Pallets`, `Barrels`, `Sign`, `Vehicle` (parked background vehicles)
COST RULES	Masses join the block meshes; furniture and parked vehicles go to the **Clutter** layer (12) in 64 m cells, culled beyond 90 m; interior light is emissive only (no real-time lights); one simple box collider per mass
RESULT (City.md)	119 of 139 ha filled: 707 building parcels, 69 service yards, 27 surface car parks, 13 multi-storey car parks, 24 parks, 30 courtyards, 26 utility compounds, 31 warehouses, 17 storage yards, 1,802 parked vehicles, 385 yard lamps

8.5 Street furniture and infrastructure (`CityProps`)

`Assets/_NeonRift/Scripts/Editor/District/CityProps.cs` places, per block side and zone:

Category	Methods
Sidewalk furniture	`Bin`, `Bench`, `Hydrant`, `Utility`, `Vending`, `Planter`, `Dumpster`, `Crates`, `Pallets`, `Barrels`, `FoodCart`, `Shelter` (bus shelters with adverts)
Signs and cameras	`KerbSigns`, `RegulatorySign` (60 / 30 km/h, NO PARKING, LOADING), `Cctv`
Facade clutter	`Facade` (AC units, drain pipes, fire escapes, LED fins)
Rooftops	`Rooftop` (tanks, HVAC, antennas with aviation lights)
Alleys	`AlleyWall`: service pipes, extract **vents**, caged **security lights**, **fire-exit doors** under green signs, **cables** slung across
Lamps	street lights via `DistrictKit` prefabs, `LampHaze` cones
Intersections	`BuildIntersections`: **traffic signals** on mast arms (far-side right of each approach), pole heads, street-name `Blade`s, corner bollards, CCTV; split into 400 m cells; the six lens renderers stay city-wide for `TrafficSignalNetwork`

Prop prefabs from `DistrictKit.CreatePrefabs`: `PF_Prop_StreetLight_{Led,Sodium,Warm}`, `PF_Prop_JerseyBarrier`, `PF_Prop_SecurityBeacon` (`Assets/_NeonRift/Prefabs/District/`, with LOD groups via `DistrictKit.AddLod`).

8.6 Landmarks (`CityLandmarks`)

`Assets/_NeonRift/Scripts/Editor/District/CityLandmarks.cs`. All static geometry; nothing animates except where noted.

Landmark	Method	Notes
Harbor Skyway (elevated, neon-edged)	`Skyway`	Graph road with ramps (y 0 → 8 m); opened by the Skyway Gate in a lockdown
Expressway median barrier	`ExpresswayMedian`	Also a 0.8 m median in the road graph
Lantern Arcade, Kowloon arches	`LanternArcade`, `Arch`	
Kowloon overhead cables and lantern strings	`KowloonCables`, `Catenary`	
S7 Car Park	`CarPark`	Slots `CityLandmarks.CarParkSlots` (used by the intro staging marker)
Spire Plaza and monument	`Plaza`	72 m glass needle with holo rings
Night market	`NightMarket`	Canopies have colliders above car height (camera clipping fix)
Harbor container yard with **gantry cranes**	`ContainerYard`, `GantryCrane`	Static cranes with aviation lights. **Cranes are not animated**
Lowtown construction site with a **tower crane**	`Construction`, `TowerCrane`	Static

Gateway and escape gantries (`SPIRE HEIGHTS ↑`, `KOWLOON MARKET`, `HARBOR YARDS`, `LOWTOWN`, `EXTRACTION`, `RIFT GATE`, `SKYWAY`): `NightRunDistrict.BuildNavigationSigns` and `Gantry`.

8.7 Skyline beyond the city

`SkylineBackdrop` (`Assets/_NeonRift/Scripts/Editor/District/SkylineBackdrop.cs`): three rings of tower silhouettes 1.7, 2.1 and 2.55 km from the centre, each hazier than the one in front, with downtown clusters and blinking aviation lights. About 3 k triangles in one draw, using the unfogged shader `NeonRift/SkylineBackdrop`. The camera far plane is 3.5 km (`SkylineBackdrop.RequiredFarClip`).

8.8 Data Core compound, gates and tunnel

Built by mission code, not the district: `NightRunBuilder.BuildCompound`, `BuildAlleyGate`, `BuildBarrier`, `BuildExtraction` (`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.cs`), and `NightRunBuilder.Heist.cs` (terminal kiosk, chamber, arms). See 20 to 22.

8.9 The crew garage in the city

`NightRunBuilder.BuildCrewGarage` (`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.Garage.cs`) places the **same hall** as Car Select (`GarageBuilder.BuildShell`) on W Avenue, in the reserved rect (−44…−7, −271…−239), with its roller door onto the avenue. It has colliding walls, its own baked probe and intro-only lights, and a runtime `CrewGarage` component (`Scripts/Gameplay/CrewGarage.cs`) that strikes the light banks and lifts the door. The mission spawn is just outside the door at (3.5, −238).

8.10 Generated textures and materials

Asset	Generator	Output
Wet asphalt, pavement, facades, shopfronts, signs, billboards, warning billboard, glow gradients, plaza, container	`DistrictTextures.Generate` (`Asphalt`, `Pavement`, `Facade`, `Shopfronts`, `Signs`, `Billboard`, `WarningBillboard`, `GlowGradient`, `GlowSoft`, `Plaza`, `Container`)	`Assets/_NeonRift/Art/District/Textures/` (PNG)
District materials (`District_*`)	`DistrictKit.CreateMaterials`, `Lit`, `Glow`	`Assets/_NeonRift/Art/District/Materials/`
Meshes	`DistrictKit.SaveMesh`	`Assets/_NeonRift/Art/District/Meshes/` (Git LFS)

8.11 Related

07 City Generation · 25 Lighting · 26 Weather and Atmosphere · 39 Performance

09

Roads and Route Network

IN THIS CHAPTER

9.1 One source of truth for "where can a car go" (Decision D24)
9.2 Road data (CityLayout.Road, Assets/_NeonRift/Scripts/Editor/District/CityLayout.cs)
9.3 Building the graph (CityLayout.BuildNetwork, line 412)
9.4 Runtime classes
9.5 Gates on the graph
9.6 Consumers
9.7 Validation

9.1 One source of truth for "where can a car go" (Decision D24)

The same road list in `CityLayout.Roads` produces two things:

1. The **visible roads**: gaps between raised pavement blocks, plus markings (`NightRunDistrict.BuildGround`, `BuildMarkings`).
2. The **runtime road graph** `RoadNetwork_NightRun.asset`, used by the HUD route, the minimap, the location readout, rival AI, traffic and race standings.

9.2 Road data (`CityLayout.Road`, `Assets/_NeonRift/Scripts/Editor/District/CityLayout.cs`)

Field	Meaning
`Name`, `Sign`	Street name (HUD readout) and sign text
`A`, `B`	Axis-aligned end points
`HalfWidth`, `Lanes`	Carriageway half width; lanes per direction
`Sidewalk`, `Median`	Pavement width; central median half width (expressway 0.8 m, Freight Lane 4.5 m)
`Class`	`RoadClass`: Arterial, Street, Alley, Service, Skyway, Tunnel
`Ground`	Ground roads cut the slabs and get markings; graph-only roads (skyway, compound, car park) don't
`BlockerId`	Gate id on this road
`SpeedLimit`	By class: Arterial 38, Skyway 40, Street 28, Tunnel 26, Alley 16, other 12 (m/s)

Examples (verified): `W Avenue` x = 0, half width 7, 2 lanes, Arterial. `Market Street` z = −100, half width 6, 1 lane. `East Expressway` x = 320, half width 9, median 0.8. `Service Alley` x = 160, z −100…60, Alley. Graph-only: `Compound Drive`, `Compound Loop` (loops round the west of the core so routes don't pass through the pedestal), `Car Park`, `Skyway West Ramp` / `Harbor Skyway` / `Skyway East Ramp` (elevated to 8 m).

9.3 Building the graph (`CityLayout.BuildNetwork`, line 412)

1. `NetworkNodes()` puts nodes at every junction and road end, plus nodes 6 m either side of each blocker (so a gate sits on its own short edge).
2. For each road, the nodes lying on it are sorted along the road, and consecutive pairs become `RoadEdge`s carrying width, lanes, median, class, street name, speed limit and **blocker id** (from `Road.BlockerId` or a `Blockers` entry within 6.5 m of the edge midpoint).
3. `CityDistrict` entries (id, display name, area, colour) are added.
4. Result: `Assets/_NeonRift/Data/World/RoadNetwork_NightRun.asset` (last build log: **72 nodes, 107 edges**). `RoadNetwork.Validate` problems are logged by `NightRunBuilder.BuildNavigation`.

9.4 Runtime classes

Class	File	Role
`RoadNetwork` (ScriptableObject)	`Scripts/World/RoadNetwork.cs`	Pure data: `RoadNode`, `RoadEdge`, `CityDistrict`; `NodePosition`, `TryGetNearestEdge`, `TryGetDistrict`, `ProjectXZ`, `Validate`
`RoadGraph`, `RoadPath`	`Scripts/World/RoadGraph.cs`	Shortest paths between **arbitrary positions**. Each end is projected onto its nearest edge, then **Dijkstra** runs over the nodes with per-query extra edge costs. The start edge is never treated as closed (a car past a gate can drive away). Allocation-free after construction apart from the result. `FindPath`, `CountReachable`, `EdgeLength`
`CityNavigation` (MonoBehaviour)	`Scripts/Gameplay/Navigation/CityNavigation.cs`	The scene's view of the graph with **live gate state**: closed gate = impassable (∞), counting down to close = +450 m (`pendingPenalty`). `Version` increments whenever a gate changes state, so drivers and the HUD re-plan. `Plan(from, to, path[, extraCost])`, `IsBlocked`, `StreetAt`, `TryGetDistrict`

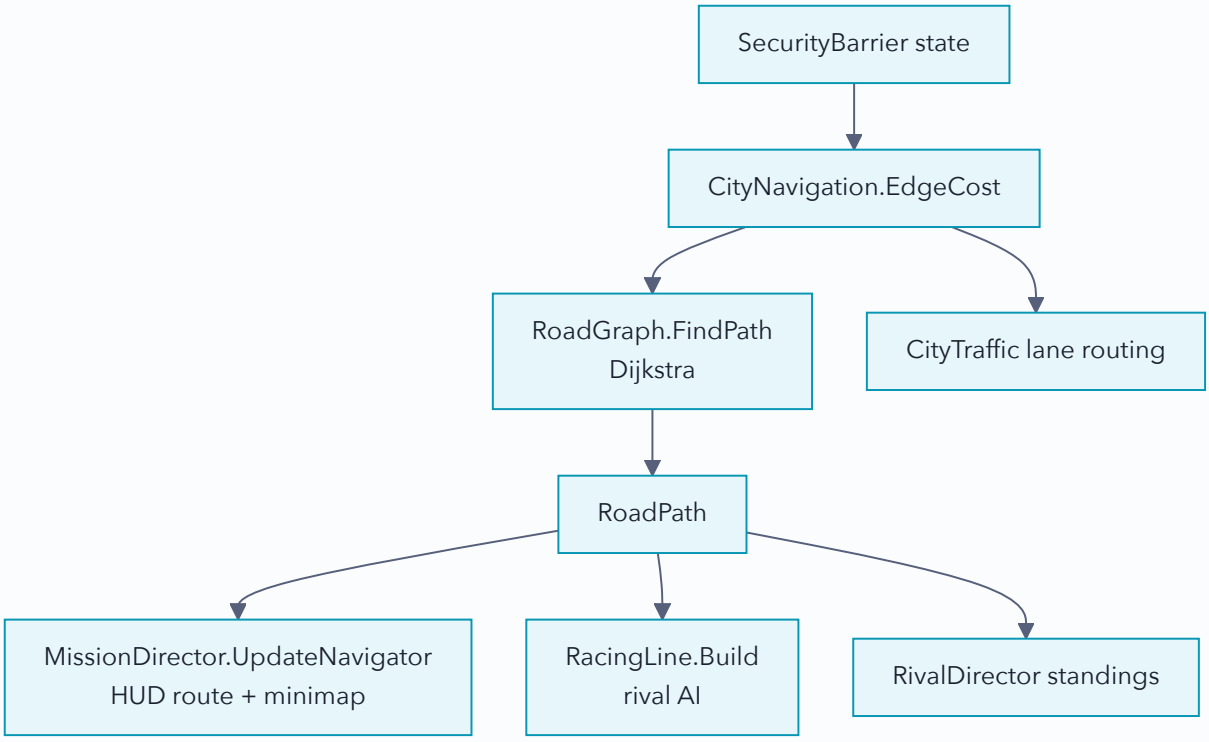


Figure 9.1 Route network: gate state to routes

9.5 Gates on the graph

Wired in `NightRunBuilder.Build` → `BuildNavigation(c, gates)`:

Gate id (`CityLayout`)	Barrier object	Default
`AlleyGateId`	AlleyGate (hackable)	Closed until hacked
`CompoundGateId`	CompoundGate (on `Compound Drive`)	Open; seals 12 s after the lockdown
`CheckpointId`	ExpresswayCheckpoint (320, 170)	Open; seals 30 s after the lockdown
`HarborCheckpointId`	HarborCheckpoint (440, −100)	Open; seals 22 s after the lockdown
`SkywayGateId`	SkywayGate (on the west ramp)	Closed; opens on `escape.start`

9.6 Consumers

Consumer	How it uses the graph
HUD route and minimap	`MissionDirector.UpdateNavigator` re-plans every `routeInterval` (0.5 s) or when `Version` changes; `MissionHud.SetNavigator`; `CityMinimap.SetRoute` draws it
Street / district readout	`CityNavigation.StreetAt` (nearest edge within 30 m), `TryGetDistrict`
Rival AI	`RacerDriver.Replan` → `RacingLine.Build(path)`; remembered blocked edges cost +250 m (`RacerDriver.AvoidCost`)
Traffic	`CityTraffic` drives kerb-side lanes of the graph; dead ends are pruned so cars don't strand at the gated compound spur

Consumer	How it uses the graph
Standings	`RivalDirector.RouteMetres`

9.7 Validation

`CityTests`: `CityNetwork_IsValidAndConnected`, `Path_SpawnToCore_FollowsRoads`, `ClosedGates_ChangeTheRoute`, `RememberedBlockedEdge_IsRoutedAround`, `Skyway_IsElevatedAndReachable` (`Assets/_NeonRift/Scripts/Tests/EditMode/CityTests.cs`).

PART III

Vehicles

Cars from data and physics to sound, visuals, selection and input.

10 Vehicle System

11 Vehicle Physics

12 Vehicle Audio

13 Vehicle Visuals

14 Car Catalog and Data

15 Car Selection

16 Player Input

10 Vehicle System

IN THIS CHAPTER

10.1 Overview
10.2 Spawning (VehicleSpawnPoint.Spawn)
10.3 Input abstraction (Scripts/Vehicles/DrivingInput.cs)
10.4 Which script controls what?
10.5 Vehicle-specific configuration
10.6 Chapters in this section

10.1 Overview

A car in Neon Rift is made of four things:

Part	What it is	Where
Definition (data)	Identity, prefabs, physics profile, audio profile, display stats, licence	`VehicleDefinition` (`Assets/_NeonRift/Scripts/Vehicles/VehicleDefinition.cs`), assets `Assets/_NeonRift/Data/Vehicles/Vehicle_*.asset`
Prefab (geometry)	Nested source model + split wheel pivots + `VehicleRig` + Rigidbody + two-box collider + `VehicleController` + `VehicleAudio` + LODGroup	`Assets/_NeonRift/Prefabs/Vehicles/PF_Vehicle_*.prefab`
Physics profile (tuning)	Everything that makes it drive the way it does	`VehiclePhysicsProfile` (`Scripts/Vehicles/Physics/VehiclePhysicsProfile.cs`), `Data/Vehicles/Physics/VehiclePhysics_*.asset`
Audio profile (sound)	Engine loops, beds, one-shots, levels	`VehicleAudioProfile` (`Scripts/Vehicles/Audio/VehicleAudioProfile.cs`), `Data/Vehicles/Audio/VehicleAudio_*.asset`

One component drives every car: **`VehicleController`** (`Scripts/Vehicles/Physics/VehicleController.cs`). Whoever spawns a car decides who drives it through **`IVehicleInputSource`**.

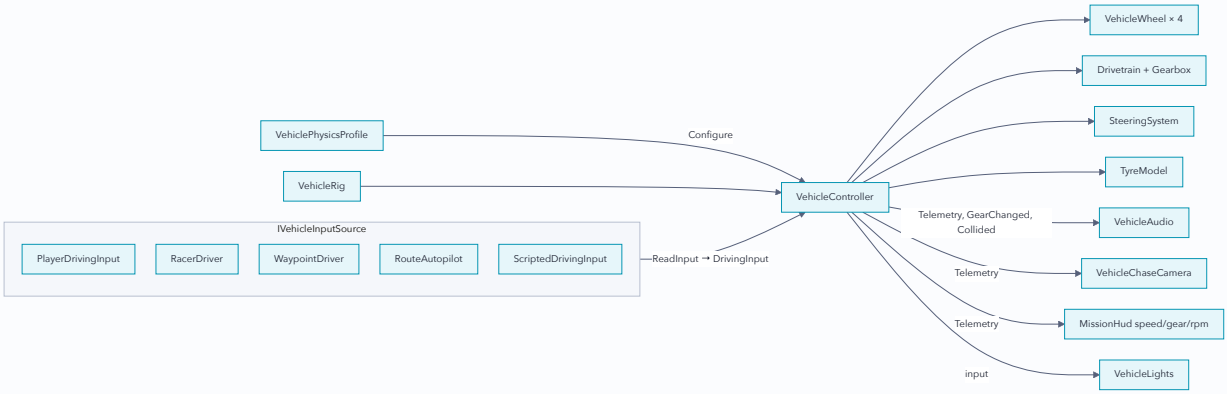


Figure 10.1 Vehicle architecture

10.2 Spawning (`VehicleSpawnPoint.Spawn`)

`Assets/_NeonRift/Scripts/Gameplay/VehicleSpawnPoint.cs:17`:

1. Instantiate `definition.GameplayPrefab` at the spawn pose.
2. `controller.Configure(definition.PhysicsProfile)`: builds the wheels, configures the body, drivetrain and steering, and makes the kinematic prefab Rigidbody dynamic.
3. Add or get `VehicleLights` and call `Build(detailedLights)`. Only the player gets shadows and full range.
4. `VehicleAudio.Configure(definition.AudioProfile, controller)`.

Then the caller sets the input source. `MissionSceneEntry` gives the player a `PlayerDrivingInput`, and `RivalDirector.Spawn` gives each rival a `RacerDriver`. Prefabs ship with a **kinematic** Rigidbody, so a prefab dropped in a scene without a profile stays inert, and Car Select can reuse it as a static model (`Docs/Architecture.md`).

10.3 Input abstraction (`Scripts/Vehicles/DrivingInput.cs`)

Type	Meaning
`DrivingInput` (struct)	Normalised intent: `Throttle`, `Brake`, `Steer`, `Handbrake` (identical for player and AI)
`IVehicleInputSource`	`DrivingInput ReadInput()`. Implemented by `PlayerDrivingInput`, `RacerDriver`, `WaypointDriver`, `RouteAutopilot`, `ScriptedDrivingInput`
`IVehicleInputReceiver`	`SetInputSource(IVehicleInputSource)`. Implemented by `VehicleController`

10.4 Which script controls what?

Concern	Script	Key members
Rigidbody step, suspension application, differentials, aero, traction control, telemetry	`Vehicles/Physics/VehicleController.cs`	`Configure` (91), `Step` (163), `ApplySuspension` (219), `ApplyDifferentials` (252), `ApplyAerodynamics` (272), `UpdateTractionControl` (289), `UpdateTelemetry` (322), `UpdateVisuals` (385), `Teleport` (119), `Recover` (141)
One corner: raycast suspension, wheel spin, tyre forces, ABS, static hold	`Vehicles/Physics/VehicleWheel.cs`	`UpdateContact` (154), `SuspensionForce` (207), `DamperForce` (220), `ApplyLoad` (227), `UpdateTyre` (241)
Tyre force curve, combined slip, friction ellipse	`Vehicles/Physics/TyreModel.cs`	`Curve`, `LoadFactor`, `CombinedSlip`, `Force`, `AvailableLongitudinal`, `ClampToEllipse`
Engine, clutch, reverse selection	`Vehicles/Physics/Drivetrain.cs`	`Update` (83), `EngineBrakeAt`, `UnlimitedDriveTorque`, `UpdateDirection`
Gear state and automatic shifting	`Vehicles/Physics/Gearbox.cs`	`Shift`, `SelectDirection`, `Tick`, `UpdateAutomatic`, event `GearChanged`
Steering lock, rate limit, Ackermann	`Vehicles/Physics/SteeringSystem.cs`	`MaxAngleAt`, `Update`
Surface grip	`Vehicles/Physics/DrivingSurface.cs`	Per-collider grip; no component = dry asphalt (grip 1)
Units and constants	`Vehicles/Physics/VehicleUnits.cs`	`Gravity`, `KphToMs`
Collision events	`Vehicles/Physics/VehicleCollision.cs`	Raised by `VehicleController.OnCollisionEnter` → `Collided`
Telemetry snapshot	`Vehicles/Physics/VehicleTelemetry.cs`	Speed, rpm, gear, load, slip, g, yaw rate …
Visual structure	`Vehicles/VehicleRig.cs`	`WheelRig` (Pivot steered about Y, Spin rotated about X, radius, position), lamps
Head and tail lights	`Vehicles/VehicleLights.cs`	`Build(detailed)`, `SetHeadlightLevel`, `SetTailOverride`
Sound	`Audio/VehicleAudio.cs`	`Configure`, `Tick`, `UpdateEngine`, `UpdateTyres`, `UpdateThumps`
Recover (R) and out-of-bounds	`Gameplay/MissionSceneEntry.cs`	`OnResetVehicle` → `Recover()`; `FixedUpdate` teleports below −30 m
Dev telemetry overlay (F3)	`Gameplay/VehicleDebugHud.cs`	`OnGUI`

10.5 Vehicle-specific configuration

Car (catalog order)	Definition	Prefab	Physics	Audio
SLS AMG (`mercedes_sls_amg_2010`)	`Vehicle_MercedesSLS.asset`	`PF_Vehicle_MercedesSLS`	`VehiclePhysics_MercedesSLS` – RWD, 1620 kg, 650 Nm, 7200 rpm, limiter 317 km/h	`VehicleAudio_MercedesSLS` (V8Road)
SLS AMG GT3 (`mercedes_sls_gt3`)	`Vehicle_MercedesSLSGT3.asset`	`PF_Vehicle_MercedesSLSGT3`	`VehiclePhysics_MercedesSLSGT3` – RWD, 1380 kg, 640 Nm, slicks, downforce	`VehicleAudio_MercedesSLSGT3` (V8Race)
Terzo Millennio (`lamborghini_terzo_millennio`)	`Vehicle_LamborghiniTerzo.asset`	`PF_Vehicle_LamborghiniTerzo`	`VehiclePhysics_LamborghiniTerzo` – AWD (40 % front), 1650 kg, 1050 Nm, single-speed EV, limiter 320 km/h	`VehicleAudio_LamborghiniTerzo` (EV)
AMG GT3 (`mercedes_amg_gt3`)	`Vehicle_MercedesAMGGT3.asset`	`PF_Vehicle_MercedesAMGGT3`	–	–

The AMG GT3 has a definition and prefab but is **not in the catalog** (material defects, CC-BY-NC-SA). `VehicleModelTests.Catalog_ExcludesAmgGt3_AndFlagsNonCommercialLicences` enforces this.

10.6 Chapters in this section

- 11 Vehicle Physics: suspension, tyres, drivetrain, assists, aero
- 12 Vehicle Audio
- 13 Vehicle Visuals: prefab build, wheels, lights, LODs
- 14 Car Catalog and Data

11

Vehicle Physics

IN THIS CHAPTER

11.1 Principles
11.2 The physics step (VehicleController.Step, line 163)
11.3 Suspension (VehicleWheel.cs)
11.4 Tyre model (TyreModel.cs, TyreSettings.cs)
11.5 Wheel spin (D15)
11.6 Drivetrain (Drivetrain.cs, Gearbox.cs, EngineSettings.cs, TransmissionSettings.cs)
11.7 Brakes and assists
11.8 Steering (SteeringSystem.cs, SteeringSettings.cs)
11.9 Aerodynamics (AeroSettings.cs)
11.10 Body
11.11 Telemetry (VehicleTelemetry)
11.12 Measured results (from Docs/VehiclePhysics.md, bench at 100 Hz)
11.13 Tools and tests
11.14 Known limitations

All paths are under `Assets/_NeonRift/Scripts/Vehicles/Physics/` unless stated. The original design notes are in `Docs/VehiclePhysics.md` and Decisions D7, D8, D15, D16 and D17.

11.1 Principles

- **Our own model, no `WheelCollider`** (D15). Unity's WheelCollider can't match the profile data and hides state that audio and AI need.
- **100 Hz fixed step** (D8): `Time.fixedDeltaTime = 0.01` (`ProjectSettings/TimeManager.asset`).
- **Physical units** (D16): kg, Nm, Hz, damping ratio, friction coefficients, C·A (m^2).
- **Nothing moves the transform directly** while driving. `Teleport` and `Recover` are the only pose writes.

11.2 The physics step (`VehicleController.Step`, line 163)

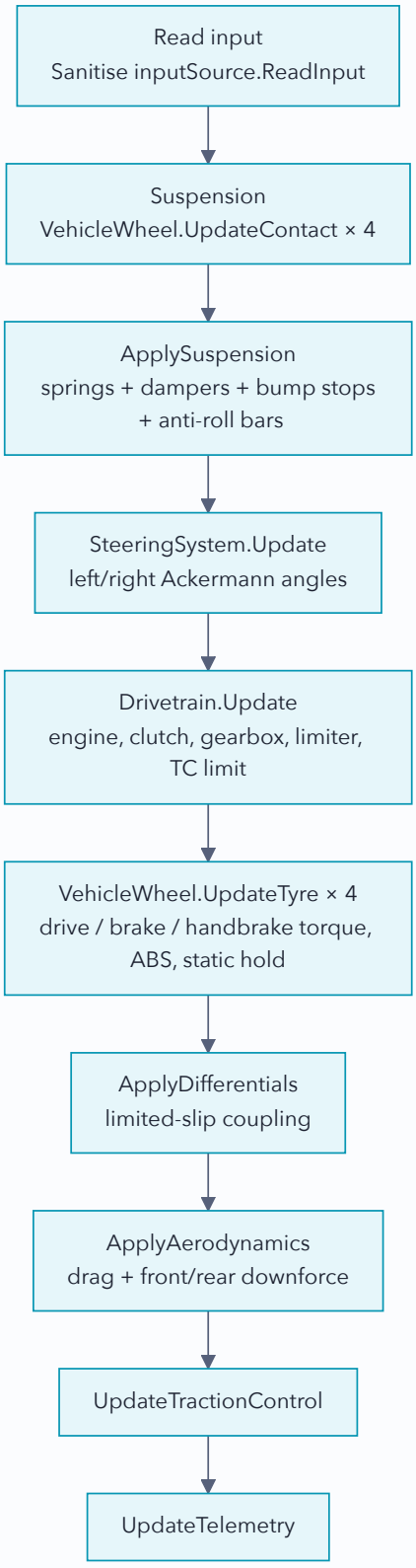


Figure 11.1 Vehicle physics pipeline (VehicleController.Step, 100 Hz)

`FixedUpdate` calls `Step(Time.fixedDeltaTime)` unless `externalStepping` is set (the bench steps cars manually).

11.3 Suspension (`VehicleWheel.cs`)

Feature	Implementation
Contact	Raycast along the suspension axis. **5 rays per wheel** (`contactSamples`, ±40° around the tyre) sample the rolling profile, so sharp edges are rolled onto rather than stepped up (`UpdateContact`, line 154)
Spring	Rate from the axle's **natural frequency** (`springFrequency`, Hz) and corner mass, preloaded so static ride height equals the model's ride height (`SuspensionForce`, line 207)
Damper	Separate bump and rebound damping ratios with a **digressive knee** (`damperKneeSpeed`, `highSpeedDamping`): linear up to the knee, then a smaller high-speed share (`DamperForce`, line 220)
Bump stop	Progressive stop (`bumpStopScale`) in series with tyre radial stiffness; `bumpTravel` / `droopTravel`
Anti-roll bars	`VehicleController.ApplySuspension`: force = `antiRollStiffness × (compression left − compression right)` per axle
Force direction	Along the contact normal (`ApplyLoad`, line 227)

Axle data: `AxleSettings.cs` (`springFrequency`, `bumpDamping`, `reboundDamping`, `damperKneeSpeed`, `highSpeedDamping`, `bumpTravel`, `droopTravel`, `bumpStopScale`, `antiRollStiffness`, `wheelMass`, plus a `TyreSettings`).

11.4 Tyre model (`TyreModel.cs`, `TyreSettings.cs`)

Combined slip with a friction ellipse. Slip ratio and slip angle are each divided by their peak values, then combined:

```
s  = sqrt( (slipRatio / peakSlipRatio)² + (tan slipAngle / tan peakSlipAngle)² )
F  = Curve(s) · LoadFactor · surfaceGrip · load / s      (per unit of normalised slip)
Fx = longitudinalGrip · F · sx        Fy = −lateralGrip · F · sy
```

- `Curve`: a **sine rise** to 1 at the peak, then a **Gaussian fall** toward `slidingGripRatio` (`breakawaySharpness`).
- `LoadFactor`: load sensitivity. More load than static gives proportionally less friction (clamped 0.5 to 1.3).
- `AvailableLongitudinal` and `ClampToEllipse`: braking or accelerating in a corner trades lateral grip for longitudinal grip.
- Surface grip comes from `DrivingSurface` on the collider (grass 0.6 and gravel 0.45 on the test track).

11.5 Wheel spin (D15)

`VehicleWheel.UpdateTyre` (line 241) integrates wheel spin **implicitly** against the tyre's stiffness (the larger of the local slope and the secant through zero slip). This keeps braking and traction stable at 100 Hz without sub-stepping and removes low-speed slip oscillation. A stopped wheel held by its brake uses **static friction** shared across held wheels (`holdMass`), so cars park on slopes without creeping.

11.6 Drivetrain (`Drivetrain.cs`, `Gearbox.cs`, `EngineSettings.cs`, `TransmissionSettings.cs`)

Part	Behaviour
Engine	Torque = peak torque × curve(rpm / max rpm) × throttle (`EngineSettings.TorqueAt`); soft rev limiter; engine braking (regen for the EV); inertia sets rev rate and is reflected onto the driven wheels
Clutch	Slips at launch (holds launch rpm), locks once the wheels catch up, opens for shifts (with rev-matching) and for the handbrake
Gearbox	Automatic with hysteresis, kick-down and a minimum shift interval (`Gearbox.UpdateAutomatic`); decisions use wheel-locked rpm. Gear −1 reverse, 0 neutral, 1..n forward. `Shift(int)` is the hook for manual shifting. Event `GearChanged(from, to)`
Reverse	**Hold brake at standstill** selects reverse (`Drivetrain.UpdateDirection`), never while the handbrake is held
Differentials	RWD / FWD / AWD with `frontTorqueShare`; limited-slip coupling per driven axle, `differentialLock` 0 (open) … 1 (locked) (`VehicleController.ApplyDifferentials`)

11.7 Brakes and assists

Feature	Implementation
Brakes	`BrakeSettings`: `maxBrakeTorque`, `frontBias`, rear `handbrakeTorque`
ABS	`antiLock` / `antiLockTarget`: each wheel's brake torque is capped at the tyre force it can actually use (including engine braking already in use)
Traction control	`VehicleController.UpdateTractionControl`. Feed-forward: limit throttle so requested drive torque does not exceed driven-tyre capacity (plus wheel inertia). Feedback: cut further when a driven wheel is already past peak slip. Recovers at `TractionRecoveryRate`
Speed limiter	`AssistSettings.speedLimiterKph` (soft band)

11.8 Steering (`SteeringSystem.cs`, `SteeringSettings.cs`)

Rate-limited (slower at speed), with a **speed-dependent lock** that falls toward the grip-limited angle (the kinematic angle for the grip-limited corner plus peak front slip angle), and **Ackermann** geometry so the inner wheel turns more. Full keyboard lock at speed therefore means "at the limit", not "plough".

11.9 Aerodynamics (`AeroSettings.cs`)

Drag (`dragArea`, CdA) at the centre of mass; downforce (`frontDownforceArea`, `rearDownforceArea`, ClA) at each axle: `q = ½ ρ v²`.

11.10 Body

`VehicleController.ConfigureBody`: mass, centre of mass from front weight share and CG height, box inertia with per-axis scale, continuous collision. The prefab's two-box body collider uses the low-friction physic material `Assets/_NeonRift/Data/Physics/PM_VehicleBody.asset`.

11.11 Telemetry (`VehicleTelemetry`)

Updated every step: speed (m/s, km/h, forward, normalised to predicted top speed), raw and effective throttle/brake/steer, steer angle, engine rpm and load, gear, shifting flag, clutch, max combined slip / slip ratio / slip angle, skid speed, grounded wheels, longitudinal/lateral g, yaw rate, traction-control factor. Per wheel: grounded, load, compression, bump-stop, rpm, slips, tyre force, surface grip. Consumers: `VehicleAudio`, `VehicleChaseCamera`, `MissionHud`, `RacerDriver`, `VehicleDebugHud`.

11.12 Measured results (from `Docs/VehiclePhysics.md`, bench at 100 Hz)

	SLS AMG	SLS AMG GT3	Terzo Millennio
Layout	RWD, 7-speed	RWD, 6-speed, slicks, wing	AWD single-speed EV
0–100 km/h	4.4 s	3.7 s	3.0 s
Top speed (predicted)	312 km/h	278 km/h	314 km/h
100–0 km/h	38.3 m	30.5 m	34.0 m
Steady grip 60 / 120 km/h	0.92 / 0.97 g	1.19 / 1.29 g	1.06 / 1.13 g

These are the numbers shown in Car Select (`VehiclePerformanceProbe.UpdateDisplayStats`).

11.13 Tools and tests

Tool	File	Purpose
`VehicleTestBench`	`Editor/Vehicles/VehicleTestBench.cs`	Isolated preview physics scene stepped manually at 100 Hz (deterministic, D17)
`VehiclePerformanceProbe`	`Editor/Vehicles/VehiclePerformanceProbe.cs`	Settle, launch, top speed, brake, ramp steer, step steer, bumps, route lap. Menu **Neon Rift ▸ Vehicles ▸ Measure Vehicle Performance**

Tool	File	Purpose
Test track and validation route	`Scenes/Dev/TestTrack.unity`, `Editor/TestTrack/ValidationRouteBuilder.cs`	1.55 km loop: acceleration, braking, banked turn, slalom, bumps, corners, gravel runoff
`RouteAutopilot`, `DrivingRoute`, `VehicleTelemetryLog`	`Scripts/Gameplay/`	Validation laps through the normal input path
`VehiclePhysicsTests`	`Tests/EditMode/VehiclePhysicsTests.cs`	Spawn at ride height without jitter, acceleration band, top speed, straight braking, cornering grip, full-lock step steer doesn't spin, bumps, wall impact and recovery, slope hold, validation lap, spawn applies profile (run per catalog car)
`VehicleModelTests`	`Tests/EditMode/VehicleModelTests.cs`	Tyre curve and ellipse, load sensitivity, steering lock and Ackermann, gearbox shifts, reverse selection, launch clutch, handbrake clutch, profile validity

11.14 Known limitations

No unsprung mass (no wheel hop), no tyre relaxation length, temperature or wear; differentials are speed-coupling approximations; two-box body collider; collision events fire only in Play Mode. Full list: `Docs/VehiclePhysics.md` and 45 Known Limitations.

12 Vehicle Audio

IN THIS CHAPTER

12.1 Idea
12.2 Pieces
12.3 What drives each sound
12.4 Clips (generated)
12.5 Mixing and voices
12.6 Validation

12.1 Idea

A car's sound is **data** (`VehicleAudioProfile`), and **one component plays every car** (`VehicleAudio`), driven entirely by the controller's telemetry and events (Decision D19). Every loop is started **once**. Afterwards only volume and pitch change. All engine and chassis sounds are **synthesised in-house**, with no samples and no licences.

12.2 Pieces

Piece	File	Role
`VehicleAudioProfile` (+ `EngineAudioSettings`, `ChassisAudioSettings`, `EngineSoundLayer`)	`Assets/_NeonRift/Scripts/Vehicles/Audio/`	Per-car data: engine loops at 5 rpm points × on/off load, levels, smoothing, pitch limits, shift and pop clips, tyre/road/wind beds, thumps, impacts
`EngineSoundModel`	`Scripts/Audio/EngineSoundModel.cs`	Pure blend maths: equal-power crossfade between neighbouring rpm layers in **log-rpm** space, equal-power on/off-load crossfade, pitch = rpm / recorded rpm; layers fade out before pitch limits instead of over-stretching (`Layer`, `Evaluate`, `Blend`)
`TyreSoundModel`	`Scripts/Audio/TyreSoundModel.cs`	Slip → squeal / scrub intensity (load- and speed-aware): `SlipIntensity`, `SquealDirectionWeight`, `Smooth`
`VehicleAudio`	`Scripts/Audio/VehicleAudio.cs`	On the prefab. `Configure` (87) creates sources and calls `Play()` once per loop; `Tick` → `UpdateEngine` (189), `UpdateTyres` (225), `UpdateThumps` (264); `OnGearChanged` (280) and `OnCollided` (288) play one-shots; `SetPlayerView` (149): the player's car is mostly 2D with no Doppler
`ShowroomEngine`	`Scripts/Frontend/ShowroomEngine.cs`	Same loops for a parked car in Car Select: idle plus `Blip`

12.3 What drives each sound

Sound	Driver
Engine layers	Audio rpm = telemetry rpm smoothed 30–50 ms; load = effective throttle (0 on the overrun), cut while shifting
Overrun pops	Random one-shots after lifting off above `popMinRpm` (combustion cars only)
Tyre squeal	Per-wheel combined slip over `skidSlipRange`, scaled by load and speed
Tyre scrub	Combined slip just below or at the peak (`scrubSlipRange`)
Loose surface	Wheels on surfaces with grip below `looseSurfaceGrip` (grass, gravel)
Road roar, wind	Speed (wind ∝ speed²)
Gear shift	`Gearbox.GearChanged`
Suspension thump	Compression speed over `thumpSpeed`, harder on the bump stop
Impacts	`VehicleController.Collided` impulse: light / medium / heavy, log-scale level

12.4 Clips (generated)

`VehicleAudioGenerator` (`Scripts/Editor/Audio/VehicleAudioGenerator.cs`, menu **Neon Rift ▸ Audio ▸ Generate Vehicle Audio**) uses `EngineSynth` (`Editor/Audio/EngineSynth.cs`) and `Dsp` (`Editor/Audio/Dsp.cs`):

Voice	Car	Model
V8Road	SLS AMG	Exhaust pulses at real firing angles (cross-plane V8 order 1-5-4-8-6-3-7-2) split across two banks, pipe resonances, comb pipe, muffler low-pass, saturation
V8Race	SLS AMG GT3	Open race exhaust, brighter, more overrun pops
EV	Terzo Millennio	Motor and gear whine orders, inverter noise, low rumble

Output folders: `Assets/_NeonRift/Audio/Generated/Engine_V8Road/`, `Engine_V8Race/`, `Engine_EV/` (10 loops each), `Engine_Pops/`, `Tyres/`, `Road/`, `Mechanical/`, `Impacts/`. Every loop holds a whole number of engine cycles (seamless). Engine loops are PCM on desktop for clean pitch-shifting; everything else is ADPCM, Decompress-On-Load, mono. `VehicleAudioPrefabs` (menu **Neon Rift ▸ Audio ▸ Add Vehicle Audio To Prefabs**) adds and routes the component.

12.5 Mixing and voices

- Routing: engine → **Engine** group, tyres → **Tires**, impacts → **SFX** (see 29 Audio).
- A car the camera is not following gets +140 priority on every source (`VehicleAudio.OtherCarPriorityOffset`), so the player's car always keeps its voices.
- On an audio device change, `VehicleAudio.OnAudioConfigurationChanged` restarts the loops.

12.6 Validation

- `VehicleAudioTests` (`Tests/EditMode/VehicleAudioTests.cs`): `EngineModel_IsEqualPower_AndCrossfadesLoad`, `EngineModel_IsContinuous_AndPitchTracksRpm`, `TyreModel_SquealsOnlyWhenSlidingAndMoving`, `CatalogVehicles_HaveValidAudioProfiles`, `LoopingClips_AreSeamless_AndOneShotsStartAndEndSilent`, `VehicleAudio_FollowsTelemetry_WithoutRestartingLoops`, `VehicleAudio_SkidFollowsTyreSlip`.
- `VehicleAudioValidator` (`Scripts/Gameplay/VehicleAudioValidator.cs`): scripted idle → launch → lift-off → brake → idle → handbrake slide → wall impact → recover, recording the listener with `AudioOutputRecorder` and measuring with `AudioSignalAnalysis` (`ClickScore`, `Pitch`, `Brightness`). Results table: `Docs/Audio.md`.
- Invariant: `VehicleAudio.PlayCalls == LoopCount` for the life of the car (no restarts).

13
Vehicle Visuals

IN THIS CHAPTER

13.1 How a Sketchfab car becomes a game car
13.2 VehicleRig (Scripts/Vehicles/VehicleRig.cs)
13.3 Wheel visuals
13.4 Lights (Scripts/Vehicles/VehicleLights.cs)
13.5 LODs (Editor/Vehicles/VehicleLodSetup.cs)
13.6 Damage
13.7 Validation

13.1 How a Sketchfab car becomes a game car

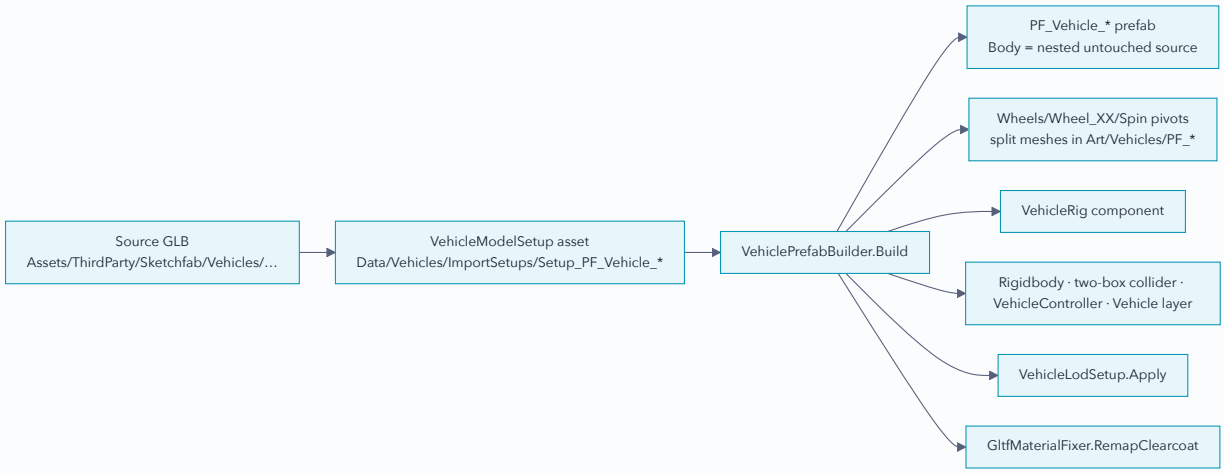


Figure 13.1 Vehicle prefab build pipeline

Step	Code
Recipes (keywords for wheels, lamps, hidden parts; orientation and scale fixes)	`ThirdPartyIntake.VehicleRecipes` (`Assets/_NeonRift/Scripts/Editor/ThirdPartyIntake.cs:90`), stored as `VehicleModelSetup` (`Editor/Vehicles/VehicleModelSetup.cs`)
Build the prefab	`VehiclePrefabBuilder.Build` (`Editor/Vehicles/VehiclePrefabBuilder.cs:56`): orient, scale and ground the model; `Classify` renderers by keyword; `SplitParts` / `QuadrantBounds` / `ExtractMesh` split merged wheel geometry into four corners; `AddPhysics` (Rigidbody, two-box collider with `PM_VehicleBody`, `VehicleController`, layer Vehicle)
Clearcoat fix	`GltfMaterialFixer.RemapClearcoat` (`Editor/GltfMaterialFixer.cs`): glTFast's clearcoat graph produced NaN (flat cyan) output, so renderers are remapped to project-owned copies on the plain glTF PBR graph (Decision D12)

Step	Code
Menus	**Neon Rift ▸ Assets ▸ Run Third-Party Intake**, **Neon Rift ▸ Assets ▸ Rebuild Vehicle Prefabs And Catalog**, **Neon Rift ▸ Vehicles ▸ Build All Vehicle Prefabs**

The source model is never modified. It stays a nested prefab, and only transforms and renderer enable states are overridden (Decision D11).

13.2 `VehicleRig` (`Scripts/Vehicles/VehicleRig.cs`)

Describes the visual structure so physics, lighting and presentation can drive any car without knowing its hierarchy. Root pivot at ground level, centred, +Z forward, metres.

Member	Meaning
`WheelRig`	Per corner: `WheelPosition`, the **Pivot** (steered about local Y), the **Spin** transform (rotated about local X), radius, width; static parts such as calipers are children of the pivot only
Lamps	Head and tail lamp positions used by `VehicleLights`

13.3 Wheel visuals

`VehicleController.UpdateVisuals` (`Vehicles/Physics/VehicleController.cs:385`) runs in `LateUpdate`: each wheel's pivot gets its steer angle and suspension travel, and the Spin transform gets its rolled angle from the wheel's angular velocity. Motion-blur rims (SLS AMG, SLS GT3, Terzo) are hidden at rest (`Docs/ThirdParty.md` vehicle audit).

13.4 Lights (`Scripts/Vehicles/VehicleLights.cs`)

Light	Detail
Headlights	Two spot lights from the rig. `Build(detailed: true)` (player): full intensity and range, and the **left** one casts soft shadows. AI cars: 60 % intensity and range, no shadows
Tail light	A 130° spot at bumper height aimed back and down. It brightens under braking and turns white in reverse. Range 7 m for the player, 5 m otherwise. It is a spot (not a point light) so the sloping rear glass sits outside its cone: this fixed a hot specular "rear-glass blob" (`Docs/Environment.md`, URP visual pass)
Overrides	`SetHeadlightLevel`, `SetTailOverride` (used by Car Select and the intro)

13.5 LODs (`Editor/Vehicles/VehicleLodSetup.cs`)

Built from the renderers the car already has (no new meshes):

- **LOD0:** everything.
- **LOD1** (below 18 % screen height): drops the interior, engine bay, brakes, damage glass and small non-light parts.
- **LOD2** (below 5 %): body panels, glass, wheels and lights only.
- **Culled** below 0.8 %. Lights stay in every level.

Triangle counts (`Docs/Environment.md`): SLS AMG 169 k → 47.5 k → 45 k; GT3 165 k → 50 k → 46 k; Terzo 103 k → 84 k → 64 k. Menu **Neon Rift ▸ Vehicles ▸ Apply Vehicle LODs To Catalog Prefabs**.

13.6 Damage

There is **no damage model**. The prefab builder hides `DAMAGE_GLASS` meshes (`ATTRIBUTION.txt` of the SLS models), and collisions only produce sound, camera shake and HUD contact counts.

13.7 Validation

`VehicleModelTests` (rig and catalog checks) and `FoundationTests.Catalog_ReportsDuplicateIdsAndMissingPrefabs` (`Assets/_NeonRift/Scripts/Tests/EditMode/`).

14 Car Catalog and Data

IN THIS CHAPTER

14.1 Adding a car is a data change (Decision D4)
14.2 VehicleCatalog (Assets/_NeonRift/Scripts/Vehicles/VehicleCatalog.cs)
14.3 VehicleDefinition (Scripts/Vehicles/VehicleDefinition.cs)
14.4 VehicleDisplayStats (Scripts/Vehicles/VehicleDisplayStats.cs)
14.5 VehiclePhysicsProfile (Scripts/Vehicles/Physics/VehiclePhysicsProfile.cs)
14.6 VehicleAudioProfile
14.7 Import setups
14.8 How to add a car
14.9 Rivals
14.10 Validation

14.1 Adding a car is a data change (Decision D4)

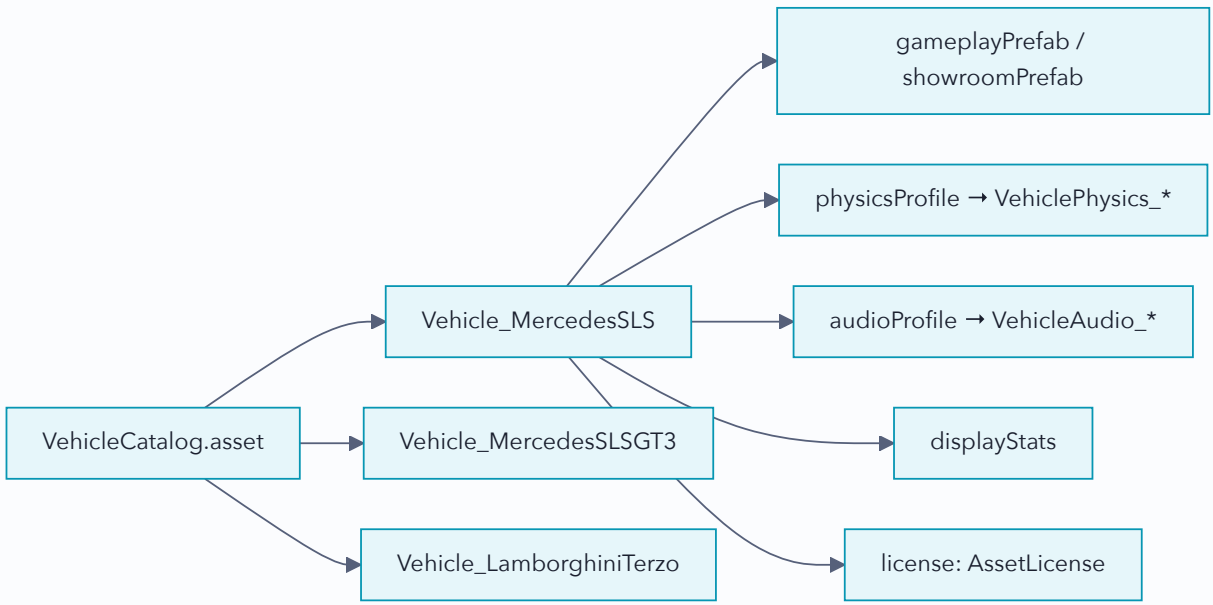


Figure 14.1 Vehicle catalog data relationships

14.2 `VehicleCatalog` (`Assets/_NeonRift/Scripts/Vehicles/VehicleCatalog.cs`)

`Assets/_NeonRift/Data/Vehicles/VehicleCatalog.asset`, in order:

#	Definition	`id`	`displayName`	`category`	Licence (`AssetLicense`)
0 (default)	`Vehicle_MercedesSLS.asset`	`mercedes_sls_amg_2010`	SLS AMG	Grand Tourer	CC-BY-4.0, commercial allowed
1	`Vehicle_MercedesSLSGT3.asset`	`mercedes_sls_gt3`	SLS AMG GT3	GT3 Race Car	CC-BY-4.0, commercial allowed
2	`Vehicle_LamborghiniTerzo.asset`	`lamborghini_terzo_millennio`	Terzo Millennio	Electric Hypercar Concept	**CC-BY-NC-4.0, non-commercial**
– (not in catalog)	`Vehicle_MercedesAMGGT3.asset`	`mercedes_amg_gt3`	AMG GT3	GT3 Race Car	CC-BY-NC-SA-4.0

API: `Vehicles`, `Count`, `Default` (= first entry), `TryGet(id)`, `IndexOf`, `Validate` (duplicate ids, missing prefabs).

14.3 `VehicleDefinition` (`Scripts/Vehicles/VehicleDefinition.cs`)

Field	Purpose
`id`	Stable string id (saves, lookups)
`displayName`, `manufacturer`, `category`, `description`	Car Select copy
`gameplayPrefab`	Spawned in missions
`showroomPrefab`	Car Select model (falls back to the gameplay prefab)
`physicsProfile`	`VehiclePhysicsProfile` (the single source of tuning)
`audioProfile`	`VehicleAudioProfile`
`displayStats`	`VehicleDisplayStats`
`displayStatsProvisional`	False once the stats are measured
`license`	`AssetLicense` (`Scripts/Core/AssetLicense.cs`): `LicenseId`, `Author`, `SourceUrl`, `CommercialUseAllowed`
`DrivetrainLabel` (computed)	"RWD" / "FWD" / "AWD" from the physics profile

14.4 `VehicleDisplayStats` (`Scripts/Vehicles/VehicleDisplayStats.cs`)

`topSpeedKph`, `zeroToHundredSeconds`, `powerHp`, `massKg`, `handlingRating` (0–10), `brakingDistanceMetres`. These are **measured on the physics bench, not manufacturer claims**: `VehiclePerformanceProbe.UpdateDisplayStats` (menu **Neon Rift ▸ Vehicles ▸ Update Car Select Stats From Physics**). The Terzo Millennio has no official figures at all.

14.5 `VehiclePhysicsProfile` (`Scripts/Vehicles/Physics/VehiclePhysicsProfile.cs`)

Assets in `Data/Vehicles/Physics/`. Sections: `ChassisSettings` (mass, weight split, CG height, inertia scale), `AxleSettings` front/rear (suspension and `TyreSettings`), `EngineSettings`, `TransmissionSettings`, `BrakeSettings`, `SteeringSettings`, `AeroSettings`, `AssistSettings`, plus *Tuning Notes* text. Methods: `Axle(front)`, `IsDriven(front)`, `EstimateTopSpeed`, `Validate`.

Profile	`VehiclePhysics_MercedesSLS`	`VehiclePhysics_MercedesSLSGT3`	`VehiclePhysics_LamborghiniTerzo`
Drive	RWD	RWD	AWD (40 % front)
Mass	1620 kg	1380 kg	1650 kg
Peak torque	650 Nm	640 Nm	1050 Nm
Max rpm	7200	7500	16000
Final drive	3.27	3.36	1.0
Diff lock	0.35	0.55	0.30
Brake torque / bias	7200 Nm / 0.64	9500 Nm / 0.60	8500 Nm / 0.62
Downforce ClA F/R	0.04 / 0.10	0.55 / 1.05	0.25 / 0.45
Limiter	317 km/h	–	320 km/h

(Values read from the assets. ABS and traction control are on for all three.)

14.6 `VehicleAudioProfile`

`Data/Vehicles/Audio/VehicleAudio_MercedesSLS.asset`, `_MercedesSLSGT3`, `_LamborghiniTerzo`. Built and linked by `VehicleAudioGenerator.Profile` / `Link`. See 12 Vehicle Audio.

14.7 Import setups

`Data/Vehicles/ImportSetups/Setup_PF_Vehicle_*.asset` (`VehicleModelSetup`): keywords and fixes used by `VehiclePrefabBuilder`. Copy text comes from `ThirdPartyIntake.VehicleRecipes` (menu **Neon Rift ▸ Vehicles ▸ Apply Vehicle Copy From Recipes**).

14.8 How to add a car

1. Put the model under `Assets/ThirdParty/Sketchfab/Vehicles/<Name>/` with an `ATTRIBUTION.txt`.
2. Add a recipe in `ThirdPartyIntake.VehicleRecipes()` (wheel, lamp and hidden-part keywords; orientation and scale).
3. Run **Neon Rift ▸ Assets ▸ Run Third-Party Intake**: prefab + definition + catalog entry.

4. Create a `VehiclePhysicsProfile`, generate audio, measure stats, run the tests.

14.9 Rivals

The cars **not** picked become the rival field: `RunSession.SelectVehicle(vehicle, catalog)` (`Scripts/Game/RunSession.cs`). Tested by `CityTests.Session_RivalsAreTheCarsNotPicked`.

14.10 Validation

`FoundationTests` (`Catalog_ReportsDuplicateIdsAndMissingPrefabs`, `Catalog_LooksUpById_AndDefaultIsFirst`, `ProjectWiring_IsValid`), `VehicleModelTests.CatalogProfiles_AreValid_AndPredictPlausibleTopSpeeds`, `CarSelectTests.CatalogVehicles_HaveCarSelectData`, `ProjectValidator` (menu **Neon Rift ▸ Validate Project**; `Warnings()` lists non-commercial assets).

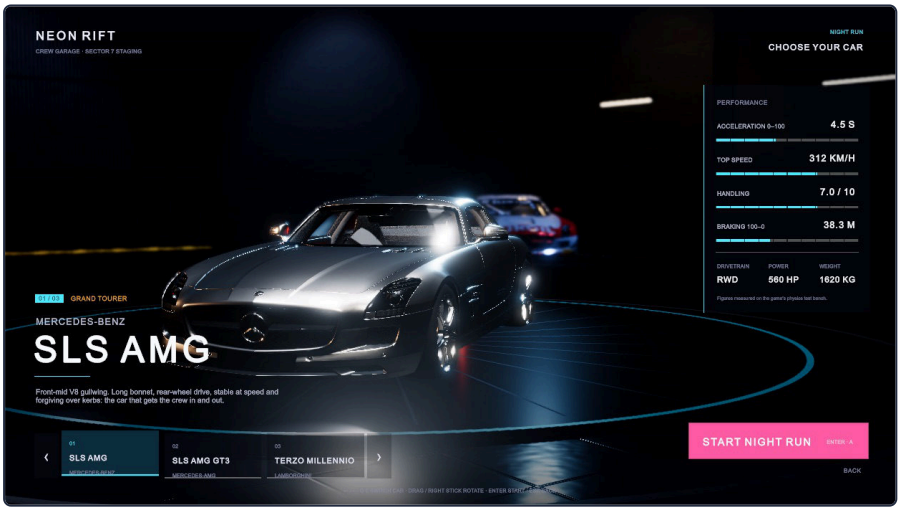
15
Car Selection

IN THIS CHAPTER

15.1 What the player sees
15.2 Where it lives
15.3 Flow
15.4 Presentation-only cars
15.5 Cameras
15.6 Lighting and garage
15.7 Persistence
15.8 Validation

15.1 What the player sees

A crew garage at night. The selected car stands on a turntable under a key light, the other two catalog cars are parked in the shadows, and a planning wall of screens shows the city map and the Data Core. Q/E or ←/→ switch cars, A/D (right stick, or dragging) turn the turntable, and Enter confirms: the turntable swings the car to the roller door, the door lifts, the engine revs, and the car drives out into the night. Then the mission loads.



Car Select hero shot

`Docs/Screenshots/CarSelect2/hero.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

15.2 Where it lives

Piece	File / asset
Scene	`Assets/_NeonRift/Scenes/CarSelect.unity` (generated)

Piece	File / asset
Scene generator	`Assets/_NeonRift/Scripts/Editor/Frontend/GarageBuilder.cs` (menu **Neon Rift ▸ Car Select ▸ Build Garage**)
Planning wall textures	`Scripts/Editor/Frontend/GaragePlanningWall.cs` : a map generated from the real `RoadNetwork` (garage, route to the core, compound, Rift Gate) and a core schematic → `Assets/_NeonRift/Art/Garage/Textures/`
Screen controller (entry point)	`Scripts/Frontend/CarSelectScreen.cs`
3D presentation	`Scripts/Frontend/VehicleShowroom.cs`
Showroom engine sound	`Scripts/Frontend/ShowroomEngine.cs`
UI	`Assets/_NeonRift/UI/CarSelect.uxml` , `CarSelect.uss` (elements `vehicle-name` , `vehicle-category` , `vehicle-maker` , `vehicle-desc` , `stats` , `chip-drive` , `chip-power` , `chip-weight` , `cards` , `prev-button` , `next-button` , `confirm-button` , `back-button` , `cs-fade`)
Post-processing and blends	`Settings/Rendering/Garage_Volume.asset` , `Garage_CameraBlends.asset`

15.3 Flow

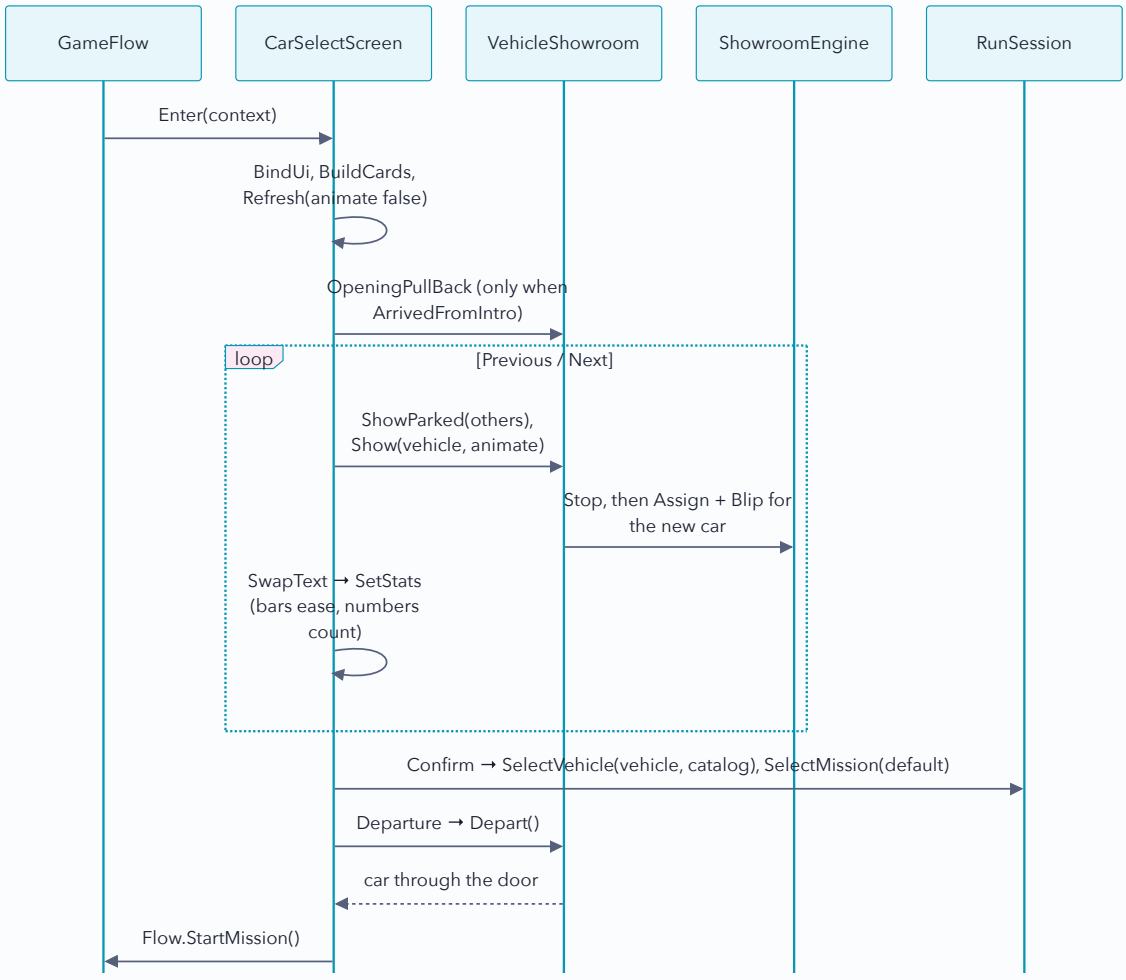


Figure 15.1 Car Selection flow

Step	Method
Enter	`CarSelectScreen.Enter` (45): binds Menu actions (`Previous` , `Next` , `Confirm` , `Back`); the start index is the session's selected car
Previous / next	`Previous` / `Next` → `Select(i)` → `Refresh` (199)
Turn the car	`Update` reads `Menu.Rotate` → `VehicleShowroom.Spin` ; pointer drag → `VehicleShowroom.Drag` ; with no input the turntable idles round
Car swap animation	`VehicleShowroom.Swap` (90): stage dims to 8 %, outgoing lights off, turntable whips to 720°/s; the new car spins in from 150° off and settles; DRLs then headlight beams, a brake-light pulse, and an engine blip from the car's own loops (`ShowroomEngine.Blip`)
Background cars	`VehicleShowroom.ShowParked` (157): the other catalog cars in the parked slots, presentation-only
Stats	`SetStats` : category, drivetrain, power, weight, measured 0–100, top speed, handling, 100–0 braking; `StatRow.Tick` eases bars and counts numbers over 0.5 s
Confirm	`Confirm` (273): `session.SelectVehicle(car, catalog)` (the others become rivals), `SelectMission( GameConfig.DefaultMission)` if none, then the `Departure` coroutine (292)
Departure	`VehicleShowroom.Depart` (246): turntable swings the nose to `exitDirection` (−Z) while the door rises; switch to the depart camera; blips; pull away with eased acceleration and spinning wheels; then `Flow.StartMission()`
Back	`Back` → `Flow.GoToFrontend()`

15.4 Presentation-only cars

`VehicleShowroom.MakePresentationOnly` (311) uses the **gameplay prefab** but makes it inert: Rigidbody kinematic, collisions off, **interpolation off**, colliders disabled, every behaviour except `VehicleRig` disabled, audio sources off. Turning interpolation off fixed the "car leaves the garage at an angle" bug: the interpolated body was writing back a stale pose that fought the turntable. `Depart` also squares the car to the door exactly before pulling away, and caps the per-frame step at 0.05 s.

15.5 Cameras

Cinemachine cameras made by `GarageBuilder` (`Cameras`) and switched by `VehicleShowroom.Set Camera` (priority):

Camera	Use
Hero	Default framing (27° lens, car slightly left of centre), slow dolly drift (`heroDrift`)
Swap	Low side shot during a car switch
Depart	Follow camera for the drive-out
Opening	Headlight close-up matching the intro's last frame (`RunSession.ArrivedFromIntro`)

15.6 Lighting and garage

`GarageBuilder.BuildShell` (99): a 26 × 30 m hall (concrete floor, steel columns, trusses), soft-box strips, wall washers, key light and two rims on the turntable, workbench, tool chests, tyre rack, planning wall, roller door, two parking bays, and a stand-in street through the door. A box-projected reflection probe is baked (`BakeProbe`), and the Volume is set up by `Volume` and `ConfigureRendering`. The **same shell** stands in the city on W Avenue (`NightRunBuilder.Garage.cs`), which is what makes the intro → Car Select match cut work.

15.7 Persistence

The selected car lives in `RunSession` (memory only, for this session). It is not saved between launches.

15.8 Validation

`CarSelectTests` (`Assets/_NeonRift/Scripts/Tests/EditMode/CarSelectTests.cs`): `Catalog Vehicles_HaveCarSelectData`, `CarSelectScene_IsAWiredGarage`. Play Mode checks with `PlaytestCapture` are recorded in `Docs/Intro.md` (switching SLS AMG → GT3 → Terzo; departure frames; Terzo confirmed as the player car in the mission).

16 Player Input

IN THIS CHAPTER

16.1 Technology
16.2 Controls (read from the .inputactions file and GameRoot)
16.3 From key press to car
16.4 Validation

16.1 Technology

Unity **Input System 1.19.0** only (`activeInputHandler: 1` in `ProjectSettings/ProjectSettings.asset`). Actions live in one asset, `Assets/_NeonRift/Settings/Input/NeonRiftControls.inputactions`, from which Unity generates the typed C# wrapper `Assets/_NeonRift/Scripts/Input/NeonRiftControls.cs` (Decision D5: no string lookups). Two action maps: **Driving** and **Menu**. Each scene enables only the maps it needs. UI navigation uses `InputSystemUIInputModule` in the Bootstrap scene.

16.2 Controls (read from the `.inputactions` file and `GameRoot`)

Driving map

Action	Type	Keyboard	Gamepad
Throttle	Value	W, ↑	Right trigger
Brake (hold at a stop to reverse)	Value	S, ↓	Left trigger
Steer	Value	A / D, ← / →	Left stick X
Handbrake	Button	Space	B / East
Interact (hold to hack; retry on the debrief)	Button	E	A / South
ResetVehicle (recover)	Button	R	Select
Pause	Button	Esc	Start

Menu map

Action	Keyboard	Gamepad
Previous	Q, ←	Left shoulder, d-pad left
Next	E, →	Right shoulder, d-pad right

Action	Keyboard	Gamepad
Rotate (Car Select turntable)	A / D	Right stick X
Confirm	Enter, keypad Enter	A / South
Back	Esc	B / East
Skip (intro)	Space	Start
Up / Down (menus)	↑ / W, ↓ / S	d-pad, left stick

Global and dev keys

Key	Where it is bound	Effect
M / gamepad **Y** (`buttonNorth`)	`GameRoot.Awake` creates an `InputAction("ToggleMusic")` in code (`Scripts/Game/GameRoot.cs`)	`GameSettings.ToggleMusic()` anywhere in the game
F3	`VehicleDebugHud` (`Scripts/Gameplay/VehicleDebugHud.cs`) reads `Keyboard.current` directly	Telemetry overlay; disabled outside debug/development builds (`Debug.isDebugBuild`)

16.3 From key press to car

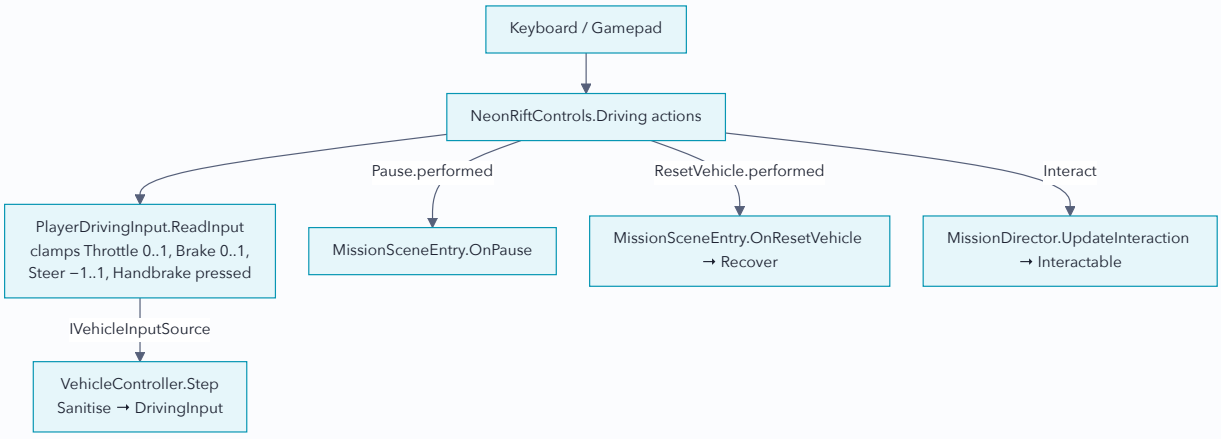


Figure 16.1 Vehicle input flow

Class	File	Role
`NeonRiftControls`	`Scripts/Input/NeonRiftControls.cs` (generated)	`DrivingActions`, `MenuActions`, `IDrivingActions`, `IMenuActions`
`PlayerDrivingInput`	`Scripts/Input/PlayerDrivingInput.cs`	Bridges Driving actions to `IVehicleInputSource`; `DescribeDevices(out canDrive)` lists the input devices
`MissionSceneEntry`	`Scripts/Gameplay/MissionSceneEntry.cs`	Enables Driving on `Enter`, logs `[Input] Driving controls enabled. Devices: …`, disables it while paused
`CarSelectScreen`, `IntroSceneEntry`, `OptionsMenu`, `TitleScreen`	Frontend / Intro / Game	Use the Menu map

Editor note (README): keyboard input only reaches the game while the Game view has focus. If `[Input]` warns that no keyboard or gamepad is connected, the editor's Input System backend has lost its devices; restart the editor.

16.4 Validation

Test / tool	File	What it checks
`InputTests` (an `InputTestFixture`)	`Scripts/Tests/EditMode/InputTests.cs`	Keyboard throttle, brake, steer, throttle+steer together, handbrake and interact; gamepad driving and buttons; disabling the Menu map leaves Driving enabled
`InputPlaytest`	`Scripts/Editor/Validation/InputPlaytest.cs`	Play Mode: writes key state events into the real device (`Hold`, `Release`, `Tap`), runs `RunStandardScript` and `RunMinimapScript`, writes `Logs/InputPlaytest.txt`. See 41 Input Playtesting

PART IV

AI and Traffic

Rival crews and city traffic.

17 Rival AI

18 Traffic System

17 Rival AI

IN THIS CHAPTER

17.1 The short answer
17.2 Files
17.3 Architecture
17.4 The crews' heist (RivalTask data, authored in NightRunBuilder.CreateMissionData)
17.5 RivalDirector key methods
17.6 RacerDriver: how a rival drives
17.7 RacerProfile: driver personality
17.8 Qualifying and results
17.9 Drones on the rivals
17.10 Validation
17.11 Intro use

17.1 The short answer

Two rival crews, **VEX** and **KADE**, drive the two catalog cars the player did not pick. Each car is controlled by a `RacerDriver`, which plugs into the car through the **same `IVehicleInputSource` interface as the player** (Decision D25): no transforms are moved and no forces are faked. `RivalDirector` gives each crew **its own heist** (relay hack → core trunk tap → extraction), keeps live standings and decides **qualifying** (only the first two out qualify).

17.2 Files

Class	File
`RivalDirector` (+ `Slot`, `Racer`)	`Assets/_NeonRift/Scripts/Gameplay/Racing/RivalDirector.cs`
`RacerDriver`	`Scripts/Gameplay/Racing/RacerDriver.cs`
`RacerProfile` (ScriptableObject)	`Scripts/Gameplay/Racing/RacerProfile.cs`; assets `Assets/_NeonRift/Data/Racing/Racer_Vex.asset`, `Racer_Kade.asset`
`RacingLine`	`Scripts/Gameplay/Racing/RacingLine.cs`
`RaceMarker`	`Scripts/Gameplay/Racing/RaceMarker.cs` (5 in the NightRun scene)
`RivalTask` (data)	`Scripts/Missions/RivalTask.cs`
Task list in the mission	`MissionDefinition.RivalTasks`, `RivalStartDelay`, `QualifyingPlaces` (`Scripts/Missions/MissionDefinition.cs`)
Scene wiring (spawns, markers, work FX, profiles)	`NightRunBuilder.BuildRivals` (`Scripts/Editor/District/NightRunBuilder.cs`)
Task data authoring	`NightRunBuilder.CreateMissionData`

17.3 Architecture

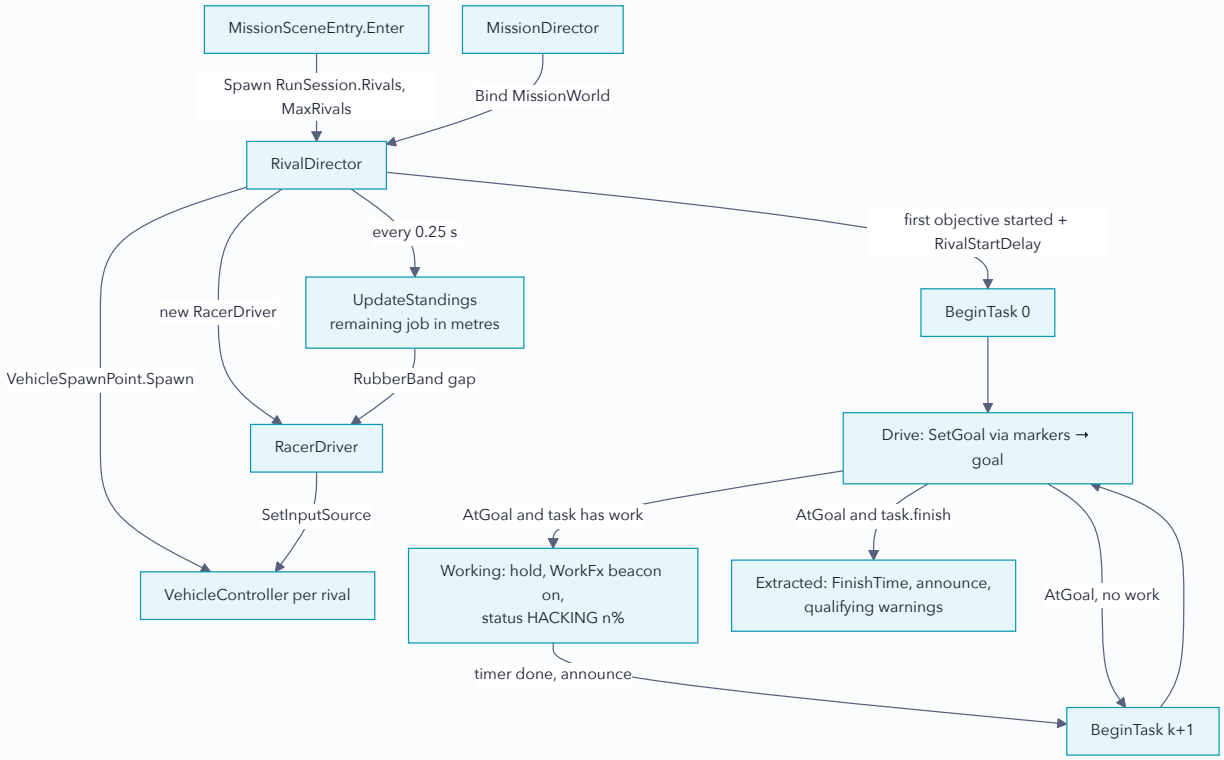


Figure 17.1 Rival AI architecture

17.4 The crews' heist (`RivalTask` data, authored in `NightRunBuilder.CreateMissionData`)

Task	VEX (slot 0)	KADE (slot 1)	Work	Labels	Done message
1 Relay hack	marker `spire_relay` (Spire Boulevard, east-bound kerb lane, x 80)	marker `kowloon_relay` (Lantern Street, northbound kerb lane, z 20)	12 s ± 15 %	TO RELAY / HACKING RELAY	"{name}: SPIRE RELAY CRACKED" / "{name}: KOWLOON RELAY CRACKED"
2 Core trunk tap	marker `trunk_tap` slot 0 (Spire Avenue south-bound, (154.75, 350))	`trunk_tap` slot 1 (North Boulevard eastbound, (110, 294.75))	16 s ± 15 %	TO THE CORE / TAPPING CORE	"{name} HAS A CORE PACKAGE · RUNNING FOR THE GATE"
3 Extraction	`extraction` (Rift Gate)	`extraction`	–	TO RIFT GATE	"{crew} EXTRACTED" (finish)

`RivalStartDelay` 0.6 s, `QualifyingPlaces` 2. Constants: `NightRunBuilder.RivalRelaySeconds = 12`, `RivalUplinkSeconds = 16`. Marker ids: `RivalRelayNorthId`, `RivalRelayWestId`, `RivalTrunkTapId`, `RivalExtractionId`, `RivalStagingId` (S7 car park, used by the intro only).



Figure 17.2 Rival heist flow (per crew)

Why kerb lanes. Every stop sits on the road graph, facing the way the crew leaves, so there are no U-turns across medians. In validation, two earlier placements failed: uplink bays inside the compound got sealed by the player's lockdown, and taps in the S7 car-park lot made the two drivers deadlock while

yielding to each other (`Docs/NightRun.md`).

Work FX. While a crew works, `Racer.WorkFx` (a magenta beacon column plus a point light, `RivalWorkFx_n`, built in `BuildRivals`) stands over its car.

17.5 `RivalDirector` key methods

Method	Line	What it does
`Spawn(vehicles, max)`	115	Spawns up to `max` rivals through `VehicleSpawnPoint.Spawn` (same prefab, physics and audio), creates a `RacerDriver`, logs `[Rivals] contact …` telemetry on collisions
`Bind` / `Unbind`	166 / 192	Reset per attempt; `PlanRoutes` caches every leg's route length
`OnObjectiveStarted`	244	The first objective starts the race (after `RivalStartDelay`); the mission end holds every car
`BeginTask(i, k)`	266	Queues via markers, sets the goal (hold if the task has work or finishes), sets the status label
`Update`	296	Via points (14 m radius), work timers, arrivals, finishing, standings every 0.25 s
`OnRivalFinished`	367	HUD: "1 QUALIFYING PLACE LEFT" / "QUALIFYING CLOSED · NO PLACE LEFT FOR YOU"
`RivalRemaining` / `PlayerRemaining`	377 / 402	**Whole remaining job** in metres: route to the goal + work left × 22 m/s (`workMetresPerSecond`) + later legs; the player gets the route to the current objective + later objective legs + 9 s per interaction objective (`playerWorkSeconds`)
`UpdateStandings`	415	Sorts finished crews by finish time, then the rest by remaining job; applies the rubber band; sets `PlayerPosition`
`FinishPosition`	450	Player's place = rivals already extracted + 1
`TryGetGap`	458	"88 M BEHIND KADE" style gap for the HUD
`DescribeAi`	439	Per-rival report line (contacts, reversals, resets, yields, task, status) for validators

Race-panel rows are rebuilt only when the status changes, and work progress moves in 10 % steps (`Racer.SetWorkStatus`), so there is no per-frame allocation.

17.6 `RacerDriver`: how a rival drives

`RacerDriver.ReadInput` (line 163) runs every physics step:

1. **Plan.** `Replan` (124) asks `CityNavigation.Plan` for a route that respects gate state and the driver's own blocked-edge memory (`AvoidCost`: +250 m for 25 s after a reversal on that edge). It builds a `RacingLine` (centreline with rounded corners, resampled every 2 m, per-sample tangent, usable lateral range and an advisory speed from curvature and the street limit). It re-plans when `CityNavigation.Version` changes (a gate moved), and at most every 1.5 s when far off the line.
2. **Arrive.** Within 4 m of the end, set `AtGoal`; hold if asked (`Park`: brake to a stop, then handbrake only, because holding brake at a standstill would select reverse).
3. **Probe.** `UpdateProbe` (515) every 0.1 s: five lateral sphere casts ahead (walls, props, gates, and the Traffic layer), **path occupancy** (`PathClearance`, 485: another car's 3.8 m body within 2.2 m of the

candidate line), **side risk** (cars alongside rule out lines that would side-swipe them), and **yield** (`ScanTraffic`, 359: a car predicted to pass within 2.8 m in the next 1.6 s makes it match speed for 0.5 s).

4. **Lateral line.** Moves `offset` toward the chosen lateral line at `LaneChangeRate`. Lane choice and overtaking are just "point + right × offset".
5. **Steer.** Pure pursuit on the offset line with look-ahead = clamp(min + time × speed), normalised by the car's speed-dependent steering lock.
6. **Speed.** A **braking-distance planner** over the line (`v² = v_next² + 2·b·d`) × `PaceScale` (rubber band), capped by the obstacle ahead and by **`FollowSpeed`** (`√(theirs² + 2·b·(gap − 0.45 s × own speed))`: never hits a leader that brakes as hard as we can).
7. **Intent rules** (`NextTurn`, `TurnDistance`): never pass a peer that is about to turn (or while about to turn yourself), with "about to turn" = max(40 m, 3.5 s at speed). A **player alongside** who is converging makes the rival drop 2 m/s below the player and hold its lane.
8. **Stuck recovery** (`Recovering`, 252): no progress for `stuckSeconds` → back out with opposite lock (never into a car behind it), remember the edge, re-plan after the second reversal. After `reversesBeforeReset` (3) failed reversals, or six "car behind" waits (jam breaker), `Reset` (320) puts it back on its line, **out of the player's view only**; in view it is just righted (`InPlayerView`).

Overtaking and merging are not separate systems: they fall out of the lateral line choice from the probe sweep plus the occupancy and side-risk rules.

17.7 `RacerProfile`: driver personality

Field	VEX	KADE
`cornerGrip` (commitment in corners)	1.00	0.92
`braking` (m/s²)	8.5	7.5
`streetSpeedScale`	1.30	1.20
`aggression`	0.85	0.55
`catchUp` / `easeOff` (rubber band)	0.05 / 0.06	0.05 / 0.06

(Set in `NightRunBuilder.BuildRivals` through `RacerProfile.EditorConfigure`.) Other fields: look-ahead min/max/time, lane-change rate, probe base/time, stuck/reverse seconds, reverses before reset, `bandDistance` (220 m).

Rubber band (`RacerProfile.RubberBand`, Decision D25): pace multiplier = 1 + `catchUp` × t when behind the player, 1 + `easeOff` × t when ahead, with t = gap / `bandDistance` clamped to ±1. So it is bounded to ±5–6 % on the target speed. Tested by `CityTests.RubberBand_IsBoundedAndSymmetricInSign`.

17.8 Qualifying and results

- `MissionDefinition.QualifyingPlaces` = 2. In `MissionDirector.BuildDebrief`, a player finishing third **still completes the mission** but is shown **NOT QUALIFIED** ("ONLY THE FIRST 2 OUT ADVANCE"); the crews list marks QUALIFIED / EXTRACTED / STILL RUNNING. `MissionRecords.MarkQualified` stores the qualification in PlayerPrefs.
- The grade's position component uses `FinishPosition` among `rivals.Count + 1` cars (36 Results).

17.9 Drones on the rivals

After the counter-intrusion or lockdown, `SecurityDrones.TargetFor` assigns one drone to each rival still running (the last drones in the list), and the rest tail the player (23 Heat and Lockdown).

17.10 Validation

- `CityTests.RacingLine_SlowsForCornersAndStops`, `RubberBand_IsBoundedAndSymmetricInSign`, `Session_RivalsAreTheCarsNotPicked`; `MissionTests.NightRunSceneProvidesEveryTargetAndReaction` (rival tasks name existing markers, crews work different sites, the last task finishes, `QualifyingPlaces` = 2, every slot has work FX).
- Play Mode: `MissionSeries` with `NightRunValidator` (contacts, reversals, resets, finishing). Final runs (`Docs/NightRun.md`): VEX extracted at 124.9 s, just ahead of the scripted player at about 125 s; in another run the player won at 127.8 s with both crews about 390 m from the gate.

17.11 Intro use

In the intro, the two bay cars leave the garage on scripted lines with **`WaypointDriver`** (`Scripts/Gameplay/Racing/WaypointDriver.cs`: pure pursuit plus speed per waypoint, real physics). `RacerDriver` is the fallback when no garage exists (`IntroSceneEntry.Depart`).

18 Traffic System

IN THIS CHAPTER

18.1 Overview
18.2 Prefabs (TrafficCarBuilder.Build)
18.3 Physics (TrafficCar.FixedUpdate, line 120)
18.4 Roaming behaviour (CityTraffic.Update, Steer)
18.5 Parked pool
18.6 Spawning and recycling (no popping)
18.7 Collisions
18.8 Audio
18.9 Traffic signals (TrafficSignalNetwork)
18.10 Intro traffic (separate)
18.11 Validation

18.1 Overview

City traffic uses the **generic passenger car pack** (10 models) as real physical cars: a Rigidbody with four raycast springs, lateral tyre grip and a drive/brake force. They are deliberately simpler than the player's `VehicleController`: traffic cars do not use `IVehicleInputSource`. There are two pools: **roaming** (14) and **parked** (26).

Piece	File
Pool manager (roaming + parked)	`Assets/_NeonRift/Scripts/Gameplay/City/CityTraffic.cs` (`IMissionWorldComponent`)
One physical car	`Scripts/Gameplay/City/TrafficCar.cs` (`enum Mode { Parked, Driving, Knocked, Stopped }`)
Prefab builder	`Scripts/Editor/District/TrafficCarBuilder.cs` → `Assets/_NeonRift/Prefabs/Traffic/PackCar_*.prefab` (10)
Scene wiring	`NightRunBuilder.BuildTraffic` (`Scripts/Editor/District/NightRunBuilder.CityLife.cs`)
Source model	`Assets/ThirdParty/Sketchfab/Vehicles/generic-passenger-car-pack/source/fab.fbx` + `textures/`



Pack-car traffic at a junction
`Docs/Screenshots/Life/pack_traffic_junction.png`



Traffic on the avenue
`Docs/Screenshots/Life/traffic_avenue.png`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

18.2 Prefabs (`TrafficCarBuilder.Build`)

The pack is one FBX with ten bodies in a ring, each with four loose wheels. The builder re-frames each body and its four nearest wheels so the car sits on the ground at the origin facing +Z. It then adds URP materials from the pack's textures (`Assets/_NeonRift/Art/Traffic/Traffic_Pack*`), a **box collider sized from the body's vertices** (read with `Mesh.AcquireReadOnlyMeshData`, because the FBX is not Read/Write enabled), a Rigidbody (1350–1900 kg) and a `TrafficCar`. Wheel transforms are named `Wheel`.

18.3 Physics (`TrafficCar.FixedUpdate`, line 120)

- Four springs at the wheel positions (ride height, pitch, roll).
- Tyres kill sideways slip up to the grip limit (about 0.95 g), so a hard hit still slides the car.
- Steering is a yaw rate toward the aim point, limited by speed (no turning on the spot).
- Drive/brake force toward the target speed; parked-and-hit, knocked or stopped cars brake to a halt.
- Tint per car: `SetTint` writes `_BaseColor` through a `MaterialPropertyBlock` on the paint renderers.

18.4 Roaming behaviour (`CityTraffic.Update`, `Steer`)

Rule	Implementation
Lanes	Kerb-side lane of the road graph (`LaneFor`); no alleys, service roads, tunnels or the Data Core compound (`InCompound`, exclusion rect (100, 55, 120 × 155))
Dead ends pruned	Nodes with a single drivable edge are removed repeatedly (`Start`, lines 102–108), so cars never strand at the compound spur
Route	At each node pick the next edge (`NextEdge`, `PeekNext`)
Speed	Cruise 8–13 m/s (`cruiseRange`), slower for corners
Distance keeping	Forward sphere cast for anything ahead (other traffic, the player, the rivals) and a braking distance
Give way at junctions	`JunctionBusy`: halts short of the line when the player or a rival (`Drivers`) will reach the junction within ~3.5 s

Rule	Implementation
Lockdown	`OnSecurity(Lockdown)` : pull over and stop
Traffic signals	**Not obeyed.** `TrafficSignalNetwork` is visual only (see below)

18.5 Parked pool

2,073 parking bays are recorded from the lots and garage decks (`spots`). The nearest 26 to the player are filled (`FillSpots` , within `parkedRadius` 170 m). Parked cars are **kinematic until hit**.

18.6 Spawning and recycling (no popping)

`Spawn` / `Acquire` : cars are placed **out of the camera's view** (`InView`) between `spawnMin` 70 m and `spawnMax` 260 m. They are recycled when beyond `recycleBeyond` 380 m, overturned, stuck, or disturbed and left behind. Placement uses `Ground` raycasts.

18.7 Collisions

- The **Traffic** layer (13) ignores itself: traffic never collides with traffic (`Physics.IgnoreLayerCollision` in `CityTraffic.Start`). It collides with the player, rivals and the world.
- `TrafficCar.OnCollisionEnter` (182): the car turns dynamic, is shunted and spun by the momentum share of the hitting car, plays an impact sound by strength and stops driving for 2.5–7 s (`Knocked`), then rejoins the nearest road (`Resume`).
- Rival obstacle probes include the Traffic layer, and the scripted validation driver brakes gently for traffic on its line (`NightRunValidator.FollowTraffic`).

18.8 Audio

A 3D engine loop pitched by speed (low voice priority) and impact one-shots on dedicated sources (`TrafficCar` fields `engine` , `sfx` , `engineLoop`).

18.9 Traffic signals (`TrafficSignalNetwork`)

`Scripts/Gameplay/City/TrafficSignalNetwork.cs` : every north–south and east–west lens shares **one runtime material per colour**, so the whole grid cycles (green → amber → all-red, north–south vs east–west) with six material updates. On `grid.anomaly` the grid **stutters amber**, and in a lockdown it **flashes red**. It works without a mission (free roam) and binds to one for the lockdown look. Signal geometry: `CityProps.BuildIntersections` .

18.10 Intro traffic (separate)

The intro does **not** use `CityTraffic` . It uses visual-only copies of the pack cars driven on Timeline-synced lanes by `IntroCityLife` (31 Cinematic Intro).

18.11 Validation

- `CityTests.TrafficCars_ArePhysicalAndIgnoreEachOther`, `NightRun_CityLifeIsSetUp`.
- Play Mode (`Docs/City.md`): 14 roaming + 26 parked live during every heist scenario, 29 cars within 150 m; a scripted crash woke four parked cars, which were shoved and came to rest upright. Final series: Boulevard runs completed with 0 player collisions and 0 rival contacts.

PART V

Missions and the Heist

Objectives, multi-stage hacks, the gate, the Data Core and the lockdown.

19 Mission System

20 Heist System

21 Security Gate

22 Data Core

23 Heat and Lockdown

24 Environmental Interactions

19 Mission System

IN THIS CHAPTER

19.1 Design (Decision D21)
19.2 Pieces
19.3 The Night Run data (Assets/_NeonRift/Data/Missions/Mission_NightRun.asset)
19.4 The state machine (MissionProgress)
19.5 The director (MissionDirector)
19.6 MissionWorld: what scene objects see
19.7 Mission flow, end to end
19.8 Validation

19.1 Design (Decision D21)

A mission is **data plus a pure state machine**; the scene **reacts to string event ids**. The director has no mission-specific code. Adding a mechanic means adding an `IMissionWorldComponent` (or an `InteractionDefinition`) and referencing event ids from data.

19.2 Pieces

Layer	Class	File
Mission data	`MissionDefinition`	`Assets/_NeonRift/Scripts/Missions/MissionDefinition.cs`
Objective data	`ObjectiveDefinition` (`ObjectiveKind`: Reach / Interact)	`Scripts/Missions/ObjectiveDefinition.cs`, `ObjectiveKind.cs`
HUD messages from events	`MissionAnnouncement` (`MessageTone`)	`Scripts/Missions/MissionAnnouncement.cs`, `MessageTone.cs`
State machine (pure C#)	`MissionProgress` (`MissionPhase`, `SecurityLevel`)	`Scripts/Missions/MissionProgress.cs`, `MissionPhase.cs`, `SecurityLevel.cs`
Per-attempt hub	`MissionWorld`	`Scripts/Gameplay/Mission/MissionWorld.cs`
Runner	`MissionDirector`	`Scripts/Gameplay/Mission/MissionDirector.cs`
Participants	`IMissionWorldComponent`, `IMissionTarget`	`Scripts/Gameplay/Mission/IMissionWorldComponent.cs`, `IMissionTarget.cs`
Reach target	`MissionZone` (Trigger layer, beacon while current)	`Scripts/Gameplay/Mission/MissionZone.cs`
Interact target	`Interactable` + `InteractionDefinition`	`Scripts/Gameplay/Mission/Interactable.cs`, `InteractionDefinition.cs`
Result	`RunResult`, `MissionOutcome`	`Scripts/Missions/RunResult.cs`

Layer	Class	File
Grade and records	`MissionGrade`, `MissionRecords`	`Scripts/Missions/MissionGrade.cs`, `MissionRecords.cs`

19.3 The Night Run data (`Assets/_NeonRift/Data/Missions/Mission_NightRun.asset`)

Authored in code by `NightRunBuilder.CreateMissionData` (`Scripts/Editor/District/NightRunBuilder.cs:174`).

#	Objective id	Kind → target	Title	Security	Timer	Events
1	`reach_core`	Reach → zone `core_compound`	INFILTRATE THE DATA CORE FACILITY	Calm	–	on complete `core.arrive`
2	`disable_security`	Interact → `core_security_terminal`	DISABLE FACILITY SECURITY	–	–	(terminal raises `core.security.off`)
3	`hack_core`	Interact → `core_uplink`	EXTRACT THE DATA CORE	Calm	–	on complete `core.acquired`
4	`escape`	Reach → zone `extraction`	ESCAPE TO THE RIFT GATE	**Lockdown**	**80 s** ("TRACE COMPLETE – YOU WERE FOUND")	`startDelay` 2.4 s; on start `core.breached`, `lockdown`, `escape.start`

Mission-level fields: events on start `mission.start`, on complete `mission.complete`, on fail `mission.failed`; `heatTimePenalty` 25 s per 1.0 heat; `minimumTimeLimit` 30 s; `maxRivals` 2; `parTime` 115 s; `qualifyingPlaces` 2; `rivalTasks` (17); crew roles "SPIRE RELAY · OWN JOB", "KOWLOON RELAY · OWN JOB"; and about 17 announcements (for example `mission.start` → "SECTOR 7 GRID ONLINE · FOLLOW THE CYAN BEACON", `core.breached` → banner "SECURITY BREACH DETECTED | SECTOR 7 LOCKDOWN INITIATED").

The dev mission `Mission_DevTestTrack.asset` drives the TestTrack scene.

19.4 The state machine (`MissionProgress`)

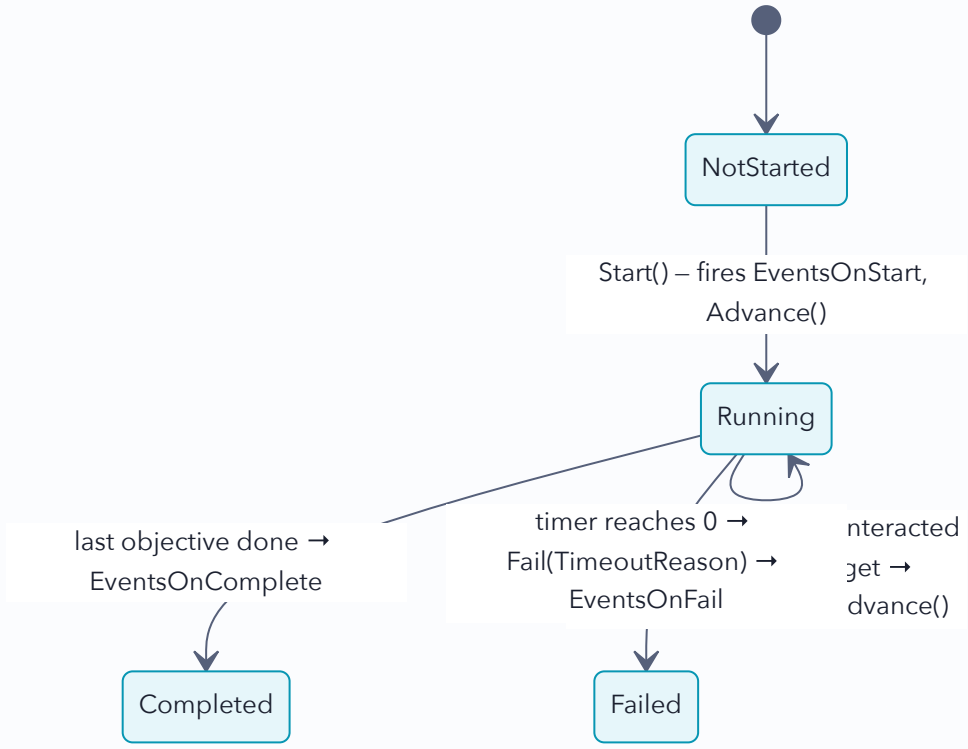


Figure 19.1 Objective state machine (MissionProgress)

Method	Behaviour
`Start`	Phase Running, fire mission start events, advance to objective 0
`Tick(dt)`	Elapsed time; objective `startDelay` countdown; timer countdown → `Fail`
`NotifyReached(zoneId)` / `NotifyInteracted(id)`	`TryComplete`: only completes when kind and target match the current objective
`Advance` → `StartObjective`	Raises security to the objective's level (**security only escalates**), starts the timer as max(min(minimumTimeLimit, limit), limit − heat × heatTimePenalty), fires the objective's start events
`AddHeat(amount, reason)`	Heat clamped to 1; a running timer loses `gained × heatTimePenalty` immediately; Calm → **Alert** on the first heat
Events out	`ObjectiveStarted`, `ObjectiveCompleted`, `SecurityChanged`, `HeatAdded`, `WorldEvent`, `PhaseChanged`

How the game knows which objective is active: `MissionProgress.ObjectiveIndex` / `Current`; `IsCurrentTarget(targetId)` answers "is this zone or interactable the one we are waiting for?". `MissionWorld.IsObjectiveTarget` exposes this to scene objects, for example so an `Interactable` only be-

comes usable when it is current.

19.5 The director (`MissionDirector`)

`Begin(context, mission, player, camera)` (line 80) creates a fresh `MissionProgress` and `MissionWorld`, wires their events, calls `BindScene` (finds every `IMissionWorldComponent` in the scene, indexes `IMissionTarget`s by id, publishes `ObjectivePositions`, calls `Bind(World)` on each), sets the Gameplay mix, initialises `MissionAudio` and the HUD, then `Progress.Start()`.

Method	Role
`Update` (206)	`Progress.Tick`, interaction focus, HUD, navigator, race panel, briefing, results timing, theft beat
`UpdateInteraction` (232)	Picks the focused `Interactable` (one in use keeps focus), feeds the Interact button (`Driving.Interact`), drives the `TerminalReadout` and the hack loop audio
`OnZoneEntered` / `OnInteractionCompleted`	→ `Progress.NotifyReached` / `NotifyInteracted`
`OnObjectiveStarted` (298)	Activates the target's beacon, refreshes interactables, notifies the world, updates the HUD objective
`OnObjectiveCompleted`	Records split time, `hud.StampObjective`
`OnSecurityChanged` (331)	Lockdown: camera kick and focus on the alert origin, **Lockdown** mixer snapshot, lockdown cue
`OnHeatAdded`	HUD toast "+n % HEAT" and heat cue
`OnWorldEvent` (388) → `RunAnnouncements`	Shows the mission's announcements for that id; `beatEvent` (`core.breached`) starts the **theft beat**
`StartBeat` / `UpdateBeat` / `EndBeat`	2.6 s camera swing (`VehicleChaseCamera.CinematicBeat`), letterbox, time scale eased to 0.45 and back
`OnPhaseChanged` (357)	Finish position, `RunSession.RecordResult`, **Results** snapshot, score outro, debrief after `resultsDelay` (2 s)
`BuildBriefing` / `BuildDebrief`	35, 36
`Retry` / `Continue` / `RestartMission`	E → `Flow.StartMission()`, Esc → `Flow.GoToCarSelect()`

19.6 `MissionWorld`: what scene objects see

`Player`, `PlayerBody`, `Security`, `Heat`, `Phase`, `Mission`, `ObjectiveIndex`, `ObjectivePositions`, `Elapsed`, `AlertOrigin`. Methods `Raise(eventId)`, `Announce(text, tone)`, `AddHeat`, `IsPlayer(collider)`, `IsObjectiveTarget`, `IsMissionTarget`, HUD countdowns (`SetCountdown`, `ClearCountdown`, `TryGetNextCountdown`), and `Matches(list, id)`. Events: `EventRaised`, `SecurityChanged`, `Announced`, `ObjectiveStarted`.

19.7 Mission flow, end to end

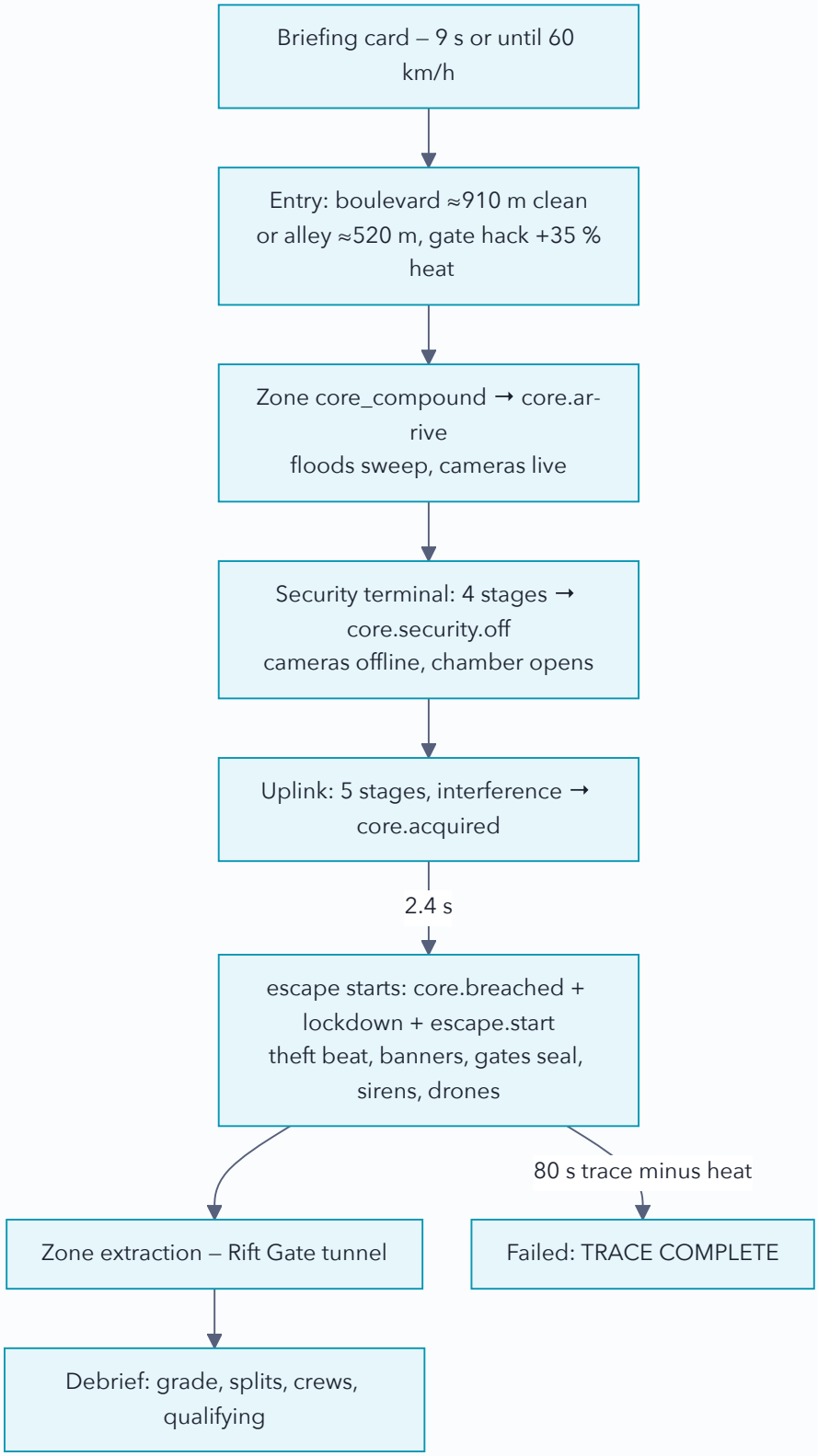


Figure 19.2 Night Run mission flow

Section details: 20 Heist, 21 Security Gate, 22 Data Core, 23 Heat and Lockdown.

19.8 Validation

- `MissionTests` (`Scripts/Tests/EditMode/MissionTests.cs`): `ObjectivesAdvanceInOrderAndFireWorldEvents`, `HeatRaisesAlertAndShortensTheEscapeTimer`, `HeatIsClampedAndTimerRespectsMinimum`, `TimerExpiryFailsTheMission`, `UntimedObjectivesNeverFail`, `NightRunMissionDataIsValid`, `NightRunSceneProvidesEveryTargetAndReaction`, grade and records tests.
- `InteractionTests.ObjectiveStartDelayHoldsTheNextObjective`.
- `NightRunValidator` scenarios `BoulevardInAlleyOut`, `AlleyInExpresswayOut`, `TraceTimeout`, `AlleyInFailRetry` (40).

20 Heist System (multi-stage interactions)

IN THIS CHAPTER

20.1 Idea
20.2 Classes
20.3 Stage kinds
20.4 The three Night Run devices (data authored in code)
20.5 Runtime flow
20.6 Presentation
20.7 Validation

20.1 Idea

Terminals, the extraction uplink and gate controllers share **one framework**: the player stops the car inside a trigger zone and works through ordered **stages**, each with its own input rule. Stages raise **world events**, so screens, locks, lights, cameras, signals and machinery react without mission-specific code. Original notes: `Docs/Interactions.md`.

20.2 Classes

Piece	Assembly	File	Role
`InteractionStep` (+ `InteractionStepKind`, `InteractionCue`)	Missions	`Assets/_NeonRift/Scripts/Missions/InteractionStep.cs`	One stage: label, detail, kind, duration, checkpoint flag, timing window / misses / heat, sustain interference, events on start/complete, progress cues. Builders `WithTiming(width, centre, misses, heatPerMiss)`, `WithInterference(at[], responseSeconds, rollback)`, `WithCues(...)`
`InteractionRun`	Missions	`Scripts/Missions/InteractionRun.cs`	**Pure C# runner**: `Tick(dt, held, engaged)`, `Reset`, `Cancel`, `Miss`, `StartStep`, `CompleteStep`; state `Idle / Running / Completed / Failed`; `Overall` progress weighted by duration; `CancelGrace` = 0.6 s
`InteractionDefinition` (asset)	Gameplay	`Scripts/Gameplay/Mission/InteractionDefinition.cs`	Prompt verb, hold seconds, **max speed (km/h)**, decay, heat on complete, messages, device name, stages, lockout, fail heat, fail/cancel events, tension level
`Interactable`	Gameplay	`Scripts/Gameplay/Mission/Interactable.cs`	Scene trigger: availability by objective and events, runs stages (`Operate`, `CreateRun`), raises events and heat, locks out on failure, `Feedback` for presentation; `{id}` in event ids is expanded to its own id (`Expand`)

Piece	Assembly	File	Role
`TerminalDisplay`	Gameplay	`Scripts/Gameplay/Mission/TerminalDisplay.cs`	The device's screens (`NeonRift/TerminalScreen` shader), light and 3D blips
`TerminalReadout`	Gameplay	`Scripts/Gameplay/Mission/TerminalReadout.cs`	HUD panel: header, link state, stage list, active widget, data stream, progress

20.3 Stage kinds

Kind	The player...	Rule
`Auto`	stays stopped in the zone	Machinery and handshakes
`Hold`	holds **E**	Releasing bleeds progress (`decayPerSecond`)
`Timing`	presses **E** while the sweeping cursor is in the window	A miss adds `missHeat`, moves the window and counts toward `maxMisses` (0 = unlimited). Reaching the limit fails the run → lockout plus fail heat. The press that starts a run never counts as an answer
`Sustain`	stays stopped and answers **interference** with E	Interference stalls progress for `responseSeconds`; ignored, it rolls back `rollback` of the stage. It never fails

Engaged means inside the trigger and slower than `maxSpeedKph`. Being disengaged for more than 0.6 s **cancels** the run back to the last `checkpoint` stage reached.

20.4 The three Night Run devices (data authored in code)

Perimeter security terminal: `Interaction_CoreSecurity.asset`

`NightRunBuilder.CoreTerminalDefinition` (`Scripts/Editor/District/NightRunBuilder.Heist.cs:30`). Device "**S7-CORE // PERIMETER SECURITY NODE**", verb HACK, max speed 8 km/h, lockout 4 s, fail heat 0.10 ("INTRUSION FILTER LOCKOUT · TRACE LOGGED"). The interactable `core_security_terminal` stands at (145, 0, 127.5), west of the core, and raises `core.security.off` on completion.

#	Stage	Kind	Duration	Detail
1	CONNECTING	Auto	1.1 s	Handshake with the perimeter node
2	AUTHENTICATING	Hold	1.8 s	Spoofing a maintenance credential: hold E
3	BYPASSING	Timing	1.5 s / sweep	Window 0.26 at 0.64; **3 misses** allowed; +4 % heat per miss
4	SECURITY OVERRIDE	Auto	1.0 s	Cameras, floods and chamber locks → crew control

Extraction uplink: `Interaction_DataCoreUplink.asset`

`NightRunBuilder.ConfigureUplinkDefinition` (`NightRunBuilder.Heist.cs:46`). Device "**S7-CORE // EXTRACTION UPLINK**", verb EXTRACT, max speed 6 km/h, lockout 3 s ("LINK COLLAPSED"), tension 1.0. The interactable `core_uplink` is the ring around the core; on completion the objective fires

`core.acquired`.

#	Stage	Kind	Duration	Checkpoint	Events
1	CONNECT EXTRACTION DEVICE	Auto	2.2 s	✔	start `core.extract.connect`
2	INITIALIZE EXTRACTION	Auto	1.6 s		start `core.extract.init`
3	STABILIZE DATA LINK	Timing	1.3 s	✔	start `core.extract.stabilize`; window 0.22 at 0.40, unlimited misses, +3 % heat per miss
4	**DATA EXTRACTION**	Sustain	**14 s**	✔	**Interference at 38 % and 74 %** (2.6 s to answer, 12 % rollback). Cues: 3 % `core.extract.begin`, **50 %** `grid.anomaly`, **80 %** `core.extract.trace`
5	EXTRACTION COMPLETE	Auto	0.9 s		–

Alley gate controller: `Interaction_HackGate.asset`

See 21 Security Gate (five stages: CONNECTING, AUTHENTICATING, BYPASSING SECURITY, OVERRIDE, GATE RELEASE).

20.5 Runtime flow

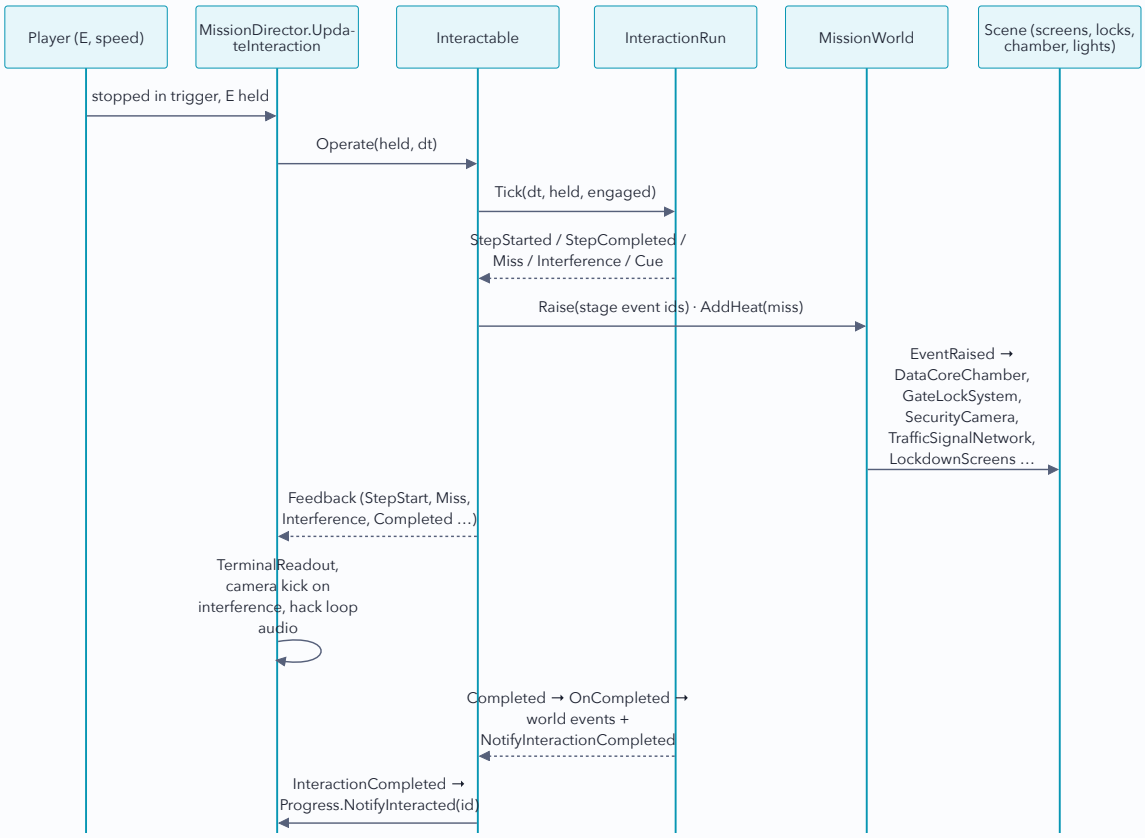


Figure 20.1 Multi-stage interaction sequence

20.6 Presentation

- **HUD** (`TerminalReadout`): link state STANDBY / LINK ACTIVE / SIGNAL WEAK / LOCKOUT / COMPLETE; numbered stages (OK / ·· / --); the active widget is a hold bar, a timing track (window + cursor, green while inside) or an interference alarm with a re-sync countdown; hex data stream; overall percent. It flashes green on completion, red on a miss and amber on interference.
- **World** (`TerminalDisplay`): `MaterialPropertyBlock` drives the stage strip, scrolling rows, progress bar and scan line. Colours: standby cyan, working, alarm red, complete green, lockout. Screens tear on misses and interference.
- **Audio**: stage blips, denied, interference, re-sync, lockout and cancel cues; a tension bed scaled by `tension × progress` (`MissionAudioSet`, generated by `HeistAudioGenerator`).
- **Camera**: `VehicleChaseCamera.SetFraming` holds a wide framing of the car and the core while the chamber opens and data is pulled; the player keeps control.

20.7 Validation

`InteractionTests` (`Scripts/Tests/EditMode/InteractionTests.cs`): `StagesRunInOrderAndRaiseTheirEvents`, `TimingPressInsideTheWindowPassesAndMissesCostHeatThenLockOut`, `InterferenceStallsUntilAnsweredAndRollsBackWhenIgnored`, `LeavingCancelsBackToTheLastCheckpoint`, `CuesFireOnceAtTheirProgress`. Play Mode (`Docs/NightRun.md`): terminal 4.8 s, uplink 20.9 s with both interference events answered, cues fired on time.

21

Security Gate (alley gate hack)

IN THIS CHAPTER

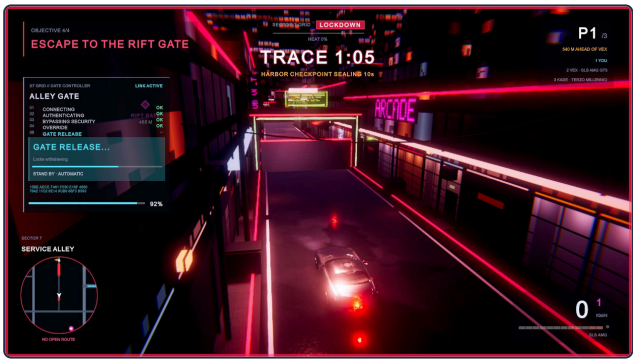
21.1 What it is
21.2 Pieces and code
21.3 The hack (Interaction_HackGate)
21.4 Physical gate (SecurityBarrier)
21.5 Lock presentation (GateLockSystem)
21.6 Lockdown re-hack
21.7 Validation

21.1 What it is

The **Service Alley** (x = 160, z −100…60) is the short, risky route into the Data Core compound. A locked sliding gate across it at (160, −20) must be **hacked** from a kiosk on the west pavement. A successful hack opens the gate but logs **+35 % heat**. In a lockdown the gate **slams shut** and can be hacked again for another +35 %.



Alley gate hack, BYPASSING stage
`Docs/Screenshots/Gate/bypass.jpg`



Re-hack after the lockdown slam
`Docs/Screenshots/Gate/lockdown_rehack.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

21.2 Pieces and code

Piece	Class	File	Built by
Sliding gate panel, countdown, warning, slam	`SecurityBarrier`	`Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityBarrier.cs`	`NightRunBuilder.BuildAlleyGate` → `BuildBarrier("AlleyGate", …)` (`Scripts/Editor/District/NightRunBuilder.cs:427`)

Piece	Class	File	Built by
Bolts, status lamps, lamp bar, status hologram, lock audio, camera framing	`GateLockSystem`	`Scripts/Gameplay/Mission/GateLockSystem.cs`	`NightRunBuilder.BuildGateLocks` (`NightRunBuilder.Heist.cs:155`)
Hack zone (Trigger, 10 × 5 × 46 m)	`Interactable` id `alley_gate_terminal`	`Scripts/Gameplay/Mission/Interactable.cs`	`BuildAlleyGate`
Kiosk screens	`TerminalDisplay`	`Scripts/Gameplay/Mission/TerminalDisplay.cs`	`TerminalKiosk` (`NightRunBuilder.Heist.cs:97`)
Hack data	`InteractionDefinition` `Interaction_HackGate.asset`	`Assets/_NeonRift/Data/Missions/`	`ConfigureGateHackDefinition` (`NightRunBuilder.Heist.cs:138`)
Graph edge	gate id `alley_gate`	`CityLayout.Blockers`	`BuildNavigation`

21.3 The hack (`Interaction_HackGate`)

`EditorConfigure("HACK", hold 0, maxSpeed 10 km/h, decay 0.6, heatOnComplete 0.35, "GATE BREACH LOGGED", "GATE RELEASED")`. Device "S7 GRID // GATE CONTROLLER". Lockout 5 s, fail heat 0.12 ("GATE CONTROLLER LOCKOUT · TRACE LOGGED"), tension 0.35. Event ids use `{id}`, expanded to `alley_gate_terminal` at runtime, so any gate terminal can reuse the definition.

#	Stage	Kind	Duration	What the world does
1	CONNECTING	Auto	0.9 s	`{id}.hack`: lock lamps and the lamp bar go amber and pulse, bolts chatter, the gate status hologram works
2	AUTHENTICATING	**Hold E**	1.3 s	"Replaying a maintenance key – hold E"; kiosk screens scroll
3	BYPASSING SECURITY	**Timing**	1.2 s / sweep	Window 0.26 at 0.58, **3 misses** allowed, **+5 % heat per miss** (raises Alert), screens tear, denied buzz. Three misses → `{id}.lockout`: lamps flash red, 5 s lockout, +12 % heat
4	OVERRIDE	Auto	0.8 s	"Override accepted – controller is ours"
5	GATE RELEASE	Auto	1.1 s	`{id}.unlock`: four bolts withdraw into the housing (ratchet + pneumatic sigh), lamps turn green

On completion the interactable raises **`alley.gate.open`** (the barrier's `openOn`), and the panel slides open. Heat +35 % is applied (`heatOnComplete`).

Cancelling: driving off (over 10 km/h, or out of the zone for 0.6 s) drops the link ("CONNECTION LOST"). `{id}.abort` (the definition's cancel event) returns the locks to sealed red; the next attempt starts from CONNECTING.

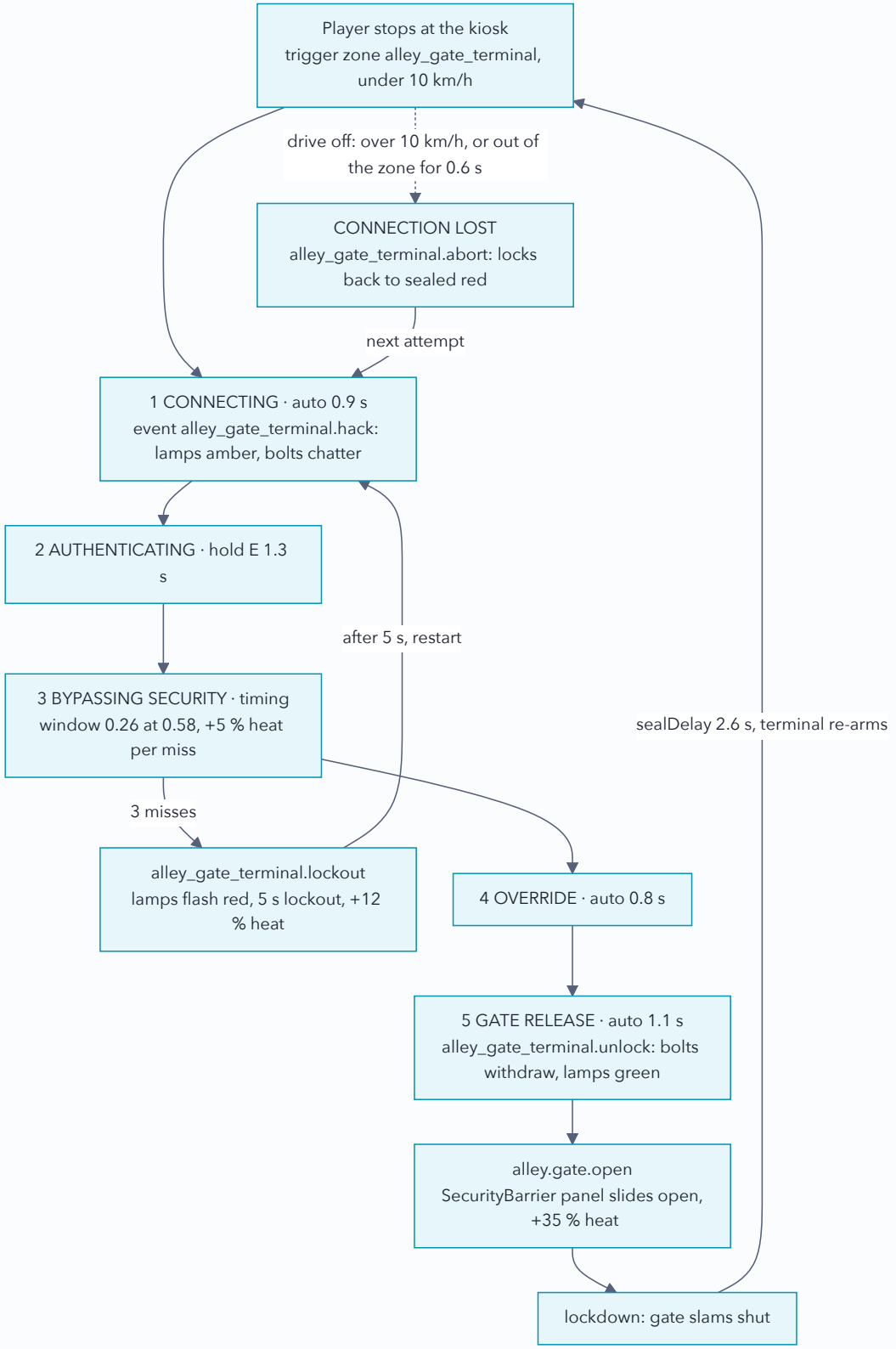


Figure 21.1 Security Gate hack flow (Interaction_HackGate)

21.4 Physical gate (`SecurityBarrier`)

Alley gate parameters: width 10 m, one panel, **closed at start**, `closeOn` = `lockdown`, `openOn` = `alley.gate.open`, warning 1 s, travel 1.5 m.

General behaviour (`SecurityBarrier`): closing is **staged** so the player can read it coming: an optional countdown (shortened by heat, shown on the HUD via `MissionWorld.SetCountdown`), then a **warning** phase with strobing lights and a klaxon, then the **kinematic** panels slide (they push cars, never teleport them) and slam. Methods: `Close` (120), `Open` (133), `Update` / `FixedUpdate` (`MovePanels`), `SetWarning`, `Finish`. State is read by `CityNavigation` (`IsOpen`, `IsClosed`, `SecondsUntilClosing`) to re-route.

21.5 Lock presentation (`GateLockSystem`)

Field / method	Behaviour
`hackOn`, `abortOn`, `lockoutOn`, `unlockOn`, `releasedOn`, `sealOn`	Event lists (take the hack's own stage events)
Colours	sealed red (2.2, 0.12, 0.2), hacking amber (2.6, 1.2, 0.15), released green (0.2, 2.4, 0.9)
`bolts`, `boltSeconds` 0.5	Bolts slide into / out of the post (`PlaceBolts`)
`sealDelay` 2.6 s	After a lockdown slam the bolts re-seat with a latch clunk and the terminal **re-arms**
`UpdateCamera`	While hacked or releasing and the car is within `frameRange` 40 m, the chase camera rises and frames the gate, staying near-straight behind the car so it does not swing into the alley walls

21.6 Lockdown re-hack

The interactable is configured with **re-arm on `lockdown`** (`Interactable.EditorConfigure(..., new[] { EventLockdown }, ...)`). After the slam the hack is available again (+35 % heat). The `BoulevardInAlleyOut` validation scenario uses exactly this escape.

21.7 Validation

`NightRunValidator` scenario `AlleyInFailRetry`: three deliberate misses → lockout (+5 % ×3 and +12 % heat, Alert); a 5 s wait; restart and drive off mid-bypass → CONNECTION LOST and abort; then a full hack → bolts released → gate open (+35 %). `GateTrick` in `Scripts/Gameplay/Mission/NightRunValidator.cs:201` performs the scripted misses. Screenshots: `Docs/Screenshots/Gate/`.

22 Data Core

IN THIS CHAPTER

22.1 What it is
22.2 Who does what
22.3 DataCoreChamber phases (from its class summary and methods)
22.4 Terminal (core_security_terminal)
22.5 Uplink (core_uplink)
22.6 Sentries
22.7 Validation

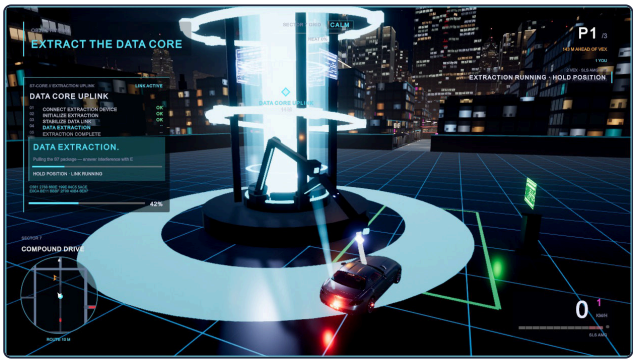
22.1 What it is

The **Sector 7 Data Core** sits in a walled compound (reserved rect x 105…215, z 60…195; core at **(160, 0, 127.5)**). The player drives in through the north **compound gate** (Access Road) or the **alley**, hacks the **perimeter security terminal** (west of the core), then parks on the **uplink ring** around the core while robotic arms dock a data cable onto the car and pull the package.



core.security.off: the chamber opens

`Docs/Screenshots/Heist/core_security_off.jpg`



core.extract.begin: data extraction

`Docs/Screenshots/Heist/core_extract_begin.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

22.2 Who does what

Concern	Class	File
Rules: stages, timing, interference	`InteractionRun` via `Interactable` (`core_security_terminal`, `core_uplink`)	`Scripts/Missions/InteractionRun.cs`, `Scripts/Gameplay/Mission/Interactable.cs`
Presentation of the whole facility	**`DataCoreChamber`** (reads world events and the uplink's `InteractionRun`; **owns no rules**)	`Assets/_NeonRift/Scripts/Gameplay/Mission/DataCoreChamber.cs`

Concern	Class	File
Terminal screens	`TerminalDisplay`	`Scripts/Gameplay/Mission/TerminalDisplay.cs`
Reach zone	`MissionZone` `core_compound`	`Scripts/Gameplay/Mission/MissionZone.cs`
Geometry (kiosk, pedestal, sleeve, rail, arms, cable, rings, holograms, strobes, 3D audio)	builder	`NightRunBuilder.BuildCompound` (`Scripts/Editor/District/NightRunBuilder.cs:257`), `BuildCoreTerminal`, `BuildChamber` (254), `BuildArm` (415) in `NightRunBuilder.Heist.cs`
Facility sounds	`HeistAudioGenerator` (`Hum`, `Servo`, `Stream`, `ChamberOpen`, `Clunk`, `Release`, `Acquired`, `Breach`)	`Scripts/Editor/Audio/HeistAudioGenerator.cs`

22.3 `DataCoreChamber` phases (from its class summary and methods)

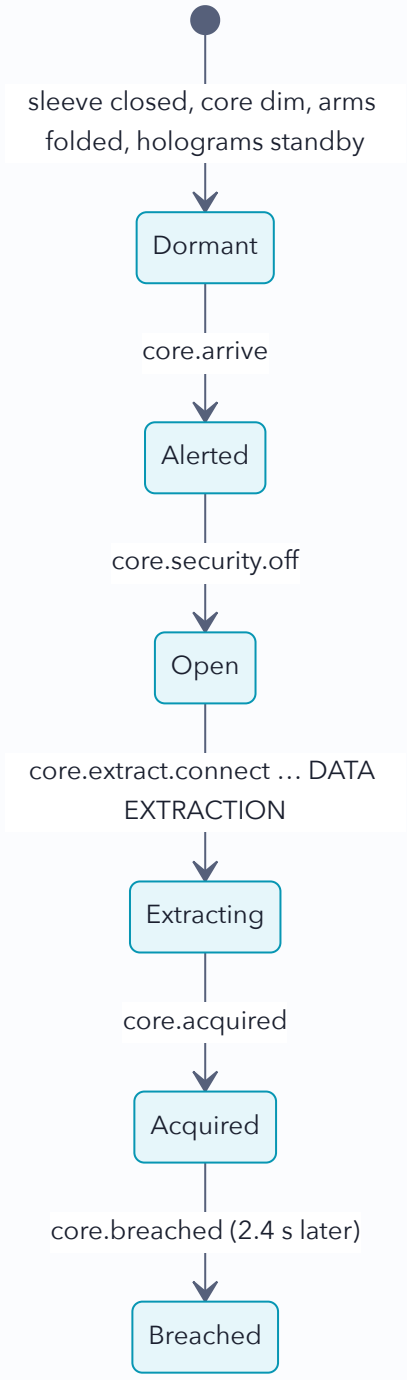


Figure 22.1 Data Core chamber phases (DataCoreChamber)

Phase / event	What moves and lights (`UpdateSleeve`, `UpdateCore`, `UpdateFacility`, `UpdateArms`, `UpdateLink`, `UpdateHolograms`, `UpdateAudio`)
Alerted (`core.arrive`)	Floodlights sweep, compound strips go amber, the compound cameras go live, a red scan plane hunts up the column, holograms turn amber, the facility hum starts

Phase / event	What moves and lights (`UpdateSleeve`, `UpdateCore`, `UpdateFacility`, `UpdateArms`, `UpdateLink`, `UpdateHolograms`, `UpdateAudio`)
Security off (`core.security.off`)	Six curved **armoured sleeve** panels unlock one after another (a small outward kick) and **sink into the pedestal over 3.2 s**; hydraulic one-shot; the column's light floods out; rings spin up. Cameras turn to watch the core
Connect	Two arms on carriages ride a **rail** round the pedestal to the car's side (`RailPoint`). The **connector arm** (two-bone IK, elbow up: `Pose`) reaches for the roof, docks with a clunk and lowers a glowing **data cable** (`BuildPath`, `Sample`) onto the car. The **scanner arm** sweeps a beam front to back
Extraction	Pulses travel core → arm → cable → car (2.5 → 16 per second with progress); the core shifts cyan → white-blue → magenta; amber strobes from 30 %; scan rings speed up; holograms show the stage strip and progress; 3D data-stream loop and 2D tension bed
Interference	Pulses freeze; link, cable, pulses and core flash red; holograms tear; camera kick
City cues	50 % `grid.anomaly`: every traffic signal stutters amber, screens within 420 m glitch. 80 % `core.extract.trace`: compound cameras reboot, compound beacons spin, **security drones launch**
Acquired (`core.acquired`)	Cable releases, core collapses to an ember; **DATA ACQUIRED** banner + stinger
Breached (`core.breached`)	Core turns the alarm colour, floods turn red; theft beat; lockdown

Camera. `UpdateCamera` (240) asks `VehicleChaseCamera.SetFraming` for a slow, wide framing of the car and the core while the chamber opens and data is pulled. The player keeps control.

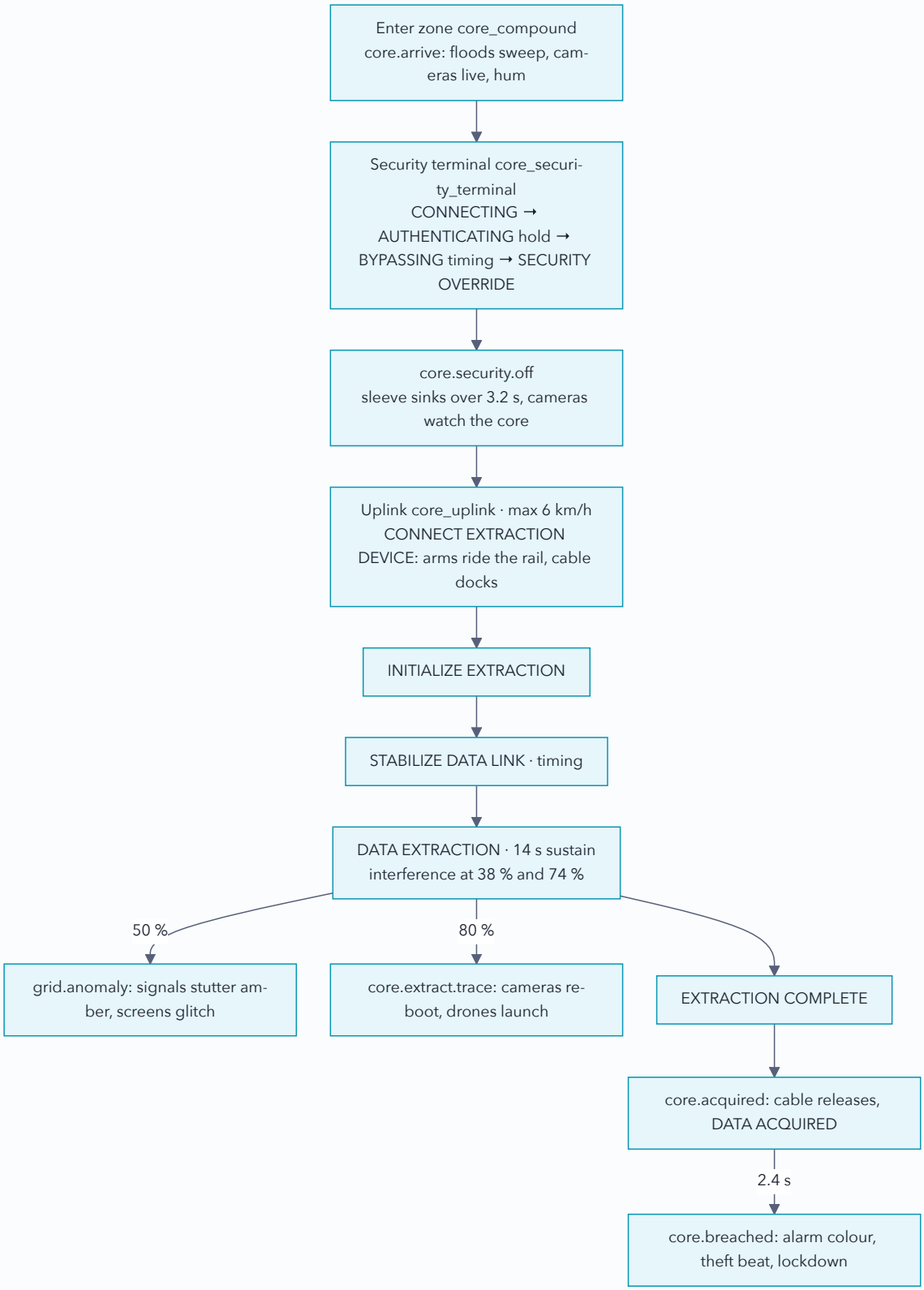


Figure 22.2 Data Core heist flow

22.4 Terminal (`core_security_terminal`)

Kiosk at (145, 0, 127.5) facing east (`TerminalKiosk`): body, main screen, status hologram, screen glow light, speaker. Four stages: CONNECTING → AUTHENTICATING (hold) → BYPASSING (timing) → SECURITY OVERRIDE (20 Heist).

22.5 Uplink (`core_uplink`)

A ring around the core. Five stages: CONNECT EXTRACTION DEVICE → INITIALIZE EXTRACTION → STABILIZE DATA LINK (timing) → DATA EXTRACTION (14 s sustain, interference at 38 % and 74 %) → EXTRACTION COMPLETE. Max speed 6 km/h.

22.6 Sentries

Two **Buster Drone** sentries (posed, baked to static meshes) hover and scan over the compound at all times (`HoverSentry`, `Scripts/Gameplay/City/HoverSentry.cs`; built by `NightRunBuilder.BuildSentries`).

22.7 Validation

`MissionTests.NightRunSceneProvidesEveryTargetAndReaction` (targets and reactions exist in the scene). Play Mode via `MissionPlaytest` (captures after chosen events and stages; images in `Docs/Screenshots/Heist/`): completed in 125.5 s, terminal 4.8 s, uplink 20.9 s.

23 Heat and Lockdown (city reaction)

IN THIS CHAPTER

23.1 Heat: the consequence currency (Decision D22)
23.2 Security levels (Assets/_NeonRift/Scripts/Missions/SecurityLevel.cs)
23.3 What triggers what (verified wiring)
23.4 Systems and files
23.5 Drones in detail (SecurityDrones)
23.6 The theft beat
23.7 Failure

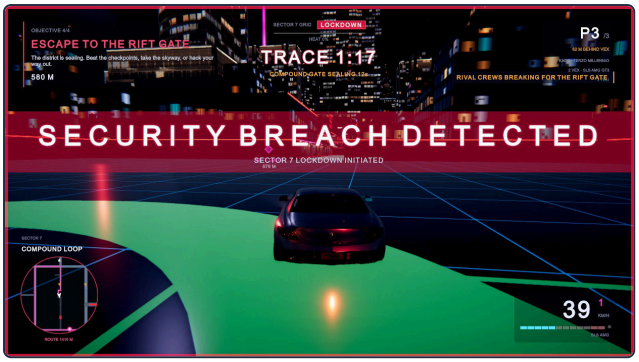
23.1 Heat: the consequence currency (Decision D22)

`MissionProgress.Heat` (0…1). One number, shown on the HUD, that couples **route choice before the theft** to **options after it**.

Source of heat	Amount	Code
Completing the alley gate hack (each time)	+35 %	`Interaction_HackGate` `heatOnComplete` 0.35
A missed timing press	+5 % (gate), +4 % (core terminal), +3 % (uplink)	`InteractionStep.missHeat`
Lockout after 3 misses	+12 % (gate), +10 % (terminal)	`InteractionDefinition` fail heat
Security camera holds the player in view for 1.1 s (once security is raised)	+8 %, 14 s cooldown	`SecurityCamera` (`range` 38 m, FOV 80°, line of sight)

What heat does (`MissionProgress.AddHeat`):

- **First heat raises security Calm → Alert.**
- **Timed objectives shrink.** The escape timer starts at max(min(30, 80), 80 − heat × 25). Heat gained **during** the escape removes time immediately.
- **Checkpoints seal sooner.** `SecurityBarrier` countdowns subtract `heatPenalty` × heat (minimum `minDelay`).
- The debrief's **Quiet** score is 25 × (1 − heat).



SECURITY BREACH DETECTED
`Docs/Screenshots/Look/breach.jpg`



Drones during the lockdown
`Docs/Screenshots/Life/drones_lockdown.png`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

23.2 Security levels (`Assets/_NeonRift/Scripts/Missions/SecurityLevel.cs`)

`Calm` → `Alert` → `Lockdown`. **Only ever escalates** during a mission. Alert comes from heat; Lockdown comes from the `escape` objective (security level Lockdown).

23.3 What triggers what (verified wiring)

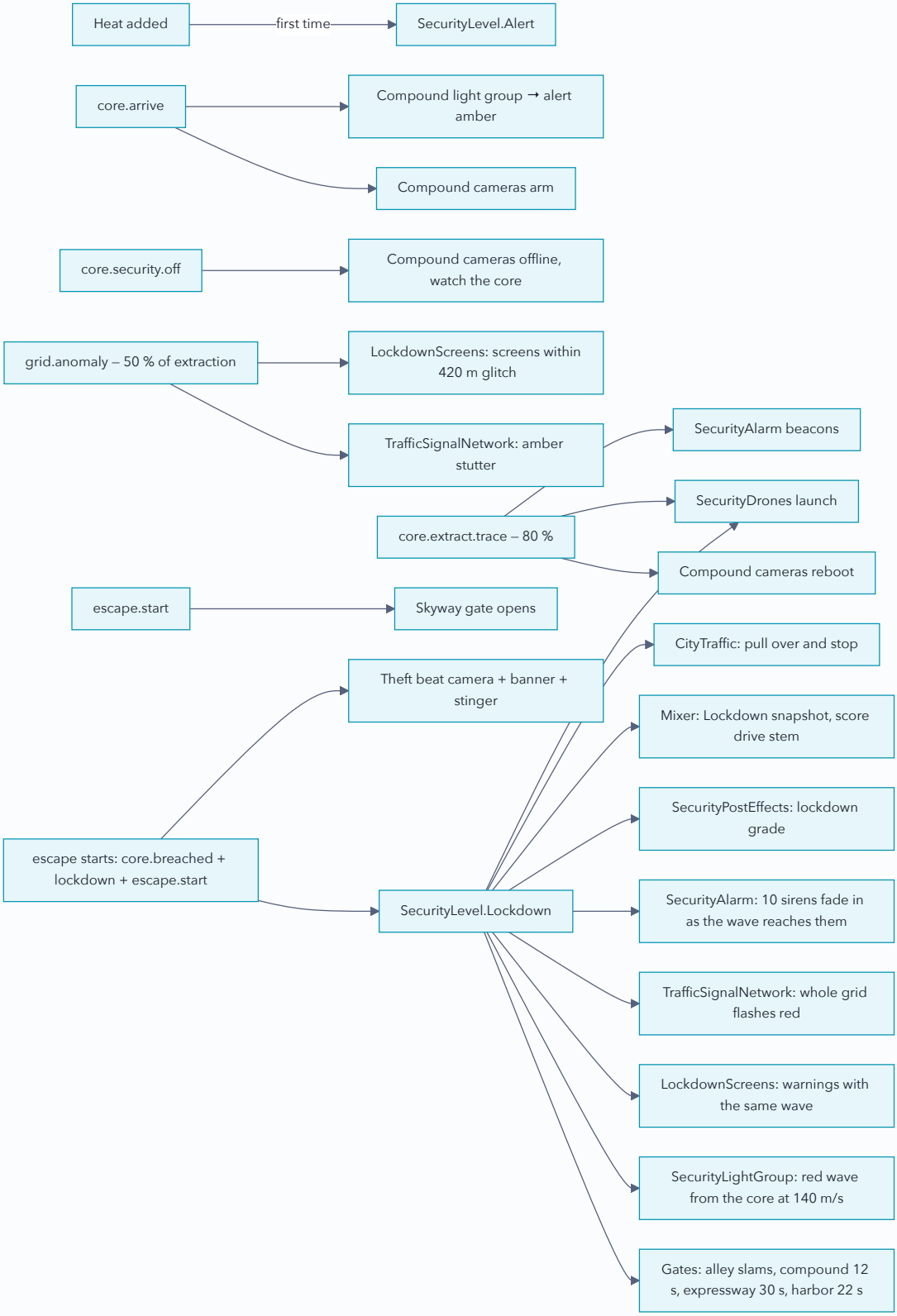


Figure 23.1 Heat and lockdown reactions

23.4 Systems and files

System	Calm	Lockdown	Class / file	Built by
Gates	Alley gate locked (hackable); skyway gate closed	Alley slams (1 s warning); **compound gate** seals in 12 s (−4 s per heat, min 7); **expressway checkpoint** 30 s (−12 per heat, min 12); **harbor checkpoint** 22 s (−8 per heat, min 10); **skyway gate opens** on `escape.start`	`SecurityBarrier` (`Scripts/Gameplay/Mission/SecurityBarrier.cs`)	`NightRunBuilder.Build` → `BuildBarrier` ×5
Road closures for AI and HUD	–	Closed gates impassable, closing gates +450 m	`CityNavigation.EdgeCost`	`BuildNavigation`
Security light strips (kerb lines, crowns, gantries; 35 groups)	Faint cyan	Red (3.6, 0.12, 0.14), switching after distance ÷ 140 m/s, pulsing	`SecurityLightGroup` (`Scripts/Gameplay/Mission/SecurityLightGroup.cs`)	`BuildSecurityGroups`
Warning screens (billboards, shelter adverts, street screens)	Adverts	Warning material, same wave; glitch on `grid.anomaly` within 420 m	`LockdownScreens` (`Scripts/Gameplay/City/LockdownScreens.cs`)	`BuildCitySystems`
Traffic signals	Green/amber/all-red cycle	Whole grid flashes red; amber stutter on `grid.anomaly`	`TrafficSignalNetwork` (`Scripts/Gameplay/City/TrafficSignalNetwork.cs`)	`BuildCitySystems`
Security cameras (12: alley, compound gate, expressway, tunnel, harbor checkpoint, skyway …)	Idle (green LED)	Track the player in range and line of sight (red LED); +8 % heat per 1.1 s exposure	`SecurityCamera` (`Scripts/Gameplay/City/SecurityCamera.cs`)	`BuildSecurityCameras`
Sirens and beacons	Silent, dark	10 sirens (gain 0.35) fade in after the wave reaches them; rotating beacons	`SecurityAlarm` (`Scripts/Gameplay/Mission/SecurityAlarm.cs`)	`BuildCityAlarm`
Lockdown grade	Base night volume	Alert volume at 0.25 weight; Lockdown volume blended over 1.2 s after 0.6 s	`SecurityPostEffects` (`Scripts/Gameplay/Mission/SecurityPostEffects.cs`) + `Settings/Rendering/NightRun_Alert.asset`, `NightRun_Lockdown.asset`	`BuildLighting`
Security drones (5 Mech Drones)	Dormant	Lift off from the core, then 3 tail the player and 1 tails each rival still running; red/blue strobes; searchlight held on the car	`SecurityDrones` (`Scripts/Gameplay/City/SecurityDrones.cs`)	`BuildDrones` (`NightRunBuilder.CityLife.cs`)
Traffic	Roaming	Pulls over and stops	`CityTraffic.OnSecurity`	`BuildTraffic`
HUD	Cyan route	Magenta route, closed gates red on the minimap, lockdown frame, trace timer, seal countdowns	`MissionHud`, `CityMinimap`	–
Audio	Gameplay mix	Lockdown snapshot, lockdown stinger and cue, score drive stem	`MissionDirector.OnSecurityChanged`, `MissionAudio`	–

23.5 Drones in detail (`SecurityDrones`)

- **Launch:** on `launchOn` events (`core.extract.trace`) or `SecurityLevel.Lockdown` (`Launch`).
- **Targets:** `TargetFor(i)`: the last drones each take one rival still running; the rest tail the player.
- **Flight:** a slot behind, beside and above the target (`trail` 9 m, `altitude` 15 m, staggered heights), leading the car by 0.45 s of velocity; climbs out of the core for the first 2.5 s; flies faster than the car (`speed` 30 m/s minimum); banks and pitches with its velocity.
- **Searchlight and beam:** aimed at the car with a 1.2 m hand-held wander, so the light on the car visibly comes from the drone. Strobes flash red and blue through a `MaterialPropertyBlock`.
- **Model:** `PF_Drone_Mech` (Mech Drone, CC BY; `Scripts/Editor/World/DroneModels.cs`).

23.6 The theft beat

On `core.breached` (`MissionDirector.beatEvent`), for 2.6 s: the camera swings 55° round and 3.4 m up on the far side of the car from the core (`VehicleChaseCamera.CinematicBeat`), letterbox bars slide in, the time scale eases to 0.45 and back, and the driving HUD steps back.

23.7 Failure

The escape timer runs out → `MissionProgress.Fail("TRACE COMPLETE — YOU WERE FOUND")`. The `TraceTimeout` validator scenario tests this (trace 71.3 s = 80 − 0.35 × 25 with 35 % heat).

24 Environmental Interactions

IN THIS CHAPTER

24.1 Not implemented

This chapter lists every way the **world itself** responds to the player, with the mechanism behind each. Everything goes through the **event-id model**: an `IMissionWorldComponent` binds to `MissionWorld` and reacts to `EventRaised` / `SecurityChanged` (19 Mission System).

Interaction	Player action	World response	Component	File
Mission zones	Drive into a trigger	Objective completes, beacon hides	`MissionZone` (Trigger layer, collides with vehicles only)	`Scripts/Gameplay/Mission/MissionZone.cs`
Terminals and uplink	Stop in the zone, work the stages with E	Screens, holograms, machinery, events	`Interactable`, `TerminalDisplay`, `DataCoreChamber`	`Scripts/Gameplay/Mission/`
Hackable gate	Hack the alley controller	Bolts withdraw, panel slides open; re-hack in a lockdown	`Interactable` + `GateLockSystem` + `SecurityBarrier`	21
Sealing gates	Steal the core	Countdown, strobes and klaxon, panels slide (they **push** cars)	`SecurityBarrier`	`Scripts/Gameplay/Mission/SecurityBarrier.cs`
Security cameras	Stay in a camera's cone with line of sight	Head tracks the car, LED red, heat after 1.1 s	`SecurityCamera`	`Scripts/Gameplay/City/SecurityCamera.cs`
Traffic	Drive near or into cars	Cars give way at junctions; hits shunt, spin and stop them	`CityTraffic`, `TrafficCar`	`Scripts/Gameplay/City/`
Parked cars	Hit them	They wake (kinematic → dynamic) and are shoved	`TrafficCar`	same
Pedestrian groups	Drive within 16 m	The group steps out of sight (never stands in a car's way)	`CrowdGroups`	`Scripts/Gameplay/City/CrowdGroups.cs`
Street light budget	Move through the city	The 56 nearest lamps (front-weighted) fade on, 2 with shadows	`LightBudget`	`Scripts/Gameplay/City/LightBudget.cs`
Signals and screens	Extraction progress, lockdown	Amber stutter, red flashing, warning wave	`TrafficSignalNetwork`, `LockdownScreens`	`Scripts/Gameplay/City/`
Drones	Counter-intrusion or lockdown	Pursuit with searchlights	`SecurityDrones`	`Scripts/Gameplay/City/SecurityDrones.cs`

Interaction	Player action	World response	Component	File
Crew garage	(intro only)	Light banks strike, roller door lifts	`CrewGarage`	`Scripts/Gameplay/CrewGarage.cs`
Out of bounds	Fall below −30 m	Car returns to the spawn point	`MissionSceneEntry.FixedUpdate`	`Scripts/Gameplay/MissionSceneEntry.cs`
Recover	Press R	Car is righted on the ground beneath it	`VehicleController.Recover`	`Scripts/Vehicles/Physics/VehicleController.cs:141`

24.1 Not implemented

- **Destructible environment:** props and buildings are static colliders; nothing breaks.
- **Vehicle damage:** none (see 13).
- **Pedestrian collision or animation:** crowd figures are static low-poly silhouettes with a slow sway, and they have no colliders.

PART VI

Rendering and Effects

Lighting, atmosphere, surfaces and effects.

25 Lighting

26 Weather and Atmosphere

27 Water and Effects

28 Particles and VFX

25 Lighting

IN THIS CHAPTER

25.1 How the night-city look is produced
25.2 Components of the look
25.3 Post-processing (URP Volumes)
25.4 Fog and haze
25.5 Performance trade-offs (measured; Docs/Environment.md, Docs/Performance.md)
25.6 Tools

25.1 How the night-city look is produced

The look is **emissive-first**. Windows, neon, signs, lamp heads and ground pools are emissive materials that bloom, and **real-time lights are budgeted** to the ones near the camera. There is **no global illumination**: `NightRun_Lighting.lighting` has baked GI and realtime GI both off (`NightRunBuilder.LightingSettingsAsset`: "Real-time lights + baked reflection probes for now; baked GI comes with the mobile lighting pass"). Wet-looking roads come from **baked box-projected reflection probes**.

All of it is set up by `NightRunBuilder.BuildLighting` (`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.cs:668`) and `ConfigurePipeline` (1143).

25.2 Components of the look

Element	Implementation	Where
Pipeline	URP Forward+, HDR, 4× MSAA (+ SMAA High on the camera), SSAO renderer feature, shadow distance 90 m, **2 cascades**, soft shadows quality **Low** (from the GPU bench, −37 % GPU), SRP Batcher, GPU Resident Drawer	`Assets/_NeonRift/Settings/Rendering/PC_RPAsset.asset`, `PC_Renderer.asset`; `NightRunBuilder.ConfigurePipeline`
Sky	`NeonRift/NightSky` shader: navy zenith, low sodium-brown light-pollution band, two-scale overcast lit from below, moon direction	`Art/Shaders/NeonRift_NightSky.shader`, `RenderSettings.skybox`
Moon	One directional light, intensity 0.16, soft shadows, rotation (34°, −35°)	`BuildLighting`
Ambient	Trilight: sky (0.04, 0.044, 0.06), equator (0.05, 0.042, 0.038), ground (0.02, 0.016, 0.012): cool sky, warm street bounce	`BuildLighting`
Street lights	787 street and flood lamps (600–680 cd, 32 m range), each with a **disabled** real-time spot; `LightBudget` keeps the **56 nearest within 150 m** on (front-weighted, `behindPenalty` 2.2, re-evaluated every 0.2 s, 0.6 s fades) and gives the **2 nearest in front** soft shadows (`maxShadowed`, with hysteresis)	`Scripts/Gameplay/City/LightBudget.cs`; prefabs `PF_Prop_StreetLight_{Led,Sodium,Warm}`

Element	Implementation	Where
Lamp kinds	LED (~4000 K neutral), Sodium, Warm (`DistrictKit.LampKind`, `LampColours`)	`Scripts/Editor/District/DistrictKit.cs`
Distant lamps	Emissive heads, ground pools and haze cones (`CityProps.LampHaze`), so streets read without real lights	`CityProps.cs`
Neon	Emissive materials up to about 2.8 HDR in cyan, magenta, warm white, amber, red (violet and lime removed)	`DistrictKit.CreateMaterials`; textures `DistrictTextures`
Towers	`NeonRift/TowerFacade`: procedural lit office windows by suite, lobby band, uplight; crown strips and washes; aviation lights	`Art/Shaders/NeonRift_TowerFacade.shader`, `TowerDressing.cs`
Shopfronts	Interiors lit like shops (warm white, fluorescent, halogen); colour stays in the sign bands	`DistrictTextures.Shopfronts`
Reflection probes	63 baked, box-projected (128 px in Sector 7 and Spire, 64 px elsewhere) along arterials and Sector 7 streets; the crew garage has its own	`NightRunDistrict.BuildReflectionProbes`, `NightRunBuilder.BakeProbes`; EXRs in `Assets/_NeonRift/Scenes/NightRun/`
Vehicle lights	Headlights (the player's left one shadowed), tail spot (brake/reverse)	`Scripts/Vehicles/VehicleLights.cs`
Mission lights	Core floods (`Flood_NW`, `Flood_SE`), tunnel, gate and beacon lights: mostly off until needed	`NightRunBuilder.BuildCompound`, `Light()` helper
Emergency lighting	Security strips turning red with the lockdown wave; rotating beacons and sirens; drone red/blue strobes and searchlights	`SecurityLightGroup`, `SecurityAlarm`, `SecurityDrones`
Garage	19 hall lights (intro only) struck in banks by `CrewGarage`; the Car Select hall has its own key light and rims	`NightRunBuilder.Garage.cs`, `GarageBuilder.cs`

25.3 Post-processing (URP Volumes)

Volume	File	Contents (from `BuildLighting` and `Docs/Environment.md`)
`NightRun_Base`	`Settings/Rendering/NightRun_Base.asset`	Bloom threshold 1.05 / intensity 0.34 / high-quality filtering; ACES tonemapping; **fixed post-exposure** (URP has no eye adaptation); contrast 14, saturation −8, white balance −4; split toning (neutral-cool shadows, warm highlights); lift/gamma/gain; vignette 0.22; film grain 0.08; `GroundHaze`
`NightRun_Alert`	`NightRun_Alert.asset`	Alert tint, blended at weight 0.25
`NightRun_Lockdown`	`NightRun_Lockdown.asset`	Colour filter (1, 0.93, 0.92), contrast 22, dark red vignette, bloom 0.55, chromatic aberration 0.07, darker haze. "The red comes from the security lights themselves, not the grade"
`Garage_Volume`	`Garage_Volume.asset`	Car Select look
Intro surveillance	`Data/Intro/Intro_Surveillance.asset`	Green CCTV grade for the intro's surveillance shot

Blending: `SecurityPostEffects` (`Scripts/Gameplay/Mission/SecurityPostEffects.cs`). **Stop NaN** is enabled on scene cameras as a safeguard (Decision D13).

25.4 Fog and haze

Exponential-squared distance fog (density 0.0011, in the colour of the horizon glow), plus the custom **ground haze**. See 26.

25.5 Performance trade-offs (measured; `Docs/Environment.md`, `Docs/Performance.md`)

- Each shadowed lamp re-renders the combined block meshes inside its cone: 2 shadowed lamps add about 0.45 M triangles and 200 shadow casters over 0. `maxShadowed` is the dial (desktop 2, mobile 0).
- Moon shadow cascades 4 → 2 cut 4.53 M → 3.13 M triangles at the spawn view.
- Soft-shadow quality High → Low: −37 % GPU in the lockdown escape, with no visible difference (mean pixel difference 0.48/255).
- MSAA 4× and SSAO were kept for the look (2× saved 7–12 %, SSAO off saved 11 %).
- Real-time lights peak at about 70 in the lockdown (beacons, strobes, drone searchlights) on top of the lamp budget.

25.6 Tools

`LookDevShots` (**Neon Rift ▸ Night Run ▸ Capture Look-Dev Shots (Play Mode)**) and `ViewCapture` (`Scripts/Editor/District/ViewCapture.cs`) for before/after captures; `GpuBench` for A/B GPU measurements (39).

26 Weather and Atmosphere

IN THIS CHAPTER

26.1 Summary
26.2 Distance fog
26.3 Ground haze (NeonRift.Rendering)
26.4 Sky and clouds
26.5 Skyline haze
26.6 Steam
26.7 Light shafts and beams
26.8 Related

26.1 Summary

Effect	Status
Rain (falling)	**Not currently implemented in the completed version.** There is no rain system, particle rain or rain audio
Weather states, wind, lightning, snow	**Not implemented**
Distance fog	Implemented (URP / built-in `RenderSettings` fog)
Ground-hugging height haze	Implemented (custom Volume override + Render Graph pass)
"After the rain" wet look	Implemented as **materials and textures only** (damp asphalt patches, glassy puddles, reflections). No wetness simulation
Steam from manholes	Implemented as static soft plume meshes
Night sky with clouds and moon	Implemented as a sky shader

26.2 Distance fog

`NightRunBuilder.BuildLighting` (`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.cs:668`): `RenderSettings.fog = true`, mode **ExponentialSquared**, density **0.0011**, coloured from the horizon light-pollution glow so distance dissolves into the sky (the older purple fog made distant towers black cut-outs).

26.3 Ground haze (`NeonRift.Rendering`)

WHAT	Exponential **height fog** layered on top of distance fog

CODE	`Assets/_NeonRift/Scripts/Rendering/GroundHaze.cs` (a `VolumeComponent` override, so profiles can blend it), `Scripts/Rendering/GroundHazeFeature.cs` (`ScriptableRendererFeature` with a Render Graph pass `HazePass`, `RecordRenderGraph`)
SHADER	`Assets/_NeonRift/Art/Shaders/NeonRift_GroundHaze.shader`
HOW	After opaques and the sky, before transparents: one full-screen triangle reads the camera depth texture and integrates exponential height fog **analytically** along each view ray, then alpha-blends over the colour target (no extra colour copy)
SETTINGS	Clear for the first 40 m; density 0.006/m at street level, falling by e every 22 m of height; light-pollution colour (fog × 1.35), capped at 78 % opacity so silhouettes never vanish. The lockdown profile tints it red and darker
REGISTERED	On both `PC_Renderer.asset` and `Mobile_Renderer.asset` (`NightRunBuilder.EnsureHazeFeature`)

26.4 Sky and clouds

`NeonRift/NightSky` (`Art/Shaders/NeonRift_NightSky.shader`): deep navy zenith, a low sodium-brown light-pollution band, a two-scale overcast lit from below, and moonlit thin cloud edges. Material `c.Kit.NightSky`, assigned as `RenderSettings.skybox`; ground colour and moon direction set by `BuildLighting`.

26.5 Skyline haze

`SkylineBackdrop` rings (1.7, 2.1, 2.55 km) are each hazier than the one in front (`_HazeColor` = fog colour), drawn unfogged by `NeonRift/SkylineBackdrop` so the far city keeps its silhouettes.

26.6 Steam

Steam vents: 30 % of Kowloon and Lowtown manholes (25 total) get two crossed soft plume quads with the additive `District_Steam` material (`Docs/Environment.md`, city detail pass). Static geometry, not particles.

26.7 Light shafts and beams

The drone searchlight beams and street-lamp haze cones are additive meshes (`NeonRift/AdditiveGlow`), not volumetric lighting. See 28.

26.8 Related

25 Lighting · 27 Water and Effects

27 Water and Effects

IN THIS CHAPTER

27.1 Water
27.2 Other effects

27.1 Water

Feature	Status
Water bodies (harbour water, rivers, water surfaces, water shader)	**Not currently implemented in the completed version.** No water surface or water shader exists in `Assets/_NeonRift/` (the Unity built-in "Water" layer is unused)
Splashes, wet tyre spray, rain ripples	**Not implemented**
Puddles on the road	**Implemented as texture and material detail only**
Fountain basin in courtyards	Static lit geometry (`CityInfill.Courtyard`), no simulated water

Wet asphalt and puddles: how it is done

`DistrictTextures.Asphalt` (`Assets/_NeonRift/Scripts/Editor/District/DistrictTextures.cs:128`) bakes the road texture procedurally. Damp patches come from low-frequency noise, and "**after rain**" **puddles** are low-frequency blobs over a threshold with soft edges (comment at line 141: "*After rain: puddles (low-frequency blobs over a threshold, soft edges) inside damp patches*"). The puddle and damp areas get **higher smoothness** (about 0.6, more glassy in puddles), so the **baked box-projected reflection probes** and real-time lamps streak lamp and sign highlights across the road. That is what reads as "wet neon streets" in the screenshots. It is a static material look; nothing dries, ripples or splashes.

27.2 Other effects

Effect	Implementation	File
Neon glow and bloom	Emissive materials + URP Bloom (threshold 1.05)	`DistrictKit` , `NightRun_Base.asset`
Light shafts, beacon columns, lamp haze cones	`NeonRift/AdditiveGlow` : additive, unlit, fog-aware glow shaped by a texture's alpha	`Assets/_NeonRift/Art/Shaders/NeonRift_AdditiveGlow.shader`

Effect	Implementation	File
Objective beacons (cyan), rival work beacons (magenta)	420 m-tall `Beacon` columns (`NightRunBuilder.Beacon`), shown only while relevant	`District_BeaconCyan`, `District_BeaconMagenta`
Terminal screens and holograms	`NeonRift/TerminalScreen`: stage strip, scrolling rows, progress bar, scan line, tearing on misses and interference, via `MaterialPropertyBlock`	`Art/Shaders/NeonRift_TerminalScreen.shader`, `Scripts/Gameplay/Mission/TerminalDisplay.cs`
Data Core pulses, scan rings, holograms, strobes	Script-animated meshes and emission	`Scripts/Gameplay/Mission/DataCoreChamber.cs`
Lockdown wave	Emission colour change by distance from the theft	`SecurityLightGroup`, `LockdownScreens`
Steam vents	Crossed soft plume quads (`District_Steam`)	`CityProps` / `DistrictKit`
Drone searchlight beams	Cone mesh `Drone_Beam` scaled to the target each frame	`SecurityDrones`, `IntroCityLife`
Mech Drone thruster flames	Additive glow material `Drone_Mech_Flame` (the model's Fire_Alpha texture)	`Scripts/Editor/World/DroneModels.cs`
Camera shake	Impact and speed shake	`VehicleChaseCamera`
Intro flashes, fades, letterbox, CCTV overlay	UI Toolkit overlay	`Scripts/Intro/IntroOverlay.cs`

See also 28 Particles and VFX.

28 Particles and VFX

IN THIS CHAPTER

28.1 Particle systems
28.2 How the visual effects are made instead
28.3 Not implemented

28.1 Particle systems

There are no Unity `ParticleSystem` components in the completed project. This was checked by counting component types in every scene file (`Bootstrap`, `Frontend`, `CarSelect`, `NightRun`, `Dev/TestTrack`) and every prefab under `Assets/_NeonRift/Prefabs/`, and by searching all scripts for `ParticleSystem`. The VFX Graph package is not installed.

> An older note in `Docs/NightRun.md` ("4 small particle systems") describes the vertical-slice build. The current generated scene contains none.

28.2 How the visual effects are made instead

All effects are **geometry + custom shaders + script-driven material properties**, which keeps them deterministic and cheap.

Category	Technique	Code / asset
Glows, light shafts, beams, haze cones, steam	Additive unlit quads and cones with `NeonRift/AdditiveGlow` (texture alpha shapes the glow; fog fades it out)	`Assets/_NeonRift/Art/Shaders/NeonRift_AdditiveGlow.shader`; materials from `DistrictKit.Glow` (`Scripts/Editor/District/DistrictKit.cs:204`)
Screens, holograms	`NeonRift/TerminalScreen` driven per renderer with `MaterialPropertyBlock`	`NeonRift_TerminalScreen.shader`, `TerminalDisplay.cs`
Strobes, LEDs, beacons, signals	Emission colour set through property blocks or shared runtime materials	`SecurityDrones`, `SecurityCamera`, `SecurityAlarm`, `TrafficSignalNetwork`
Lockdown spread	Per-group delayed colour fades (140 m/s wave)	`SecurityLightGroup`, `LockdownScreens`
Data Core extraction	Pulses moved along a cable path, scan rings, colour ramps, flashes on interference	`DataCoreChamber.UpdateLink`, `UpdateCore`, `UpdateHolograms`
Atmosphere	Full-screen Render Graph pass	`GroundHazeFeature` + `NeonRift_GroundHaze.shader`

Category	Technique	Code / asset
Post FX	Bloom, ACES, split toning, vignette, grain, chromatic aberration (lockdown)	URP Volumes
Screen-space UI effects	Flashes, fades, letterbox, banners, toasts	`IntroOverlay`, `MissionHud` (USS transitions)

28.3 Not implemented

Tyre smoke, sparks on impact, exhaust smoke, debris, rain and splashes.

PART VII

Audio and Music

The mixer, sound sources and the score.

29 Audio System

30 Music System

29

Audio System

IN THIS CHAPTER

29.1 Architecture
29.2 Mixer (Assets/_NeonRift/Audio/Mixer/NeonRiftMixer.mixer)
29.3 Voices, priorities and headroom
29.4 Mission audio (MissionAudio + MissionAudioSet)
29.5 Generated sound library (all synthesised in-house)
29.6 Where each sound comes from
29.7 M music toggle
29.8 Tests and tools

29.1 Architecture

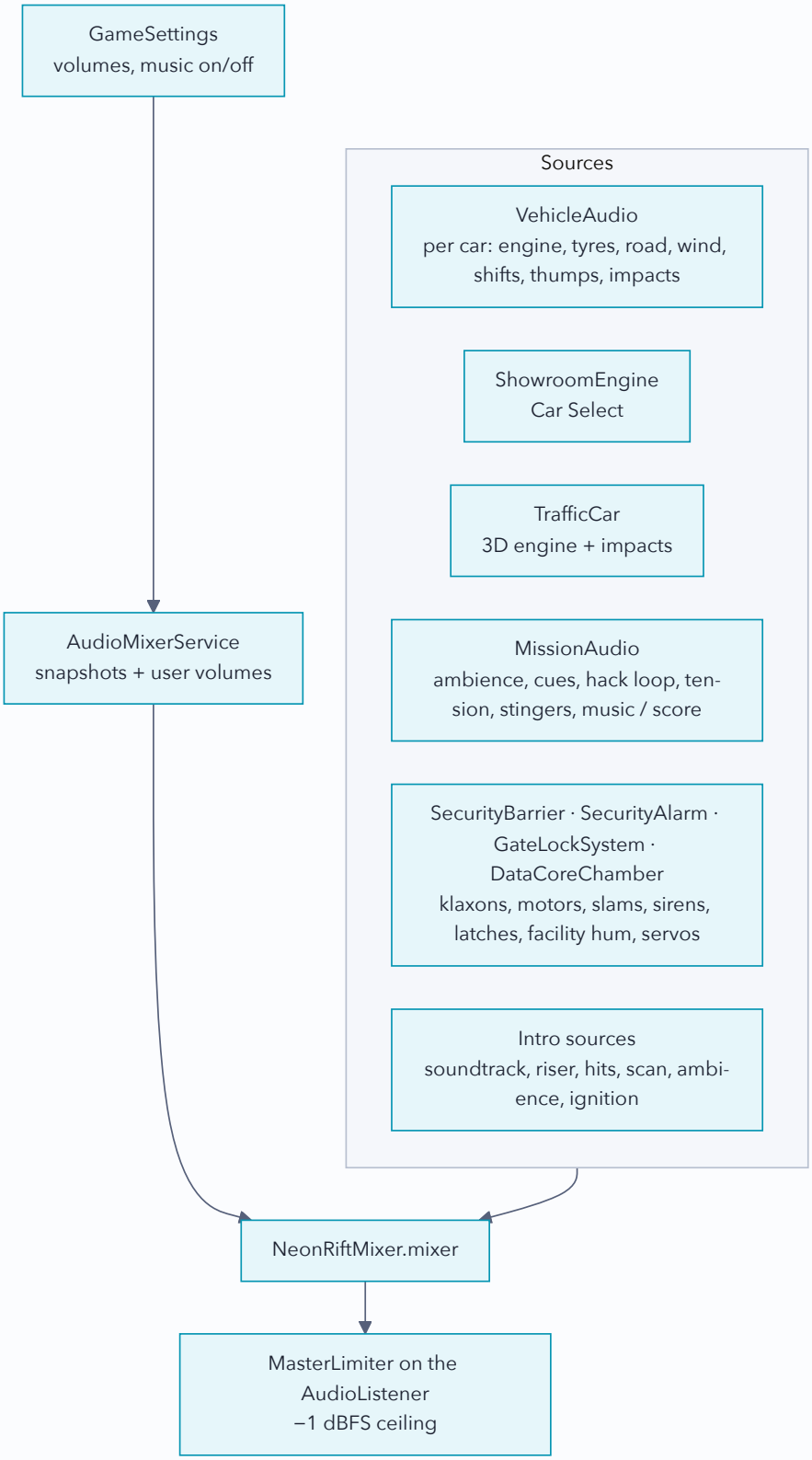


Figure 29.1 Audio architecture

29.2 Mixer (`Assets/_NeonRift/Audio/Mixer/NeonRiftMixer.mixer`)

Built by `AudioMixerBuilder` (`Assets/_NeonRift/Scripts/Editor/Audio/AudioMixerBuilder.cs`, menu **Neon Rift ▸ Audio ▸ Build Audio Mixer**). Unity has no public mixer-authoring API, so it drives the editor's mixer controller through reflection (Decision D20). It is described at runtime by `AudioMixerConfig` (`Scripts/Audio/AudioMixerConfig.cs`, asset `Data/Audio/AudioMixerConfig.asset`).

```
Master            [MasterVolume]
├─ Gameplay       [EffectsVolume]
│  ├─ Engine
│  ├─ Tires
│  ├─ SFX
│  └─ Ambience
├─ Music          [MusicVolume]
│  └─ Score
└─ UI             [UIVolume]
```

User volumes are exposed parameters on **parent** groups. State **snapshots** act on **leaf** groups, so the two never conflict.

Snapshot (`MixerState`)	Use	Offsets (dB, `Docs/Audio.md`)
Gameplay	Driving, intro	Tires −3, Ambience −4
Menu	Title, Car Select	Engine −12, Tires −20, SFX −6, Ambience −15
Results	Debrief	Engine −18, Tires −25, SFX −8, Ambience −12
Lockdown	City lockdown	Engine −2, Ambience −4
Ducked	Pause menu	Engine −8, Tires −8, Ambience −8, Score −10

`AudioMixerService` (`Scripts/Audio/AudioMixerService.cs`): `TransitionTo(state, seconds)`, `SetVolume` / `GetVolume`, dB↔linear helpers. Created by `GameRoot`; the mixer runs on **unscaled time** so pause ducking fades while time is frozen. `GameRoot.OnStateChanged` picks Gameplay vs Menu; `MissionDirector` switches to Lockdown and Results; `MissionSceneEntry.OpenPause` uses Ducked.

29.3 Voices, priorities and headroom

- Real voices **64** (`ProjectSettings/AudioManager.asset`), up from 32: with three cars at about 22 loops each, the score and ambience used to be virtualised (measured about 35 dB down).
- Priorities: score and outro 0; mission cues and stingers 4; ambience 8; tension 16; interaction loop 24; gate klaxon/motor 96; sirens 112; non-camera cars +140 (`VehicleAudio.OtherCarPriorityOffset`).
- **`MasterLimiter`** (`Scripts/Audio/MasterLimiter.cs`) on the listener (NightRun and CarSelect scenes): −1 dBFS ceiling, instant attack on each frame's peak, 150 ms release. A safety net, not the mix.
- Device change: `MissionAudio` and `VehicleAudio` listen to `AudioSettings.OnAudioConfigurationChanged`, restart loops and re-sync the score.

29.4 Mission audio (`MissionAudio` + `MissionAudioSet`)

Player	`Scripts/Gameplay/Mission/MissionAudio.cs`: 2D sources created once; loops started once; `Play(Cue)`, `SetTension`, `SetInteract`, `StartScore`, `SetScoreSecurity`, `EndScore`, `OnWorldEvent` (stingers)
Data	`Scripts/Gameplay/Mission/MissionAudioSet.cs`, asset `Data/Missions/MissionAudio_NightRun.asset`: ambience bed (0.55), interaction loop (0.6), cues (objective, interact complete, heat, lockdown, countdown tick, message, success, failure), heist stage cues (step start/done, miss, interference, resync, lockout, cancel), tension loop (0.7), `eventStingers`, `musicTrack` (0.30), score stems and outros
Routing	Ambience → Ambience group; cues and interaction loop → UI/SFX; music → Score

29.5 Generated sound library (all synthesised in-house)

Generator	Menu	Output (`Assets/_NeonRift/Audio/Generated/`)
`VehicleAudioGenerator` + `EngineSynth` + `Dsp`	Neon Rift ▸ Audio ▸ Generate Vehicle Audio	`Engine_V8Road`, `Engine_V8Race`, `Engine_EV`, `Engine_Pops`, `Tyres`, `Road`, `Mechanical`, `Impacts`
`MissionAudioGenerator`	Neon Rift ▸ Audio ▸ Generate Mission Audio (also run by the Night Run build)	`Mission/`: `Siren_Loop`, `Klaxon_Loop`, `BarrierMotor_Loop`, `Barrier_Slam`, `City_AmbienceLoop`, `Hack_Loop`, `Cue_*`, `Gate_LockRelease`, `Gate_Latch`
`HeistAudioGenerator`	(called by the mission generator)	`Mission/Heist_*`: StepStart, StepDone, Denied, Interference, Resync, Lockout, Cancel, Tension_Loop, FacilityHum_Loop, Servo_Loop, DataStream_Loop, ChamberOpen, Dock, Release, DataAcquired, Breach
`ScoreComposer`	(called by the mission generator)	`Mission/Score_Bed`, `Score_Pulse`, `Score_Arp`, `Score_Drive`, `Score_OutroSuccess`, `Score_OutroFailure`
`IntroAudioGenerator`	Neon Rift ▸ Intro ▸ Generate Intro Audio	`Intro/`: `Intro_Bed`, `Intro_Groove`, `Intro_Riser`, `Intro_Impact`, `Intro_Whoosh`, `Intro_Scan`, `Intro_Ignition`

All are deterministic and re-runnable. Loops contain whole numbers of periods (seamless). The only third-party audio is the two supplied music tracks in `Assets/ThirdParty/SoundAudio/` (30).

29.6 Where each sound comes from

Sound	Component
Engine, gear, tyre squeal and scrub, loose surface, road, wind, suspension thumps, impacts	`VehicleAudio` (12)
Traffic engines and impacts	`TrafficCar`
Sirens	`SecurityAlarm` (10 positions in `NightRunBuilder.BuildCityAlarm`)
Gate klaxon, motor, slam	`SecurityBarrier`
Lock release ratchet, latch	`GateLockSystem`

Sound	Component
Facility hum, servos, data stream, chamber hydraulics, dock, release	`DataCoreChamber` (3D)
Terminal stage blips	`TerminalDisplay` (3D speaker)
Hacking loop, tension bed, HUD cues, stingers, ambience, music	`MissionAudio`
Garage light clunks, door motor	`CrewGarage`

29.7 M music toggle

`GameRoot` binds **M** / gamepad **Y** to `GameSettings.ToggleMusic()` (`Scripts/Game/GameSettings.cs`). This sets the Music bus to −80 dB or back to the music volume, and the choice is saved in PlayerPrefs. The HUD shows `M · MUSIC ON/OFF` (`MissionHud.SetMusicHint`).

29.8 Tests and tools

`VehicleAudioTests` (mixer groups, snapshots and volumes; score stems sample-locked; `ScoreMix` follows mission state and lands on the grid; `MasterLimiter` holds the ceiling); `AudioOutputRecorder` and `AudioSignalAnalysis` (`Scripts/Audio/`) for recording and measuring the real mix; recorded excerpts in `Docs/Audio/*.wav`.

Not checked by ear: the score balance and the intro mix were measured but have not been judged on speakers by a person (`Docs/Audio.md`).

30 Music System

IN THIS CHAPTER

30.1 Two layers
30.2 Supplied tracks
30.3 Procedural adaptive score (ScoreComposer + ScoreMix)
30.4 Intro music fallback
30.5 Music on/off
30.6 Tests

30.1 Two layers

1. **Supplied music tracks** (used by default): `Assets/ThirdParty/SoundAudio/`.
2. **A procedural adaptive score** (original, generated by `ScoreComposer`): the **fallback** when no track is assigned, and the source of the intro's procedural fallback theme.

30.2 Supplied tracks

Track	Length	Use	Where it is wired
`machine-speed-sport-aggressive-electronic.mp3`	111 s	**Gameplay** background music, looping on the Score group at volume **0.30**, streamed. Fades in at the start and out at the end when the outro plays. Ignores the listener pause, so it keeps playing (ducked) under the pause menu	`MissionAudioSet.musicTrack` in `Assets/_NeonRift/Data/Missions/MissionAudio_NightRun.asset` (set by `MissionAudioSet.EditorConfigureMusic`)
`danger-chaos-cinematic-trailer-hybrid.mp3`	102 s	**Opening cinematic** soundtrack: Timeline track "Soundtrack", 0–84 s, 0.4 s fade-in, 2.5 s fade-out into Car Select	`IntroBuilder.BuildTimeline` (`Assets/_NeonRift/Scripts/Editor/Intro/IntroBuilder.cs:322`)

Licence for both: **not recorded in the repository** (no licence file; `Docs/ThirdParty.md` asks for the source and licence to be recorded). See 44.

30.3 Procedural adaptive score (`ScoreComposer` + `ScoreMix`)

Composer	`Scripts/Editor/Audio/ScoreComposer.cs`: one 8-bar theme in **A minor at about 96 bpm** (Am(add9) – Fmaj7 – Dm9 – Esus4 → E, two bars each), rendered as four stems of exactly the same length (882,048 samples, a whole number of 128-sample ADPCM blocks, so the loop never gains a gap). Tempo derived from that length: 95.995 bpm

Stems	**Bed** (detuned-saw pad + sub), **Pulse** (soft kick, sidechained eighth-note bass, hats), **Arp** (sixteenth plucks, dotted-eighth echo), **Drive** (four-on-the-floor, claps, rolling bass, alarm stab, tom fills); outros **success** (resolves to A major with a bell arpeggio) and **failure** (low A/Bb/E cluster sinking a semitone)
Mixing rules (pure)	`Scripts/Missions/ScoreMix.cs`: `ScoreLayer`, `Targets(security, tension)`, `Step`, `NextGridTime`
Runtime	`MissionAudio.StartScore`: all stems start on one scheduled DSP tick (`PlayScheduled`) so they stay sample-locked. Layers rise 0.8/s and fall 0.4/s; the drive stem slams in at 4/s. Security changes land on the **next half bar** (`SetScoreSecurity`), so the lockdown hits on the grid with the stinger. `EndScore` fades over 1.2 s and plays the outro on the next beat

Stem	Calm	Heist work (× tension)	Alert	Lockdown
Bed	0.80	0.80	0.55	0.20
Pulse	0.45	→ 0.22	0.85	0.55
Arp	0	→ 0.80	0.35 → 0.85	0.25 × tension
Drive	0	0	0	1.00

When `MissionAudioSet.musicTrack` is assigned (as it is now), the supplied track **replaces** the stems.

30.4 Intro music fallback

If the trailer track is missing, `IntroBuilder` falls back to the game's own theme: `Intro_Bed` / `Intro_Groove` from `IntroAudioGenerator` (`Scripts/Editor/Audio/IntroAudioGenerator.cs`). The groove joins at the bed's position in the loop, so both stay on the same bar and chord.

30.5 Music on/off

M / gamepad **Y** anywhere (`GameRoot`, `GameSettings.ToggleMusic`); also in Settings (master, music, effects and interface volumes plus a music on/off switch). See 29.

30.6 Tests

`VehicleAudioTests.Score_StemsAreSampleLockedSeamlessLoops`, `ScoreMix_FollowsTheMissionState`, `ScoreMix_ChangesLandOnTheGrid` (`Assets/_NeonRift/Scripts/Tests/EditMode/VehicleAudioTests.cs`).

PART VIII

Cinematics

The real-time intro, its Timeline and cameras.

31 Cinematic Intro

32 Timeline and Camera

31
Cinematic Intro

IN THIS CHAPTER

31.1 Purpose
31.2 The parts
31.3 How it runs
31.4 Timeline structure (IntroBuilder.BuildTimeline, line 322)
31.5 Cue timeline (IntroBuilder.Cues, line 293)
31.6 Shot-by-shot breakdown (IntroBuilder.Shots, line 247)
31.7 Generated / runtime elements
31.8 Validation

31.1 Purpose

An **84 s real-time cinematic**, rendered in the actual Night Run city, that tells the premise without dialogue: a crew in a hidden garage on W Avenue goes after the Sector 7 Data Core. It ends on the NEON RIFT / NIGHT RUN title, looking into the same garage that Car Select uses, so pressing Enter becomes a **match cut** into Car Select. All story, cards and procedural sound are original.



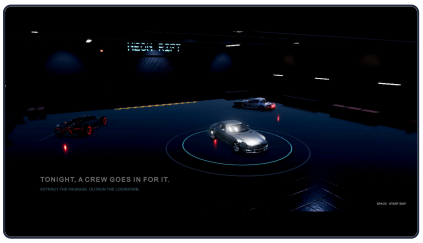
Shot 1 · 01_City (3.0 s)
`Docs/Screenshots/Intro2/01_03.0s.jpg`



Shot 8 · 05b_Core (37.0 s)
`Docs/Screenshots/Intro2/05_37.0s.jpg`



Shot 9 · 06_Surveillance (41.0 s)
`Docs/Screenshots/Intro2/06_41.0s.jpg`



Shot 11 · 07b_Reveal (48.5 s)
`Docs/Screenshots/Intro2/07_48.5s.jpg`



Shot 12 · 08a_Headlight (53.5 s)
`Docs/Screenshots/Intro2/08_53.5s.jpg`



Shot 25 · 12_Title (80.5 s)
`Docs/Screenshots/Intro2/13_80.5s.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

31.2 The parts

Part	Class / asset	File
Builder (generates the rig and the Timeline)	`IntroBuilder` (+ `IntroBuilder.CityLife.cs`)	`Assets/_NeonRift/Scripts/Editor/Intro/IntroBuilder.cs`
Runtime entry point (`IIntroEntryPoint`)	`IntroSceneEntry`	`Scripts/Intro/IntroSceneEntry.cs`
Timeline asset	`Intro_NightRun.playable`	`Assets/_NeonRift/Data/Intro/Intro_NightRun.playable`
Custom cue track	`IntroCueTrack`, `IntroCueClip`, `IntroCueBehaviour`, `IntroCueKind`, `IIntroCueHandler`	`Scripts/Intro/IntroCueTrack.cs`, `IntroCueClip.cs`, `IntroCue.cs`
2D overlay (fades, flashes, letterbox, cards, title, skip hint, CCTV frame)	`IntroOverlay` + `UI/IntroOverlay.uxml`, `IntroOverlay.uss`	`Scripts/Intro/IntroOverlay.cs`
Street and sky life	`IntroCityLife`	`Scripts/Intro/IntroCityLife.cs`
Garage lights and door	`CrewGarage`	`Scripts/Gameplay/CrewGarage.cs`
Settings	`IntroSettings` (`Policy` FirstLaunchOnly / Always / Never, `Mode` Realtime / Video, `Skippable`, seen key `NeonRift.IntroSeen`)	`Scripts/Game/IntroSettings.cs`, `Data/Config/IntroSettings.asset`
Surveillance grade	Volume profile	`Data/Intro/Intro_Surveillance.asset`
Audio	`IntroAudioGenerator` + the trailer track	`Scripts/Editor/Audio/IntroAudioGenerator.cs`, `Assets/ThirdParty/SoundAudio/danger-chaos-cinematic-trailer-hybrid.mp3`
Review tool	`IntroCapture.Begin(folder, times…)`	`Scripts/Editor/Intro/IntroCapture.cs`

Menus: **Neon Rift ▸ Intro ▸ Build Intro (open Night Run scene)** (also run by every Night Run build), **Neon Rift ▸ Intro ▸ Reset 'Intro Seen' (play it on next launch)**, **Neon Rift ▸ Intro ▸ Generate Intro Audio**.

31.3 How it runs

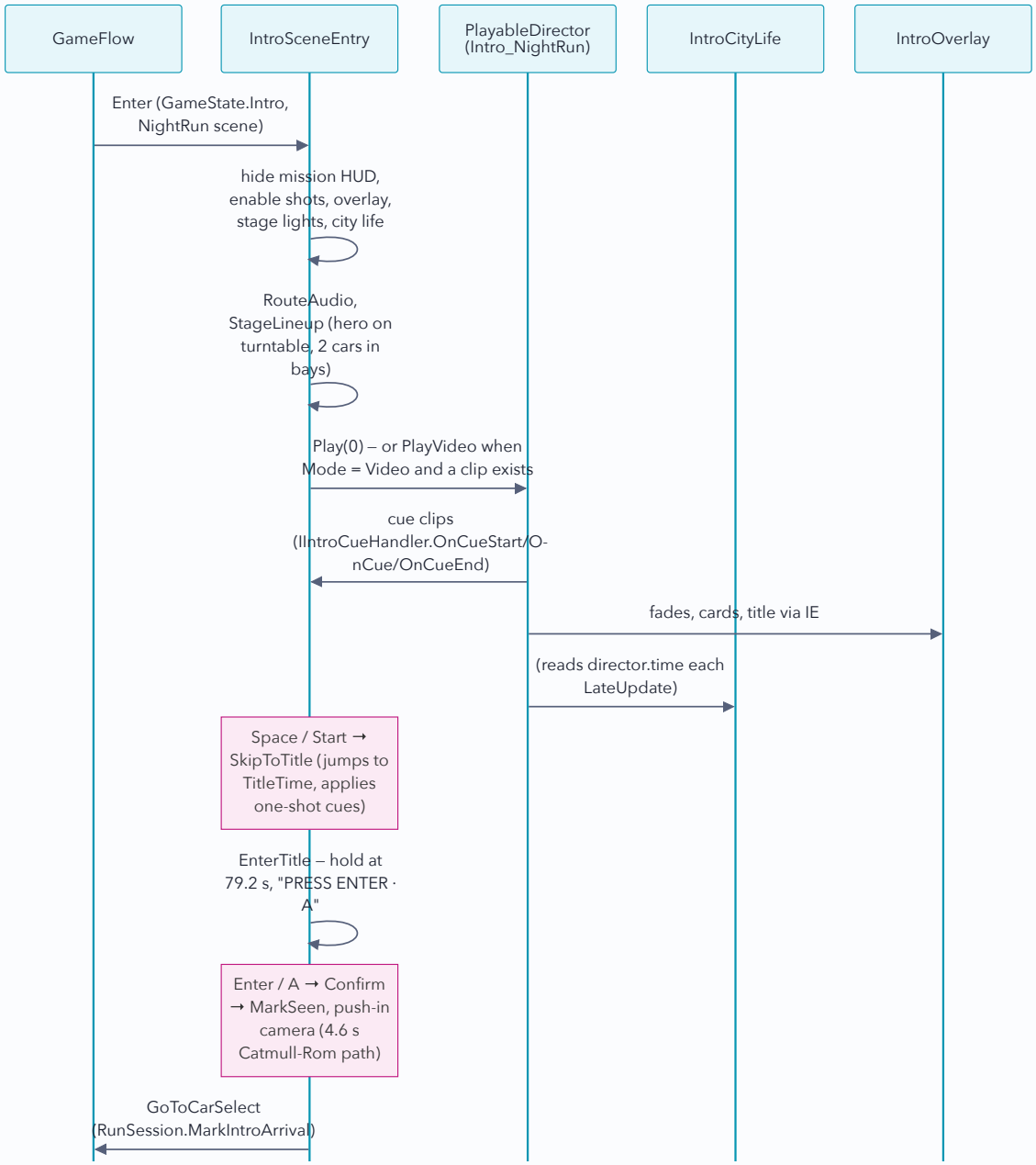


Figure 31.1 Cinematic intro runtime flow

- **Lineup:** the cars are the catalog's **gameplay prefabs** spawned through `VehicleSpawnPoint` (same models, physics, lights and engines). The hero is the session's selected car or the catalog default (`StageLineup`, line 148).
- **Departure:** on the `Departure` cue, the two bay cars drive out on scripted lines with `WaypointDriver` (real physics), through the door and up W Avenue (`Depart`, line 467).
- **Skip:** `OnSkip` → `SkipToTitle` (242) jumps to `TitleTime` (75.2 s); one-shot cues are applied so the world lands in a consistent state. A skip pressed before the Timeline's first evaluation is deferred a frame.

- **Confirm:** `Confirm` (285): `IntroSettings.MarkSeen`, push-in through the garage door, round the hero, ending exactly on Car Select's opening frame (`UpdatePushIn`, `CatmullRom`); Car Select opens on that close-up and pulls back.
- **Exit:** restores the time scale, destroys the lineup, resets the garage, disables the rig. The rig is inert in missions (tested by `IntroTests.NightRunScene_HasAWiredIntro`).
- **Video path:** `IntroSettings.Mode.Video` + a `VideoClip` plays a pre-rendered intro through a `VideoPlayer` (`PlayVideo`, 188), and the real-time title takes over at its end. **Implemented but untested: no clip is supplied.** Without a clip it falls back to real time (`IntroTests.VideoModeWithoutAClip_FallsBackToRealtime`).

31.4 Timeline structure (`IntroBuilder.BuildTimeline`, line 322)

Track	Content
Animation track per shot camera	Eased position plus a look direction keyed as a quaternion (`MoveClip`); each shot camera has an `Animator`
Cinemachine track	Cuts and blends between the **25** shot cameras
`IntroCueTrack`	Story beats (table below), bound to `IntroSceneEntry`
Audio tracks	Ambience (city loop 0–84 s), **Soundtrack** (trailer track 0–84 s, or the procedural Score bed and groove as fallback), Riser (TitleTime − 6 s), Hits, Scan (39.7–43.7 s)

31.5 Cue timeline (`IntroBuilder.Cues`, line 293)

Time (s)	Cue	Effect
0–84	Letterbox	Cinema bars
1.2–6	FadeIn	From black
2–75.2	SkipHint	"SPACE · START SKIP"
25.5–29.8	Card	*IN NEON RIFT, INFORMATION IS POWER. / AND THE GRID WATCHES EVERY STREET.*
34.2–39.2	Card	*AT ITS HEART: THE SECTOR 7 DATA CORE. / EVERY SECRET IN THE CITY PASSES THROUGH IT.*
39.5–43.5	Surveillance	CCTV overlay + grade: *UNUSUAL ACTIVITY · W AVENUE · UNIT 7*
43.2, 46.7	Dip	Soft cuts
47.6	GarageLights	`CrewGarage.LightsOn`: banks strike with flicker and clunks
48.4–52.2	Card	*TONIGHT, A CREW GOES IN FOR IT. / EXTRACT THE PACKAGE. OUTRUN THE LOCKDOWN.*
53	Headlights	Lineup headlights on
60.2–63.6	Card	*THREE CARS. ONE RUN. / NO SECOND CHANCE.*
62.6	Ignition	Engines start (ignition sound)
64.2	DoorOpen	`CrewGarage.OpenDoor`
65.6	Departure	Bay cars drive out
71–75	Flash × 6	Montage flashes every 0.7 s

Time (s)	Cue	Effect
73.1–73.8	Surveillance	*VEHICLE MOVEMENT · SECTOR 7 PERIMETER*
75.2	Flash	Title flash
75.7–84	Title	**NEON RIFT**, then **NIGHT RUN**, then **PRESS ENTER · A** (hold at 79.2 s)

31.6 Shot-by-shot breakdown (`IntroBuilder.Shots`, line 247)

World anchors are read from the built city: the core position (the director's alert origin), the compound gate (by name), the signal head lens on W Avenue (`SignalHead`) and the CCTV head near (8, −108). `G(x, y, z)` = a garage-local point.

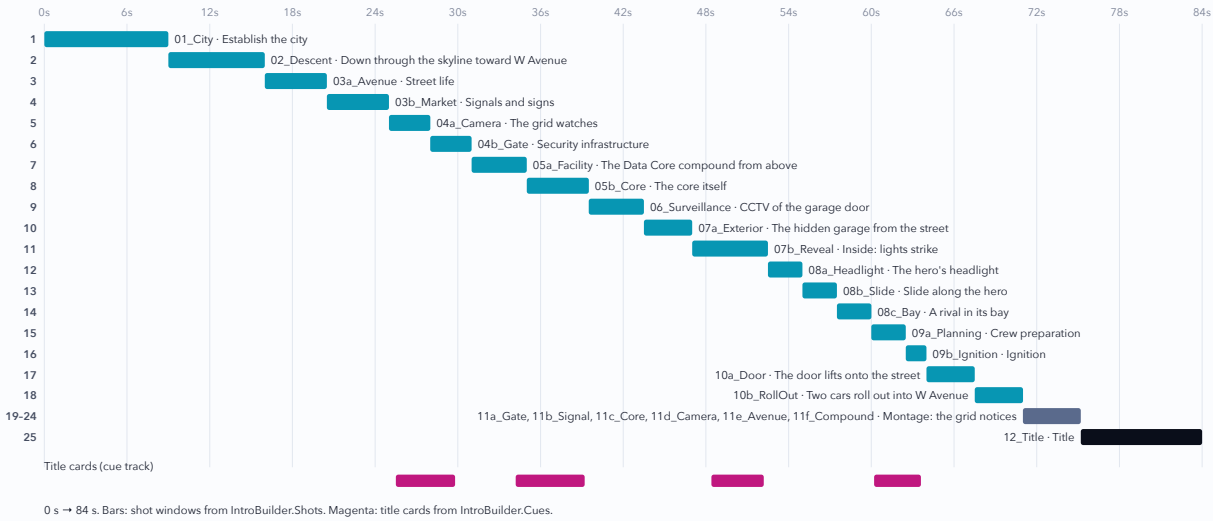


Figure 31.2 Intro timeline: the 25 shots (shots 19–24 are the 0.7 s montage) and the title cards

IMPLEMENTATION

Every shot below is authored in `IntroBuilder.Shots` (`Assets/_NeonRift/Scripts/Editor/Intro/IntroBuilder.cs`, line 247). The builder generates one Cinemachine camera with an `Animator` per shot and keys its move on the Timeline (`IntroBuilder.BuildTimeline`, line 322); street and sky life in a shot window is driven by `IntroCityLife`. "Action / on screen" is the source table's **Objects** column. A dash means the source table leaves that cell empty.

SHOT 1
`01_City`
0–9 s

PURPOSE	Establish the city
CAMERA	Far aerial (−700, 320, −1050) drifting in, looking toward downtown
ACTION / ON SCREEN	Whole city, skyline rings
LIGHTING	Night grade, fade-in
AUDIO	Ambience, soundtrack
IMPLEMENTATION	Entry `01_City` in `IntroBuilder.Shots`

SHOT 2 `02_Descent` 9-16 s	
PURPOSE	Down through the skyline toward W Avenue
CAMERA	(−260, 180, −560) → (−62, 34, −352)
ACTION / ON SCREEN	Towers; **sky-traffic drones** with searchlights (`IntroCityLife` flights Sky_A–C)
LIGHTING	Tower facades, aviation lights
AUDIO	–
IMPLEMENTATION	Entry `02_Descent` in `IntroBuilder.Shots`; also `IntroCityLife`

SHOT 3 `03a_Avenue` 16-20.5 s	
PURPOSE	Street life
CAMERA	Low dolly up W Avenue, x −4.5, z −215 → −185
ACTION / ON SCREEN	Shopfronts, steam; **traffic lanes** both ways; **patrol drone**; pavement **crowds**
LIGHTING	Street lamps (budgeted), neon
AUDIO	–
IMPLEMENTATION	Entry `03a_Avenue` in `IntroBuilder.Shots`

SHOT 4 `03b_Market` 20.5-25 s	
PURPOSE	Signals and signs
CAMERA	Over Market Street (8.5 → 16, z −103) looking east
ACTION / ON SCREEN	Traffic both ways, crowds, patrol drone
LIGHTING	Signal lenses, signs
AUDIO	–
IMPLEMENTATION	Entry `03b_Market` in `IntroBuilder.Shots`

SHOT 5 `04a_Camera` 25-28 s	
PURPOSE	The grid watches
CAMERA	Close on a real street CCTV head
ACTION / ON SCREEN	Security camera
LIGHTING	–
AUDIO	Card 1
IMPLEMENTATION	Entry `04a_Camera` in `IntroBuilder.Shots`

SHOT 6 `04b_Gate` 28-31 s	
PURPOSE	Security infrastructure
CAMERA	The compound gate from the street
ACTION / ON SCREEN	Gate; **two guard drones**
LIGHTING	Security strips
AUDIO	–
IMPLEMENTATION	Entry `04b_Gate` in `IntroBuilder.Shots`

SHOT 7 `05a_Facility` 31-35 s	
PURPOSE	The Data Core compound from above
CAMERA	gate + (50, 30, 51) → (32, 23, 33)
ACTION / ON SCREEN	Compound, **3 orbiting drones**, Buster sentries
LIGHTING	Floods, core beam
AUDIO	–
IMPLEMENTATION	Entry `05a_Facility` in `IntroBuilder.Shots`

SHOT 8 05b_Core 35-39.5 s	
PURPOSE	The core itself
CAMERA	Around the core at 4–6 m
ACTION / ON SCREEN	Core column, rings
LIGHTING	Core glow
AUDIO	Card 2
IMPLEMENTATION	Entry `05b_Core` in `IntroBuilder.Shots`

SHOT 9 06_Surveillance 39.5-43.5 s	
PURPOSE	CCTV of the garage door
CAMERA	High fixed camera (8.6, 7.6, −273)
ACTION / ON SCREEN	Roller door "UNIT 7"; traffic passing
LIGHTING	Green CCTV grade
AUDIO	Scan sound
IMPLEMENTATION	Entry `06_Surveillance` in `IntroBuilder.Shots`

SHOT 10 07a_Exterior 43.5-47 s	
PURPOSE	The hidden garage from the street
CAMERA	(6, 1, −232) → (1.5, 1.25, −243)
ACTION / ON SCREEN	Door, southbound traffic
LIGHTING	Door lamp, sodium street lamps
AUDIO	—
IMPLEMENTATION	Entry `07a_Exterior` in `IntroBuilder.Shots`

SHOT 11 07b_Reveal 47-52.5 s	
PURPOSE	Inside: lights strike
CAMERA	Garage interior high corner
ACTION / ON SCREEN	Three cars
LIGHTING	Banks strike with flicker
AUDIO	Clunks, card 3
IMPLEMENTATION	Entry `07b_Reveal` in `IntroBuilder.Shots`

SHOT 12 08a_Headlight 52.5-55 s	
PURPOSE	The hero's headlight
CAMERA	Close-up (Car Select's opening look)
ACTION / ON SCREEN	Hero car
LIGHTING	Headlights on
AUDIO	—
IMPLEMENTATION	Entry `08a_Headlight` in `IntroBuilder.Shots`

SHOT 13 08b_Slide 55-57.5 s	
PURPOSE	Slide along the hero
CAMERA	Low side slide
ACTION / ON SCREEN	Hero
LIGHTING	—
AUDIO	—
IMPLEMENTATION	Entry `08b_Slide` in `IntroBuilder.Shots`

SHOT 14 08c_Bay 57.5-60 s	
PURPOSE	A rival in its bay
CAMERA	In front of the left bay
ACTION / ON SCREEN	Bay car
LIGHTING	—
AUDIO	—
IMPLEMENTATION	Entry `08c_Bay` in `IntroBuilder.Shots`

SHOT 15 `09a_Planning` 60-62.5 s	
PURPOSE	Crew preparation
CAMERA	Planning wall
ACTION / ON SCREEN	Map from the real road data, core schematic
LIGHTING	Screen glow
AUDIO	Card 4
IMPLEMENTATION	Entry `09a_Planning` in `IntroBuilder.Shots`

SHOT 16 `09b_Ignition` 62.5-64 s	
PURPOSE	Ignition
CAMERA	Low behind the hero
ACTION / ON SCREEN	Cars
LIGHTING	–
AUDIO	Ignition, engines
IMPLEMENTATION	Entry `09b_Ignition` in `IntroBuilder.Shots`

SHOT 17 `10a_Door` 64-67.5 s	
PURPOSE	The door lifts onto the street
CAMERA	Inside looking at the door
ACTION / ON SCREEN	Roller door rising
LIGHTING	Street light spills in
AUDIO	Door motor
IMPLEMENTATION	Entry `10a_Door` in `IntroBuilder.Shots`

SHOT 18 `10b_RollOut` 67.5-71 s	
PURPOSE	Two cars roll out into W Avenue
CAMERA	Street level outside the door
ACTION / ON SCREEN	Bay cars (`WaypointDriver`)
LIGHTING	–
AUDIO	Engines
IMPLEMENTATION	Entry `10b_RollOut` in `IntroBuilder.Shots` ; also `WaypointDriver`

SHOT 19-24 `11a_Gate, 11b_Signal, 11c_Core, 11d_Camera, 11e_Avenue, 11f_Compound` 71-75.2 s	
PURPOSE	Montage: the grid notices
CAMERA	0.7 s cuts with flashes
ACTION / ON SCREEN	Gate, signal, core, camera, avenue, compound
LIGHTING	–
AUDIO	Hits, riser
IMPLEMENTATION	Entries `11a_Gate` , `11b_Signal` , `11c_Core` , `11d_Camera` , `11e_Avenue` , `11f_Compound` in `IntroBuilder.Shots`

SHOT 25 `12_Title` 75.2-84 s	
PURPOSE	Title
CAMERA	Across W Avenue looking into the open, lit garage
ACTION / ON SCREEN	Hero on the turntable; the odd passing car
LIGHTING	Garage lit
AUDIO	Title, soundtrack fade
IMPLEMENTATION	Entry `12_Title` in `IntroBuilder.Shots`

–	
Push-in (Confirm)	
4.6 s	
PURPOSE	Match cut into Car Select
CAMERA	`PushCamera` on a Catmull-Rom path through the door to Car Select's opening frame
ACTION / ON SCREEN	Hero
LIGHTING	–
AUDIO	Fade
IMPLEMENTATION	`IntroSceneEntry` push-in on Confirm (not a Timeline shot); also `PushCamera`

31.7 Generated / runtime elements

- **City life** (`IntroCityLife`): 9 traffic lanes with 36 visual-only pack cars (wheels turning), 8 drone flights (sky, avenue, market, garage, two gate guards) and 3 core orbiters (Mech Drones with searchlights and beams). Each element is active only inside its shot window, driven by `director.time` in `LateUpdate`, so every play and every skip looks identical.
- **Crowds:** extra pavement groups along W Avenue and Market Street (`NightRunBuilder.BuildCrowds`).
- **Sentries:** two Buster Drones over the compound (`HoverSentry`).

31.8 Validation

`IntroTests` (`FirstLaunchOnly_PlaysUntilSeen`, `AlwaysAndNever_IgnoreTheSeenFlag`, `VideoModeWithoutAClip_FallsBackToRealtime`, `GameConfig_PointsAtTheIntro`, `NightRunScene_HasAWiredIntro`). Play Mode frame captures with `IntroCapture` (`Docs/Screenshots/Intro2/`). Known: the intro mix has not been judged by ear. In the editor, the first play after a rebuild can show cars flat cyan while shaders compile asynchronously.

32 Timeline and Camera (cinematics)

IN THIS CHAPTER

32.1 Where Timeline is used
32.2 The Timeline is generated from code
32.3 The custom cue track (Assets/_NeonRift/Scripts/Intro/)
32.4 Cinemachine in cinematics
32.5 Scripted camera beats outside Timeline

This chapter covers how **Timeline** and **Cinemachine** are used for cinematic camera work. Gameplay cameras are in 38 Camera System.

32.1 Where Timeline is used

Only in the **intro**: one `PlayableDirector` in the NightRun scene (on the `Intro` root object), playing `Assets/_NeonRift/Data/Intro/Intro_NightRun.playable`. It is configured as `playOnAwake = false`, `extrapolationMode = Hold`, `timeUpdateMode = GameTime` (`IntroBuilder.Build`).

32.2 The Timeline is generated from code

`IntroBuilder` keeps a **shot table** (`Shot` struct: name, start, end, ease, FOV, from, to, look-from, look-to) and a **cue table** (`Cue` struct: kind, start, end, text, subtitle). `BuildTimeline` (`Assets/_NeonRift/Scripts/Editor/Intro/IntroBuilder.cs:322`) rebuilds the asset **in place**, so it keeps its GUID:

Track	Built from	Notes
One **AnimationTrack per shot camera**	`MoveClip` (436): an `AnimationClip` keying position and a look rotation (quaternion), eased	Each `Shot_*` GameObject has a `CinemachineCamera` and an `Animator`
CinemachineTrack	One shot per camera, cut or blended	Bound to the main camera's `CinemachineBrain`
IntroCueTrack	`IntroCueClip` per cue	Bound to `IntroSceneEntry` (`IIntroCueHandler`)
Audio tracks	Ambience, Soundtrack, Riser, Hits, Scan	Sources created by `IntroBuilder` and routed at runtime (`IntroSceneEntry.RouteAudio`)

32.3 The custom cue track (`Assets/_NeonRift/Scripts/Intro/`)

Type	Role
`IntroCueKind`	`FadeIn`, `Dip`, `Card`, `Title`, `Surveillance`, `Headlights`, `Ignition`, `Departure`, `GarageLights`, `DoorOpen`, `Flash`, `SkipHint`, `Letterbox`
`IntroCueTrack`	`TrackAsset` of cue clips, bound to the handler
`IntroCueClip`	`PlayableAsset` with `kind`, `text`, `subtitle`; `CreatePlayable`
`IntroCue Behaviour`	`ProcessFrame` → `IIntroCueHandler.OnCueStart / OnCue(progress) / OnCueEnd`; `OnBehaviourPause`
`IIntroCueHandler`	Implemented by `IntroSceneEntry` (`OnCueStart` 374, `OnCue` 401, `OnCueEnd` 437)

Everything is driven explicitly each frame from the Timeline time, so **scrubbing in the editor and skipping at runtime both land on the right state** (`IntroOverlay` class summary).

32.4 Cinemachine in cinematics

Camera	Where	Use
25 shot cameras (`Shot_*`, priority 0)	`Intro/Shots`	Timeline-animated, Cinemachine-cut
`PushCamera` (with `CinemachineHardLookAt`)	`Intro/PushCamera`	Confirm push-in, a 4.6 s Catmull-Rom path through the garage door (`IntroSceneEntry.UpdatePushIn`, `CatmullRom`)
Car Select cameras (Hero, Swap, Depart, Opening)	`CarSelect` scene	Switched by priority (`VehicleShowroom.SetCamera`); blends in `Settings/Rendering/Garage_CameraBlends.asset`; the Opening camera is entered with a **Cut** (the brain's default blend is temporarily set to Cut) so the first frame matches the intro's last
Chase camera	NightRun mission	`VehicleChaseCamera` writes the pose into a `CinemachineCamera` before the brain updates (Decision D18)

All shot cameras use the far clip `SkylineBackdrop.RequiredFarClip` (3.5 km), so the skyline rings are visible.

32.5 Scripted camera beats outside Timeline

Beat	Code
Theft beat on `core.breached` (2.6 s swing, letterbox, time scale 0.45)	`MissionDirector.StartBeat` / `UpdateBeat` → `VehicleChaseCamera.CinematicBeat`
Data Core extraction framing	`DataCoreChamber.UpdateCamera` → `VehicleChaseCamera.SetFraming`
Gate hack framing	`GateLockSystem.UpdateCamera` → `SetFraming`
Lockdown focus on the alert origin	`MissionDirector.OnSecurityChanged` → `VehicleChaseCamera.Kick`, `Focus`

PART IX

Interface and Presentation

HUD, minimap, briefing and debrief.

33 UI and HUD

34 Minimap

35 Objectives and Mission Presentation

36 Results and Debrief

33

UI and HUD

IN THIS CHAPTER

33.1 Technology
33.2 Mission HUD elements (UI/MissionHud.uxml) and their setters (MissionHud.cs)
33.3 Data flow
33.4 Options and pause menu (OptionsMenu)
33.5 Title and intro UI
33.6 UI animations

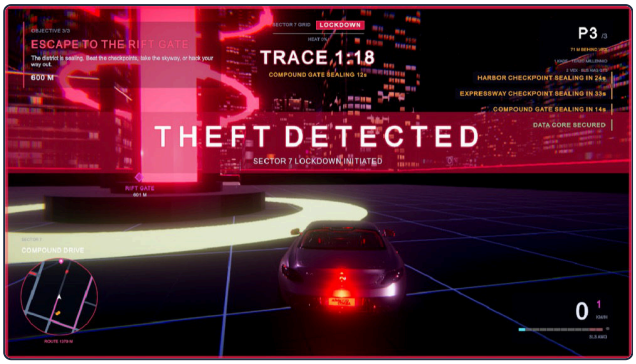
33.1 Technology

All UI is **UI Toolkit** (Decision D6): UXML layouts and USS styles in `Assets/_NeonRift/UI/`, a shared theme (`NeonRift.uss`, `NeonRiftTheme.tss`) and one `NeonRiftPanelSettings.asset` (1920×1080 reference, match 0.5). Views are **passive**: controllers push state in, and text is only rewritten when the value changes (no per-frame allocation).

Screen	UXML / USS	Controller
Loading fade	`UI/LoadingOverlay.uxml`	`Scripts/Game/LoadingOverlay.cs`
Title	`UI/TitleScreen.uxml` (START, WATCH INTRO, OPTIONS, QUIT, `M · MUSIC ON / OFF`, version)	`Scripts/Frontend/TitleScreen.cs`
Options / pause menu	Built in code (pages of rows: buttons, sliders, choices)	`Scripts/Game/OptionsMenu.cs`
Car Select	`UI/CarSelect.uxml`, `CarSelect.uss`	`Scripts/Frontend/CarSelectScreen.cs`
Intro overlay	`UI/IntroOverlay.uxml`, `IntroOverlay.uss`	`Scripts/Intro/IntroOverlay.cs`
Mission HUD, briefing, debrief	`UI/MissionHud.uxml`, `MissionHud.uss`	`Scripts/Gameplay/Mission/MissionHud.cs`
Heist readout	inside the HUD (`terminal`)	`Scripts/Gameplay/Mission/TerminalReadout.cs`
Minimap	inside the HUD (`minimap-frame`)	`Scripts/Gameplay/Mission/CityMinimap.cs`



Mission HUD in Lowtown
`Docs/Screenshots/CameraHud/hud_lowtown.jpg`



Theft beat banner
`Docs/Screenshots/CameraHud/theft_beat.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

33.2 Mission HUD elements (`UI/MissionHud.uxml`) and their setters (`MissionHud.cs`)

Element(s)	Shows	Setter (line)	Fed by
`objective`, `objective-step`, `-title`, `-detail`, `-distance`	Current objective, n/4, detail, distance	`SetObjective` (330), `SetObjectiveDistance` (351), `HideObjective`	`MissionDirector.OnObjectiveStarted`, `UpdateHud`
`objective-complete`	OBJECTIVE COMPLETE + split time	`StampObjective` (239)	`OnObjectiveCompleted`
`waypoint`, `-arrow`, `-label`, `-distance`	Screen-space waypoint, edge-clamped	`SetWaypoint` (360)	`UpdateHud`
`security`, `security-state`, `heat-fill`, `heat-label`	CALM / ALERT / LOCKDOWN and heat %	`SetSecurity` (399)	`UpdateHud`
`timer`	Trace timer	`SetTimer` (413)	`Progress.TimeRemaining`
`countdown`	Gate seal countdowns ("COMPOUND GATE SEALING IN 14s")	`SetCountdown` (425)	`MissionWorld.TryGetNextCountdown`
`race`, `race-position`, `-count`, `-gap`, `-rows`	P1/3, gap to the next car, standings rows with crew status	`SetRace` (290)	`MissionDirector.UpdateRace` ← `RivalDirector`
`navigator`, `nav-district`, `nav-street`, `nav-route`, `minimap-frame`	District, street, route length, minimap	`SetNavigator` (258), `SetRoadNetwork` (251)	`UpdateNavigator` ← `CityNavigation`
`terminal`	Multi-stage interaction readout	`SetTerminal` (222), `FlashTerminal` (228)	`UpdateInteraction`
`prompt`, `prompt-key`, `-text`, `-fill`, `-hint`	"E · HACK" interaction prompt and progress	`SetPrompt` (436)	`UpdateInteraction`
`toasts`	Short messages (+heat, rival radio lines)	`Toast` (475)	`OnHeatAdded`, `OnAnnounced`

Element(s)	Shows	Setter (line)	Fed by
`banner`, `banner-title`, `-subtitle`	DATA ACQUIRED / SECURITY BREACH DETECTED / EXTRACTED / TRACED	`Banner` (491)	announcements with `banner: true`
`lockdown-frame`	Red screen frame in a lockdown	`SetSecurity`	
`extract-frame`	Glow frame during sustain stages	`SetExtractionFrame` (231)	
`speedo`, `speed`, `gear`, `rpm-fill`, `rpm-ticks`, `shift-light`	Speed, gear, 12-step RPM bar with redline and shift light	`SetSpeed` (446)	player telemetry
`vehicle-name`	Selected car	`SetVehicleName` (328)	`Begin`
`music-hint`	`M · MUSIC ON/OFF` (amber when off)	`SetMusicHint` (59)	`GameSettings`
`letterbox-top/bottom`	Theft beat bars	`SetLetterbox` (468)	`UpdateBeat`
`briefing` (`brief-*`)	Operation card	`ShowBriefing` (504), `HideBriefing`	35
`results` (`results-*`, `retry-button`, `continue-button`)	Debrief	`ShowResults` (560)	36

Compass: there is no separate compass widget. North is the white tick on the minimap rim and the player arrow's heading (34).

Extraction indicator: the extraction zone's beacon column in the world, plus the objective, waypoint, minimap target and trace timer on the HUD.

33.3 Data flow

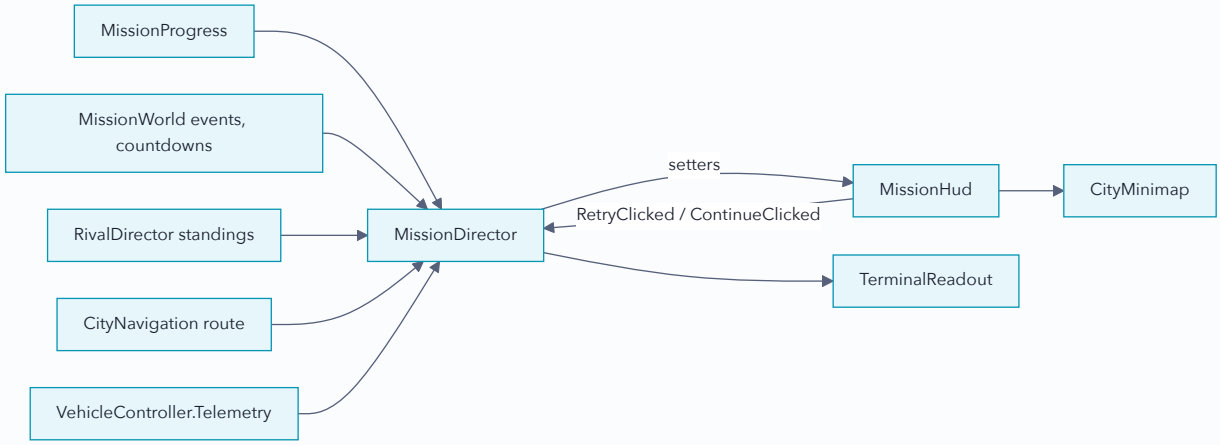


Figure 33.1 HUD data flow

33.4 Options and pause menu (`OptionsMenu`)

Rows are buttons, sliders (volumes, 10 % steps) and choices. Pages: the root, **Settings** (`PushSettings`: master / music / effects / interface volume, music on/off, graphics quality, fullscreen, V-sync) and **Controls** (`PushControls`: every keyboard and gamepad binding). Navigation: ↑↓ / W S / d-pad / stick move, ←→ change a value, Enter / A confirm, Esc / B back; the mouse works too. Persistence: `GameSettings` (PlayerPrefs).

33.5 Title and intro UI

`TitleScreen` binds `start-button` → `Flow.GoToCarSelect`, `intro-button` → `Flow.PlayIntro`, `options-button` → `OptionsMenu`, `quit-button` → `Flow.QuitGame`. `IntroOverlay` provides `SetFade`, `SetFlash`, `SetLetterbox`, `ShowCard` / `HideCard`, `SetTitle`, `SetPrompt`, `SetSkipHint`, `SetSurveillance`.

33.6 UI animations

USS transitions and class toggles: Car Select text slide-out and slide-in (`nr-cs--departing`, `nr-cs--intro`), stat bars easing and numbers counting (`StatRow.Tick`), briefing rows revealing one by one, debrief rows counting in, then the grade landing, toast and banner fades, and objective stamps.

34 Minimap

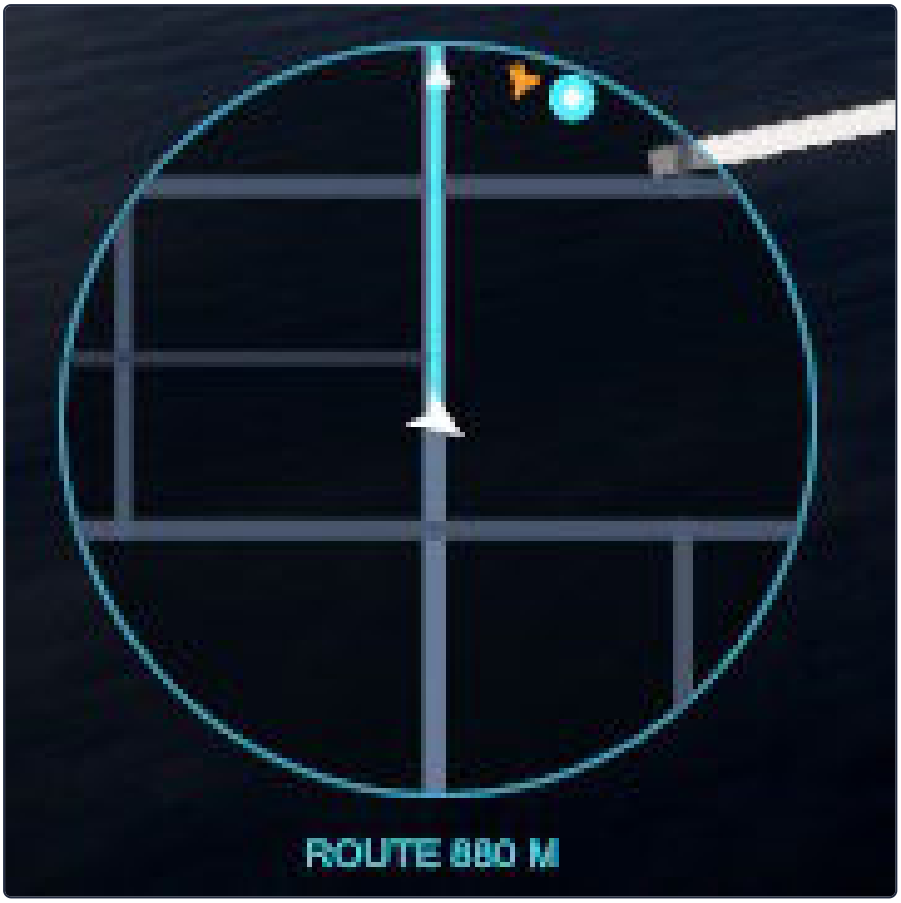
IN THIS CHAPTER

34.1 What it is
34.2 Code
34.3 Map source
34.4 World-to-map conversion
34.5 Markers and route
34.6 Validation

34.1 What it is

A **north-up** minimap in the HUD's `minimap-frame`, centred on the player. The map **never rotates**: north (+Z) is always up and east (+X) is right. Only the player arrow and the rival chevrons turn with their world headings. A white tick on the rim marks north.

> The README's feature list still says "heading-up minimap". That was the earlier version; the current code is north-up (`Docs/NightRun.md`, *Minimap (north-up)*).



Minimap capture
`Docs/Screenshots/CityInfill/minimap_heading128.jpg`

Screenshots are real captures from the project, taken during development with the project's capture tools; they are stored in `Docs/Screenshots/` and may predate small later changes.

34.2 Code

Class	File	Role
`CityMinimap` (a UI Toolkit `VisualElement`)	`Assets/_NeonRift/Scripts/Gameplay/Mission/CityMinimap.cs`	Draws with **`Painter2D`** in `Draw` (75): roads by class, closed gates in red, the GPS route, the objective marker, rival chevrons, the player arrow (`Arrow`); helpers `OutOfReach`, `Clamp`, `Dot`. Setters `SetNetwork` (42), `SetView(position, heading, lockdown)` (49), `SetRoute` (56), `SetTarget` (62), `SetRivals` (69). Default `Range` 240 m
`MinimapProjection` (readonly struct, pure maths)	`Scripts/Gameplay/Mission/MinimapProjection.cs`	`ToMap(world)` (panel y grows downward, so north is −y), `ToWorld(map)`, `Scale` (pixels per metre = width / 2 / Range), `Heading(forward)` (compass degrees: 0 = north, 90 = east, clockwise), `MarkerRotation`
`MissionHud.SetRoadNetwork`, `SetNavigator`	`Scripts/Gameplay/Mission/MissionHud.cs:251`, `:258`	Hands the network and the gate-closed predicate (`CityNavigation.IsBlocked`) to the minimap; pushes view, route and target
`MissionDirector.UpdateNavigator`	`Scripts/Gameplay/Mission/MissionDirector.cs:512`	Re-plans the route every 0.5 s or when gates change; collects rival positions and headings (index loop, no boxing)

34.3 Map source

There is no pre-rendered map texture: the map is **drawn live from the road graph** (`RoadNetwork_NightRun.asset`). Line widths come from road class, and closed gates turn red in real time.

34.4 World-to-map conversion

```
scale   = size.x / 2 / range
map.x   = size.x / 2 + (world.x − centre.x) · scale
map.y   = size.y / 2 − (world.z − centre.z) · scale      (panel y grows downward)
heading = atan2(forward.x, forward.z) in degrees, normalised to 0…360 (0 = north, clockwise)
```

34.5 Markers and route

Marker	Behaviour
Player	Arrow at the centre, rotated to its compass heading
Rivals	Small orange arrows with their own headings
Objective	Marker at its true position; **off-map objectives stay clamped to the rim** in their true direction
GPS route	The `RoadPath` from `CityNavigation.Plan`: cyan normally, **magenta in a lockdown**
Closed gates	Red road segments

34.6 Validation

- `MinimapTests` (`Assets/_NeonRift/Scripts/Tests/EditMode/MinimapTests.cs`): `North_IsUp_And_East_IsRight`, `Projection_DoesNotDependOnPlayerHeading` (0 / −90 / 90 / 180 / 270), `ToWorld_RoundTrips`, `Heading_IsCompassClockwiseFromNorth`, `PlayerArrow_PointsAlongWorldHeading`.
- Play Mode: `InputPlaytest.RunMinimapScript` drove straight, left, right and a sustained left with real keyboard events; headings logged 0° → 250° → 267° → 128°, and W Avenue stayed vertical on the map in every capture (`Docs/Screenshots/CityInfill/minimap_heading250.jpg`, `minimap_heading128.jpg`).

35

Objectives and Mission Presentation

IN THIS CHAPTER

35.1 The operation briefing card
35.2 Objective display
35.3 Message tones (Scripts/Missions/MessageTone.cs)
35.4 Key Night Run announcements (from NightRunBuilder.CreateMissionData)
35.5 Presentation beats

35.1 The operation briefing card

When	First frame of the mission, once the rival crews are on the grid (`briefingPending` in `MissionDirector.Begin`)
Built by	`MissionDirector.BuildBriefing` (`Assets/_NeonRift/Scripts/Gameplay/Mission/MissionDirector.cs:631`) → `MissionHud.Briefing`
Shown by	`MissionHud.ShowBriefing` (504); rows reveal one by one; the objective panel and minimap step aside while it is up
Contents	Kicker "OPERATION · SECTOR 7", the mission name, tagline ("INFILTRATE SECTOR 7 · STEAL THE DATA CORE · GET OUT") and briefing text from `MissionDefinition`; the four objective titles; the crew list (YOU · car = DRIVER, then each rival with `MissionDefinition.CrewRoleFor(i)`: "SPIRE RELAY · OWN JOB", "KOWLOON RELAY · OWN JOB"); personal best and best grade (`MissionRecords`); "DRIVE TO BEGIN"
Leaves	After `briefingSeconds` (9 s), or once the player passes `briefingExitKph` (60 km/h) after 2.5 s (`UpdateBriefing`, 611)

35.2 Objective display

Moment	What happens	Code
Objective starts	Title, detail and "n/4" slide in; the target's beacon shows; the waypoint and minimap target update; objective cue sound (from objective 2 on)	`MissionDirector.OnObjectiveStarted` → `MissionHud.SetObjective`; `IMissionTarget.SetObjectiveActive`
Distance	Route distance to the objective	`MissionHud.SetObjectiveDistance`
Completion	Stamp "OBJECTIVE COMPLETE" with the **split time** (e.g. `0:48.20`), then the next objective slides in	`OnObjectiveCompleted` → `MissionHud.StampObjective`
Timed objective	Trace timer; heat removes time	`MissionHud.SetTimer`

Moment	What happens	Code
Announcements	Data-driven HUD lines and banners per world event (`MissionAnnouncement`: event id, text, tone, delay, banner)	`MissionDirector.OnWorldEvent` → `RunAnnouncements` → `Toast` / `Banner`

35.3 Message tones (`Scripts/Missions/MessageTone.cs`)

`Info`, `Warning`, `Danger`, `Success`: the colour of toasts and banners.

35.4 Key Night Run announcements (from `NightRunBuilder.CreateMissionData`)

Event	Text
`mission.start` (+4.5 s)	SECTOR 7 GRID ONLINE · FOLLOW THE CYAN BEACON
`mission.start` (+9 s)	RIVAL CREWS RUNNING THEIR OWN JOBS · FIRST TWO OUT QUALIFY
`alley.gate.open`	ALLEY GATE OPEN
`core.arrive`	FACILITY SECURITY ACTIVE · INTRUDER SCAN RUNNING
`core.security.off`	LOCAL SECURITY OFFLINE · CORE CHAMBER OPENING
`core.extract.begin`	EXTRACTION RUNNING · HOLD POSITION
`grid.anomaly`	S7 GRID: ANOMALOUS DATA FLOW · NODES REROUTING
`core.extract.trace`	COUNTER-INTRUSION ONLINE · CAMERAS REBOOTING
`core.acquired`	**banner** DATA ACQUIRED / S7 CORE PACKAGE · 2.4 TB SECURED
`core.breached`	**banner** SECURITY BREACH DETECTED / SECTOR 7 LOCKDOWN INITIATED; then CAMERAS ARMED · STAY OUT OF SIGHT
`escape.start`	EXTRACTION: RIFT GATE, SOUTH-EAST TUNNEL · CREW: SKYWAY GATE OPEN · HARBOR ROUTE CLEAR · ALLEY GATE CAN BE RE-HACKED · COSTS HEAT

Rival radio lines ("VEX: SPIRE RELAY CRACKED", "KADE HAS A CORE PACKAGE · RUNNING FOR THE GATE", "VEX EXTRACTED", qualifying warnings) come from `RivalDirector` through `MissionWorld.Announce`.

35.5 Presentation beats

Beat	Code
Data Core framing camera	`DataCoreChamber.UpdateCamera`
Theft beat (camera swing, letterbox, slow motion 0.45 for 2.6 s)	`MissionDirector.StartBeat` / `UpdateBeat` / `EndBeat`
Lockdown frame, magenta route, red gates	`MissionHud.SetSecurity`, `CityMinimap`

Beat	Code
End banner (EXTRACTED / TRACED), then the debrief after 2 s	`OnPhaseChanged` → `UpdateResults`

Screenshots: `Docs/Screenshots/Presentation/briefing.png`, `debrief.png`.

36
Results and Debrief

IN THIS CHAPTER

36.1 When it appears
36.2 The debrief (MissionDirector.BuildDebrief, line 656)
36.3 The grade (MissionGrade, Assets/_NeonRift/Scripts/Missions/MissionGrade.cs)
36.4 Records (MissionRecords, Scripts/Missions/MissionRecords.cs)
36.5 Tests

36.1 When it appears

`MissionProgress.PhaseChanged` → Completed or Failed → `MissionDirector.OnPhaseChanged` (`Assets/_NeonRift/Scripts/Gameplay/Mission/MissionDirector.cs:357`):

1. `FinishPosition = rivals.FinishPosition()` (rivals already extracted + 1).
2. `RunSession.RecordResult(Progress.ToResult())` (`RunResult`: outcome, time, security).
3. Mixer → **Results** snapshot; the score outro plays (success or failure), or a plain cue.
4. Banner EXTRACTED / TRACED; the player car is parked (`ParkPlayer`).
5. After `resultsDelay` (2 s): `BuildDebrief` → `MissionHud.ShowResults` (560). The driving HUD fades out behind it.

36.2 The debrief (`MissionDirector.BuildDebrief`, line 656)

Section	Content
Header	DEBRIEF · NIGHT RUN; **MISSION COMPLETE** / **MISSION FAILED**; reason: "EXTRACTION CONFIRMED · DATA CORE SECURED · QUALIFIED", or "DATA CORE SECURED · NOT QUALIFIED · ONLY THE FIRST 2 OUT ADVANCE", or the fail reason
Grade badge	Letter S/A/B/C/D and points (completed runs only)
RUN	TIME, POSITION n/3, QUALIFYING (TOP 2 · QUALIFIED / NOT QUALIFIED), HEAT %, SECURITY, HARD CONTACTS, OBJECTIVES n/4
SPLITS	Cumulative time per objective (`--` for ones not reached)
CREWS	Finishing order: crews that extracted first (QUALIFIED if in the first two, else EXTRACTED), the player (QUALIFIED / NOT QUALIFIED), crews still running (STILL RUNNING); "-- YOU · TRACED" on a fail
Best	FIRST CLEAR · PERSONAL BEST SET, NEW PERSONAL BEST · x FASTER, or PERSONAL BEST m:ss.cc · +gap

Section	Content
Buttons	**RETRY (E)** → `Retry` → `Flow.StartMission()`; **GARAGE (Esc)** → `Continue` → `Flow.GoToCarSelect()`

Rows count in, then the grade lands (USS animation). A log line records it, e.g. `[Mission] debrief: P1 qualified, grade A (77: pace 33, quiet 12, position 20, clean 13) …`.

36.3 The grade (`MissionGrade`, `Assets/_NeonRift/Scripts/Missions/MissionGrade.cs`)

Pure logic, out of **100**:

Part	Max	Rule (`Evaluate`)
Pace	40	Full at or under par (`ParTime`, Night Run **115 s**), zero at 1.6 × par (184 s), linear between
Quiet	25	25 × (1 − final heat)
Position	20	(field − position) / (field − 1): 1st = 20, last = 0; full marks with no race
Clean	15	15 × (1 − contacts / 6): 0 at 6 hard contacts (a hard contact is a collision at ≥ 4 m/s, `contactSpeed`)

Letters (`LetterFor`): **S ≥ 90, A ≥ 75, B ≥ 60, C ≥ 45, otherwise D**. A failed run gets no grade.

Worked example (from a real run): 3:20 time → pace 0; heat 73 % → quiet 7; 2nd of 3 → position 10; 12 contacts → clean 0; total 17 = **D**.

36.4 Records (`MissionRecords`, `Scripts/Missions/MissionRecords.cs`)

PlayerPrefs keys `NeonRift.Best.<missionId>.time`, `.grade`, `.qualified`. Only completed runs count; a slower run can still raise the best grade (`Submit`). `MarkQualified` / `HasQualified` record qualifying. `FormatTime` gives m:ss.cc, rounded to centiseconds first.

36.5 Tests

`MissionTests.Grade_RewardsPaceQuietPositionAndCleanDriving`, `Grade_LettersFollowTheScoreBands`, `Records_KeepTheBestTimeAndScore`, `Records_FormatTimeRoundsToCentiseconds`.

PART X

Animation and Camera

Everything that moves on screen, and the gameplay camera.

37 Animation Systems

38 Camera System

37 Animation Systems

IN THIS CHAPTER

37.1 Overview
37.2 Catalogue
37.3 Not animated

37.1 Overview

Almost every moving thing in Neon Rift is **procedural animation in C#**: transforms or material properties set each frame from game state (time, progress, events). Unity animation assets are used in only three places:

1. **Timeline AnimationTracks** on the intro's shot cameras.
2. A **legacy** `Animation` loop on the Mech Drone model.
3. A **sampled pose** of the Buster Drone (baked once, then static).

There are no Animator controllers, no character animation and no hand-keyed object animation.

37.2 Catalogue

What moves	How it moves	System responsible	Code	Asset / prefab
Car body	Rigidbody physics	`VehicleController`	`Scripts/Vehicles/Physics/VehicleController.cs`	`Prefabs/Vehicles/PF_Vehicle_*`
Wheels (steer, suspension travel, spin)	`LateUpdate` sets each pivot's position and steer rotation and each Spin transform's roll angle from angular velocity	`VehicleController.UpdateVisuals` (385)	same	`Wheels/Wheel_XX/Spin` pivots (`VehiclePrefabBuilder`)
Vehicle lights (brake brightening, reverse white, headlight levels)	Intensity and colour from input each frame	`VehicleLights.Update`	`Scripts/Vehicles/VehicleLights.cs`	–
Traffic cars (body via springs, wheels)	Physics + wheel roll; intro copies: positions and wheel rotations along lanes from Timeline time	`TrafficCar`, `IntroCityLife.Apply`	`Scripts/Gameplay/City/TrafficCar.cs`, `Scripts/Intro/IntroCityLife.cs`	`Prefabs/Traffic/PackCar_*`

What moves	How it moves	System responsible	Code	Asset / prefab
Turntable (idle spin, drag momentum, whip on car switch, swing to the door)	Rotation integrated from angular velocity; eased coroutines	`VehicleShowroom.Update`, `Swap`, `Depart`	`Scripts/Frontend/VehicleShowroom.cs`	CarSelect scene
Car Select departure (car pulls out of the garage)	Eased acceleration along the exit heading, wheels spun by distance, step capped at 0.05 s	`VehicleShowroom.Depart`	same	–
Car Select garage door	Eased rise (`SmoothStep`)	`VehicleShowroom.Depart`	same	–
Crew garage in the city (light banks striking with flicker, roller door)	Bank intensities over time; smoothstep door travel 5.4 m over 3 s	`CrewGarage.Update`, `PlaceDoor`	`Scripts/Gameplay/CrewGarage.cs`	NightRun `CrewGarage`
Security gates (sliding panels, warning strobes)	Kinematic panels moved in `FixedUpdate` (they push cars), eased	`SecurityBarrier.MovePanels`, `SetWarning`, `Ease`	`Scripts/Gameplay/Mission/SecurityBarrier.cs`	5 barriers
Gate bolts, lamps, hologram	Bolts slide into / out of the post (0.5 s); lamp pulse, flash, colour	`GateLockSystem.Update`, `PlaceBolts`	`Scripts/Gameplay/Mission/GateLockSystem.cs`	AlleyGate
Data Core armoured sleeve	Six panels unlock in sequence with an outward kick and sink into the pedestal over 3.2 s	`DataCoreChamber.UpdateSleeve`	`Scripts/Gameplay/Mission/DataCoreChamber.cs`	built by `NightRunBuilder.BuildChamber`
Data Core arms (hydraulic connector and scanner)	Carriages ride a circular rail (`RailPoint`); the connector arm is solved with **two-bone IK, elbow up** (`Pose`) toward the car roof; the scanner sweeps a beam	`DataCoreChamber.UpdateArms`	same	`BuildArm`
Data cable and pulses	Curved path sampled each frame (`BuildPath`, `Sample`); pulses travel along it at 2.5–16 per second	`DataCoreChamber.UpdateLink`	same	–
Core column, scan rings, holograms, strobes	Colour ramps, ring spin, scan-plane sweeps, hologram tearing	`UpdateCore`, `UpdateFacility`, `UpdateHolograms`	same	–
Terminal screens	Shader parameters (stage strip, scroll, progress, scan line, tear)	`TerminalDisplay.Update`	`Scripts/Gameplay/Mission/TerminalDisplay.cs` + `NeonRift_TerminalScreen.shader`	kiosks
Security cameras	Head yaw/pitch tracking the player in its cone; LED colour	`SecurityCamera.Update`, `SetTracking`	`Scripts/Gameplay/City/SecurityCamera.cs`	12 poles
Security drones (lockdown)	Steering toward a chase slot, bank and pitch from velocity, searchlight aim, strobes	`SecurityDrones.Update`	`Scripts/Gameplay/City/SecurityDrones.cs`	`PF_Drone_Mech`
Mech Drone model (hover/limb motion)	**Legacy** `Animation` playing the FBX's "Take 001" clip in a loop, culled by renderers	set up by `DroneModels.Mech`	`Scripts/Editor/World/DroneModels.cs`	`Prefabs/Drones/PF_Drone_Mech.prefab`

What moves	How it moves	System responsible	Code	Asset / prefab
Buster Drone sentries	Pose **sampled once** at 14 s of its lift-off clip and baked to static meshes; at runtime only a bob and a scanning yaw	`DroneModels.Buster` + `HoverSentry.Update`	`Scripts/Editor/World/DroneModels.cs`, `Scripts/Gameplay/City/HoverSentry.cs`	`PF_Drone_Buster.prefab`
Intro drones and traffic	Flights (lerp + bob), orbits around the core, lanes, all from Timeline time	`IntroCityLife.Apply`	`Scripts/Intro/IntroCityLife.cs`	`Intro/CityLife`
Intro shot cameras	**Timeline AnimationTracks** (eased position + look quaternion) cut by a Cinemachine track	`IntroBuilder.MoveClip`, `BuildTimeline`	`Scripts/Editor/Intro/IntroBuilder.cs`	`Data/Intro/Intro_NightRun.playable`
Intro push-in	Catmull-Rom camera path	`IntroSceneEntry.UpdatePushIn`	`Scripts/Intro/IntroSceneEntry.cs`	–
Chase camera	See 38	`VehicleChaseCamera`	`Scripts/Gameplay/VehicleChaseCamera.cs`	–
Lockdown wave	Per-group delayed emission and light colour fades, pulsing	`SecurityLightGroup.Update`	`Scripts/Gameplay/Mission/SecurityLightGroup.cs`	35 groups
Screens / billboards	Material swap with the wave; glitch flicker	`LockdownScreens.Update`	`Scripts/Gameplay/City/LockdownScreens.cs`	–
Traffic signals	Shared-material cycle; amber stutter; red flashing	`TrafficSignalNetwork.Update`	`Scripts/Gameplay/City/TrafficSignalNetwork.cs`	–
Sirens and beacons	Beacon rotation; siren fade-in after the wave	`SecurityAlarm.Update`	`Scripts/Gameplay/Mission/SecurityAlarm.cs`	–
Crowds	Slow sway; groups step out of sight within 16 m	`CrowdGroups.Update`	`Scripts/Gameplay/City/CrowdGroups.cs`	–
Aviation lights	Blink in the `NeonRift/SkylineBackdrop` shader's aviation mode	shader	`Art/Shaders/NeonRift_SkylineBackdrop.shader`	–
UI	USS transitions and class toggles; counting numbers; easing bars; Intro overlay fades and flashes	`MissionHud`, `CarSelectScreen.StatRow.Tick`, `IntroOverlay`	`Scripts/Gameplay/Mission/MissionHud.cs`, `Scripts/Frontend/CarSelectScreen.cs`, `Scripts/Intro/IntroOverlay.cs`	`UI/*.uss`

37.3 Not animated

Gantry cranes, the tower crane, the Spire monument and the skyway are static geometry. Pedestrians have no walk cycles.

38 Camera System

IN THIS CHAPTER

38.1 Gameplay chase camera: VehicleChaseCamera
38.2 Special framings during the mission
38.3 Car Select cameras
38.4 Intro cameras
38.5 Test track

38.1 Gameplay chase camera: `VehicleChaseCamera`

`Assets/_NeonRift/Scripts/Gameplay/VehicleChaseCamera.cs`, `[DefaultExecutionOrder(-50)]`, on a `CinemachineCamera`. It **computes the pose itself** and writes it into the Cinemachine camera before the `CinemachineBrain` updates; Cinemachine remains the blending layer (Decision D18). It follows the body rigidly in position and **never damps roll or pitch**, so you see what the chassis is doing.

Behaviour	Field (code default)	Night Run value (set by `NightRunBuilder.BuildMissionRig`, lines 931–941)
Distance / height behind the car	`distance` 5.6 m, `height` 1.75 m	**6.2 m / 2.05 m**
Speed pull-back / height change	`speedDistance` 1.2, `speedHeight` −0.25	–
Heading swing toward the direction of travel when sliding	`travelHeadingWeight` 0.55 above `travelHeadingMinSpeed` 4 m/s, `yawSmoothTime` 0.22 s	–
Look-ahead (leads forward travel only, fading in over 0–3 m/s)	`lookAheadTime` 0.18 s, `maxLookAhead` 6 m	**0.26 s / 8 m**
Speed FOV	`baseFieldOfView` 58°, `speedFieldOfView` +9°	**60° + 12°**
Acceleration lag	`accelerationLag` 0.7, smooth 0.25 s	–
Collision	Sphere cast (`collisionRadius` 0.25, `minDistance` 1.6) against `obstacleLayers` = Drivable (7) + Environment (8)	–
Cornering bank	`rollPerG` 1.6° per g of lateral acceleration, `maxRoll` 3°, eased 0.3 s, faded out below 8 m/s	–
Framing (heist and gate)	`frameSwing` 38°, `frameRise` 3.6 m, `framePullBack` 5.5 m, `frameBlend` 1.4 s	–
Theft beat	`beatSwing` 55°, `beatRise` 3.4 m, `beatPullBack` 5 m, `beatBlend` 0.5 s	–

Behaviour	Field (code default)	Night Run value (set by `NightRunBuilder.BuildMissionRig`, lines 931–941)
Shake	`impactShake` 0.05, `speedShake` 0.012, `shakeDecay` 3.5, `maxShake` 0.45	–

Public API: `SetTarget`, `Snap` (after a teleport or recovery), `Kick` (impact shake), `Focus(point, seconds, weight)`, `CinematicBeat`, `SetFraming(target, weight)`; `OnCollided` reacts to car impacts. **Perspective** projection only; the camera far clip is extended for the skyline (`SkylineBackdrop.RequiredFarClip` = 3.5 km), and `CameraCullDistances` sets per-layer culling (Detail 180 m, Clutter 90 m).

38.2 Special framings during the mission

Moment	Caller	Effect
Data Core chamber opening and extraction	`DataCoreChamber.UpdateCamera`	Slow, wide framing of the car and the core; the player keeps control
Gate hack / release within 40 m	`GateLockSystem.UpdateCamera`	Camera rises and frames the gate, near-straight behind the car
Interference	`MissionDirector.OnInteractionFeedback`	`Kick(0.12)`
Lockdown	`MissionDirector.OnSecurityChanged`	`Kick(0.35)`, `Focus` on the alert origin + 20 m
Theft beat (`core.breached`)	`MissionDirector.StartBeat`	Swing to the far side of the car from the core, letterbox, time scale 0.45 for 2.6 s (unscaled timing, restored at the end)
Recover / out of bounds	`MissionSceneEntry`	`Snap()`

38.3 Car Select cameras

`VehicleShowroom` switches Cinemachine cameras by priority (`SetCamera`): **Hero** (27° lens, slow dolly drift `heroDrift` ±9° over 26 s), **Swap** (low side shot during a switch), **Depart** (follow camera for the drive-out), **Opening** (headlight close-up that matches the intro's final frame; entered with a Cut blend). Blends: `Assets/_NeonRift/Settings/Rendering/Garage_CameraBlends.asset`. Cameras are created by `GarageBuilder` (15).

38.4 Intro cameras

25 Timeline-animated shot cameras cut by a Cinemachine track, plus the push-in camera (31, 32).

38.5 Test track

`Scenes/Dev/TestTrack.unity` uses the same `VehicleChaseCamera`, wired by `ValidationRouteBuilder.ConfigureScene`.

PART XI

Engineering, Quality and Release

Performance, testing, builds, repository, licensing and limits.

39 Performance

40 Testing and Validation

41 Input Playtesting

42 Build and Release

43 Git and Repository

44 Assets and Licensing

45 Known Limitations

39 Performance

IN THIS CHAPTER

39.1 Optimisations that exist
39.2 Measurement tools
39.3 Recorded results
39.4 Remaining hotspots (recorded)

Every number in this chapter comes from `Docs/Performance.md`, `Docs/Environment.md` or `Docs/City.md`, where the measurements were recorded. Editor numbers include editor overhead and drift between runs; decisions were made on interleaved A/B measurements, and absolute numbers come from the player build.

39.1 Optimisations that exist

Area	Technique	Where
Geometry batching	One combined mesh **per material per block** (`BlockMeshes`), static batching flags, SRP Batcher, GPU Resident Drawer	`Scripts/Editor/District/BlockMeshes.cs`, `DistrictKit.Renderer`, `Settings/Rendering/PC_RPAsset.asset`
Cell splitting for culling	Intersection furniture and tower dressing in 400 m cells; infill clutter in 64 m cells	`CityProps.BuildIntersections`, `TowerDressing.Cell`, `CityInfill.ClutterAt`
Layer cull distances	Detail layer beyond 180 m, Clutter layer beyond 90 m (spherical, so turning doesn't pop)	`Scripts/Gameplay/CameraCullDistances.cs`
LODs	Vehicle LOD0/1/2 + cull (`VehicleLodSetup`); hero buildings drop interiors below 15 % (`HeroLod`); prop LODs (`DistrictKit.AddLod`)	`Scripts/Editor/Vehicles/VehicleLodSetup.cs`, `NightRunDistrict.HeroLod`
Light budget	787 lamps with disabled spots; the 56 nearest within 150 m on, 2 shadowed	`Scripts/Gameplay/City/LightBudget.cs`
Emissive-only interiors and far lamps	Infill lighting and distant streets use emission and ground pools, not lights	`CityInfill.YardLamp`, `CityProps.LampHaze`
Shadows	Moon cascades 4 → 2, shadow distance 90 m; **soft-shadow quality Low** (−37 % GPU, no visible change); cars and vans cast no shadows in infill; drones and sentries cast none	`NightRunBuilder.ConfigurePipeline`, `DroneModels`
Skyline	About 3 k-triangle backdrop rings in one draw; Sketchfab skyline clusters removed (< 0.2 % pixel change)	`SkylineBackdrop.cs`, `CityLayout.Skyline` (empty)

Area	Technique	Where
Physics	100 Hz; Traffic ignores Traffic; parked cars kinematic until hit; one simple box collider per mass; no mesh colliders on the city	`TimeManager.asset`, `CityTraffic.Start`, `TrafficCar`
Allocation-free frame	HUD strings rebuilt only on change; index loops instead of `foreach` over `IReadOnlyList` (no boxed enumerators); race rows in 10 % steps	`MissionDirector`, `MissionHud`, `VehicleAudio`, `RacerDriver`, `RivalDirector`
Audio voices	64 real voices, explicit priorities, other cars +140	`ProjectSettings/AudioManager.asset`, `VehicleAudio`
Intro rig	Inert during missions; intro city life only active in its shot windows	`IntroSceneEntry`, `IntroCityLife`
Sentry drone	Posed skinned mesh baked to a static mesh (no per-frame skinning)	`DroneModels.Buster`

Kept for the look: MSAA 4× (2× saved 7–12 % inconsistently and softened neon edges) and SSAO (off saved 11 %).

39.2 Measurement tools

Tool	File	What it does
`RoutePerf`	`Scripts/Editor/Validation/RoutePerf.cs`	Rides along a Play Mode scenario; per frame: PlayerLoop and main thread, GPU (`FrameTimingManager`), GC, SetPass, triangles, shadow casters, lights, phase, position → `Logs/Perf/<prefix>_seconds.csv` and `_summary.md` per segment (approach, heist, escape)
`GpuBench`	`Scripts/Editor/Validation/GpuBench.cs`	Freezes the scene and A/B tests URP variants (soft shadows, MSAA, SSAO …), 120 settle + 300 measured frames, three interleaved rounds
`PerfBuild` + `PerfRouteRunner` + `PerfRouteArgs`	`Scripts/Editor/Validation/PerfBuild.cs`, `Scripts/Gameplay/PerfRouteRunner.cs`, `Scripts/Game/PerfRouteArgs.cs`	**Neon Rift ▸ Validation ▸ Build Perf Player (macOS)** builds a release player; `-perfroute <folder>` boots into the mission, drives the BoulevardInAlleyOut route and writes `perfroute_summary.md` / `perfroute_seconds.csv`
Unity Profiler	–	Hotspot attribution (GC per sample, time per script)

39.3 Recorded results

GPU bench (lockdown escape, frozen, 1080p): baseline 21.06 ms → soft shadows Low **13.32 ms (−37 %)**; soft shadows off −31 %; MSAA 2× −7 %.

Player build (release macOS, Apple M3, 1920×1080 windowed, V-sync off, full route; measured **before** the pack-car traffic replaced the earlier block traffic):

Segment	FPS avg	Frame avg	p95	CPU main	GPU avg	GC/frame
Approach	35	28.9 ms	47.1	26.2 ms	23.5 ms	0
Heist	38	26.0 ms	38.4	24.2 ms	17.5 ms	0
Escape	39	25.4 ms	39.1	23.7 ms	22.1 ms	0

Segment	FPS avg	Frame avg	p95	CPU main	GPU avg	GC/frame
Whole route	**38**	26.5 ms	41.4	24.5 ms	21.3 ms	**0**

Scene stats at the spawn view (editor, `UnityStats`): 3.82 M triangles (phase 1) → 2.94 M after vehicle LODs and 2 cascades → 4.06 M after the block infill (mostly shadow and depth passes over the new masses).

No newer player-build measurement is recorded after the pack traffic, the rival heists, the drones or the intro city life were added.

39.4 Remaining hotspots (recorded)

Mission start hitches (shader first use; a warm-up collection would help), tower triangles (towers have no LODs: 2.6–4.2 M triangles in view), and about 70 real-time lights in the lockdown (beacons, strobes, drone searchlights).

40 Testing and Validation

IN THIS CHAPTER

40.1 Automated tests (EditMode)
40.2 Play Mode validation tools (editor)
40.3 Test → purpose → location (selected highlights)
40.4 Recorded validation runs
40.5 Unity console validation

40.1 Automated tests (EditMode)

- **Location:** `Assets/_NeonRift/Scripts/Tests/EditMode/` (assembly `NeonRift.Tests.EditMode`).
- **Count:** 98 test methods in 11 files. Physics tests run once per catalog car (`[TestCaseSource]`), giving **125 test cases**. The last recorded run (`Logs/TestResults-EditMode.txt`, local, not committed) shows `passed=125 failed=0 skipped=0`.
- **Run them: Window ▸ General ▸ Test Runner ▸ EditMode**, or **Neon Rift ▸ Tests ▸ Run EditMode Tests (write report)**. The second option uses `Scripts/Editor/TestRunReporter.cs` and writes a plain-text report.
- **Note:** after the tests the editor is left on an untitled scene, so reopen `NightRun.unity` before pressing Play.

Test file	Methods	What it covers
`FoundationTests.cs`	5	Catalog duplicate ids and missing prefabs, lookup and default, session readiness, result record/clear, project wiring (`ProjectValidator`)
`InputTests.cs`	8 (+ fixture setup)	Keyboard throttle, brake, steer, combined; handbrake and interact; gamepad driving and buttons; Menu map disable leaves Driving on (an `InputTestFixture`)
`VehicleModelTests.cs`	16	Tyre curve, slip direction, peak and sliding, combined slip, friction ellipse, load sensitivity, surface grip; steering lock and Ackermann; gearbox shifts and hysteresis; reverse selection; launch clutch; handbrake clutch; profile validity; AMG GT3 excluded and NC licences flagged
`VehiclePhysicsTests.cs`	11 (× cars)	Spawn at ride height without jitter, acceleration band, top speed, straight braking, cornering grip, full-lock step steer (slides, no spin), speed bumps, wall impact and recovery, slope hold, validation-route lap, spawn applies the profile (on the deterministic `VehicleTestBench`)
`VehicleAudioTests.cs`	12	Engine model equal-power and continuity; tyre squeal; profiles valid; loops seamless, one-shots clean; mixer groups, snapshots and volumes; `VehicleAudio` follows telemetry without restarting loops; skid follows slip; score stems sample-locked; `ScoreMix` rules and grid timing; `MasterLimiter`

Test file	Methods	What it covers
`CityTests.cs`	13	Network valid and connected; city size; every block has a zone and pavement; heroes avoid roads; spawn → core path; closed gates re-route; remembered blocked edges; skyway; racing line slows for corners; rubber band bounded; rivals are the cars not picked; city life set up; traffic cars physical and ignoring each other
`MissionTests.cs`	11	Objectives advance and fire events; heat → Alert and shorter timer; heat clamp and minimum timer; timer failure; untimed objectives never fail; Night Run data valid; **scene provides every target and reaction** (including rival tasks and markers); grade; records
`Interaction Tests.cs`	6	Stages in order with events; timing hits and misses, heat, lockout; interference stall and rollback; leaving cancels to the checkpoint; cues fire once; objective start delay
`IntroTests.cs`	5	First-launch policy; Always/Never; Video without a clip falls back; config points at the intro; NightRun scene has a wired, inert intro
`MinimapTests.cs`	5	North-up / east-right; projection independent of heading; round trip; compass headings; player arrow direction
`CarSelectTests.cs`	2	Catalog cars have Car Select data; CarSelect scene is a wired garage

40.2 Play Mode validation tools (editor)

Tool	File	Purpose
`NightRunValidator`	`Scripts/Gameplay/Mission/NightRunValidator.cs`	Plays the real mission through the player's interfaces: the autopilot drives a `DrivingRoute`, stops at the interactable in focus, holds Interact until it completes. Scenarios: `BoulevardInAlleyOut`, `AlleyInExpresswayOut`, `TraceTimeout`, `AlleyInFailRetry` (`GateTrick` misses on purpose). Reports a timeline of objectives, security, heat, events, collisions and `RivalDirector.DescribeAi()`
`MissionPlaytest`	`Scripts/Editor/Validation/MissionPlaytest.cs`	Starts a scenario, captures the Game view after chosen world events and stages (`"event@delay"`, `"stage:LABEL@delay"`, `rival0:` / `aerial:` prefixes), writes `Logs/Playtest/<prefix>_report.txt`
`MissionSeries`	`Scripts/Editor/Validation/MissionSeries.cs`	Several scenarios back to back via the normal retry path → `Logs/Playtest/<prefix>_series.txt` (player collisions, rival contacts with positions, reversals, resets, rivals finished)
`PlaytestCapture`	`Scripts/Editor/Validation/PlaytestCapture.cs`	Timed captures from the editor loop (used for Car Select)
`IntroCapture`	`Scripts/Editor/Intro/IntroCapture.cs`	Captures the intro at given Timeline times; writes `_done.txt`
`LookDevShots`	`Scripts/Editor/District/LookDevShots.cs`	Fixed viewpoints for look comparisons
`InputPlaytest`	`Scripts/Editor/Validation/InputPlaytest.cs`	Real keyboard events in Play Mode (41)

Tool	File	Purpose
`VehicleAudioValidator`	`Scripts/Gameplay/VehicleAudioValidator.cs`	Scripted audio run with listener recording and signal analysis
`VehiclePerformanceProbe`, `VehicleTestBench`	`Scripts/Editor/Vehicles/`	Deterministic physics measurements
`RouteAutopilot`, `VehicleTelemetryLog`, `VehicleDebugHud`	`Scripts/Gameplay/`	Validation laps, per-section telemetry, F3 overlay
`RoutePerf`, `GpuBench`, `PerfBuild`	`Scripts/Editor/Validation/`	Performance (39)
`ProjectValidator`	`Scripts/Editor/ProjectValidator.cs`	**Neon Rift ▸ Validate Project**: config, catalogs, build scenes; warnings for non-commercial assets

40.3 Test → purpose → location (selected highlights)

Test	Purpose	Location
`MissionTests.NightRunSceneProvidesEveryTargetAndReaction`	The generated scene contains every target, reaction, gate, rival marker and work FX the data names	`Tests/EditMode/MissionTests.cs:157`
`VehiclePhysicsTests.CompletesValidationRouteLap_OnLine_AndUpright`	Every catalog car laps the 1.55 km route on the autopilot	`Tests/EditMode/VehiclePhysicsTests.cs:142`
`CityTests.ClosedGates_ChangeTheRoute`	Navigation respects gates	`Tests/EditMode/CityTests.cs:86`
`IntroTests.NightRunScene_HasAWiredIntro`	Intro wiring is complete and inert in missions	`Tests/EditMode/IntroTests.cs:75`
`MinimapTests.Projection_DoesNotDependOnPlayerHeading`	North-up guarantee	`Tests/EditMode/MinimapTests.cs:29`
`VehicleAudioTests.VehicleAudio_FollowsTelemetry_WithoutRestartingLoops`	Loops are never restarted	`Tests/EditMode/VehicleAudioTests.cs:142`

40.4 Recorded validation runs

Recorded in `Docs/NightRun.md`, `Docs/City.md`, `Docs/Intro.md` and `Docs/Audio.md`: e.g. rival regression series with 0 contacts after the intent rules; final rival-heist runs (VEX 124.9 s vs the player at about 125 s; the player won at 127.8 s); `TraceTimeout` failing as designed at 71.3 s; the intro played to the title and Skip and Confirm verified; the departure heading held at 180°.

40.5 Unity console validation

Systems log with prefixes (`[GameFlow]` , `[Mission]` , `[Rivals]` , `[Input]` , `[Intro]` , `[CarSelect]` , `[Spawn]` , `[NightRun]`). `[Rivals] contact …` lines record the geometry of every rival contact for diagnosis.

41

Input Playtesting

IN THIS CHAPTER

41.1 Why it exists
41.2 InputPlaytest (Assets/_NeonRift/Scripts/Editor/Validation/InputPlaytest.cs)
41.3 Recorded uses
41.4 Related

41.1 Why it exists

In October 2026 "WASD does nothing" turned out to be the editor's Input System backend having **lost all devices** (`InputSystem.devices.Count == 0`), not a code bug. `InputPlaytest` was written to drive the **real keyboard device** from tooling, so the full input path is exercised: device → action maps → enable/disable lifecycle → `PlayerDrivingInput` → `VehicleController`.

41.2 `InputPlaytest` (`Assets/_NeonRift/Scripts/Editor/Validation/InputPlaytest.cs`)

Member	What it does
`Hold(keys…)`, `Release(keys…)`, `ReleaseAll()`, `Tap(key)`, `Push(gamepad values)`	Write **state events** into the real `Keyboard` / `Gamepad` devices (the same path as physical presses)
`Step`	One timed step: hold exactly these keys (and gamepad values) for a duration, then log a snapshot
`RunStandardScript`	Drives W, D, Space, arrows, etc., and logs vehicle state per step
`RunMinimapScript`	Straight, left, right, sustained left: logs headings and captures the minimap
`Run`, `Tick`, `Advance`, `Finish`, `Snapshot`, `InteractState`	Sequencer on the editor loop; writes **`Logs/InputPlaytest.txt`**

Requirement: the Game view (the Unity app) must be in front, or Unity's background behaviour resets the keyboard.

41.3 Recorded uses

- Driving script (README): W, D, Space and the arrows move the car.
- Pause menu (`Docs/NightRun.md`): Esc paused (time scale 0, listener paused); ↓↓↓ Enter opened Settings; M switched music off (mixer −80 dB, setting saved, HUD hint amber); Esc ↑ Enter opened Controls; Esc Esc resumed (time scale 1).
- Minimap: headings 0° → 250° → 267° → 128°; W Avenue stayed vertical on the map.

41.4 Related

- Unit tests of bindings: `InputTests` (40).
- Device diagnostics at mission start: `PlayerDrivingInput.DescribeDevices` logs `[Input] Driving controls enabled. Devices: …` (16).

42 Build and Release

IN THIS CHAPTER

42.1 Requirements
42.2 Opening and running the project
42.3 Scenes in the build (ProjectSettings/EditorBuildSettings.asset, all enabled)
42.4 Regenerating content (if needed)
42.5 Building a player
42.6 Release checks

42.1 Requirements

Unity	**6000.6.4f1** (or a later 6000.6.x; `ProjectSettings/ProjectVersion.txt`)
Packages	Resolved automatically from `Packages/manifest.json` (03)
Git LFS	**Required** before cloning. About 4,900 files are LFS objects (models, textures, audio, EXR probes, screenshots, generated district meshes) under 49 `.gitattributes` LFS rules
Input	Input System only (`activeInputHandler: 1`)
Product	"Neon Rift: Night Run", version 0.1.0, company "DefaultCompany" (`ProjectSettings/ProjectSettings.asset`)

42.2 Opening and running the project

```
git lfs install
```

```
git clone https://github.com/soyebmohammad03-dev/NEON-RIFT.git
```

1. Open the folder in Unity 6000.6.x (the Hub resolves the packages).
2. Open `Assets/_NeonRift/Scenes/Bootstrap.unity` and press Play: the intro (first launch) or the title → Car Select → Night Run.
3. Or press Play in `NightRun.unity` to jump straight into the mission with the first catalog car (`BootstrapLoader` + `GameFlow.AdoptScene`).
4. **Neon Rift ▸ Validate Project** checks config, catalogs and build scenes.
5. To see the intro again: **Neon Rift ▸ Intro ▸ Reset 'Intro Seen'**, or set `Data/Config/IntroSettings.asset` Policy to `Always`.

42.3 Scenes in the build (`ProjectSettings/EditorBuildSettings.asset`, all enabled)

Index	Scene
0	`Assets/_NeonRift/Scenes/Bootstrap.unity`
1	`Assets/_NeonRift/Scenes/Frontend.unity`
2	`Assets/_NeonRift/Scenes/CarSelect.unity`
3	`Assets/_NeonRift/Scenes/Dev/TestTrack.unity`
4	`Assets/_NeonRift/Scenes/NightRun.unity`

`NightRunBuilder.RegisterInProject` keeps the build list and `GameConfig` wired. The dev mission (`Mission_DevTestTrack`) can be marked `developmentOnly` so release builds hide it (`GameConfig.IsPlayable`).

42.4 Regenerating content (if needed)

Content	Menu
Night Run city, mission, intro, probes	Neon Rift ▸ Night Run ▸ Build Night Run (district + mission) (about 40 s)
Car Select garage	Neon Rift ▸ Car Select ▸ Build Garage
Vehicle prefabs and catalog	Neon Rift ▸ Assets ▸ Run Third-Party Intake / Rebuild Vehicle Prefabs And Catalog
Vehicle LODs	Neon Rift ▸ Vehicles ▸ Apply Vehicle LODs To Catalog Prefabs
Car Select stats	Neon Rift ▸ Vehicles ▸ Update Car Select Stats From Physics
Audio	Neon Rift ▸ Audio ▸ Generate Vehicle Audio / Generate Mission Audio / Build Audio Mixer / Add Vehicle Audio To Prefabs; Neon Rift ▸ Intro ▸ Generate Intro Audio
Test track route	Neon Rift ▸ Test Track ▸ Build Validation Route

42.5 Building a player

- **Standard:** File ▸ Build Profiles, the target platform (desktop), the scenes above. The PC quality level uses `PC_RPAsset`; Mobile has its own RP asset.
- **Performance player: Neon Rift ▸ Validation ▸ Build Perf Player (macOS)** → `PerfBuild.Build` (`Assets/_NeonRift/Scripts/Editor/Validation/PerfBuild.cs`): enabled build scenes, `BuildTarget.StandaloneOSX`, `BuildOptions.None` (a release, not development), output under `Builds/PerfMac` (git-ignored). Run it with `-perfroute <folder>` for the automated route profile (39).

Only a macOS player build is documented as built and measured in the repository. No CI pipeline is configured.

42.6 Release checks

- Non-commercial assets: the **Terzo Millennio** (CC-BY-NC-4.0) is in the catalog, the **AMG GT3** (CC-BY-NC-SA) is excluded, and the **Drone Design** model (CC-BY-NC-ND) is unused. `Project Validator.Warnings()` lists non-commercial definitions. The two music tracks and the passenger car pack have **no licence recorded** (44).
- Trademarks (Mercedes-Benz/AMG, Lamborghini and liveries) would need de-branding for a commercial release (`Docs/ThirdParty.md`).

43
Git and Repository

IN THIS CHAPTER

43.1 Repository
43.2 Git LFS and merging
43.3 Never committed (.gitignore)
43.4 History (29 commits, oldest at the bottom)
43.5 Project licence

43.1 Repository

Remote	`https://github.com/soyebmohammad03-dev/NEON-RIFT` (public)
Branch	`main` (single branch)
Author	All commits are authored by **Soyeb Mohammad** (the first two under the local name `soyebmohammad`)
Guide	`Docs/SourceControl.md`

43.2 Git LFS and merging

- `.gitattributes` has 49 LFS rules: models (FBX, GLB, OBJ, blend), textures, audio, video, EXR reflection probes, screenshots, zips, and the generated district meshes (`Assets/_NeonRift/Art/District/Meshes/*.asset`). About 4,900 files are stored in LFS.
- Unity YAML (scenes, prefabs, assets, materials, mixers, playables, lighting) is text with `merge=unityyamlmerge`. Register Unity's SmartMerge once per clone (`Docs/SourceControl.md`):

```
git config merge.unityyamlmerge.name "Unity SmartMerge"
```

```
git config merge.unityyamlmerge.driver "'<Unity.app>/Contents/Tools/UnityYAMLMerge' merge -p %O %B %A %A"
```

43.3 Never committed (`.gitignore`)

`Library/`, `Temp/`, `Obj/`, `Build/`, `Builds/`, `Logs/`, `UserSettings/`, `MemoryCaptures/`, `Recordings/`, `ProfilerCaptures/`, IDE and solution files (`*.csproj`, `*.sln`, `*.slnx`), `SourceArchives/` (original downloaded archives) and the stray `My project/` folder.

43.4 History (29 commits, oldest at the bottom)

Commit	Date	Summary
`602c7130`	2026-10-04	Rival heists and qualifying, drone pursuit, Car Select exit fix, busier intro
`26d7db2d`	2026-10-04	Pack-car traffic, licensed music, pause menu and settings, full-route performance
`3ba63dd6`	2026-10-04	Adaptive procedural score, voice priorities and master limiter
`90cdba32`	2026-10-04	Mission presentation: operation briefing, split stamps, graded debrief
`c9e7ae9a`	2026-10-04	City life: pooled ambient traffic, plaza crowds, security drones
`3e96fbee`	2026-10-04	URP visual pass: restrained lockdown grade, security red, rear-glass fix
`f9925bb9`	2026-10-04	Car Select premium pass
`cb96f3e3`	2026-10-04	Cinematic intro rework: 84 s story through the crew garage
`c54fd8b0`	2026-10-04	Three-car heist and rival collision fix
`a70e49b8`	2026-10-04	Security gate hack: five-stage controller hack with physical locks
`54ac9c00`	2026-10-04	Data Core heist: multi-stage interactions, extraction chamber
`4b6b6a76`	2026-10-04	Car Select: generated crew garage, turntable, departure
`1a11bafc`	2026-10-04	North-up minimap and block-interior city fill
`c4cfa0a6`	2026-10-03	Opening cinematic (first version)
`5573b5c9`	2026-10-03	Camera and HUD polish
`795c55d3`	2026-10-03	LODs and performance
`2355d226`	2026-10-03	City detail
`62487243`	2026-10-03	City lighting and atmosphere: tower facades, crowns, ground haze
`2b072ff7`	2026-10-03	Docs: rival validation runs
`4e6a57a0`	2026-10-03	Rival AI: traffic awareness, path occupancy, blocked-route memory
`b187d1af`	2026-10-03	Input: verify and harden keyboard/gamepad driving
`dcf63430`	2026-10-03	Night look foundation
`16285452`	2026-10-03	Licence and ownership notice, source-control guide
`62ebb897`	2026-10-03	Night Run city: five districts, rival AI, night lighting, navigator HUD
`de7bbb83`	2026-10-02	Night Run vertical slice
`3d5ae392`	2026-10-02	Engine audio
`9e77107f`	2026-10-02	Vehicle physics
`97530885`	2026-10-02	Asset intake
`be0ec0be`	2026-10-02	Phase 1: project foundation

History note (`Docs/SourceControl.md`): five early commits (`be0ec0b`, `9753088`, `9e77107`, `3d5ae39`, `de7bbb8`) carry a `Co-Authored-By: Claude` trailer added by the AI coding tool used during development, and were published unchanged on purpose. Later commits carry no such trailer.

43.5 Project licence

`LICENSE.md` at the repository root (ownership notice, added in `16285452`).

44

Assets and Licensing

IN THIS CHAPTER

44.1 First-party content
44.2 Third-party assets
44.3 How licences travel with content (Decision D14)
44.4 Rules (Decision D9, Docs/Conventions.md)
44.5 Trademarks
44.6 Overall commercial status
44.7 Gaps

Licence information below is copied from the repository's own records: each folder's `ATTRIBUTION.txt`, `Docs/ThirdParty.md`, `LICENSE.md` and the `AssetLicense` fields on the vehicle definitions. Nothing here is inferred.

Licensing at a glance (a regrouping of the facts in this chapter; nothing added)

Category	What it covers	Licence status
Project-created content	Source code, scenes, UI, shaders (`LICENSE.md`)	First-party, "All rights reserved"; public for viewing, no open-source licence
Procedural / generated content	Generated meshes and textures, the signage font (`PixelFont`), all synthesised audio: engines, sirens, cues, score, intro sounds	First-party (covered by `LICENSE.md`)
Third-party, licence known	SLS AMG, SLS GT3, the 99.Miles buildings, Mech Drone, Buster Drone (CC BY 4.0); Terzo Millennio (CC-BY-NC-4.0); AMG GT3 Red Bull, unused (CC-BY-NC-SA-4.0); Drone Design, unused (CC BY-NC-ND 4.0)	Recorded in each `ATTRIBUTION.txt` and `Docs/ThirdParty.md`
Third-party, provenance / licence undocumented	Generic passenger car pack (10 cars); `machine-speed-sport-aggressive-electronic.mp3`; `danger-chaos-cinematic-trailer-hybrid.mp3`	**Not recorded**: supplied by the project owner, no licence file
Rejected	dasy444 "city at night low poly skyscrapers"	Not imported

44.1 First-party content

`LICENSE.md`: "Copyright (c) 2026 Soyeb Mohammad. All rights reserved." This covers the source code, scenes, generated meshes and textures, the signage font (`PixelFont`), shaders, UI and **all audio not listed as third-party** (the engines, sirens, cues, score and intro sounds are synthesised in-house). The repository is public for viewing; no open-source licence is granted.

44.2 Third-party assets

Asset	Source	Licence	Location	Usage	Commercial restriction
2010 Mercedes SLS AMG (Dave Love SketchFab)	sketchfab.com/3d-models/2010-mercedes-sls-amg-fa3fd5eeea674f37bb03283f2c53d563	CC-BY-4.0	`Assets/ThirdParty/Sketchfab/Vehicles/Mercedes_SLS_AMG_2010/`	Player/rival car (catalog)	Commercial allowed with attribution
Mercedes SLS GT3 (Dave Love SketchFab)	…/mercedes-sls-gt3-5ec93bea4816494cbe3e7333bbfca5f6	CC-BY-4.0	`…/Vehicles/Mercedes_SLS_GT3/`	Player/rival car (catalog)	Commercial allowed with attribution
(FREE) Lamborghini Terzo Millennio (SDC PERFORMANCE)	…/free-lamborghini-terzo-millennio-7ad3dffa9d344c3c978eafcc220cb709	**CC-BY-NC-4.0**	`…/Vehicles/Lamborghini_Terzo_Millennio/`	Player/rival car (catalog)	**Non-commercial only**
Mercedes-AMG GT3 Red Bull Racing (VTX)	…/mercedes-amg-gt3-red-bull-racing-035f061cb6624effbc483d5f9b5ecaba	**CC-BY-NC-SA-4.0**	`…/Vehicles/Mercedes_AMG_GT3_RedBull/`	Prefab built, **kept out of the catalog** (material defects)	**Non-commercial, share-alike**
Generic passenger car pack (10 cars)	Supplied by the project owner	**Not recorded** (no licence or author file in the folder; `Docs/ThirdParty.md`: "Record the source and licence here before any distribution")	`…/Vehicles/generic-passenger-car-pack/`	City traffic (`Prefabs/Traffic/PackCar_*`) and intro traffic	**Unknown**
[FREE] London Skyscraper (99.Miles)	…/free-london-skyscraper-52b73f6ea18a440cb42734840d5edc72	CC-BY-4.0	`Assets/ThirdParty/Sketchfab/Buildings/London_Skyscraper/`	Hero landmark (Spire Plaza)	Commercial allowed with attribution
Singapore Office Skyscraper [FREE] (99.Miles)	…/singapore-office-skyscraper-free-2305f0fe03ba44229a1f768dcb48bd8f	CC-BY-4.0	`…/Buildings/Singapore_Office_Skyscraper/`	Hero towers	Commercial allowed with attribution
Asian Themed Low Poly Night City Buildings (99.Miles)	…/asian-themed-low-poly-night-city-buildings-9f0343aff4814b758dc6e905aba5b5e0	CC-BY-4.0 (Sketchfab page)	`…/Buildings/Asian_Night_City_Buildings/`	19 tower prefabs	Commercial allowed with attribution
Low Poly Night City Building Skyline (99.Miles)	…/low-poly-night-city-building-skyline-b0035b8713b048bb8ddf311ee67c28c8	CC-BY-4.0 (Sketchfab page)	`…/Buildings/Night_City_Building_Skyline/`	Catalog entry only (clusters removed from the city)	Commercial allowed with attribution

Asset	Source	Licence	Location	Usage	Commercial restriction
Mech Drone (Willy Decarpentrie)	…/mech-drone-8d06874aac5246c59edb4adbe3606e0e	CC BY 4.0	`Assets/ThirdParty/Sketchfab/Drones/Mech_Drone/`	Security drones, intro drones (`PF_Drone_Mech`)	Commercial allowed with attribution
Buster Drone (LaVADraGoN)	…/buster-drone-294e79652f494130ad2ab00a13fdbafd	CC BY 4.0	`…/Drones/Buster_Drone/`	Compound sentries (`PF_Drone_Buster`)	Commercial allowed with attribution
Drone Design (3DHaupt)	…/drone-design-592c49fb806c42dba309e4d7a31d4a57	**CC BY-NC-ND 4.0**	`…/Drones/Drone_Design/`	**Not used in the game**; kept unmodified for reference	**Non-commercial, no derivatives**
`machine-speed-sport-aggressive-electronic.mp3`	Supplied by the project owner	**Not recorded** (no licence file)	`Assets/ThirdParty/SoundAudio/`	Gameplay music	**Unknown**
`danger-chaos-cinematic-trailer-hybrid.mp3`	Supplied by the project owner	**Not recorded** (no licence file)	`Assets/ThirdParty/SoundAudio/`	Intro soundtrack	**Unknown**
dasy444 "city at night low poly skyscrapers"	Sketchfab	Sketchfab "Free Standard"	Not imported (archive in `SourceArchives/`, git-ignored)	**Rejected**: photo textures of unverifiable provenance	–

Each Sketchfab model folder except the passenger car pack has an `ATTRIBUTION.txt` (title, author, source, licence, origin and the changes made). The drone licence pages were captured as `Docs/Licenses/mech_drone_sketchfab_licence.png`, `buster_drone_sketchfab_licence.png` and `drone_design_sketchfab_licence.png` (committed in `602c7130`).

44.3 How licences travel with content (Decision D14)

`AssetLicense` (`Assets/_NeonRift/Scripts/Core/AssetLicense.cs`: `LicenseId`, `Author`, `SourceUrl`, `CommercialUseAllowed`, `AttributionLine`) is stored on every `VehicleDefinition` and `BuildingDefinition`. `ProjectValidator.Warnings()` lists non-commercial ones so a release build can exclude them. `VehicleModelTests.Catalog_ExcludesAmgGt3_AndFlagsNonCommercialLicences` checks the catalog.

44.4 Rules (Decision D9, `Docs/Conventions.md`)

Vendor packs keep their own import root and are never edited in place. Changes (orientation, scale, material copies, split wheel meshes, baked drone meshes) are made in `_NeonRift` prefabs and assets.

44.5 Trademarks

Mercedes-Benz/AMG, Lamborghini, Red Bull, Lukoil, Blancpain, Michelin and other liveries on the car models are third-party trademarks. A commercial release would need de-branded liveries and fictional names (`Docs/ThirdParty.md`, `LICENSE.md`).

44.6 Overall commercial status

Because the **Terzo Millennio** (CC-BY-NC) is in the player catalog, and three supplied assets have no recorded licence, **the project as it stands must not be used commercially** (`LICENSE.md` says the same about the non-commercial models).

44.7 Gaps

- `LICENSE.md`'s third-party table lists the cars and buildings but **not** the two CC BY drones, the passenger car pack or the music.
- The source and licence of the passenger car pack and the two music tracks still need to be recorded.

45 Known Limitations

IN THIS CHAPTER

45.1 Not implemented
45.2 Vehicle physics (Docs/VehiclePhysics.md)
45.3 AI
45.4 Audio
45.5 Visual
45.6 Performance
45.7 Data and licensing
45.8 Documentation drift

An honest list, collected from `Docs/*.md` and checks made while writing this book.

45.1 Not implemented

Feature	Notes
Rain, weather states, water bodies, splashes	Only an "after rain" asphalt material look (26, 27)
Particle effects (smoke, sparks, debris)	No `ParticleSystem` anywhere (28)
Vehicle damage, destructible props	Collisions give sound, shake and contact counts only
Animated pedestrians or cranes	Crowds are static silhouettes that sway; cranes are static
Global illumination	Baked and realtime GI both off; real-time lights + baked reflection probes
More missions / "next level"	Only Night Run. Qualifying is recorded (`MissionRecords.HasQualified`), but nothing unlocks yet
Manual gear shifting	`Gearbox.Shift(int)` exists as a hook; there is no input binding
Saved car choice between launches	`RunSession` is memory only
Traffic obeying signals	Signals are visual; traffic yields at junctions to the player and rivals
Pre-rendered intro video	The `IntroSettings.Mode.Video` path is implemented but untested (no clip)

45.2 Vehicle physics (`Docs/VehiclePhysics.md`)

No unsprung mass (no wheel hop); no tyre relaxation, temperature or wear; a 1.5 m/s low-speed slip floor; differentials are speed-coupling approximations; a two-box body collider; no anti-dive or anti-squat; the GT3 scrapes 10 cm bumps at ~55 km/h; collision events fire only in Play Mode.

45.3 AI

- Rivals occasionally make racing contact with the player where routes cross (e.g. at the compound gate). Timing-dependent contacts are logged (`[Rivals] contact …`).
- Kade's pace varies with traffic on Lantern Street; the rubber band is deliberately small (±5–6 %).
- The validation driver `RouteAutopilot` is not race AI (no countersteer, fixed braking assumption).

45.4 Audio

Procedural engines are convincing but not recordings of real cars; mild formant shift between layers; the EV is silent at standstill; no occlusion or reverb zones; **the score balance and the intro mix were measured but not judged by ear**.

45.5 Visual

- Towers have no LODs (2.6–4.2 M triangles in view at times).
- Car paint has no clear coat (glTFast clearcoat remapped, D12); NaN safeguards (Stop NaN) remain on cameras (D13).
- In the editor, the first play after a rebuild can show flat-cyan materials while shaders compile asynchronously (not in builds).
- The garage exterior is a plain box (`Docs/Intro.md`).

45.6 Performance

- No player-build profile recorded after the pack traffic, rival heists, drones and intro city life were added.
- Mission start hitches on first shader use; about 70 real-time lights at the lockdown peak.

45.7 Data and licensing

- Three supplied assets (the passenger car pack and two music tracks) have no recorded licence; `LICENSE.md` doesn't yet list the drones, the car pack or the music.
- The Terzo Millennio (CC-BY-NC) is in the catalog, so the project is non-commercial as it stands.

45.8 Documentation drift

Some older notes describe superseded behaviour (rival "orders" and `rivalGoalId`, crew roles NORTH GATE OVERWATCH / EXPRESSWAY SCOUT, the heading-up minimap in the README, "4 small particle systems", "62 s Timeline / 18 shot cameras" in the first part of `Docs/Intro.md`, D26 in `Docs/Decisions.md`). This book documents the current code. See DOCUMENTATION_AUDIT.

PART XII

Reference

Quick reference, viva preparation and glossary.

46 Developer Quick Reference

47 Teacher Q&A Guide

48 Glossary

46 Developer Quick Reference

IN THIS CHAPTER

46.1 System → scripts → scene → prefab → data
46.2 Interfaces
46.3 ScriptableObjects
46.4 Managers, controllers, services, builders, state machines
46.5 Menu cheat-sheet (all under Neon Rift)
46.6 World event ids (Night Run)
46.7 Target and marker ids

Paths: `S/` = `Assets/_NeonRift/Scripts/`, `D/` = `Assets/_NeonRift/Data/`, `P/` = `Assets/_NeonRift/Prefabs/`.

46.1 System → scripts → scene → prefab → data

System	Primary script	Secondary scripts	Scene	Prefab	Data
Boot / flow	`S/Game/GameRoot.cs`	`GameFlow`, `BootstrapLoader`, `GameContext`, `RunSession`, `LoadingOverlay`	Bootstrap	–	`D/Config/GameConfig.asset`
Settings / options	`S/Game/GameSettings.cs`	`OptionsMenu`	all	–	PlayerPrefs `NeonRift.Settings.*`
Title	`S/Frontend/TitleScreen.cs`	`OptionsMenu`	Frontend	–	`UI/TitleScreen.uxml`
Car Select	`S/Frontend/CarSelectScreen.cs`	`VehicleShowroom`, `ShowroomEngine`, `S/Editor/Frontend/GarageBuilder.cs`, `GaragePlanningWall`	CarSelect	`P/Vehicles/PF_Vehicle_*`	`D/Vehicles/VehicleCatalog.asset`
Vehicle physics	`S/Vehicles/Physics/VehicleController.cs`	`VehicleWheel`, `TyreModel`, `Drivetrain`, `Gearbox`, `SteeringSystem`	all driving scenes	`P/Vehicles/PF_Vehicle_*`	`D/Vehicles/Physics/VehiclePhysics_*.asset`, `D/Physics/PM_VehicleBody.asset`
Vehicle audio	`S/Audio/VehicleAudio.cs`	`EngineSoundModel`, `TyreSoundModel`, `S/Editor/Audio/VehicleAudioGenerator.cs`, `EngineSynth`	–	–	`D/Vehicles/Audio/VehicleAudio_*.asset`
Vehicle visuals	`S/Vehicles/VehicleRig.cs`	`VehicleLights`, `S/Editor/Vehicles/VehiclePrefabBuilder.cs`, `VehicleLodSetup`, `GltfMaterialFixer`	–	`P/Vehicles/`	`D/Vehicles/ImportSetups/`

System	Primary script	Secondary scripts	Scene	Prefab	Data
Spawn	`S/Gameplay/VehicleSpawnPoint.cs`	`MissionSceneEntry`	NightRun, TestTrack	–	`D/Vehicles/Vehicle_*.asset`
Input	`S/Input/PlayerDrivingInput.cs`	`NeonRiftControls` (generated)	all	–	`Settings/Input/NeonRiftControls.inputactions`
Mission	`S/Gameplay/Mission/MissionDirector.cs`	`MissionProgress`, `MissionWorld`, `MissionZone`	NightRun	–	`D/Missions/Mission_NightRun.asset`
Heist interactions	`S/Gameplay/Mission/Interactable.cs`	`InteractionRun`, `InteractionStep`, `InteractionDefinition`, `TerminalDisplay`, `TerminalReadout`	NightRun	–	`D/Missions/Interaction_*.asset`
Data Core	`S/Gameplay/Mission/DataCoreChamber.cs`	`S/Editor/District/NightRunBuilder.Heist.cs`	NightRun	–	`Interaction_DataCoreUplink`, `Interaction_CoreSecurity`
Gates	`S/Gameplay/Mission/SecurityBarrier.cs`	`GateLockSystem`	NightRun	–	`Interaction_HackGate`
Lockdown reactions	`S/Gameplay/Mission/SecurityLightGroup.cs`	`SecurityAlarm`, `SecurityPostEffects`, `S/Gameplay/City/LockdownScreens.cs`, `TrafficSignalNetwork`, `SecurityCamera`, `SecurityDrones`	NightRun	`P/Drones/PF_Drone_Mech`	`Settings/Rendering/NightRun_Alert.asset`, `NightRun_Lockdown.asset`
Navigation	`S/Gameplay/Navigation/CityNavigation.cs`	`S/World/RoadGraph.cs`, `RoadNetwork`	NightRun	–	`D/World/RoadNetwork_NightRun.asset`
Rival AI	`S/Gameplay/Racing/RivalDirector.cs`	`RacerDriver`, `RacingLine`, `RaceMarker`, `RacerProfile`, `RivalTask`	NightRun	–	`D/Racing/Racer_*.asset`, `Mission_NightRun.rivalTasks`
Traffic	`S/Gameplay/City/CityTraffic.cs`	`TrafficCar`, `S/Editor/District/TrafficCarBuilder.cs`	NightRun	`P/Traffic/PackCar_*`	–
Crowds, sentries	`S/Gameplay/City/CrowdGroups.cs`	`HoverSentry`	NightRun	`P/Drones/PF_Drone_Buster`	–
Lights budget	`S/Gameplay/City/LightBudget.cs`	`CameraCullDistances`	NightRun	`P/District/PF_Prop_StreetLight_*`	–
City generation	`S/Editor/District/NightRunBuilder.cs`	`NightRunDistrict`, `CityLayout`, `CityInfill`, `CityProps`, `CityLandmarks`, `TowerDressing`, `SkylineBackdrop`, `DistrictKit`, `DistrictTextures`, `MeshBuilder`, `BlockMeshes`	NightRun (generated)	`P/Buildings`, `P/District`	`D/World/Building*.asset`
Atmosphere	`S/Rendering/GroundHazeFeature.cs`	`GroundHaze`	–	–	`Settings/Rendering/PC_Renderer.asset`
HUD	`S/Gameplay/Mission/MissionHud.cs`	`CityMinimap`, `MinimapProjection`, `TerminalReadout`	NightRun	–	`UI/MissionHud.uxml`, `MissionHud.uss`

System	Primary script	Secondary scripts	Scene	Prefab	Data
Camera	`S/Gameplay/VehicleChaseCamera.cs`	`CameraCullDistances`	NightRun, TestTrack	–	–
Mission audio / music	`S/Gameplay/Mission/MissionAudio.cs`	`MissionAudioSet`, `ScoreMix`, `S/Editor/Audio/ScoreComposer.cs`, `MissionAudioGenerator`, `HeistAudioGenerator`	NightRun	–	`D/Missions/MissionAudio_NightRun.asset`
Mixer	`S/Audio/AudioMixerService.cs`	`AudioMixerConfig`, `MasterLimiter`, `S/Editor/Audio/AudioMixerBuilder.cs`	–	–	`D/Audio/AudioMixerConfig.asset`, `Audio/Mixer/NeonRiftMixer.mixer`
Intro	`S/Intro/IntroSceneEntry.cs`	`IntroOverlay`, `IntroCue*`, `IntroCityLife`, `CrewGarage`, `S/Editor/Intro/IntroBuilder.cs`	NightRun	–	`D/Intro/Intro_NightRun.playable`, `D/Config/IntroSettings.asset`
Grade / records	`S/Missions/MissionGrade.cs`	`MissionRecords`	–	–	PlayerPrefs `NeonRift.Best.*`

46.2 Interfaces

Interface	File	Implemented by
`ISceneEntryPoint`	`S/Game/GameContext.cs`	`TitleScreen`, `CarSelectScreen`, `MissionSceneEntry`, `IntroSceneEntry`
`IIntroEntryPoint`	`S/Game/IGameFlow.cs`	`IntroSceneEntry`
`IGameFlow`	`S/Game/IGameFlow.cs`	`GameFlow`
`IVehicleInputSource`	`S/Vehicles/DrivingInput.cs`	`PlayerDrivingInput`, `RacerDriver`, `WaypointDriver`, `RouteAutopilot`, `ScriptedDrivingInput`
`IVehicleInputReceiver`	`S/Vehicles/DrivingInput.cs`	`VehicleController`
`IMissionWorldComponent`	`S/Gameplay/Mission/IMissionWorldComponent.cs`	15 classes (see 02)
`IMissionTarget`	`S/Gameplay/Mission/IMissionTarget.cs`	`MissionZone`, `Interactable`
`IIntroCueHandler`	`S/Intro/IntroCue.cs`	`IntroSceneEntry`

46.3 ScriptableObjects

`GameConfig`, `IntroSettings`, `VehicleCatalog`, `VehicleDefinition`, `VehiclePhysicsProfile`, `VehicleAudioProfile`, `MissionDefinition`, `InteractionDefinition`, `MissionAudioSet`, `RacerProfile`, `RoadNetwork`, `BuildingCatalog`, `BuildingDefinition`, `AudioMixerConfig`, `VehicleModelSetup` (editor).

46.4 Managers, controllers, services, builders, state machines

Role	Classes
Composition root / services	`GameRoot`; `AudioMixerService`, `GameSettings`, `GameFlow`
Managers / directors	`MissionDirector`, `RivalDirector`, `CityTraffic`, `LightBudget`, `TrafficSignalNetwork`, `SecurityDrones`, `CrowdGroups`
Controllers	`VehicleController`, `VehicleChaseCamera`, `CarSelectScreen`, `VehicleShowroom`, `IntroSceneEntry`
Pure state machines / logic	`MissionProgress`, `InteractionRun`, `Gearbox`, `Drivetrain`, `MissionGrade`, `ScoreMix`, `EngineSoundModel`, `TyreModel`, `MinimapProjection`, `RoadGraph`
Factories / spawners	`VehicleSpawnPoint.Spawn`, `RivalDirector.Spawn`, `CityTraffic.Create/Spawn`
Editor builders (generators)	`NightRunBuilder`, `NightRunDistrict`, `GarageBuilder`, `IntroBuilder`, `ValidationRouteBuilder`, `VehiclePrefabBuilder`, `BuildingPrefabBuilder`, `TrafficCarBuilder`, `DroneModels`, `ThirdPartyIntake`, `AudioMixerBuilder`, all `*AudioGenerator`s, `ScoreComposer`

46.5 Menu cheat-sheet (all under **Neon Rift**)

Validate Project · Night Run ▸ Build Night Run (district + mission) · Night Run ▸ Capture Look-Dev Shots (Play Mode) · Car Select ▸ Build Garage · Intro ▸ Build Intro / Generate Intro Audio / Reset 'Intro Seen' · Assets ▸ Run Third-Party Intake / Rebuild Vehicle Prefabs And Catalog · Vehicles ▸ Build All Vehicle Prefabs / Apply Vehicle LODs To Catalog Prefabs / Apply Vehicle Copy From Recipes / Measure Vehicle Performance / Update Car Select Stats From Physics · Audio ▸ Generate Vehicle Audio / Generate Mission Audio / Build Audio Mixer / Add Vehicle Audio To Prefabs · Test Track ▸ Build Validation Route · Tests ▸ Run EditMode Tests (write report) · Validation ▸ Build Perf Player (macOS).

46.6 World event ids (Night Run)

`mission.start`, `mission.complete`, `mission.failed`, `core.arrive`, `core.security.off`, `core.extract.connect`, `core.extract.init`, `core.extract.stabilize`, `core.extract.begin`, `grid.anomaly`, `core.extract.trace`, `core.acquired`, `core.breached`, `lockdown`, `escape.start`, `alley.gate.open`, `alley_gate_terminal.hack|abort|lockout|unlock` (constants in `S/Editor/District/NightRunBuilder.cs` and `NightRunBuilder.Heist.cs`).

46.7 Target and marker ids

Zones `core_compound`, `extraction`; interactables `core_security_terminal`, `core_uplink`, `alley_gate_terminal`; gates `alley_gate`, `compound_gate`, `expressway_checkpoint`, `harbor_checkpoint`, `skyway_gate`; race markers `spire_relay`, `kowloon_relay`, `trunk_tap`, `extraction`, `core_staging`.

47

Teacher Q&A Guide (viva cheat sheet)

IN THIS CHAPTER

Q1 What is Neon Rift?
Q2 How is the project architected?
Q3 How did you make the city?
Q4 How did you generate the buildings?
Q5 How do the roads and routing work?
Q6 How does the vehicle physics work?
Q7 How does the suspension work?
Q8 How do the tyres work?
Q9 How do cars get their sound?
Q10 How does car selection work?" / "How do you switch between cars?
Q11 Where does the chosen car come from in the mission?
Q12 How does the rival AI work?
Q13 Do the rivals cheat?
Q14 How does traffic work?
Q15 How does the mission system know which objective is active?
Q16 How does the Data Core work?
Q17 What are the interaction stage types?
Q18 How does the security gate work?
Q19 What is heat?
Q20 How does lockdown work?
Q21 How do the drones follow the cars?
Q22 How did you make the cinematic?
Q23 How did you make the fog / neon / rain effects?
Q24 How is the city lit without killing performance?
Q25 How does the minimap work?
Q26 How does the HUD get its data?
Q27 How is the grade calculated?
Q28 What happens if you finish third?
Q29 How does the camera work?
Q30 How do you handle input?
Q31 Where is the animation implemented?
Q32 How did you optimise performance?
Q33 How did you test it?
Q34 What Unity packages did you use?
Q35 Where did the assets come from, and are they licensed?
Q36 Where is the code for X?

`S/` = `Assets/_NeonRift/Scripts/`. Each answer points to the chapter with the full detail.

Each card gives a **short answer** you can say in one breath, the **detailed answer** when the examiner asks for more, and exactly **where the implementation lives**. Paths are written out in full; the `S/` shorthand of the source Markdown is expanded.

Q1 What is Neon Rift?

SHORT ANSWER

A Unity 6 URP cyberpunk racing-heist game: drive into a generated night city, steal a data core in multi-stage hacks, and escape a city-wide lockdown before two AI rival crews.

DETAILED ANSWER

One mission, Night Run, with 4 data-driven objectives, 3 player cars, 2 rival crews running their own heists, heat and lockdown consequences, an 84 s real-time cinematic intro and a graded debrief.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Data/Missions/Mission_NightRun.asset` `Assets/_NeonRift/Scripts/Gameplay/Mission/MissionDirector.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Data/Missions/Mission_NightRun.asset`
RELATED CHAPTER	**01** Project Overview

Q2 How is the project architected?

SHORT ANSWER

A persistent Bootstrap scene with one composition root, additive content scenes that receive a `GameContext` , ScriptableObject data, and missions that communicate by string event ids.

DETAILED ANSWER

`GameRoot` creates the services; `GameFlow` loads one content scene at a time and calls `ISceneEntryPoint.Enter(context)` . There are no singletons (domain reload is disabled). 13 assemblies with one-way dependencies.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Game/GameRoot.cs`
	`Assets/_NeonRift/Scripts/Game/GameFlow.cs`
	`Assets/_NeonRift/Scripts/Game/GameContext.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Game/GameRoot.cs`
KEY CLASS / METHOD	`GameFlow.TransitionAsync`
RELATED CHAPTER	**02** Architecture **06** Bootstrap and Scenes

Q3 How did you make the city?

SHORT ANSWER

It is generated by an editor tool from layout data; nothing is hand-placed in the scene.

DETAILED ANSWER

`CityLayout` lists roads, lots, reserved areas, heroes and districts. `Decompose` turns the road lines into a grid, and union-find merges cells into blocks. `NightRunDistrict` then builds the ground, markings, podium street walls, towers, props, landmarks, skyline and reflection probes; `CityInfill` fills block interiors. Meshes are combined per material per block. One menu rebuilds everything in about 40 s.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Editor/District/CityLayout.cs`
	`Assets/_NeonRift/Scripts/Editor/District/NightRunDistrict.cs`
	`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Editor/District/CityLayout.cs`
KEY CLASS / METHOD	`NightRunBuilder.Build`, `CityLayout.Decompose`
RELATED CHAPTER	**07** City and World Generation

Q4 How did you generate the buildings?

SHORT ANSWER

Three layers: procedural podium street walls, Sketchfab towers placed from a catalog, and procedural block infill.

DETAILED ANSWER

`NightRunDistrict.BuildStreetWall` emits shopfront podiums with generated textures and a pixel font for signs. `BuildingPrefabBuilder` wraps the Sketchfab models into `PF_Bld_*` prefabs with `BuildingDefinition` data, and `PlaceHeroes` / `FillTowers` place them. `TowerDressing` + the `NeonRift/TowerFacade` shader add lit windows, crowns and aviation lights. `CityInfill` partitions free space with a largest-rectangle algorithm into yards, car parks, parks and back buildings.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Editor/District/NightRunDistrict.cs`
	`Assets/_NeonRift/Scripts/Editor/District/CityInfill.cs`
	`Assets/_NeonRift/Scripts/Editor/District/TowerDressing.cs`
	`Assets/_NeonRift/Scripts/Editor/World/BuildingPrefabBuilder.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Editor/District/NightRunDistrict.cs`
RELATED CHAPTER	**08** Buildings and Environment

Q5 How do the roads and routing work?

SHORT ANSWER

The same road list produces the visible roads and a runtime graph; Dijkstra finds routes, and closed gates make edges impassable.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/World/RoadGraph.cs`
	`Assets/_NeonRift/Scripts/World/RoadNetwork.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Navigation/CityNavigation.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/World/RoadGraph.cs`
KEY CLASS / METHOD	`RoadGraph.FindPath`, `CityNavigation.EdgeCost`
RELATED CHAPTER	**09** Roads and Route Network

Q6 How does the vehicle physics work?

SHORT ANSWER

A custom Rigidbody car at 100 Hz with raycast suspension, a combined-slip tyre model, an engine/clutch/gearbox drivetrain, anti-roll bars, aero, ABS and traction control. No `WheelCollider`.

DETAILED ANSWER

`VehicleController.Step`: suspension contact (5 rays per wheel) → springs, dampers, bump stops and anti-roll → steering → drivetrain → tyre forces per wheel (implicit wheel-spin integration) → differentials → aero → traction control → telemetry. All tuning comes from a `VehiclePhysicsProfile` in physical units.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleController.cs`
	`Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleWheel.cs`
	`Assets/_NeonRift/Scripts/Vehicles/Physics/TyreModel.cs`
	`Assets/_NeonRift/Scripts/Vehicles/Physics/Drivetrain.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleController.cs`
KEY CLASS / METHOD	`VehicleController.Step` (line 163)
RELATED CHAPTER	**11** Vehicle Physics

Q7 How does the suspension work?

SHORT ANSWER

Per-wheel raycast springs whose rate comes from a natural frequency, with digressive dampers, bump stops and anti-roll bars.

DETAILED ANSWER

`VehicleWheel.UpdateContact` casts 5 rays across the tyre profile; `SuspensionForce` = spring (preloaded to the model's ride height) + `DamperForce` (separate bump and rebound, linear to a knee then softer) + a progressive bump stop; `ApplySuspension` adds anti-roll = stiffness × compression difference per axle.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleWheel.cs` `Assets/_NeonRift/Scripts/Vehicles/Physics/AxleSettings.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Vehicles/Physics/VehicleWheel.cs`
RELATED CHAPTER	11 Vehicle Physics

Q8 How do the tyres work?

SHORT ANSWER

Slip ratio and slip angle are normalised by their peaks and combined; the force follows a sine rise and Gaussian fall, limited to a friction ellipse, with load sensitivity.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Vehicles/Physics/TyreModel.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Vehicles/Physics/TyreModel.cs`
KEY CLASS / METHOD	`TyreModel.Force`, `ClampToEllipse`
RELATED CHAPTER	11 Vehicle Physics

Q9 How do cars get their sound?

SHORT ANSWER

Telemetry-driven layered loops: 5 rpm points × on/off load, crossfaded equal-power in log-rpm and pitched by rpm. All synthesised in-house from a physical exhaust model.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Audio/VehicleAudio.cs`
	`Assets/_NeonRift/Scripts/Audio/EngineSoundModel.cs`
	`Assets/_NeonRift/Scripts/Editor/Audio/EngineSynth.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Audio/VehicleAudio.cs`
RELATED CHAPTER	12 Vehicle Audio

Q10 How does car selection work?" / "How do you switch between cars?

SHORT ANSWER

`CarSelectScreen` browses the `VehicleCatalog`; `VehicleShowroom` swaps the car on a turntable with a light, camera and engine-blip transition; Confirm stores the choice in `RunSession` and plays the garage departure.

DETAILED ANSWER

Previous/Next → `Select` → `VehicleShowroom.Show` (whip-out and spin-in) and `SetStats` (counting bars). Confirm → `RunSession.SelectVehicle(car, catalog)` (the other cars become rivals) → `VehicleShowroom.Depart` → `Flow.StartMission()`.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Frontend/CarSelectScreen.cs`
	`Assets/_NeonRift/Scripts/Frontend/VehicleShowroom.cs`
	`Assets/_NeonRift/Scripts/Game/RunSession.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Frontend/CarSelectScreen.cs`
KEY CLASS / METHOD	`CarSelectScreen.Confirm`, `VehicleShowroom.Swap`, `Depart`
RELATED CHAPTER	**15** Car Selection

Q11 Where does the chosen car come from in the mission?

SHORT ANSWER

`MissionSceneEntry.Enter` spawns `Session.SelectedVehicle` through `VehicleSpawnPoint.Spawn`, which instantiates the prefab and applies the physics and audio profiles.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/MissionSceneEntry.cs` `Assets/_NeonRift/Scripts/Gameplay/VehicleSpawnPoint.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/MissionSceneEntry.cs`
RELATED CHAPTER	**10** Vehicle System

Q12 How does the rival AI work?

SHORT ANSWER

Each rival is a `RacerDriver` that feeds the same input interface as the player: it plans over the road graph, follows a racing line with pure pursuit and a braking-distance speed planner, sweeps ahead for traffic, yields, follows and recovers when stuck. `RivalDirector` gives each crew its own heist tasks and computes standings and qualifying.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/Racing/RacerDriver.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Racing/RivalDirector.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Racing/RacingLine.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Racing/RacerProfile.cs`
	`Assets/_NeonRift/Scripts/Missions/RivalTask.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/Racing/RacerDriver.cs`
KEY CLASS / METHOD	`RacerDriver.ReadInput`, `RivalDirector.BeginTask`, `UpdateStandings`
RELATED CHAPTER	**17** Rival AI

Q13 Do the rivals cheat?

SHORT ANSWER

No teleporting or fake forces while driving. The only non-physical action is the same Recover the player has, used out of view after repeated failed reversals. A bounded rubber band changes their target speed by at most about ±5–6 %.

KEY CLASS / METHOD	`RacerProfile.RubberBand`, `RacerDriver.Reset`, `InPlayerView`
RELATED CHAPTER	**17** Rival AI

Q14 How does traffic work?

SHORT ANSWER

14 roaming and 26 parked physical pack cars with spring suspension and lateral grip; they keep their distance, give way at junctions to the player and rivals, pull over in a lockdown, and are recycled out of view.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/City/CityTraffic.cs` `Assets/_NeonRift/Scripts/Gameplay/City/TrafficCar.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/City/CityTraffic.cs`
RELATED CHAPTER	**18** Traffic System

Q15 How does the mission system know which objective is active?

SHORT ANSWER

`MissionProgress` (pure C#) holds `ObjectiveIndex / Current`; zones and interactables report completions with their ids, and `TryComplete` only accepts the id and kind of the current objective.

DETAILED ANSWER

`MissionDirector` routes `MissionZone` entries to `NotifyReached` and interaction completions to `NotifyInteracted`. Objectives fire world events on start and complete. Scene objects check `MissionWorld.IsObjectiveTarget(id)`.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Missions/MissionProgress.cs` `Assets/_NeonRift/Scripts/Gameplay/Mission/MissionDirector.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Missions/MissionProgress.cs`
KEY CLASS / METHOD	`MissionProgress.TryComplete`, `Advance`, `IsCurrentTarget`
RELATED CHAPTER	**19** Mission System

Q16 How does the Data Core work?

SHORT ANSWER

Two multi-stage interactions (a 4-stage security terminal, a 5-stage extraction uplink) run by a pure `InteractionRun`; `DataCoreChamber` animates the sleeve, the IK arms, the cable, pulses and lights from events and progress.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/Mission/DataCoreChamber.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Mission/Interactable.cs`
	`Assets/_NeonRift/Scripts/Missions/InteractionRun.cs`
	`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.Heist.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/Mission/DataCoreChamber.cs`
KEY CLASS / METHOD	`ConfigureUplinkDefinition`, `DataCoreChamber.UpdateArms`
RELATED CHAPTER	**20** Heist System **22** Data Core

Q17 What are the interaction stage types?

SHORT ANSWER

Auto (stay stopped), Hold (hold E; decays when released), Timing (press E in a moving window; misses cost heat and can lock you out), Sustain (answer interference with E or lose progress).

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Missions/InteractionStep.cs` `Assets/_NeonRift/Scripts/Missions/InteractionRun.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Missions/InteractionStep.cs`
RELATED CHAPTER	**20** Heist System

Q18 How does the security gate work?

SHORT ANSWER

A five-stage hack on a kiosk; the gate's bolts, lamps and hologram react to the hack's events; success opens the sliding gate and adds 35 % heat; a lockdown slams it and re-arms the hack.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/Mission/GateLockSystem.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityBarrier.cs`
	`Assets/_NeonRift/Scripts/Editor/District/NightRunBuilder.Heist.cs` (`ConfigureGateHackDefinition`)

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/Mission/GateLockSystem.cs`
RELATED CHAPTER	**21** Security Gate

Q19 What is heat?

SHORT ANSWER

One 0–1 number for risk. Gate hacks, missed timing presses, lockouts and cameras add heat; heat raises security to Alert, shortens the escape timer and gate countdowns, and lowers the debrief grade.

KEY CLASS / METHOD	`MissionProgress.AddHeat`
RELATED CHAPTER	**23** Heat and Lockdown

Q20 How does lockdown work?

SHORT ANSWER

The escape objective has security level Lockdown and fires `core.breached`, `lockdown` and `escape.start`. Every reactive object hears these: gates seal on staged countdowns, a red wave spreads at 140 m/s, signals flash, screens show warnings, sirens fade in, drones chase, traffic pulls over, the grade and mix change, and the skyway opens.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityBarrier.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityLightGroup.cs`
	`Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityAlarm.cs`
	`Assets/_NeonRift/Scripts/Gameplay/City/SecurityDrones.cs`
	`Assets/_NeonRift/Scripts/Gameplay/City/TrafficSignalNetwork.cs`
	`Assets/_NeonRift/Scripts/Gameplay/City/LockdownScreens.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/Mission/SecurityBarrier.cs`
RELATED CHAPTER	**23** Heat and Lockdown

Q21 How do the drones follow the cars?

SHORT ANSWER

`SecurityDrones` launches on the counter-intrusion or lockdown; three drones tail the player and one tails each rival still running, flying a slot above and behind the car and holding a searchlight on it.

KEY CLASS / METHOD	`SecurityDrones.TargetFor`, `Update`
RELATED CHAPTER	**23** Heat and Lockdown

Q22 How did you make the cinematic?

SHORT ANSWER

A Timeline generated from a shot table in code: 25 Cinemachine shot cameras animated by AnimationTracks, a custom cue track for story beats, audio tracks, a UI Toolkit overlay, and real cars driving with real physics in the actual city.

DETAILED ANSWER

`IntroBuilder.Shots` and `Cues` → `BuildTimeline`; `IntroSceneEntry` runs it, handles Skip (jump to the title with one-shot cues applied) and Confirm (Catmull-Rom push-in matched to Car Select's first frame); `IntroCityLife` adds traffic and drones synced to the Timeline clock.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Editor/Intro/IntroBuilder.cs`
	`Assets/_NeonRift/Scripts/Intro/IntroSceneEntry.cs`
	`Assets/_NeonRift/Scripts/Intro/IntroCue.cs`
	`Assets/_NeonRift/Scripts/Intro/IntroCityLife.cs`
	`Assets/_NeonRift/Data/Intro/Intro_NightRun.playable`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Editor/Intro/IntroBuilder.cs`
RELATED CHAPTER	**31** Cinematic Intro **32** Timeline and Camera

Q23 How did you make the fog / neon / rain effects?

SHORT ANSWER

Neon is emissive materials with HDR bloom; fog is exponential-squared distance fog plus a custom Render Graph ground-haze pass. Rain is **not implemented**: the wet look is the asphalt texture's damp patches and puddles with high smoothness reflecting baked probes.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Rendering/GroundHazeFeature.cs`
	`Assets/_NeonRift/Scripts/Rendering/GroundHaze.cs`
	`Assets/_NeonRift/Scripts/Editor/District/DistrictTextures.cs` (`Asphalt`)
	`Assets/_NeonRift/Settings/Rendering/NightRun_Base.asset`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Rendering/GroundHazeFeature.cs`
RELATED CHAPTER	**25** Lighting **28** Particles and VFX

Q24 How is the city lit without killing performance?

SHORT ANSWER

787 lamps carry disabled lights; `LightBudget` turns on the 56 nearest (2 with shadows) with fades; everything else is emissive. No GI; baked box-projected reflection probes.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/City/LightBudget.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/City/LightBudget.cs`
RELATED CHAPTER	**25** Lighting

Q25 How does the minimap work?

SHORT ANSWER

A UI Toolkit element drawn with `Painter2D` from the road graph, north-up and centred on the player; only the player and rival arrows rotate; off-map targets are clamped to the rim.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/Mission/CityMinimap.cs` `Assets/_NeonRift/Scripts/Gameplay/Mission/MinimapProjection.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/Mission/CityMinimap.cs`
KEY CLASS / METHOD	`MinimapProjection.ToMap`, `Heading`
RELATED CHAPTER	**34** Minimap

Q26 How does the HUD get its data?

SHORT ANSWER

`MissionDirector` pushes state into the passive `MissionHud` (UI Toolkit) through setters, only rewriting text when values change.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/Mission/MissionHud.cs` `Assets/_NeonRift/UI/MissionHud.uxml`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/Mission/MissionHud.cs`
RELATED CHAPTER	**33** UI and HUD

Q27 How is the grade calculated?

SHORT ANSWER

Out of 100: pace 40 (par 115 s → 0 at 184 s), quiet 25 × (1 − heat), position 20, clean 15 (0 at 6 hard contacts). S ≥ 90, A ≥ 75, B ≥ 60, C ≥ 45, else D.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Missions/MissionGrade.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Missions/MissionGrade.cs`
KEY CLASS / METHOD	`Evaluate`, `LetterFor`
RELATED CHAPTER	**36** Results and Debrief

Q28 What happens if you finish third?

SHORT ANSWER

The mission is completed but marked NOT QUALIFIED (only the first two out qualify); the qualified flag is not stored.

KEY CLASS / METHOD	`MissionDirector.BuildDebrief`, `MissionRecords.MarkQualified`
RELATED CHAPTER	**17** Rival AI **36** Results and Debrief

Q29 How does the camera work?

SHORT ANSWER

A custom chase camera computes the pose (heading swing on slides, look-ahead, speed FOV, acceleration lag, collision, cornering bank, shake) and writes it into a Cinemachine camera; special framings for the heist, the gate and the theft beat.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Gameplay/VehicleChaseCamera.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/VehicleChaseCamera.cs`
RELATED CHAPTER	**38** Camera System

Q30 How do you handle input?

SHORT ANSWER

Input System actions (Driving and Menu maps) generate a typed wrapper; `PlayerDrivingInput` converts them to `DrivingInput` for the car. M toggles music globally; F3 shows telemetry in dev builds.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Settings/Input/NeonRiftControls.inputactions` `Assets/_NeonRift/Scripts/Input/PlayerDrivingInput.cs`
PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Input/PlayerDrivingInput.cs`
RELATED CHAPTER	**16** Player Input

Q31 Where is the animation implemented?

SHORT ANSWER

Almost entirely procedural C# (wheels, gates, bolts, arms with IK, doors, drones, lights, UI). Timeline animation only for the intro cameras; a legacy `Animation` loop for the Mech Drone.

RELATED CHAPTER	**37** Animation Systems

Q32 How did you optimise performance?

SHORT ANSWER

Combined meshes per block and material, cell-based culling and layer cull distances, vehicle LODs, a lamp light budget, 2 shadow cascades and Low soft shadows (−37 % GPU measured), allocation-free per-frame code, and 64 audio voices with priorities. The release macOS player averaged 38 fps at 1080p on an M3 with zero GC (measured before the latest content additions).

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Editor/Validation/RoutePerf.cs`
	`Assets/_NeonRift/Scripts/Editor/Validation/GpuBench.cs`
	`Assets/_NeonRift/Scripts/Editor/Validation/PerfBuild.cs`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Editor/Validation/RoutePerf.cs`
RELATED CHAPTER	**39** Performance

Q33 How did you test it?

SHORT ANSWER

125 EditMode test cases (physics on a deterministic bench, tyres, gearbox, mission state machine, interactions, navigation, minimap, audio, intro and scene wiring) plus Play Mode validators that play the real mission through the player's interfaces.

WHERE TO FIND THE IMPLEMENTATION	`Assets/_NeonRift/Scripts/Tests/EditMode/`
	`Assets/_NeonRift/Scripts/Gameplay/Mission/NightRunValidator.cs`
	`Assets/_NeonRift/Scripts/Editor/Validation/`

PRIMARY SCRIPT / ASSET	`Assets/_NeonRift/Scripts/Gameplay/Mission/NightRunValidator.cs`
RELATED CHAPTER	**40** Testing and Validation

Q34 What Unity packages did you use?

SHORT ANSWER

URP 17.6, Input System 1.19, Cinemachine 6.6, Timeline 6.6, glTFast 6.20 (model import), Test Framework, Video module, uGUI (EventSystem only). Splines, ProBuilder and AI Navigation are installed but **not used** by the game.

WHERE TO FIND THE IMPLEMENTATION	`Packages/manifest.json`
PRIMARY SCRIPT / ASSET	`Packages/manifest.json`
RELATED CHAPTER	**03** Tech Stack

Q35 Where did the assets come from, and are they licensed?

SHORT ANSWER

Cars, buildings and drones are Sketchfab models with recorded CC licences (one car is non-commercial); all sound effects and the fallback score are synthesised in-house; two music tracks and the traffic car pack were supplied without a recorded licence.

WHERE TO FIND THE IMPLEMENTATION	`Assets/ThirdParty/**/ATTRIBUTION.txt`
	`Docs/ThirdParty.md`
	`LICENSE.md`

PRIMARY SCRIPT / ASSET	`Assets/ThirdParty/**/ATTRIBUTION.txt`
RELATED CHAPTER	**44** Assets and Licensing

Q36 Where is the code for X?

Use the **Find It Fast** table in 00_INDEX or 46 Developer Quick Reference.

48 Glossary

Term	Meaning	See
Ackermann	Steering geometry where the inner front wheel turns more than the outer	11
Alert	Middle security level, reached on the first heat	23
Alert origin	The theft position (Data Core) the lockdown wave spreads from	`MissionWorld.AlertOrigin`
Announcement	A data-driven HUD message or banner triggered by a world event	`MissionAnnouncement`
Block	Merged grid cells between road lines; the unit of street walls, meshes and infill	07
Bootstrap	Always-loaded scene with the composition root	06
Checkpoint (gate)	Expressway or harbor barrier that seals in a lockdown	`SecurityBarrier`
Checkpoint (interaction)	A stage a cancelled run returns to	`InteractionStep.checkpoint`
Clutter / Detail layers	Small props culled beyond 90 m / 180 m	`CameraCullDistances`
Combined slip	Normalised slip ratio and slip angle combined into one value for the tyre force	`TyreModel`
Composition root	The one place long-lived services are created (`GameRoot`)	02
Core trunk tap	A rival crew's second task, hacking the core's data line from the kerb	17
Crew	The player's team; also each rival team (VEX, KADE)	–
Crew garage / Unit 7	The hidden garage on W Avenue: the Car Select hall placed in the city	`CrewGarage`
Data Core	The heist target in the Sector 7 compound	22
Debrief	The end-of-run results screen with a grade	36
Engaged	Inside an interactable's trigger and below its speed limit	20
Event id	String such as `core.breached` that scene objects react to	19
Extraction	Final zone, the Rift Gate tunnel; also the uplink stage that pulls the data	–
Friction ellipse	Limit on the combined longitudinal and lateral tyre force	`TyreModel.ClampToEllipse`
Heat	0–1 risk value that raises security and shortens timers	23

Term	Meaning	See
Hero building	A hand-placed landmark model from the catalog	`CityLayout.Heroes`
Infill	Procedural content behind the street walls	`CityInfill`
Interactable	A "stop and work the device" trigger with stages	`Interactable`
Interference	A Sustain-stage disruption the player must answer with E	`InteractionStep.WithInterference`
Kerb lane	The outer lane next to the pavement; where rivals stop to work	17
Light budget	Keeping only the nearest 56 lamps lit as real lights	`LightBudget`
Lockdown	Highest security level after the theft	23
Lockout	Temporary block after too many timing misses	`InteractionDefinition`
Match cut	The intro's push-in ending exactly on Car Select's first frame	31
Mission world	The per-attempt hub objects bind to	`MissionWorld`
Par	Target time (115 s) for full pace marks	`MissionDefinition.parTime`
Podium / street wall	Generated low-rise frontage along every block side	08
Pure pursuit	Steering toward a look-ahead point on a path	`RacerDriver`, `RouteAutopilot`
Qualifying	Only the first two crews out advance	17
Race marker	A named place rivals drive to (child slots per rival)	`RaceMarker`
Racing line	A rounded, resampled path with lateral range and advisory speed	`RacingLine`
Relay hack	A rival crew's first task	17
Rift Gate	The south-east extraction tunnel	–
Rival task	One stage of a rival crew's own heist	`RivalTask`
Road graph / network	Nodes and edges describing where cars can drive	09
Rubber band	Small bounded pace adjustment for rivals by gap to the player	`RacerProfile.RubberBand`
Sector 7 (S7)	The hero district with the Data Core	–
Sentry	Hovering Buster Drone over the compound	`HoverSentry`
Skyway	Elevated harbor road opened during the escape	23
Snapshot	AudioMixer state (Gameplay, Menu, Results, Lockdown, Ducked)	29
Split	Time at which each objective was completed	`MissionHud.StampObjective`
Stage	One step of a multi-stage interaction (Auto, Hold, Timing, Sustain)	`InteractionStep`
Telemetry	Per-step snapshot of a car's state	`VehicleTelemetry`
Theft beat	2.6 s slow-motion camera swing on `core.breached`	38
Trace	The 80 s escape timer before you are found	–
Uplink	The ring around the core where the data is extracted	22

Term	Meaning	See
VEX / KADE	The two rival crews and their driver profiles	17
Work FX	Magenta beacon and light over a rival car while it hacks	`RivalDirector.Racer.WorkFx`

A

Script Index

Generated from each type's XML documentation comment in the source (`Assets/_NeonRift/Scripts/`). Long summaries are shortened with "…".

`Scripts/Audio/`

File	Main type	Purpose
`AudioMixerConfig.cs`	`AudioMixerConfig`	The game's mixer, its routing groups, snapshot names and exposed volume parameters.
`AudioMixerService.cs`	`AudioMixerService`	Runtime front for the mixer: user volume channels and state snapshots (menus, results, lockdown, ducking). Owned by the composition root and handed to scenes through the game context.
`AudioOutputRecorder.cs`	`AudioOutputRecorder`	Development tool: records the final mix at the AudioListener (mono) into a fixed buffer, for automated checks (clicks, pitch tracking) and for saving a WAV to listen to. Add it to the …
`AudioSignalAnalysis.cs`	`AudioSignalAnalysis`	Signal measurements used to validate audio without listening: clicks, level, pitch, brightness.
`EngineSoundModel.cs`	`EngineSoundModel`	Turns engine speed and load into a volume and pitch for every loop of an `EngineAudioSettings`. Neighbouring rpm layers are crossfaded with an equal-power law in log-rpm space; on- and …
`MasterLimiter.cs`	`MasterLimiter`	Final-stage safety limiter on the AudioListener: the summed mix never exceeds the ceiling (no digital clipping when sirens, a klaxon, the engine at the limiter and the lockdown score …
`MixerState.cs`	`MixerState`	Mixer snapshots the game can be in. Each maps to a snapshot in the mixer asset.
`TyreSoundModel.cs`	`TyreSoundModel`	Maps tyre state to sound intensities (0..1). Pure; shared by every car.
`VehicleAudio.cs`	`VehicleAudio`	Plays a vehicle's sound from its telemetry: layered engine loops, tyre squeal/scrub, loose-surface, road and wind beds, gear shifts, overrun pops, suspension thumps and impacts. Every loop …

`Scripts/Core/`

File	Main type	Purpose
`AssetLicense.cs`	`AssetLicense`	Provenance of a third-party asset. Kept with the data that uses it so release checks can filter.

`Scripts/Editor/Audio/`

File	Main type	Purpose
`AudioMixerBuilder.cs`	`AudioMixerBuilder`	Builds the game's AudioMixer: group tree, state snapshots and exposed user-volume parameters, then writes the `AudioMixerConfig` and hooks it into `GameConfig`. Unity has no public API for …
`Dsp.cs`	`Dsp`	Small offline DSP toolkit for the procedural sound generator (editor only).
`EngineSynth.cs`	`EngineSynth`	Offline engine sound synthesis. A combustion engine is modelled as per-cylinder exhaust pulses at the real firing angles, routed to two exhaust banks, each shaped by pipe resonances, a pipe …
`HeistAudioGenerator.cs`	`HeistAudioGenerator`	The heist sounds: terminal stage blips, denials, interference, re-sync, lockout and cancel cues; the tension bed under the extraction; the facility hum, arm servos, data stream, chamber …
`IntroAudioGenerator.cs`	`IntroAudioGenerator`	Original, procedural audio for the opening cinematic. Everything is synthesised here (no samples, no licensed music). The music is the game's own theme from `ScoreComposer` (A minor, 96 …
`MissionAudioGenerator.cs`	`MissionAudioGenerator`	Re-runnable recipe for the mission sounds: lockdown siren, barrier klaxon/motor/slam, the city ambience bed, the hacking loop and HUD/story cues. Synthesised in-house and deterministic like …
`ScoreComposer.cs`	`ScoreComposer`	The Neon Rift score: original, fully procedural music (no samples, no licensed material). One 8-bar theme in A minor at about 96 bpm (Am(add9) – Fmaj7 – Dm9 – Esus4 → E, two bars each), …
`VehicleAudioGenerator.cs`	`VehicleAudioGenerator`	Re-runnable recipe for every vehicle sound: synthesises the engine loops, tyre/road/wind beds and one-shots (deterministic, so re-running reproduces identical files), sets import settings, …
`VehicleAudioPrefabs.cs`	`VehicleAudioPrefabs`	Adds and routes the `VehicleAudio` component on vehicle prefabs.

`Scripts/Editor/District/`

File	Main type	Purpose
`BlockMeshes.cs`	`BlockMeshes`	One `MeshBuilder` per material for a group of geometry (a block, a landmark), emitted as one static renderer per material: the district's batching unit. Builders are created on first use.
`CityInfill.cs`	`CityInfill`	Fills the inside of a block (everything behind the podium street walls that no tower, lot or sidewalk uses) with urban fabric that has a reason to be there: back buildings, service yards, …
`CityLandmarks.cs`	`CityLandmarks`	Hand-composed structures that give each district an identity and the player something to navigate by: the elevated Harbor Skyway, the Lantern Arcade and Kowloon's overhead cables and …
`CityLayout.cs`	`CityLayout`	The layout of the Night Run city (≈ 1.1 × 1.4 km of streets around Sector 7). Everything here is plain data: roads, reserved areas, special lots, hero buildings and districts. `Decompose` …
`CityProps.cs`	`CityProps`	Street furniture and infrastructure the city is made believable with: sidewalk furniture per zone (bins, benches, hydrants, utility cabinets, vending machines, planters, dumpsters, pallets, …
`DistrictKit.cs`	`DistrictKit`	Materials, saved meshes and reusable prop prefabs for the district. Every asset is created on first use and updated in place afterwards, so re-running the builder keeps GUIDs (and scene …
`DistrictTextures.cs`	`DistrictTextures`	Procedural, tileable textures for the Night Run district (wet asphalt, pavement, facades, shopfronts, neon signage, glow shapes). Deterministic, generated in-house, saved as PNG so they …

File	Main type	Purpose
`LookDevShots.cs`	`LookDevShots`	Captures a fixed set of Night Run viewpoints in Play Mode (lamps and the HUD are only live at runtime), so before/after look-dev shots are comparable. Street shots move the player car; …
`MeshBuilder.cs`	`MeshBuilder`	Accumulates quads for one material, so a whole block's facades become a single mesh (one draw per material). UVs are in metres divided by the texture's real-world tile size, so textures …
`NightRunBuilder.CityLife.cs`	`NightRunBuilder`	Signs of life: physical city traffic from the car pack (`CityTraffic` : roaming and parked), pedestrian silhouettes in groups (`CrowdGroups`) and security drones for the lockdown …
`NightRunBuilder.Garage.cs`	`NightRunBuilder`	The crew's garage on W Avenue: the Car Select hall (`BuildShell`) placed in the city with its roller door onto the avenue, walls and door that collide, a turntable you can stand a car on, …
`NightRunBuilder.Heist.cs`	`NightRunBuilder`	The Data Core heist: the interaction data (perimeter security terminal, extraction uplink) and the scene pieces that perform it – the terminal kiosk, the armoured sleeve, rail-mounted …
`NightRunBuilder.cs`	`NightRunBuilder`	Builds the Night Run vertical slice: the Sector 7 district (via `NightRunDistrict`), the Data Core compound, security gates, the extraction tunnel, lighting and post-processing, the HUD, …
`NightRunDistrict.cs`	`NightRunDistrict`	Builds the Night Run city from `CityLayout` : one asphalt ground plane, raised pavement blocks with kerbs, zone-styled podium street walls (shopfronts, lit floors, neon, signage, rooftop …
`PixelFont.cs`	`PixelFont`	A first-party 5×7 bitmap font used to bake neon signage textures (no third-party font files needed). Uppercase letters, digits, a few symbols and arrows.
`SkylineBackdrop.cs`	`SkylineBackdrop`	The metropolis beyond the playable city: three rings of tower silhouettes (1.7, 2.1 and 2.55 km from the centre) drawn with NeonRift/SkylineBackdrop. Each ring is hazier than the one in …
`TowerDressing.cs`	`TowerDressing`	Night dressing for the kit towers so they stop reading as black masses from the street and the air: swaps the kit materials for NeonRift/TowerFacade (procedural office windows, lobby band, …
`TrafficCarBuilder.cs`	`TrafficCarBuilder`	Turns the generic passenger car pack (one FBX: ten bodies in a ring, each with four loose wheels) into ten traffic prefabs: body and its four nearest wheels re-framed so the car sits on the …
`ViewCapture.cs`	`ViewCapture`	Renders the scene's main camera from a given pose into a PNG (look-dev and review from tools).

`Scripts/Editor/Frontend/`

File	Main type	Purpose
`GarageBuilder.cs`	`GarageBuilder`	Generates the Car Select scene: the crew's garage in Sector 7. A 26 × 30 m workshop hall with a polished concrete floor, steel columns and roof trusses, soft-box strip lights, a turntable …
`GaragePlanningWall.cs`	`GaragePlanningWall`	The crew's planning wall: textures generated from the real city data. A map of the streets (from `Roads`) with the garage, the planned route to the Data Core, the compound and the Rift Gate …

`Scripts/Editor/`

File	Main type	Purpose
`GltfMaterialFixer.cs`	`GltfMaterialFixer`	glTFast's clearcoat Shader Graph outputs invalid colour (flat cyan, i.e. NaN red) at runtime in this URP setup. Renderers using it are remapped to project-owned copies on the plain glTF PBR …

File	Main type	Purpose
`ProjectValidator.cs`	`ProjectValidator`	Checks the wiring that would otherwise only fail at runtime: config, catalogs and build scenes.
`TestRunReporter.cs`	`TestRunReporter`	Runs the EditMode suite and writes a plain-text report to Logs/TestResults-EditMode.txt (for tooling and CI).
`ThirdPartyIntake.cs`	`ThirdPartyIntake`	Re-runnable recipe for every supplied third-party model: vehicle prefabs + definitions, building materials + prefabs + definitions, and both catalogs. Audit notes: Docs/ThirdParty.md.

`Scripts/Editor/Intro/`

File	Main type	Purpose
`IntroBuilder.CityLife.cs`	`IntroBuilder`	Builds the intro's street and sky life (`IntroCityLife`): traffic lanes, drone flights and orbits
`IntroBuilder.cs`	`IntroBuilder`	Generates the opening cinematic into the Night Run scene: a rig (shot cameras, car lineup slots, stage lights, surveillance grade, audio sources, overlay) and a Timeline asset built from …
`IntroCapture.cs`	`IntroCapture`	Review tool: while the intro plays in Play Mode, captures the Game view (with overlay) when the Timeline passes each requested time. Writes _done.txt when finished or when the intro reaches …

`Scripts/Editor/TestTrack/`

File	Main type	Purpose
`ValidationRouteBuilder.cs`	`ValidationRouteBuilder`	Builds the connected physics-validation loop on the dev test track and wires the scene's vehicle camera and dev tools. Re-runnable: the "ValidationRoute" object is rebuilt from scratch each …

`Scripts/Editor/Validation/`

File	Main type	Purpose
`GpuBench.cs`	`GpuBench`	Controlled GPU A/B bench (Play Mode): with the scene held still, applies each URP variant in turn, lets it settle, then averages `FrameTimingManager` GPU time over a fixed number of frames. …
`InputPlaytest.cs`	`InputPlaytest`	Play Mode input checks driven from tooling (the unity CLI / MCP). Keys are written as state events into the real `Keyboard` device, so they travel the same path as physical key presses from …
`MissionPlaytest.cs`	`MissionPlaytest`	Play Mode harness around `NightRunValidator` : starts a scenario on the running mission, captures the Game view (with the HUD) a moment after chosen world events and interaction stages, and …
`MissionSeries.cs`	`MissionSeries`	Runs several `NightRunValidator` scenarios back to back in one Play Mode session (restarting the mission through its normal retry path between runs) and writes a summary table plus every …
`PerfBuild.cs`	`PerfBuild`	Builds the macOS player used for the performance run (release, not development: the numbers should be a player's). Run it with -perfroute (see PerfRouteRunner).
`PlaytestCapture.cs`	`PlaytestCapture`	Timed Play Mode screenshots taken from inside the editor loop: optionally taps a key at t = 0, then captures the Game view (with UI Toolkit) at the given unscaled times. Tool calls stall …
`RoutePerf.cs`	`RoutePerf`	Full-route performance profile: rides along with a running `NightRunValidator` scenario (or any Play Mode session) and samples every frame with `ProfilerRecorder` s – PlayerLoop and …

`Scripts/Editor/Vehicles/`

File	Main type	Purpose
`VehicleLodSetup.cs`	`VehicleLodSetup`	Distance LODs for a gameplay vehicle, built from the renderers it already has (no new meshes): LOD0: everything (the player's car in the chase camera, close-ups); LOD1 below 18 % screen …
`VehicleModelSetup.cs`	`VehicleModelSetup`	Editor-only recipe that turns a third-party car model into a Neon Rift vehicle prefab. Keywords are matched case-insensitively against each renderer's hierarchy path and material names.
`VehiclePerformanceProbe.cs`	`VehiclePerformanceProbe`	Standard manoeuvres run on `VehicleTestBench`: settle, launch, top speed, braking, ramp steer, step steer and bumps. Results feed the automated tests and the "Measure Vehicle Performance" …
`VehiclePrefabBuilder.cs`	`VehiclePrefabBuilder`	Builds a vehicle prefab from a `VehicleModelSetup`: orients and scales the source model, grounds it at the root pivot, splits wheel geometry into four pivoted wheels and records a …
`VehicleTestBench.cs`	`VehicleTestBench`	Isolated, deterministic physics scene for one vehicle: a preview scene with its own physics world, stepped manually at the game's 100 Hz. Used by automated tests and by the performance …

`Scripts/Editor/World/`

File	Main type	Purpose
`BuildingPrefabBuilder.cs`	`BuildingPrefabBuilder`	Wraps third-party building models in prefabs with a ground-centre pivot, optional box collider, and URP materials. Source models are referenced, never modified.
`DroneModels.cs`	`DroneModels`	Prefabs for the two CC BY drone models (see each folder's ATTRIBUTION.txt): the Mech Drone (security pursuit drone: textured URP material, its looping hover animation as a legacy clip, …

`Scripts/Frontend/`

File	Main type	Purpose
`CarSelectScreen.cs`	`CarSelectScreen`	Car Select in the crew garage: browse the vehicle catalog (cards, performance from the physics bench, copy), preview the car on the `VehicleShowroom` turntable, and confirm into the …
`ShowroomEngine.cs`	`ShowroomEngine`	Engine sound for a car standing in the garage, from the car's own recordings in its `VehicleAudioProfile`: the lowest off-load loop idles at the profile's idle rpm, and a blip (selection, …
`TitleScreen.cs`	`TitleScreen`	Title screen entry point: START, WATCH INTRO, OPTIONS, QUIT
`VehicleShowroom.cs`	`VehicleShowroom`	The crew garage behind Car Select. Presents the selected car on a turntable under the stage lights, the rest of the catalog parked in the shadows, and the transitions between them: on a …

`Scripts/Game/`

File	Main type	Purpose
`BootstrapLoader.cs`	`BootstrapLoader`	Guarantees the Bootstrap scene is present no matter which scene Play Mode starts from, so every scene is always entered through the game flow.
`GameConfig.cs`	`GameConfig`	Top-level data the composition root needs: content catalogs and front-end scene names.
`GameContext.cs`	`GameContext`	Everything a scene is allowed to know about the running game. Handed to each scene's `ISceneEntryPoint` by the flow – scenes never look it up globally.

File	Main type	Purpose
`GameFlow.cs`	`GameFlow`	Owns the single loaded content scene. The Bootstrap scene stays loaded underneath; content scenes are loaded additively, activated, then handed the `GameContext`.
`GameRoot.cs`	`GameRoot`	Composition root. Lives in the Bootstrap scene for the whole session and builds the objects every scene shares. This is the only place long-lived services are created.
`GameSettings.cs`	`GameSettings`	Player settings (volumes, music on/off, graphics), kept in PlayerPrefs and applied to the mixer and the quality settings. Shared by the pause menu, the title screen and the global music …
`IGameFlow.cs`	`IGameFlow`	High-level navigation between game states. Requests made during a transition are ignored.
`IntroSettings.cs`	`IntroSettings`	When and how the opening cinematic plays. Data-driven so it never gets in the way during development: play it on the first launch only (default), always, or never; the Title screen can …
`LoadingOverlay.cs`	`LoadingOverlay`	Full-screen fade that hides scene loads. Starts opaque so the empty Bootstrap scene is never seen.
`OptionsMenu.cs`	`OptionsMenu`	The game's menu overlay (UI Toolkit, built in code): a stack of pages of rows. Used as the pause menu in a mission and as the options menu on the title screen. Rows are buttons, sliders …
`PerfRouteArgs.cs`	`PerfRouteArgs`	Command line for the player-build performance run: -perfroute [output folder] boots straight into the default mission with the first car and lets PerfRouteRunner drive the route and write …
`RunSession.cs`	`RunSession`	Choices and results that travel between scenes for one play session.

`Scripts/Gameplay/`

File	Main type	Purpose
`CameraCullDistances.cs`	`CameraCullDistances`	Per-layer cull distances for a camera (Unity does not serialize `layerCullDistances`). Small street furniture on the Detail layer disappears beyond a distance where it is a few pixels …
`CrewGarage.cs`	`CrewGarage`	The crew's garage in the city (the same building as Car Select): its lights come on in banks, its roller door lifts and closes. Used by the intro; dark and shut during missions (its lights …
`DrivingRoute.cs`	`DrivingRoute`	A closed driving line with advisory speeds and named sections. Used by autopilot/AI drivers and by the validation tools that report per-section results. Points are in world space at …
`MissionSceneEntry.cs`	`MissionSceneEntry`	Entry point for any scene that hosts a mission: spawns the selected vehicle, gives it the player's controls, points the camera at it and, if the scene has one, starts the `MissionDirector` …
`PerfRouteRunner.cs`	`PerfRouteRunner`	Player-build performance run: launched with -perfroute (see `PerfRouteArgs`), the game boots straight into the mission, drives the full BoulevardInAlleyOut route with …
`RouteAutopilot.cs`	`RouteAutopilot`	Drives a vehicle along a `DrivingRoute` through the same `IVehicleInputSource` a player uses: pure-pursuit steering and a braking-distance speed planner. Used for physics validation laps; …
`VehicleAudioValidator.cs`	`VehicleAudioValidator`	Development tool: drives a vehicle through a scripted audio test (idle, launch, lift-off, braking to a stop, idle, handbrake slide, wall impact, recovery) through the normal input …

File	Main type	Purpose
`VehicleChaseCamera.cs`	`VehicleChaseCamera`	Chase camera that positions a `CinemachineCamera` behind a vehicle. Heading swings toward the direction of travel when the car slides, the look point leads the velocity, the lens widens …
`VehicleDebugHud.cs`	`VehicleDebugHud`	Development overlay with live vehicle telemetry and per-wheel suspension/tyre state. F3 toggles it. Not part of the player HUD; development builds only.
`VehicleSpawnPoint.cs`	`VehicleSpawnPoint`	Where a vehicle enters a mission scene. Forward (blue axis) is the driving direction.
`VehicleTelemetryLog.cs`	`VehicleTelemetryLog`	Development recorder: samples a vehicle every physics step and summarises the run per route section (speeds, accelerations, slip, body attitude, suspension use, contacts). Used for physics …

`Scripts/Gameplay/City/`

File	Main type	Purpose
`CityTraffic.cs`	`CityTraffic`	City traffic from the generic passenger car pack (`TrafficCar`: real Rigidbodies). Two pools: Roaming: driving the road graph's kerb-side lanes around the player, keeping their distance …
`CrowdGroups.cs`	`CrowdGroups`	Pedestrian silhouettes in small groups (static low-poly figures, no colliders) on plazas, at the night market and on pavements. They give the streets a sense of people at a distance; a …
`HoverSentry.cs`	`HoverSentry`	A large sentry drone holding station over the Data Core compound: a slow bob and a slow scanning yaw. Pure presentation (always on, intro and missions); one transform per sentry.
`LightBudget.cs`	`LightBudget`	Keeps a city's worth of lamp lights affordable: every lamp has a (disabled) real-time light, and only the `maxActive` nearest the viewer (with a preference for what is in front of it) are …
`LockdownScreens.cs`	`LockdownScreens`	Billboards, bus-shelter adverts and street screens that the security grid takes over in a lockdown: each one switches to the warning material as the lockdown wave reaches it (distance from …
`SecurityCamera.cs`	`SecurityCamera`	A street security camera. Idle while the grid is calm; once security is raised (Alert or Lockdown) it tracks the player inside its cone with line of sight, and holding them in view for …
`SecurityDrones.cs`	`SecurityDrones`	Security drones. Dormant until the counter-intrusion or a lockdown; then they lift off from the Data Core and hunt the crews: most of them tail the player, the rest each lock on to a rival …
`TrafficCar.cs`	`TrafficCar`	A physical city car (generic passenger car pack). Rigidbody with four raycast springs holding the body up, lateral tyre grip and a drive/brake force, so it rolls, pitches, gets shunted and …
`TrafficSignalNetwork.cs`	`TrafficSignalNetwork`	Runs the city's traffic signals: every north–south and east–west lens shares one runtime material per colour, so the whole grid cycles (green, amber, all-red) with six material updates. In …

`Scripts/Gameplay/Mission/`

File	Main type	Purpose
`CityMinimap.cs`	`CityMinimap`	North-up minimap drawn with Painter2D: roads of the `RoadNetwork` by class, closed gates in red, the GPS route to the objective, rival and target markers. The map is centred on the player …

File	Main type	Purpose
DataCoreChamber.cs	DataCoreChamber	Presentation of the Data Core heist. Reads world events and the uplink's InteractionRun ; owns no gameplay rules. Phases: Dormant: armoured sleeve closed, core dim, arms folded, holograms …
GateLockSystem.cs	GateLockSystem	The physical locks and status display of a hackable gate (or door, bridge, lift …). World events from the hack drive it: the status lamps go from sealed red to a pulsing amber while a hack …
IMissionTarget.cs	IMissionTarget	Something an objective can point at: a zone to reach or an interactable to use.
IMissionWorld Component.cs	IMissionWorld Component	Implemented by scene objects that take part in a mission (zones, interactables, barriers, lights, alarms). The MissionDirector finds them in its scene and binds them to the running …
Interactable.cs	Interactable	A world object the player uses from the car (terminals, uplinks, payloads): stop inside its trigger and work through its stages with Interact. Its kind comes from an InteractionDefinition …
Interaction Definition.cs	Interaction Definition	What kind of interaction an Interactable is (hack a terminal, extract a core …): its stages, how each is performed and what it costs. Shared by every interactable of that kind, so new …
MinimapProjection. cs	MinimapProjection	North-up minimap projection: world XZ around a centre point maps to panel pixels with +Z (north) pointing up and +X (east) pointing right. The map never rotates; only the player marker …
MissionAudio.cs	MissionAudio	Plays the mission's 2D sounds through the game mixer: ambience on the Ambience group, cues and the interaction loop on UI/SFX. Sources are created once; loops are started once and only …
MissionAudioSet.cs	MissionAudioSet	Non-vehicle sounds of a mission: ambience bed, interaction loop and HUD/story cues.
MissionDirector.cs	MissionDirector	Runs one attempt of a data-driven mission in its scene. Owns the MissionProgress state machine and the MissionWorld hub, binds every IMissionWorldComponent in the scene, routes zone …
MissionHud.cs	MissionHud	The Night Run gameplay HUD (UI Toolkit). A passive view: the MissionDirector pushes state in and receives the two results-screen choices back. Text is only rewritten when the value …
MissionWorld.cs	MissionWorld	What scene objects can see of a running mission: the player vehicle, security state, world events and HUD countdowns. Created by the MissionDirector per attempt and handed to every …
MissionZone.cs	MissionZone	A trigger volume the player can reach (objective destinations, extraction). Lives on the Trigger layer, which only collides with vehicles. Shows its beacon while it is the current objective.
NightRunValidator. cs	NightRunValidator	Development tool: plays a mission end to end in Play Mode through the same interfaces a player uses – the autopilot drives a DrivingRoute , stops at whatever interactable comes into focus …
SecurityAlarm.cs	SecurityAlarm	Lockdown sirens and rotating beacons. Silent and dark until security reaches activeFrom , then each siren fades in after the lockdown wave reaches it. Updates only while active.
SecurityBarrier.cs	SecurityBarrier	A physical barrier the city controls: sliding gates, checkpoint shutters, blast walls. World events open or close it. Closing is staged so the player can read it coming: an optional …
SecurityLightGroup. cs	SecurityLight Group	A set of security-coloured emissive strips and lights (building crowns, kerb lines, gantries) that change colour with the security level. A lockdown spreads outward from the theft: each …

File	Main type	Purpose
`SecurityPostEffects.cs`	`SecurityPostEffects`	Blends a lockdown grade (red push, heavier vignette) over the base night grade when security escalates.
`TerminalDisplay.cs`	`TerminalDisplay`	The physical side of an `Interactable`: its screens (NeonRift/Terminal-Screen) show the stage strip, scrolling data and progress, change colour with the state (standby, working, denied, …
`TerminalReadout.cs`	`TerminalReadout`	HUD view of a multi-stage interaction: device header, the stage list with status, the active stage with its own widget (hold bar, timing track with target window and cursor, interference …

`Scripts/Gameplay/Navigation/`

File	Main type	Purpose
`CityNavigation.cs`	`CityNavigation`	The scene's view of the `RoadNetwork`: plans routes that respect the current state of security gates (a closed gate removes its edge; a pending or closing one is avoided unless nothing else …

`Scripts/Gameplay/Racing/`

File	Main type	Purpose
`RaceMarker.cs`	`RaceMarker`	A place rival drivers head for (named by mission data); its children are the slots they stop in.
`RacerDriver.cs`	`RacerDriver`	AI driver for a `VehicleController`, plugged in through the same `IVehicleInputSource` the player uses: no transforms are moved and no forces are faked. It plans a route over the city's …
`RacerProfile.cs`	`RacerProfile`	How an AI driver drives: commitment in corners and braking, lane craft, obstacle handling, recovery and (optional) rubber-banding. Same physics as the player; only the decisions come from …
`RacingLine.cs`	`RacingLine`	A drivable line built from a `RoadPath`: the road centreline with rounded corners, resampled every 2 m, with per-sample tangent, usable lateral range (kerbs, median) and an advisory speed …
`RivalDirector.cs`	`RivalDirector`	Rival crews in a mission: spawns the cars the player did not pick through the normal spawn path (same prefab, physics profile and audio), gives each a `RacerDriver`, and runs each crew …
`WaypointDriver.cs`	`WaypointDriver`	Drives a car along a short scripted polyline (cinematics: cars pulling out of a garage onto the street) through the normal input path: pure-pursuit steering and a speed target per waypoint …

`Scripts/Input/`

File	Main type	Purpose
`NeonRiftControls.cs`	`NeonRiftControls`	Generated Input System wrapper for NeonRiftControls.inputactions
`PlayerDrivingInput.cs`	`PlayerDrivingInput`	Bridges the Driving action map to the device-agnostic `IVehicleInputSource`.

`Scripts/Intro/`

File	Main type	Purpose
`IntroCityLife.cs`	`IntroCityLife`	Street and sky life for the opening cinematic, driven by the intro Timeline's clock so every shot looks the same on every play (and after a Skip): lanes of traffic (visual-only copies of …

File	Main type	Purpose
`IntroCue.cs`	`IntroCueKind`	What an intro cue clip does while the Timeline plays it.
`IntroCueClip.cs`	`IntroCueClip`	Timeline clip asset for one intro cue (kind, text, subtitle)
`IntroCueTrack.cs`	`IntroCueTrack`	Timeline track of intro cues, bound to the object that handles them.
`IntroOverlay.cs`	`IntroOverlay`	The cinematic's 2D layer (UI Toolkit): black fades, white flashes, letterbox, text cards, the title reveal, the skip hint and the security-camera view. Everything is driven explicitly by …
`IntroSceneEntry.cs`	`IntroSceneEntry`	Runs the opening cinematic in the city scene (the mission rig stays idle): stages the car lineup, plays the generated Timeline (camera shots, cues, audio), handles Skip, holds on the NEON …

`Scripts/Missions/`

File	Main type	Purpose
`InteractionRun.cs`	`InteractionRun`	Runtime of one multi-stage interaction (hack a terminal, extract a core …): walks an ordered list of `InteractionStep`s, each with its own input rule, and reports what happens through …
`InteractionStep.cs`	`InteractionStep`	One stage of a multi-stage interaction (CONNECTING, AUTHENTICATING, BYPASSING …). Pure data; run by `InteractionRun`. Steps raise world events as they start and finish, so the environment …
`MessageTone.cs`	`MessageTone`	How the HUD presents a mission message.
`MissionAnnouncement.cs`	`MissionAnnouncement`	A HUD message shown when a world event fires (mission narrative lives in data, not code).
`MissionDefinition.cs`	`MissionDefinition`	A playable mission: which scene hosts it, how it is presented, and its ordered objectives. The scene supplies the targets (zones, interactables) and the reactions to world events; this …
`MissionGrade.cs`	`MissionGrade`	The debrief grade of a finished run: pace against the mission's par time, how quiet the job stayed (heat), the crew finishing order and clean driving (hard contacts), as a 0–100 score and a …
`MissionPhase.cs`	`MissionPhase`	NotStarted / Running / Completed / Failed
`MissionProgress.cs`	`MissionProgress`	Runtime state of one mission attempt: current objective, security level, heat and the objective timer. Plain C#, driven by the scene through `Tick`, `NotifyReached`, `NotifyInteracted` and …
`MissionRecords.cs`	`MissionRecords`	Personal bests per mission (completed runs only), kept in PlayerPrefs.
`ObjectiveDefinition.cs`	`ObjectiveDefinition`	One step of a mission. Objectives run in order; each one names a scene target (zone or interactable id), what it means for security, and which world events it fires. Scene objects react to …
`ObjectiveKind.cs`	`ObjectiveKind`	Reach (zone) or Interact (interactable)
`RivalTask.cs`	`RivalTask`	One stage of a rival crew's own heist, run in parallel with the player's: drive to a race marker (per crew, "via>goal" routes allowed), stop there and work for a while (a relay hack, an …
`RunResult.cs`	`RunResult`	Outcome of one mission attempt, handed from the mission scene to the results screen.
`ScoreMix.cs`	`ScoreMix`	How the adaptive score is mixed from the mission state (pure logic). Calm driving is the pad with a light groove; heist work brings the arpeggio in with the interaction's tension; an alert …
`SecurityLevel.cs`	`SecurityLevel`	How aware the city's security grid is of the player. Only ever escalates during a mission.

`Scripts/Rendering/`

File	Main type	Purpose
`GroundHaze.cs`	`GroundHaze`	Ground-hugging haze (exponential height fog) layered on top of URP's distance fog: the driving foreground stays clear (`startDistance`), haze thickens with distance close to the ground, and …
`GroundHaze Feature.cs`	`GroundHaze Feature`	Draws `GroundHaze` after opaques and the sky, before transparents: one full-screen triangle that reads the camera depth texture and alpha-blends haze over the colour target. No extra colour …

`Scripts/Tests/EditMode/`

File	Main type	Purpose
`CarSelectTests.cs`	`CarSelectTests`	Car Select garage: scene wiring (generated by GarageBuilder) and the catalog data it presents.
`CityTests.cs`	`CityTests`	City layout and road graph (pure data/logic), racing lines, the rubber band and the rival field.
`FoundationTests.cs`	`FoundationTests`	Catalog, session and project-wiring tests
`InputTests.cs`	`InputTests`	Drives virtual devices through the real Driving action map and checks the values that reach `PlayerDrivingInput` , the source the player's VehicleController reads every step.
`InteractionTests.cs`	`InteractionTests`	The multi-stage interaction runner (pure logic) and objective start delays.
`IntroTests.cs`	`IntroTests`	Intro policy, video fallback and scene wiring tests
`MinimapTests.cs`	`MinimapTests`	North-up minimap projection: the map never rotates, only the markers do.
`MissionTests.cs`	`MissionTests`	Mission state machine (pure logic) and the Night Run data/scene wiring.
`VehicleAudio Tests.cs`	`VehicleAudioTests`	Engine blend model, tyre model, generated clips, mixer set-up, and VehicleAudio driven by real bench telemetry.
`VehicleModel Tests.cs`	`VehicleModelTests`	Pure-model tests: tyre, steering, gearbox, drivetrain and profile data. No physics scene.
`VehiclePhysics Tests.cs`	`VehiclePhysics Tests`	Every catalog vehicle driven on the deterministic bench (isolated physics scene, 100 Hz). Bands are wide enough for tuning but catch broken physics: instability, jitter, NaNs, spins from …

`Scripts/Vehicles/Audio/`

File	Main type	Purpose
`ChassisAudio Settings.cs`	`ChassisAudio Settings`	Tyre, road, wind, suspension and impact sounds.
`EngineAudio Settings.cs`	`EngineAudio Settings`	How an engine's loops are blended from telemetry.
`EngineSound Layer.cs`	`EngineSoundLayer`	One seamless engine loop recorded (or synthesised) at a fixed rpm and load.
`VehicleAudio Profile.cs`	`VehicleAudio Profile`	A car's sound. Referenced from `VehicleDefinition` and played by the audio module from the vehicle's telemetry, so any car can use any profile and no car has its own audio code.

`Scripts/Vehicles/`

File	Main type	Purpose
`DrivingInput.cs`	`DrivingInput`	Normalised driver intent, identical for player and AI drivers.
`VehicleCatalog.cs`	`VehicleCatalog`	Ordered list of vehicles available to the player. Adding a car = adding a definition here.
`VehicleDefinition.cs`	`VehicleDefinition`	Identity and presentation data for one selectable vehicle.
`VehicleDisplayStats.cs`	`VehicleDisplayStats`	Player-facing figures shown in Car Select. Authored for now; once the physics profile exists these must be checked against measured test-track results.
`VehicleLights.cs`	`VehicleLights`	Real lights for a car at night, placed from its `VehicleRig` : two headlight spots (the player's left one casts shadows), and a red tail light that brightens under braking and turns white in …
`VehicleRig.cs`	`VehicleRig`	Describes the visual structure of a vehicle prefab (body, wheels, lamps) so physics, lighting and presentation systems can drive it without knowing the source model's hierarchy. Root pivot: …

`Scripts/Vehicles/Physics/`

File	Main type	Purpose
`AeroSettings.cs`	`AeroSettings`	Aerodynamic coefficients multiplied by reference area (C·A, m²). Forces scale with speed squared.
`AssistSettings.cs`	`AssistSettings`	Driver aids. They act through the same physical channels a real car uses (throttle, brake torque).
`AxleSettings.cs`	`AxleSettings`	Suspension, wheel and tyre set-up for one axle. Ride height comes from the model's wheel positions.
`BrakeSettings.cs`	`BrakeSettings`	Brake torque, front bias, handbrake torque, ABS settings
`ChassisSettings.cs`	`ChassisSettings`	Mass properties of the sprung body.
`DriveType.cs`	`DriveType`	RearWheelDrive / FrontWheelDrive / AllWheelDrive
`Drivetrain.cs`	`Drivetrain`	Engine, clutch and gearbox. Turns pedal demand into drive torque at the driven wheels and keeps a physical engine speed: locked to the wheels when the clutch is engaged, free-revving …
`DrivingSurface.cs`	`DrivingSurface`	Tyre-relevant properties of a drivable collider (on the collider or a parent). Colliders without one count as dry asphalt.
`EngineSettings.cs`	`EngineSettings`	Engine or traction motor. Torque = peak torque × curve(rpm / max rpm) × throttle.
`Gearbox.cs`	`Gearbox`	Gear state and automatic shift logic. Gear −1 = reverse, 0 = neutral, 1..n = forward. A shift takes `shiftTime` with the clutch open; the new gear is reported immediately.
`ScriptedDrivingInput.cs`	`ScriptedDrivingInput`	Input source whose values are set from code: tests, cutscenes, autopilots.
`SteeringSettings.cs`	`SteeringSettings`	Steering angle = smoothed input × speed-dependent limit. The limit falls with speed toward the angle the front tyres can actually use (kinematic angle for the grip-limited corner + peak …
`SteeringSystem.cs`	`SteeringSystem`	Rate-limited steering with a speed-dependent lock and Ackermann geometry. Angles are road-wheel angles in degrees, positive = right.
`TransmissionSettings.cs`	`TransmissionSettings`	Gearbox, final drive and differentials. Shifting is automatic; manual shifting can call the same gearbox.

File	Main type	Purpose
`TyreModel.cs`	`TyreModel`	Combined-slip tyre force. Longitudinal and lateral slip are normalised by their peak values and combined into one slip magnitude, so braking or accelerating while cornering trades lateral …
`TyreSettings.cs`	`TyreSettings`	Combined-slip tyre: force rises to a peak at the peak slip ratio / angle, then falls toward the sliding grip. Grip values are friction coefficients on a reference asphalt surface (grip 1).
`VehicleCollision.cs`	`VehicleCollision`	A body collision reported by `Collided` (damage, audio, camera shake).
`VehicleController.cs`	`VehicleController`	Rigidbody vehicle with raycast suspension, a combined-slip tyre model, engine/gearbox drivetrain, anti-roll bars and aerodynamics. All tuning comes from a `VehiclePhysicsProfile`; all …
`VehiclePhysicsProfile.cs`	`VehiclePhysicsProfile`	Everything that gives a car its driving personality. Referenced by `VehicleDefinition` and applied to the spawned prefab by `Configure`. Geometry (wheel positions, radii, ride height) comes …
`VehicleTelemetry.cs`	`VehicleTelemetry`	Snapshot of a vehicle's state after its last physics step. Intended for engine/tyre audio, HUD, camera and AI. Per-wheel detail is on `Wheels`.
`VehicleUnits.cs`	`VehicleUnits`	Unit constants (gravity, km/h ↔ m/s)
`VehicleWheel.cs`	`VehicleWheel`	One corner: raycast suspension (spring, damper, bump stop), wheel spin and tyre forces. State is public for telemetry and debugging; only `VehicleController` advances it.

`Scripts/World/`

File	Main type	Purpose
`BuildingCatalog.cs`	`BuildingCatalog`	All building prefabs available to city layout, grouped by tier.
`BuildingDefinition.cs`	`BuildingDefinition`	One placeable building prefab and the facts the city layout needs about it.
`RoadGraph.cs`	`RoadGraph`	Shortest paths over a `RoadNetwork` between arbitrary positions (each end is projected onto its nearest edge). Edges can be blocked or penalised per query, so routes follow gates as they …
`RoadNetwork.cs`	`RoadNetwork`	The drivable road graph of a city: nodes at intersections and road ends, edges with width, lanes, class, street name and an optional blocker id (security gates). Generated with the …

B

Documentation Audit

IN THIS APPENDIX

Method
System coverage
Checks requested
Findings (outside the book)
Could not be fully traced
PDF edition (NEON_RIFT_COMPLETE_TECHNICAL_BOOK.pdf)

Audit of `Docs/NEON_RIFT_BOOK/` against the repository at commit `602c7130` (Unity 6000.6.4f1), carried out on 2026-10-08.

Method

1. **Inventory:** all 198 C# files were parsed for their types and XML summaries; method signatures with line numbers were extracted; scenes were parsed for component counts and project MonoBehaviours; prefabs, data assets, UI files, shaders, packages, project settings and the input-actions asset were listed.
2. **Source reading:** the key runtime and builder code was read directly (game flow, spawn, vehicle step, tyre model, rival director and driver, mission state machine, director, interaction definitions, builders for gates, barriers, lighting, rivals, intro shots and cues).
3. **Automated reference checks** over every chapter:
   - **831** backticked file paths checked for existence. Only intentional exceptions remain: git-ignored or local folders (`Temp/`, `Logs/`, `Builds/`), folder names given relative to a stated parent (`Engine_EV/`, `Mission/`), and the three `Docs/Licenses/*.png` files (see Findings).
   - **427** `Class.Member` references checked against the defining source files: **0 missing**.
   - Bare backticked identifiers checked against the code and asset names. The only names not found are Unity engine types (`WheelCollider`, `ParticleSystem`, `EventSystem`, `InputSystemUIInputModule`, `UnityStats`) and the scene object name `RivalWorkFx_n`, all used correctly.
   - Every `](….md)` link between chapters resolves.

System coverage

System	Documented?	Primary files verified?	Scene verified?	Data verified?	Cross-references verified?
Boot, flow, scenes	✔ (05, 06)	✔ `GameRoot`, `GameFlow`, `BootstrapLoader`	✔ build list, Bootstrap contents	✔ `GameConfig.asset`	✔
City generation	✔ (07)	✔ `NightRunBuilder.Build` order read	✔ NightRun component counts	✔ `CityLayout` data	✔
Buildings and environment	✔ (08)	✔	✔	✔ building catalog and prefabs	✔
Roads and navigation	✔ (09)	✔ `RoadGraph`, `CityNavigation`	✔	✔ network asset, gate ids	✔
Vehicle system and physics	✔ (10, 11)	✔ `VehicleController.Step` read	✔	✔ profile values read from assets	✔
Vehicle audio	✔ (12)	✔	–	✔ audio profiles, generated folders	✔
Vehicle visuals	✔ (13)	✔ `UpdateVisuals`, lights, LODs	–	✔ prefabs	✔
Catalog and data	✔ (14)	✔	–	✔ ids, names, categories, licences from assets	✔
Car Select	✔ (15)	✔	✔ CarSelect components	✔ UXML element names	✔
Input	✔ (16)	✔	–	✔ bindings parsed from `.inputactions`; M binding in `GameRoot`	✔
Rival AI	✔ (17)	✔ `RivalDirector`, `RacerDriver.ReadInput`	✔ 5 race markers	✔ task data, profiles	✔
Traffic	✔ (18)	✔	✔	✔	✔
Mission system	✔ (19)	✔ `MissionProgress` read in full	✔ 2 zones, 3 interactables	✔ `Mission_NightRun.asset` read	✔
Heist interactions	✔ (20)	✔ stage definitions read from the builder	✔	✔	✔
Security gate	✔ (21)	✔	✔	✔	✔
Data Core	✔ (22)	✔ (phases from the class summary and method list)	✔	✔	✔
Heat and lockdown	✔ (23)	✔ event wiring grepped in the builder	✔ 5 barriers, 12 cameras, 35 light groups, 2 alarms	✔	✔
Environmental interactions	✔ (24)	✔	✔	–	✔
Lighting	✔ (25)	✔ `BuildLighting`, GI settings	✔ 862 lights, 63 probes	✔ RP assets, volumes	✔

System	Documented?	Primary files verified?	Scene verified?	Data verified?	Cross-references verified?
Weather / atmosphere	✔ (26): rain stated **not implemented**	✔	–	✔	✔
Water and effects	✔ (27): water stated **not implemented**	✔ asphalt puddle code	–	–	✔
Particles and VFX	✔ (28): **no ParticleSystem anywhere** (scenes, prefabs, scripts checked)	✔	✔	–	✔
Audio	✔ (29)	✔	✔ limiter in NightRun and CarSelect	✔ mixer config, audio set fields, clip lists	✔
Music	✔ (30)	✔	–	✔	✔
Cinematic intro	✔ (31)	✔ shot table and cue table read from `IntroBuilder`	✔ 25 Animators, 1 PlayableDirector	✔ Timeline, settings	✔
Timeline and cinematic camera	✔ (32)	✔	✔	✔	✔
UI and HUD	✔ (33)	✔ setters with line numbers	–	✔ UXML element names	✔
Minimap	✔ (34)	✔	–	–	✔
Mission presentation	✔ (35)	✔	–	✔ announcements from the builder	✔
Results and debrief	✔ (36)	✔ `MissionGrade.Evaluate` read	–	–	✔
Animation systems	✔ (37)	✔	✔	✔	✔
Camera	✔ (38)	✔ serialized defaults + builder overrides	✔	–	✔
Performance	✔ (39): numbers only from recorded docs	✔ tools	–	✔ RP asset settings	✔
Testing	✔ (40, 41)	✔ 98 methods / 125 cases counted	–	–	✔
Build and release	✔ (42)	✔ `PerfBuild`	✔ build list	✔ project settings	✔
Git and repository	✔ (43)	–	–	✔ 29 commits, LFS rules	✔
Assets and licensing	✔ (44)	–	–	✔ every `ATTRIBUTION.txt`, `LICENSE.md`, definition licences	✔

Checks requested

Check	Result
Every major system has a chapter	✔
Every chapter points to actual files	✔ (automated)
Every important system has a "where is the code" section	✔
No fictional files or classes	✔ (automated, 0 missing members)
No obsolete implementation presented as current	✔. Obsolete items in older notes are flagged (below)
Unity and package information correct	✔ from `ProjectVersion.txt` and `manifest.json`; unused packages marked
Scene names correct	✔ Bootstrap, Frontend, CarSelect, Dev/TestTrack, NightRun
Controls correct	✔ parsed from the input asset and `GameRoot`
Vehicle names correct	✔ SLS AMG, SLS AMG GT3, Terzo Millennio (catalog); AMG GT3 excluded
Mission flow correct	✔ from `Mission_NightRun.asset` and `CreateMissionData`
Rival systems correct	✔ current `RivalTask` system (not the removed `RivalOrder`)
Licensing correct	✔ copied from attribution files; unknown licences stated as unknown

Findings (outside the book)

1. **Older notes describe superseded behaviour.** In `Docs/Decisions.md` D26 (rival goals per objective via `rivalGoalId`, removed in `602c7130`); parts of `Docs/NightRun.md` (rival orders, crew roles NORTH GATE OVERWATCH / EXPRESSWAY SCOUT, "4 small particle systems", rivals parking in the S7 car park); the first section of `Docs/Intro.md` (62 s / 18 cameras, the earlier version); `Docs/Architecture.md` (Frontend list without `ShowroomEngine`); and the README ("heading-up minimap"). Per the brief, these files were **not modified**; the book documents the current code.
2. **Licence screenshots deleted in the working tree.** `Docs/Licenses/*.png` are committed (in `602c7130`) but currently deleted in the local working copy. This pre-existing change was left untouched and is not part of the documentation commit.
3. **Licences not recorded** for the generic passenger car pack and the two music tracks; `LICENSE.md` does not list the drones, the car pack or the music.
4. **Installed but unused packages:** Splines, ProBuilder, AI Navigation.

Could not be fully traced

- **Exact per-frame behaviour inside `DataCoreChamber`** (668 lines) is described from its class summary, method structure and the recorded heist documentation, not line by line.
- **`CityInfill` and `CityProps` internals** are summarised at the algorithm level (method names verified); numeric counts come from the recorded build logs in `Docs/City.md`.

- **Performance figures** are reproduced from recorded measurements; none were re-measured for this book, and no player-build profile exists after the latest content additions.
- **Mixer snapshot dB offsets** are taken from `Docs/Audio.md`; the `.mixer` asset itself was not decoded.

PDF edition (`NEON_RIFT_COMPLETE_TECHNICAL_BOOK.pdf`)

The PDF is compiled from the Markdown chapters in this folder. Its content is the same as theirs; only the presentation changes. Notes on the compilation:

- **Verification status.** The 831-path and 427-member checks above were made against `602c7130`. The commits since then change only `README.md` and `Docs/` (`git diff --stat 602c7130 HEAD` shows no code or asset change), so those results still apply. The PDF was not re-verified line by line. Two checks were made during compilation:
  - **Paths:** Find It Fast and the Q&A cards spell out the `S/` and `D/` shorthand, and expand 35 bare file names (for example `CityLayout.cs`) to full paths when exactly one tracked file has that name. All **98** full paths shown in those two sections exist in the repository.
  - **Destinations:** every chapter, part and figure in the TOC has a link destination in the PDF.
- **Added diagrams.** Three diagrams have no Markdown source: 17.2 Rival heist flow, 21.1 Security Gate hack flow and 22.2 Data Core heist flow. Each one only restates the steps, events, durations and ids written in the text of its chapter. The intro timeline (31.2) is drawn from the shot and cue tables in chapter 31.
- **Layout-only changes.**
  - Some diagrams are laid out left-to-right instead of top-to-bottom (or the reverse) so they fit the page.
  - The 10-column physics-profile table in chapter 14 is transposed.
  - The shot table in chapter 31 is shown as one card per shot. The column "Objects" is labelled "Action / on screen".
  - Section headings that carried their own number ("1. High-level shape") use the book's section number instead.
  - The "A. Table of contents" of `00_INDEX.md` is replaced by the PDF's own Contents.
- **Licensing at a glance** (chapter 44) regroups the chapter's existing licence facts into categories. It adds no new licence information.
- **Screenshots** come from `Docs/Screenshots/`: real captures made with the project's capture tools during development. Some may predate small later changes. No image was created or edited for the book.
- **Discrepancies found while compiling:** none in the technical content. The cue table and the shot table in chapter 31 agree on the 25 shots (shots 19–24 being the montage). The question counts (36), figure counts and chapter list (48 plus two appendices) match the Markdown.